

O'REILLY®

Compliments of
neo4j

Graph Algorithms

Practical Examples in Apache Spark & Neo4j



**Mark Needham &
Amy E. Hodler**

(Data Scientists)-[:LOVE]->(Neo4j)



POWERFUL • PRACTICAL • INTELLIGENT

Neo4j is the #1 platform for **connected data**, with native graph analytics, storage and processing. Graph algorithms in Neo4j reveal hidden patterns and enhance machine learning predictions.

[Try Neo4j Now](#)



Algoritmos gráficos

Ejemplos prácticos en Apache Spark y Neo4j

Mark Needham y Amy E. Hodler

Algoritmos gráficos de Mark Needham y Amy E. Hodler

Copyright © 2019 Amy Hodler y Mark Needham. Todos los derechos reservados.

Impreso en los Estados Unidos de América.

Publicado por O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

Los libros de O'Reilly pueden adquirirse con fines educativos, comerciales o promocionales. También hay ediciones en línea disponibles para la mayoría de los títulos (<http://oreilly.com>). Para más información, contacte con nuestro departamento de ventas corporativas/institucionales: 800-998-9938 o corporate@oreilly.com.

Editor de adquisiciones: Jonathan Hassell

Editor: Jeff Bleiel

Editora de producción: Deborah Baker

Editor de copias: Tracy Brown

Correctora: Rachel Head

Indexador: Judy McConville

Diseñador de interiores: David Futato

Diseñador de portada: Karen Montgomery

Ilustradora: Rebecca Demarest

Mayo de 2019: Primera edición

Historial de revisiones de la primera edición

15/04/2019: Primer lanzamiento

Consulte <http://oreilly.com/catalog/errata.csp?isbn=9781492047681> para obtener detalles del lanzamiento.

El logotipo de O'Reilly es una marca registrada de O'Reilly Media, Inc. Graph Algorithms, la imagen de portada de una araña de jardín europea y la imagen comercial relacionada son marcas comerciales de O'Reilly Media, Inc.

Si bien el editor y los autores se han esforzado de buena fe para garantizar la precisión de la información y las instrucciones contenidas en esta obra, declinan toda responsabilidad por errores u omisiones, incluyendo, entre otros, la responsabilidad por daños derivados del uso o la confianza depositada en esta obra. El uso de la información y las instrucciones contenidas en esta obra es bajo su propia responsabilidad. Si algún ejemplo de código u otra tecnología que esta obra contiene o describe está sujeta a licencias de código abierto o a derechos de propiedad intelectual de terceros, es su responsabilidad asegurarse de que su uso cumpla con dichas licencias o derechos.

Este trabajo forma parte de una colaboración entre O'Reilly y Neo4j. Consulte nuestra [declaración de independencia editorial](#).

978-1-492-05781-9

[LSI]

Tabla de contenido

Prefacio. ...	ix
Prólogo. ...	xiii
1. Introducción. ...	1
¿Qué son los gráficos?	2
¿Qué son el análisis de gráficos y los algoritmos?	3
Procesamiento de gráficos, bases de datos, consultas y algoritmos	6
OLTP y OLAP	7
¿Por qué deberíamos preocuparnos por los algoritmos gráficos?	8
Conclusión sobre los casos de uso del	12
análisis de gráficos	14
2. Teoría y conceptos de grafos. ...	
Terminología	15
Tipos y estructuras de gráficos	16
Estructuras aleatorias, de mundo pequeño y sin escala	17
Sabores de gráficos	18
Gráficos conectados versus gráficos desconectados	19
Gráficos no ponderados versus gráficos ponderados	19
Grafos no dirigidos versus grafos dirigidos	21
Gráficos acíclicos versus gráficos cíclicos	22
Gráficos dispersos versus gráficos densos	23
Gráficos monopartitos, bipartitos y k-partitos	24
Tipos de algoritmos gráficos	27
Búsqueda de caminos	27
Centralidad	27
Detección de la comunidad	27

Resumen	28
3. Plataformas gráficas y procesamiento. ...	
Consideraciones sobre la plataforma gráfica y el procesamiento	29
Consideraciones sobre la plataforma	29
Consideraciones de procesamiento	30
Plataformas representativas	31
Seleccionando nuestra plataforma	31
Apache Spark	32
Plataforma de gráficos Neo4j	34
Resumen	37
4. Algoritmos de búsqueda de rutas y grafos. ...	
Datos de ejemplo: El gráfico de transporte	41
Importación de datos a Apache Spark	43
Importando los datos a Neo4j	44
Búsqueda en amplitud	45
Búsqueda en amplitud con Apache Spark	46
Búsqueda en profundidad	48
Ruta más corta	49
¿Cuándo debo utilizar la ruta más corta?	50
Ruta más corta con Neo4j Ruta	51
más corta (ponderada) con Neo4j Ruta más corta	53
(ponderada) con Apache Spark Variación de la ruta más	54
corta: A* Variación de la ruta más	56
corta: k-rutas más cortas de Yen	58
La ruta más corta de todos los	60
pares Una mirada más de cerca a la ruta más corta de todos	60
los pares ¿Cuándo debería usar la ruta más corta de todos los pares?	62
La ruta más corta de todos los pares con Apache Spark	62
La ruta más corta de todos los pares con	63
Neo4j La ruta más corta de una sola	65
fuelle ¿Cuándo debería utilizar la ruta más corta de una sola fuente?	67
Ruta más corta de una sola fuente con Apache Spark Ruta	67
más corta de una sola fuente con Neo4j Árbol de	69
expansión mínimo ¿Cuándo debo	70
utilizar el árbol de expansión mínimo?	71
Árbol de expansión mínimo con paseo aleatorio de	72
Neo4j ¿Cuándo	73
debo utilizar paseo aleatorio?	74
Resumen de paseo aleatorio con	74
Neo4j	75

5. Algoritmos de centralidad.	77
Ejemplo de datos de gráficos: El gráfico social	79
Importar los datos a Apache Spark	80
Importar los datos a Neo4j	81
Alcance de centralidad	81
de grado ¿Cuándo debo	81
utilizar la centralidad de grado?	82
Centralidad de grado con Apache Spark	83
Centralidad de cercanía	84
¿Cuándo debo utilizar la centralidad de cercanía?	85
Centralidad de cercanía con Apache Spark	86
Centralidad de cercanía con Neo4j	88
Variación de centralidad de cercanía: Wasserman y Faust	89
Variación de centralidad de cercanía: centralidad armónica	91
Centralidad de intermediación	92
¿Cuándo debo utilizar la centralidad de intermediación?	94
Centralidad de intermediación con Neo4j	95
Variación de centralidad de intermediación: Brandes aleatorio-aproximado	98
PageRank	99
Iteración de	99
la fórmula de PageRank,	100
navegantes aleatorios y sumideros de rango ¿Cuándo	102
debo usar PageRank?	103
PageRank con Apache Spark	103
PageRank con Neo4j	105
Variación de PageRank: Resumen de PageRank	107
personalizado	108
 6. Algoritmos de detección de la comunidad. ...	 112
Ejemplo de datos de gráfico: el gráfico de dependencia del software	112
Importar los datos a Apache Spark	114
Importar los datos a Neo4j	114
Recuento de triángulos	114
y coeficiente de agrupamiento	114
Coeficiente de agrupamiento local	115
Coeficiente de agrupamiento global	116
¿Cuándo debo utilizar el recuento de triángulos y el coeficiente de agrupamiento?	116
Conteo de triángulos con Apache Spark	117
Triángulos con Neo4j	117
Coeficiente de agrupamiento local con Neo4j	118
Componentes fuertemente conectados	119
¿Cuándo debo utilizar componentes fuertemente conectados?	120
Componentes fuertemente conectados con Apache Spark	120

Componentes fuertemente conectados con Neo4j	122
Componentes conectados	124
¿Cuándo debo utilizar componentes conectados?	124
Componentes conectados con Apache Spark	125
Componentes conectados con Neo4j	126
Propagación de	127
etiquetas: aprendizaje semisupervisado y etiquetas de	129
semillas ¿Cuándo debo utilizar la propagación de etiquetas?	129
Propagación de etiquetas con Apache Spark	130
Propagación de etiquetas con Neo4j	131
Modularidad de Louvain	133
¿Cuándo debo usar Louvain?	137
Louvain con Neo4j	138
Validando comunidades	143
Resumen	143

7. Algoritmos gráficos en la práctica. ...

Análisis de datos de Yelp con Neo4j	145
Red social Yelp	146
Importación de datos	147
Modelo gráfico	147
Una descripción general rápida de los datos de Yelp	148
Aplicación de planificación de viajes	152
Consultoría de negocios de viajes	157
Encontrar categorías similares	162
Análisis de datos de vuelos de aerolíneas con Apache Spark	166
Análisis exploratorio	168
Aeropuertos populares	168
Retrasos de ORD	170
Mal día en SFO	172
Aeropuertos interconectados por aerolínea	174
Resumen	181

8. Uso de algoritmos gráficos para mejorar el aprendizaje automático. ...

Aprendizaje automático y la importancia del contexto	183
Gráficos, contexto y precisión	184
Extracción y selección de características conectadas	185
Características de Graphy	187
Características del algoritmo gráfico	188
Gráficos y aprendizaje automático en la práctica: predicción de enlaces	190
Herramientas y datos	190
Importando los datos a Neo4j	192

El gráfico de coautoría	193
Creación de conjuntos de datos de entrenamiento y prueba equilibrados	194
Cómo predecimos los eslabones perdidos	199
Creación de una canalización de aprendizaje automático	200
Predicción de enlaces: características básicas de los gráficos	201
Predicción de vínculos: Triángulos y el coeficiente de agrupamiento	214
Predicción de vínculos: detección de comunidades	218
Resumen	224
Resumiendo	224
A. Información y recursos adicionales. ...	
Índice. ...	

Prefacio

El mundo se rige por conexiones, desde los sistemas financieros y de comunicación hasta los procesos sociales y biológicos. Descubrir el significado de estas conexiones impulsa avances en diferentes sectores, desde la identificación de redes de fraude y la optimización de recomendaciones hasta la evaluación de la solidez de un grupo y la predicción de fallos en cascada.

A medida que la conectividad continúa acelerándose, no sorprende que el interés en los algoritmos de grafos se haya disparado, ya que se basan en matemáticas desarrolladas explícitamente para obtener información de las relaciones entre los datos. El análisis de grafos puede revelar el funcionamiento de sistemas y redes complejos a gran escala, para cualquier organización.

Nos apasiona la utilidad e importancia del análisis de grafos, así como el placer de descubrir el funcionamiento interno de escenarios complejos. Hasta hace poco, adoptar el análisis de grafos requería gran experiencia y determinación, ya que las herramientas y las integraciones eran complejas y pocos sabían cómo aplicar algoritmos de grafos a sus problemas. Nuestro objetivo es ayudar a cambiar esto. Escribimos este libro para ayudar a las organizaciones a aprovechar mejor el análisis de grafos para que puedan realizar nuevos descubrimientos y desarrollar soluciones inteligentes con mayor rapidez.

¿Qué hay en este libro?

Este libro es una guía práctica para iniciarse en algoritmos de grafos, dirigida a desarrolladores y científicos de datos con experiencia en Apache Spark™ o Neo4j. Si bien nuestros ejemplos de algoritmos utilizan las plataformas Spark y Neo4j, este libro también será útil para comprender conceptos más generales sobre grafos, independientemente de la tecnología que elija.

Los dos primeros capítulos ofrecen una introducción al análisis de grafos, los algoritmos y la teoría. El tercer capítulo abarca brevemente las plataformas utilizadas en este libro antes de profundizar en tres capítulos centrados en algoritmos de grafos clásicos: búsqueda de rutas, centralidad y detección de comunidades. Concluimos el libro con dos capítulos que muestran cómo...

Los algoritmos gráficos se utilizan dentro de los flujos de trabajo: uno para el análisis general y otro para el aprendizaje automático.

Al inicio de cada categoría de algoritmos, hay una tabla de referencia que te ayudará a acceder rápidamente al algoritmo correspondiente. Para cada algoritmo, encontrarás:

- Una explicación de lo que hace el algoritmo • Casos de uso

para el algoritmo y referencias donde puede obtener más información • Código de ejemplo que proporciona

formas concretas de usar el algoritmo en Spark, Neo4j o ambos

Convenciones utilizadas en este libro

En este libro se utilizan las siguientes convenciones tipográficas:

Cursiva

Indica nuevos términos, URL, direcciones de correo electrónico, nombres de archivos y extensiones de archivos.

Ancho constante

Se utiliza para listados de programas, así como dentro de párrafos para hacer referencia a elementos del programa como nombres de variables o funciones, bases de datos, tipos de datos, variables de entorno, declaraciones y palabras clave.

Negrita de ancho constante

Muestra comandos u otro texto que el usuario debe escribir literalmente.

Cursiva de ancho constante

Muestra texto que debe reemplazarse con valores proporcionados por el usuario o por valores determinados por el contexto.



Este elemento significa un consejo o sugerencia.



Este elemento significa una nota general.



Este elemento indica una advertencia o precaución.

Uso de ejemplos de código

El material complementario (ejemplos de código, ejercicios, etc.) está disponible para descargar en <https://bit.ly/2FPgGVV>.

Este libro está aquí para ayudarle a realizar su trabajo. En general, si se ofrece código de ejemplo con este libro, puede usarlo en sus programas y documentación. No necesita contactarnos para solicitar permiso a menos que esté reproduciendo una parte significativa del código. Por ejemplo, escribir un programa que utilice varios fragmentos de código de este libro no requiere permiso. Vender o distribuir un CD-ROM con ejemplos de los libros de O'Reilly sí requiere permiso. Responder a una pregunta citando este libro y citando código de ejemplo no requiere permiso. Incorporar una cantidad significativa de código de ejemplo de este libro en la documentación de su producto sí requiere permiso.

Agradecemos, pero no exigimos, la atribución. Una atribución suele incluir el título, el autor, la editorial y el ISBN. Por ejemplo: «Algoritmos de grafos por Amy E. Hodler y Mark Needham (O'Reilly). Copyright 2019 Amy E. Hodler y Mark Needham, 978-1-492-05781-9».

Si considera que su uso de ejemplos de código no se ajusta al uso justo o al permiso otorgado anteriormente, no dude en contactarnos a permissions@oreilly.com.

Aprendizaje en línea de O'Reilly



Durante casi 40 años, O'Reilly ha brindado capacitación, conocimientos y perspectivas tecnológicas y empresariales para ayudar a las empresas a tener éxito.

Nuestra red única de expertos e innovadores comparte sus conocimientos y experiencia a través de libros, artículos, conferencias y nuestra plataforma de aprendizaje en línea. La plataforma de aprendizaje en línea de O'Reilly le ofrece acceso bajo demanda a cursos de capacitación en vivo, rutas de aprendizaje en profundidad, entornos de programación interactivos y una amplia colección de textos y videos de O'Reilly y más de 200 editoriales. Para más información, visite <http://oreilly.com>.

Cómo contactarnos

Por favor, dirija sus comentarios y preguntas sobre este libro al editor:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (en Estados Unidos o Canadá)
707-829-0515 (internacional o local)
707-829-0104 (fax)

Contamos con una página web para este libro, donde enumeramos erratas, ejemplos e información adicional. Puede acceder a esta página en <http://bit.ly/graph-algorithms>.

Para comentar o hacer preguntas técnicas sobre este libro, envíe un correo electrónico a bookquestions@oreilly.com.

Para obtener más información sobre nuestros libros, cursos, conferencias y noticias, visite nuestro sitio web en <http://www.oreilly.com>.

Encuéntrenos en Facebook: <http://facebook.com/oreilly>

Síguenos en Twitter: <http://twitter.com/oreillymedia>

Míranos en YouTube: <http://www.youtube.com/oreillymedia>

Expresiones de gratitud

Disfrutamos muchísimo recopilando el material para este libro y agradecemos a todos los que nos ayudaron. Agradecemos especialmente a Michael Hunger por su orientación, a Jim Webber por sus invaluables correcciones y a Tomaz Bratanic por su profunda investigación. Finalmente, agradecemos enormemente a Yelp por permitirnos usar su rico conjunto de datos para ejemplos impactantes.

Prefacio

¿Qué tienen en común los siguientes aspectos: análisis de atribución de marketing, análisis de prevención del blanqueo de capitales (AML), modelado del recorrido del cliente, análisis de factores causales de incidentes de seguridad, descubrimiento basado en la literatura, detección de redes de fraude, análisis de nodos de búsqueda en internet, creación de aplicaciones de mapas, análisis de conglomerados de enfermedades y análisis del rendimiento de una obra de William Shakespeare? Como habrán adivinado, lo que todos ellos tienen en común es el uso de gráficos, lo que demuestra que Shakespeare tenía razón cuando afirmó: "¡Todo el mundo es un gráfico!".

Está bien, el Bardo de Avon en realidad no escribió gráfico en esa oración, escribió escenario. Sin embargo, tenga en cuenta que todos los ejemplos enumerados anteriormente involucran entidades y las relaciones entre ellas, incluidas tanto relaciones directas como indirectas (transitivas). Las entidades son los nodos del grafo; pueden ser personas, eventos, objetos, conceptos o lugares. Las relaciones entre los nodos son las aristas del grafo. Por lo tanto, ¿no es la esencia misma de una obra de Shakespeare la representación activa de las entidades (los nodos) y sus relaciones (las aristas)? En consecuencia, tal vez Shakespeare podría haber escrito «grafo» en su famosa declaración.

Lo que hace que los algoritmos y bases de datos de grafos sean tan interesantes y poderosos no es la simple relación entre dos entidades, donde A está relacionada con B. Después de todo, el modelo relacional estándar de bases de datos instancia este tipo de relaciones en su fundación hace décadas, en el diagrama entidad-relación (ERD). Lo que hace que los grafos sean tan notablemente importantes son las relaciones direccionales y las relaciones transitivas. En las relaciones direccionales, A puede causar B, pero no lo contrario. En las relaciones transitivas, A puede estar directamente relacionada con B y B puede estar directamente relacionada con C, mientras que A no está directamente relacionada con C, por lo que, en consecuencia, A está transitivamente relacionada con C.

Con estas relaciones de transitividad, particularmente cuando son numerosas y diversas, con muchos patrones de relación/red posibles y grados de separación entre las entidades, el modelo gráfico descubre relaciones entre entidades que de otra manera podrían parecer desconectadas o no relacionadas, y que no son detectadas por un modelo relacional.

Base de datos. Por lo tanto, el modelo gráfico se puede aplicar de forma productiva y eficaz en muchos casos de análisis de redes.

Considere este caso práctico de atribución de marketing: la persona A ve la campaña de marketing; la persona A habla de ella en redes sociales; la persona B se conecta con la persona A y ve el comentario; y, posteriormente, la persona B compra el producto. Desde la perspectiva del gestor de campañas de marketing, el modelo relacional estándar no identifica la atribución, ya que B no vio la campaña y A no respondió a ella.

La campaña parece un fracaso, pero su éxito real (y su ROI positivo) lo descubre el algoritmo de análisis de gráficos a través de la relación transitiva entre la campaña de marketing y la compra del cliente final, a través de un intermediario (entidad intermedia).

A continuación, considere un caso de análisis de prevención del blanqueo de capitales (ALD): las personas A y C son sospechosas de tráfico ilícito. Cualquier interacción entre ambas (por ejemplo, una transacción financiera en una base de datos financiera) sería detectada por las autoridades y sometida a un riguroso escrutinio. Sin embargo, si A y C nunca realizan transacciones comerciales juntas, sino que realizan operaciones financieras a través de la autoridad financiera B, segura, respetada y sin marcas, ¿qué podría detectar la transacción? ¡El algoritmo de análisis de grafos! El motor de grafos descubriría la relación transitiva entre A y C a través del intermediario B.

En las búsquedas en internet, los principales motores de búsqueda utilizan un algoritmo de red de hipervínculos (basado en grafos) para encontrar el nodo de autoridad central en todo internet para cualquier conjunto de palabras de búsqueda. La direccionalidad del borde es vital en este caso, ya que el nodo de autoridad en la red es al que apuntan muchos otros nodos.

Con el descubrimiento basado en la literatura (LBD), una aplicación de red de conocimiento (basada en grafos) que permite descubrimientos significativos en la base de conocimiento de miles (o incluso millones) de artículos de revistas de investigación, el "conocimiento oculto" se descubre solo a través de la conexión entre los resultados de investigación publicados que pueden tener muchos grados de separación (relaciones transitivas) entre ellos. El LBD se está aplicando a los estudios de investigación del cáncer, donde la enorme base de conocimiento médico semántico de síntomas, diagnósticos, tratamientos, interacciones farmacológicas, marcadores genéticos, resultados a corto plazo y consecuencias a largo plazo podría estar "ocultando" curas previamente desconocidas o tratamientos beneficiosos para los casos más impenetrables. El conocimiento podría ya estar en la red, pero necesitamos conectar los puntos para encontrarlo.

Se pueden ofrecer descripciones similares del poder de los gráficos para los demás casos de uso mencionados anteriormente, todos ejemplos de análisis de redes mediante algoritmos de grafos. Cada caso involucra profundamente entidades (personas, objetos, eventos, acciones, conceptos y lugares) y sus relaciones (puntos de contacto, tanto causales como simples).

Al considerar el poder de los gráficos, debemos tener en cuenta que quizás el nodo más poderoso en un modelo gráfico para casos de uso del mundo real podría ser el "contexto".

El contexto puede incluir tiempo, ubicación, eventos relacionados, entidades cercanas y más. Incor

De este modo, la incorporación del contexto en el gráfico (como nodos y como bordes) puede generar capacidades de análisis predictivo y prescriptivo impresionantes.

El proyecto Graph Algorithms de Mark Needham y Amy Hodler tiene como objetivo ampliar nuestro conocimiento y nuestras capacidades en torno a estos importantes tipos de análisis de gráficos, incluidos algoritmos, conceptos y aplicaciones prácticas de aprendizaje automático de los algoritmos.

Desde conceptos básicos hasta algoritmos fundamentales, plataformas de procesamiento y casos de uso prácticos, los autores han compilado una guía instructiva e ilustrativa del maravilloso mundo de los gráficos.

— Kirk Borne, doctor

Científico de datos principal y asesor ejecutivo

Booz Allen Hamilton

Marzo de 2019

Introducción

Los grafos son uno de los pilares de la informática: una representación abstracta que describe la organización de los sistemas de transporte, las interacciones humanas y las redes de telecomunicaciones. El hecho de que se puedan modelar tantas estructuras diferentes mediante un único formalismo es una fuente de gran poder para el programador cualificado.

—El manual de diseño de algoritmos, por Steven S. Skiena (Springer), profesor distinguido de Ciencias de la Computación en la Universidad de Stony Brook

Los desafíos de datos más urgentes de la actualidad se centran en las relaciones, no solo en la tabulación de datos discretos. Las tecnologías gráficas y el análisis proporcionan herramientas potentes para datos conectados que se utilizan en investigación, iniciativas sociales y soluciones empresariales, como:

- Modelar entornos dinámicos desde mercados financieros hasta servicios de TI •

- Pronosticar la propagación de epidemias, así como demoras e interrupciones del servicio • Encontrar características predictivas para el aprendizaje automático para combatir delitos financieros •

- Descubrir patrones para experiencias y recomendaciones personalizadas

A medida que los datos se vuelven cada vez más interconectados y los sistemas cada vez más sofisticados, es esencial aprovechar las relaciones ricas y cambiantes dentro de nuestros datos.

Este capítulo ofrece una introducción al análisis de grafos y a los algoritmos de grafos. Comenzaremos con un breve repaso del origen de los grafos antes de presentar los algoritmos de grafos y explicar la diferencia entre las bases de datos de grafos y el procesamiento de grafos.

Exploraremos la naturaleza misma de los datos modernos y cómo la información contenida en las conexiones es mucho más sofisticada que la que podemos obtener con métodos estadísticos básicos. El capítulo concluirá con un análisis de casos prácticos donde se pueden emplear algoritmos de grafos.

¿Qué son los gráficos?

Los grafos tienen una historia que se remonta a 1736, cuando Leonhard Euler resolvió el problema de los "Siete Puentes de Königsberg". El problema planteaba si era posible visitar las cuatro zonas de una ciudad conectadas por siete puentes, cruzando cada uno solo una vez. No fue posible.

Con la idea de que solo las conexiones en sí mismas eran relevantes, Euler sentó las bases de la teoría de grafos y sus matemáticas. La Figura 1-1 muestra la progresión de Euler con uno de sus bocetos originales, del artículo «*Solutio problematis ad geometriam situs pertinentis*».

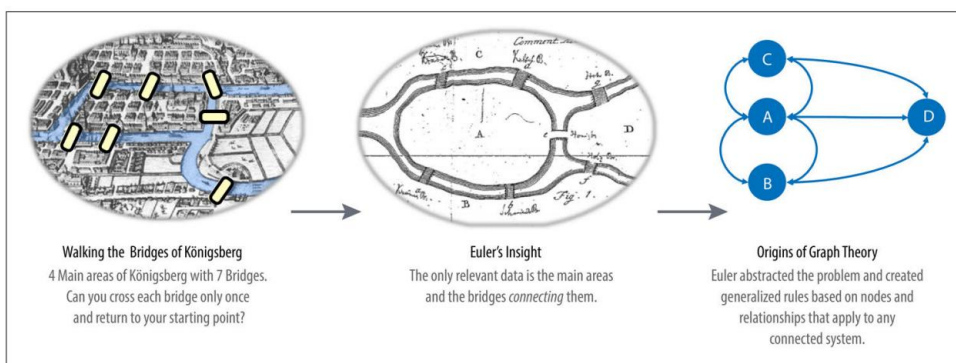


Figura 1-1. Los orígenes de la teoría de grafos. La ciudad de **Königsberg** incluía dos grandes islas conectadas entre sí y con las dos partes continentales de la ciudad por siete puentes. El problema consistía en crear un paseo por la ciudad, cruzando cada puente una sola vez.

Si bien los grafos se originaron en las matemáticas, también son una forma pragmática y de alta fidelidad de modelar y analizar datos. Los objetos que componen un grafo se llaman nodos o vértices, y las conexiones entre ellos se conocen como relaciones, enlaces o aristas. En este libro, utilizamos los términos nodos y relaciones: los nodos son los sustantivos en las oraciones y las relaciones los verbos que contextualizan los nodos. Para evitar confusiones, los grafos que tratamos en este libro no tienen nada que ver con la representación gráfica de ecuaciones o tablas como las de la Figura 1-2.

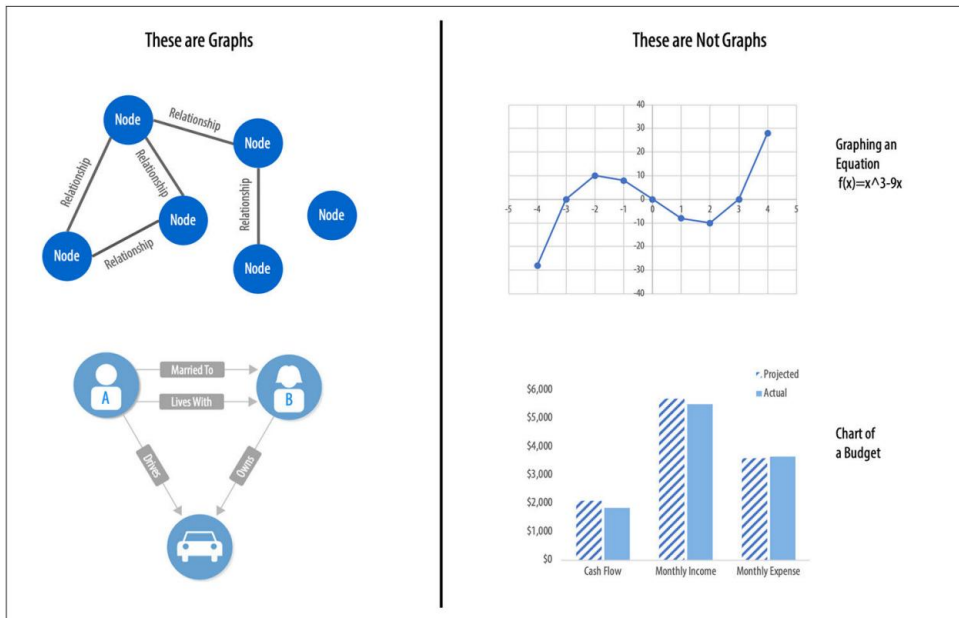


Figura 1-2. Un gráfico es una representación de una red, a menudo ilustrada con círculos para representar entidades que llamamos nodos y líneas para representar relaciones.

Al observar el gráfico de personas de la Figura 1-2, podemos construir fácilmente varias oraciones que lo describan. Por ejemplo, la persona A vive con la persona B, quien tiene un auto, y la persona A conduce un auto que pertenece a la persona B. Este enfoque de modelado es atractivo porque se adapta fácilmente al mundo real y es muy fácil de usar en la pizarra. Esto ayuda a alinear el modelado y el análisis de datos.

Pero modelar gráficos es solo la mitad de la historia. También podríamos querer procesarlos para revelar información que no es inmediatamente obvia. Este es el dominio de los algoritmos de grafos.

¿Qué son el análisis de gráficos y los algoritmos?

Los algoritmos de grafos son un subconjunto de herramientas para el análisis de grafos. El análisis de grafos es algo que hacemos: consiste en utilizar cualquier enfoque basado en grafos para analizar datos conectados. Existen varios métodos que podemos utilizar: podemos consultar los datos del gráfico, usar estadísticas básicas, explorar visualmente los gráficos o incorporarlos en nuestras tareas de aprendizaje automático. Las consultas basadas en patrones de grafos se suelen utilizar para el análisis de datos locales, mientras que los algoritmos computacionales de grafos suelen referirse a un análisis más global e iterativo.

Si bien existe superposición en cómo se pueden emplear estos tipos de análisis, utilizamos el término algoritmos gráficos para referirnos a estos últimos usos, más propios de la analítica computacional y la ciencia de datos.

Los algoritmos de grafos proporcionan uno de los enfoques más potentes para analizar datos conectados, ya que sus cálculos matemáticos están diseñados específicamente para operar con relaciones. Describen los pasos necesarios para procesar un grafo y descubrir sus cualidades generales o cantidades específicas. Basados en las matemáticas de la teoría de grafos, los algoritmos de grafos utilizan las relaciones entre nodos para inferir la organización y la dinámica de sistemas complejos. Los científicos de redes utilizan estos algoritmos para descubrir información oculta, probar hipótesis y realizar predicciones sobre el comportamiento.

Ciencia de redes

La ciencia de redes es un campo académico con fuertes raíces en la teoría de grafos que se ocupa de los modelos matemáticos de las relaciones entre objetos. Los científicos de redes se basan en algoritmos de grafos y sistemas de gestión de bases de datos debido al tamaño, la conectividad y la complejidad de sus datos.

Existen muchos recursos fantásticos sobre complejidad y ciencia de redes. Aquí tienes algunas referencias para que las explores.

- [Network Science](#), de Albert-László Barabási, es un libro electrónico introductorio
- [Complexity Explorer](#) ofrece cursos en línea
- [The New England Complex Systems Institute](#) ofrece varios recursos y papeles

Los algoritmos de grafos tienen un amplio potencial, desde la prevención del fraude y la optimización del enrutamiento de llamadas hasta la predicción de la propagación de la gripe. Por ejemplo, podríamos querer puntuar nodos específicos que podrían corresponder a condiciones de sobrecarga en un sistema eléctrico. O podríamos querer descubrir agrupaciones en el grafo que correspondan a la congestión en un sistema de transporte.

De hecho, en 2010, los sistemas de viajes aéreos de EE. UU. experimentaron dos eventos graves que involucraron múltiples aeropuertos congestionados que luego se estudiaron mediante análisis de grafos. Los científicos de redes P. Fleurquin, JJ Ramasco y VM Eguíluz utilizaron algoritmos de grafos para confirmar los eventos como parte de retrasos sistemáticos en cascada y usar esta información para asesoramiento correctivo, como se describe en su artículo, "[Propagación de retrasos sistemáticos en la red de aeropuertos de EE. UU.](#)"

Para visualizar la red que sustenta el transporte aéreo, Martin Grandjean creó [la Figura 1-3](#) para su artículo [«Un mundo conectado: desentrañando la red de tráfico aéreo»](#). Esta ilustración muestra claramente la estructura altamente conectada de los clústeres de transporte aéreo. Muchos sistemas de transporte presentan una distribución concentrada de enlaces con claros patrones radiales que influyen en los retrasos.

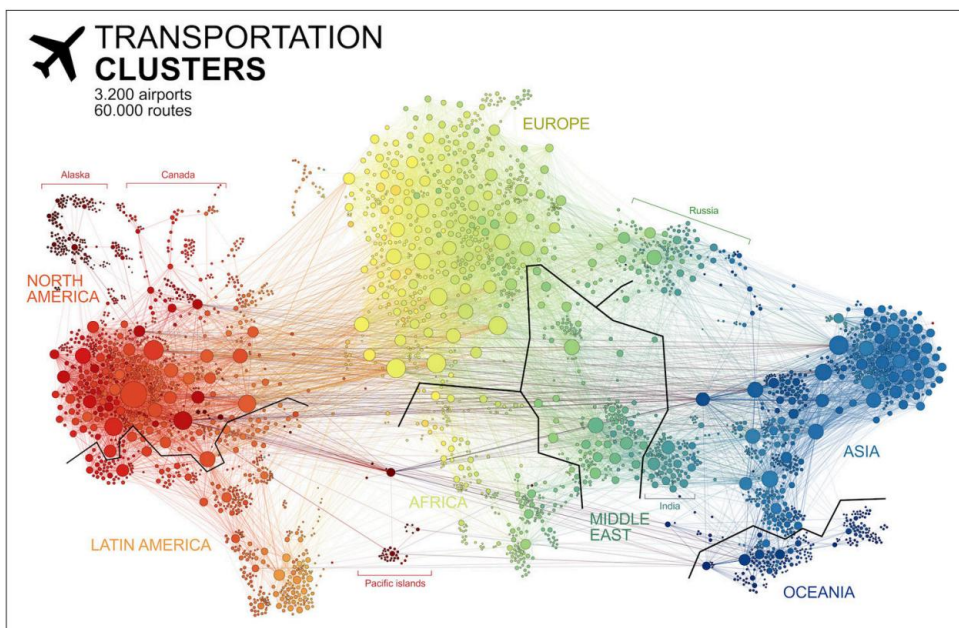


Figura 1-3. Las redes de transporte aéreo ilustran estructuras radiales que evolucionan a múltiples escalas. Estas estructuras contribuyen a la fluidez de los viajes.

Los grafos también ayudan a descubrir cómo interacciones y dinámicas muy pequeñas conducen a mutaciones globales. Vinculan las escalas micro y macro al representar con precisión qué elementos interactúan dentro de las estructuras globales. Estas asociaciones se utilizan para pronosticar el comportamiento y determinar los eslabones perdidos. La Figura 1-4 muestra una red alimentaria de interacciones entre especies de pastizales que utilizó el análisis de grafos para evaluar la organización jerárquica y las interacciones entre especies y, posteriormente, predecir las relaciones faltantes, como se detalla en el artículo de A. Clauset, C. Moore y MEJ Newman, «Estructura jerárquica y la predicción de eslabones perdidos en la red».

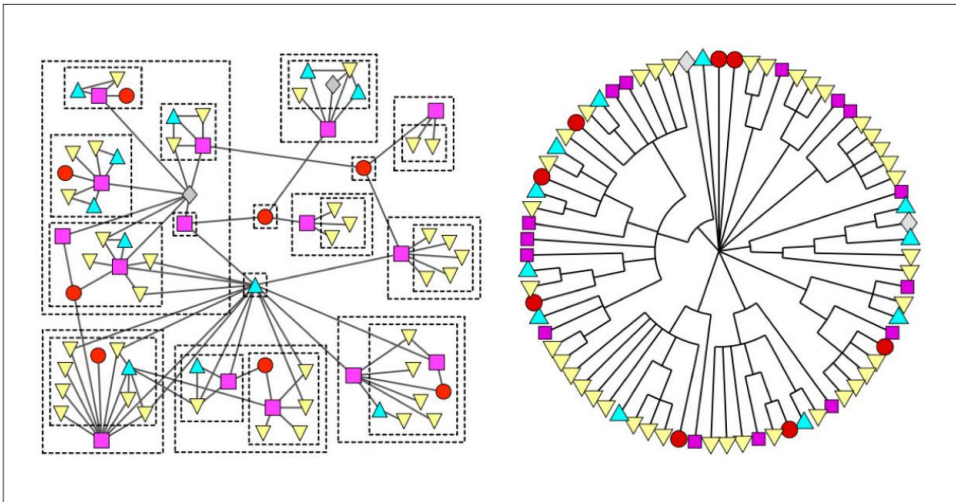


Figura 1-4. Esta red alimentaria de especies de pastizales utiliza gráficos para correlacionar interacciones a pequeña escala con la formación de estructuras más grandes.

Procesamiento de gráficos, bases de datos, consultas y algoritmos

El procesamiento de grafos incluye los métodos mediante los cuales se ejecutan las cargas de trabajo y las tareas de grafos. La mayoría de las consultas de grafos consideran partes específicas del grafo (p. ej., un nodo inicial), y el trabajo suele centrarse en el subgrafo circundante. Este tipo de trabajo se denomina "local de grafos" e implica la consulta declarativa de la estructura de un grafo, como se explica en el libro "[Bases de Datos de Grafos](#)" de Ian Robinson, Jim Webber y Emil Eifrem (O'Reilly). Este tipo de procesamiento local de grafos se utiliza a menudo para transacciones en tiempo real y consultas basadas en patrones.

Al hablar de algoritmos de grafos, solemos buscar patrones y estructuras globales. La entrada del algoritmo suele ser el grafo completo, y la salida puede ser un grafo enriquecido o algún valor agregado, como una puntuación. Este procesamiento se clasifica como grafo global, lo que implica procesar la estructura de un grafo mediante algoritmos computacionales (a menudo de forma iterativa). Este enfoque permite comprender la naturaleza general de una red a través de sus conexiones. Las organizaciones tienden a utilizar algoritmos de grafos para modelar sistemas y predecir su comportamiento basándose en la diseminación de elementos, los componentes importantes, la identificación de grupos y la robustez general del sistema.

Puede haber cierta superposición en estas definiciones (a veces podemos usar el procesamiento de un algoritmo para responder una consulta local, o viceversa), pero, hablando de manera simple, las operaciones de gráficos completos se procesan mediante algoritmos computacionales y las operaciones de subgráficos se consultan en bases de datos.

Tradicionalmente, el procesamiento y el análisis de transacciones han estado aislados. Esta división no era natural debido a las limitaciones tecnológicas. Nuestra opinión es que el análisis de gráficos...

Impulsa transacciones más inteligentes, lo que genera nuevos datos y oportunidades para un mayor análisis. Recientemente, se ha observado una tendencia a integrar estos silos para una toma de decisiones más rápida.

OLTP y OLAP

Las operaciones de procesamiento de transacciones en línea (OLTP) suelen ser actividades breves, como reservar un billete, abonar una cuenta, registrar una venta, etc. OLTP implica un procesamiento de consultas voluminoso y de baja latencia, así como una alta integridad de los datos. Aunque OLTP puede implicar solo una pequeña cantidad de registros por transacción, los sistemas procesan muchas transacciones simultáneamente.

El procesamiento analítico en línea (OLAP) facilita consultas y análisis más complejos sobre datos históricos. Estos análisis pueden incluir múltiples fuentes, formatos y tipos de datos. Detectar tendencias, realizar escenarios hipotéticos, realizar predicciones y descubrir patrones estructurales son casos de uso típicos de OLAP. En comparación con OLTP, los sistemas OLAP procesan menos transacciones, pero de mayor duración, en muchos registros.

Los sistemas OLAP están orientados a una lectura más rápida sin la expectativa de actualizaciones transaccionales presentes en OLTP, y la operación orientada a lotes es común.

Sin embargo, recientemente, la línea entre OLTP y OLAP ha comenzado a difuminarse. Las aplicaciones modernas con uso intensivo de datos ahora combinan operaciones transaccionales en tiempo real con análisis. Esta fusión de procesamiento se ha visto impulsada por varios avances en software, como una gestión de transacciones más escalable y el procesamiento incremental de flujos, así como por hardware de bajo costo y gran capacidad de memoria.

La integración de la analítica y las transacciones permite el análisis continuo como parte natural de las operaciones habituales. A medida que se recopilan datos —desde terminales de punto de venta (TPV), sistemas de fabricación o dispositivos del Internet de las Cosas (IoT)—, la analítica permite realizar recomendaciones y tomar decisiones en tiempo real durante el procesamiento. Esta tendencia se observó hace varios años, y los términos que describen esta fusión incluyen translytics y procesamiento transaccional y analítico híbrido (HTAP). La Figura 1-5 ilustra cómo se pueden utilizar las réplicas de solo lectura para integrar estos diferentes tipos de procesamiento.

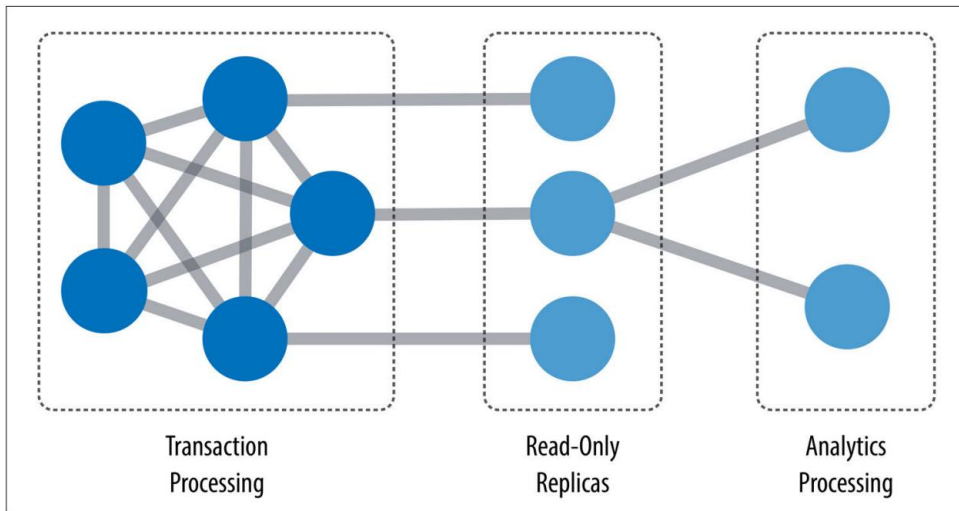


Figura 1-5. Una plataforma híbrida admite el procesamiento de consultas de baja latencia y la alta integridad de datos requerida para las transacciones, a la vez que integra análisis complejos sobre grandes cantidades de datos.

Según **Gartner**:

[HTAP] podría redefinir la forma en que se ejecutan algunos procesos de negocio, ya que la analítica avanzada en tiempo real (por ejemplo, planificación, previsión y análisis hipotéticos) se convierte en una parte integral del proceso en sí, en lugar de una actividad independiente realizada posteriormente. Esto permitiría nuevas formas de toma de decisiones empresariales en tiempo real.

proceso de toma de decisiones. En última instancia, HTAP se convertirá en una arquitectura clave que habilitará operaciones comerciales inteligentes.

A medida que OLTP y OLAP se integran más y comienzan a admitir funcionalidades que antes se ofrecían de forma aislada, ya no es necesario utilizar diferentes productos o sistemas de datos para estas cargas de trabajo; podemos simplificar nuestra arquitectura utilizando la misma plataforma para ambas. Esto significa que nuestras consultas analíticas pueden aprovechar los datos en tiempo real y podemos optimizar el proceso iterativo de análisis.

¿Por qué deberíamos preocuparnos por los algoritmos gráficos?

Los algoritmos de grafos se utilizan para comprender datos conectados. Observamos relaciones en sistemas del mundo real, desde interacciones de proteínas hasta redes sociales, desde sistemas de comunicación hasta redes eléctricas, y desde experiencias de compra hasta la planificación de misiones a Marte. Comprender las redes y sus conexiones ofrece un potencial increíble para el conocimiento y la innovación.

Los algoritmos de grafos son especialmente adecuados para comprender estructuras y revelar patrones en conjuntos de datos altamente conectados. En ninguna parte se encuentra la conectividad y la interacción.

La actividad es tan evidente que en el big data. La cantidad de información que se ha recopilado, combinado y actualizado dinámicamente es impresionante. Aquí es donde los algoritmos de grafos pueden ayudar a comprender nuestros volúmenes de datos, con análisis más sofisticados que aprovechan las relaciones y mejoran la información contextual de la inteligencia artificial.

A medida que nuestros datos se conectan más, es cada vez más importante comprender sus relaciones e interdependencias. Los científicos que estudian el crecimiento de las redes han observado que la conectividad aumenta con el tiempo, pero no de forma uniforme. La vinculación preferencial es una teoría que explica cómo la dinámica del crecimiento impacta la estructura. Esta idea, ilustrada en la [Figura 1-6](#), describe la tendencia de un nodo a conectarse con otros nodos que ya cuentan con numerosas conexiones.

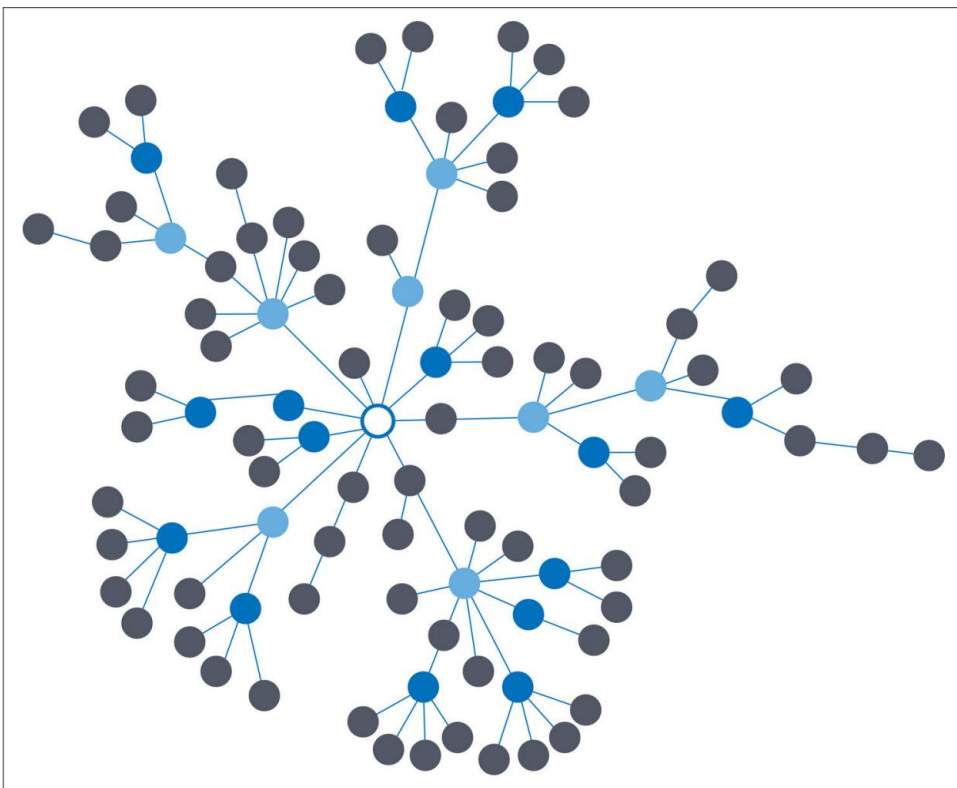


Figura 1-6. La conexión preferencial es el fenómeno según el cual, cuanto más conectado esté un nodo, mayor será la probabilidad de que reciba nuevos enlaces. Esto genera concentraciones y centros desiguales.

En su libro, *Sync: How Order Emerges from Chaos in the Universe, Nature, and Daily Life* (Hachette), Steven Strogatz ofrece ejemplos y explica diferentes maneras en que los sistemas de la vida real se autoorganizan. Independientemente de las causas subyacentes, muchos investigadores...

Creemos que el crecimiento de las redes es inseparable de sus formas y jerarquías resultantes. Se tienden a desarrollar grupos muy densos y redes de datos abultadas, y la complejidad aumenta junto con el tamaño de los datos. Observamos esta agrupación de relaciones en la mayoría de las redes del mundo real actual, desde internet hasta redes sociales como la comunidad de videojuegos que se muestra en [la Figura 1-7](#).

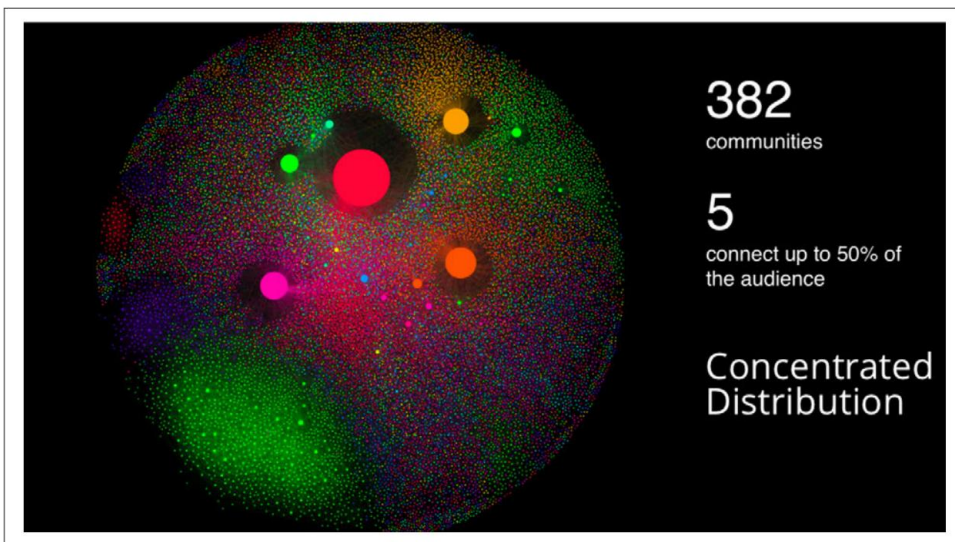


Figura 1-7. Este [análisis de la comunidad de juegos](#) muestra una concentración de conexiones en tan solo 5 de las 382 comunidades.

El análisis de red que se muestra en [la Figura 1-7](#) fue creado por Francesco D'Orazio de Pulsar para ayudar a predecir la viralidad del contenido e informar las estrategias de distribución. D'Orazio [encontró](#) una correlación entre la concentración de la distribución de una comunidad y la velocidad de difusión de un contenido.

Esto es significativamente diferente de lo que predeciría un modelo de distribución promedio, donde la mayoría de los nodos tendrían el mismo número de conexiones. Por ejemplo, si la World Wide Web tuviera una distribución promedio de conexiones, todas las páginas tendrían aproximadamente el mismo número de enlaces entrantes y salientes. Los modelos de distribución promedio afirman que la mayoría de los nodos están igualmente conectados, pero muchos tipos de grafos y muchas redes reales presentan concentraciones. La web, al igual que grafos como los de viajes y redes sociales, tiene una distribución de ley de potencia con unos pocos nodos altamente conectados y la mayoría de los nodos moderadamente conectados.

Ley de potencia

Una ley de potencia (también llamada ley de escala) describe la relación entre dos cantidades, donde una varía como una potencia de la otra. Por ejemplo, el área de un cubo está relacionada con la longitud de sus lados por una potencia de 3. Un ejemplo bien conocido es la distribución de Pareto o "regla del 80/20", utilizada originalmente para describir la situación en la que el 20 % de una población controlaba el 80 % de la riqueza. Observamos diversas leyes de potencia en el mundo natural y en las redes.

Intentar promediar una red generalmente no es eficaz para investigar relaciones ni realizar pronósticos, ya que las redes reales presentan distribuciones desiguales de nodos y relaciones. En la [Figura 1-8](#), podemos observar fácilmente cómo usar un promedio de características para datos desiguales generaría resultados incorrectos.

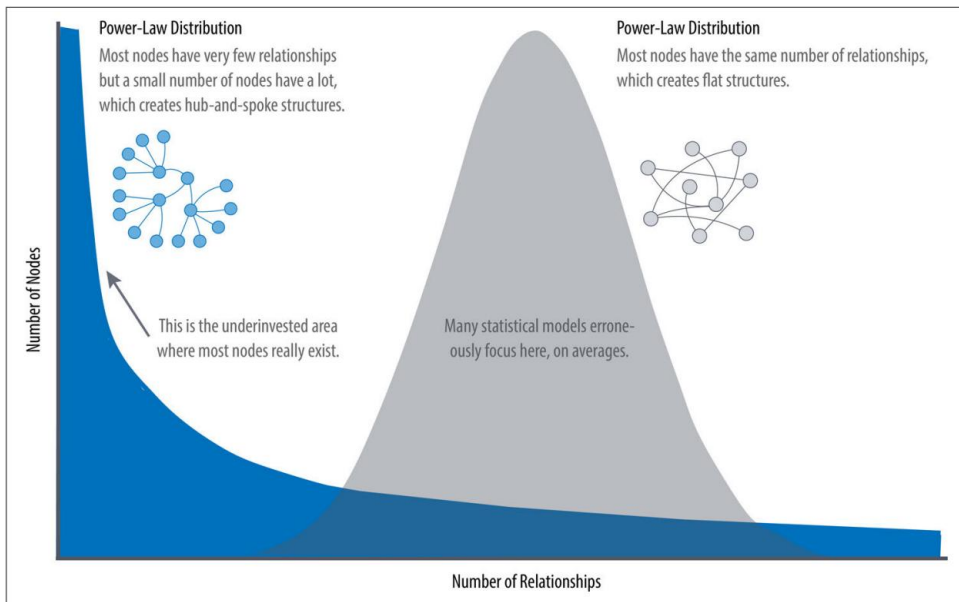


Figura 1-8. Las redes del mundo real presentan distribuciones desiguales de nodos y relaciones, representadas en extremo por una distribución de ley de potencia. Una distribución promedio asume que la mayoría de los nodos tienen el mismo número de relaciones y da como resultado una red aleatoria.

Debido a que los datos altamente conectados no se adhieren a una distribución promedio, los científicos de redes utilizan análisis de gráficos para buscar e interpretar estructuras y distribuciones de relaciones en datos del mundo real.

No existe ninguna red en la naturaleza que conozcamos que pueda describirse mediante el modelo de red aleatoria.

—Albert-László Barabási, Director del Centro de Investigación de Redes Complejas,
Universidad del Noreste, y autor de numerosos libros sobre ciencia de redes

El desafío para la mayoría de los usuarios radica en que analizar datos densamente conectados y de forma desigual resulta complicado con herramientas analíticas tradicionales. Puede que exista una estructura, pero es difícil de encontrar. Es tentador adoptar un enfoque de promedios para datos desordenados, pero hacerlo ocultará patrones y garantizará que nuestros resultados no representen a ningún grupo real. Por ejemplo, si promedia la información demográfica de todos sus clientes y ofrece una experiencia basada únicamente en promedios, seguramente pasará por alto la mayoría de las comunidades: las comunidades tienden a agruparse en torno a factores relacionados como la edad y la ocupación o el estado civil y la ubicación.

Además, el comportamiento dinámico, en particular en torno a eventos repentinos y estallidos, no se puede observar con una instantánea. Por ejemplo, si imagina un grupo social con relaciones crecientes, también esperaría más comunicaciones. Esto podría conducir a un punto de inflexión en la coordinación y una coalición posterior o, alternativamente, a la formación de subgrupos y polarización en, por ejemplo, las elecciones. Se requieren métodos sofisticados para pronosticar la evolución de una red a lo largo del tiempo, pero podemos inferir el comportamiento si comprendemos las estructuras e interacciones dentro de nuestros datos. El análisis de grafos se utiliza para predecir la resiliencia del grupo debido al enfoque en las relaciones.

Casos de uso de análisis de gráficos

En el nivel más abstracto, el análisis de grafos se aplica para pronosticar el comportamiento y prescribir acciones para grupos dinámicos. Para ello, es necesario comprender las relaciones y la estructura dentro del grupo. Los algoritmos de grafos logran esto examinando la naturaleza general de las redes a través de sus conexiones. Con este enfoque, se puede comprender la topología de los sistemas conectados y modelar sus procesos.

Hay tres grupos generales de preguntas que indican si se justifican los análisis de gráficos y los algoritmos, como se muestra en la [Figura 1-9](#).

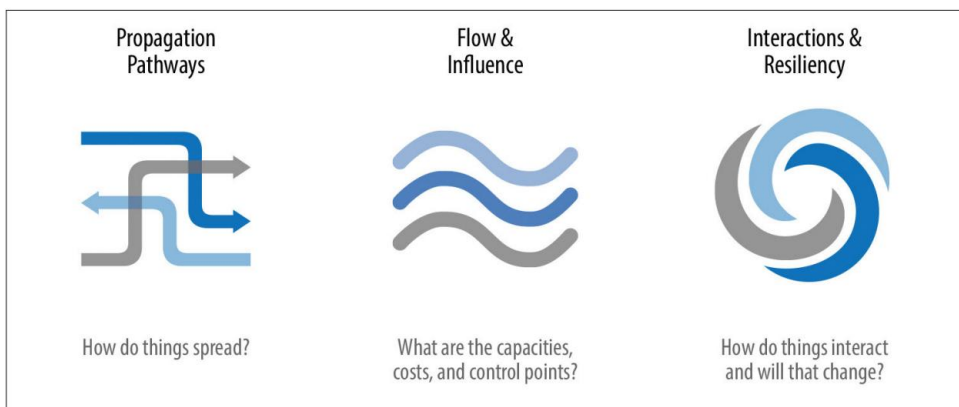


Figura 1-9. Tipos de preguntas que responde el análisis gráfico.

A continuación, se presentan algunos tipos de desafíos donde se emplean algoritmos gráficos. ¿Son similares los tuyos?

- Investigue la ruta de una enfermedad o un fallo de transporte en cascada. • Descubra los componentes más vulnerables o dañinos en un ataque de red. • Identifique la forma más económica o rápida de enrutar información o recursos. • Prediga los enlaces faltantes en sus datos. • Localice la influencia directa e indirecta en un sistema complejo. • Descubra jerarquías y dependencias ocultas. • Prediga si los grupos se fusionarán o se separarán. • Encuentre cuellos de botella o quién tiene el poder de denegar/proporcionar más recursos. • Descubra comunidades basadas en el comportamiento para obtener recomendaciones personalizadas. • Reduzca los falsos positivos en la detección de fraudes y anomalías. • Extraiga más funciones predictivas para el aprendizaje automático.

Conclusión

En este capítulo, analizamos la gran interconexión de los datos actuales y sus implicaciones. Si bien existen prácticas científicas sólidas para el análisis de dinámicas y relaciones grupales, estas herramientas no siempre son comunes en las empresas. Al evaluar técnicas de análisis avanzado, debemos considerar la naturaleza de nuestros datos y si necesitamos comprender los atributos de la comunidad o predecir comportamientos complejos. Si nuestros datos representan una red, debemos evitar la tentación de reducir los factores a un promedio. En su lugar, debemos utilizar herramientas que se ajusten a nuestros datos y a la información que buscamos.

En el próximo capítulo, cubriremos los conceptos y la terminología de los gráficos.

Teoría de grafos y conceptos

En este capítulo, establecemos el marco y cubrimos la terminología para los algoritmos gráficos. Se explican los conceptos básicos de la teoría de grafos, centrándose en los conceptos que son más relevantes para un profesional.

Describiremos cómo se representan los grafos y luego explicaremos los diferentes tipos de grafos y sus atributos. Esto será importante más adelante, ya que las características de nuestro grafo nos ayudarán a elegir nuestros algoritmos y a interpretar los resultados. Finalizaremos el capítulo con una descripción general de los tipos de algoritmos de grafos que se detallan en este libro.

Terminología

El gráfico de propiedades etiquetadas es una de las formas más populares de modelar datos gráficos.

Una etiqueta marca un nodo como parte de un grupo. En la **Figura 2-1**, tenemos dos grupos de nodos: Persona y Coche. (Aunque en la teoría de grafos clásica una etiqueta se aplica a un solo nodo, ahora se usa comúnmente para referirse a un grupo de nodos).

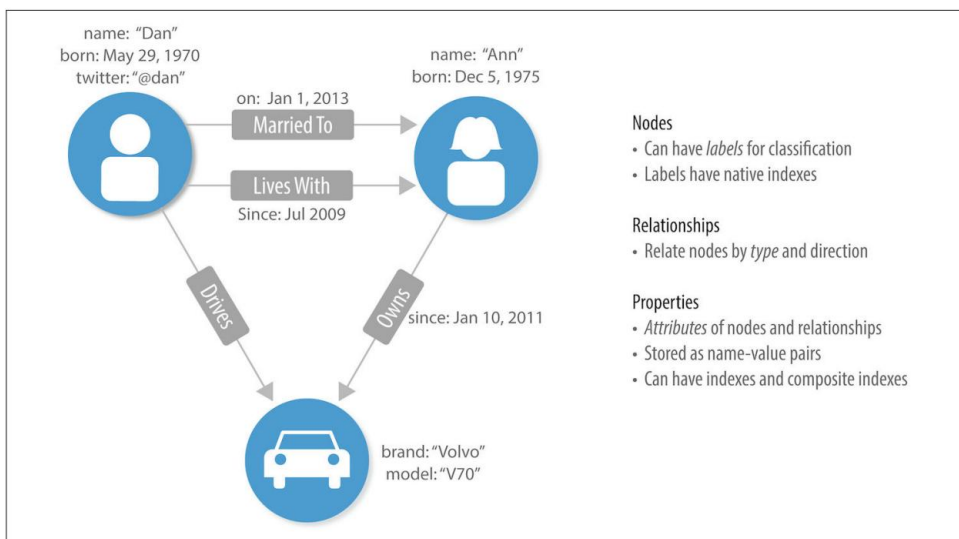


Figura 2-1. Un modelo de gráfico de propiedades etiquetadas es una forma flexible y concisa de representar datos conectados.

Las relaciones se clasifican según el tipo de relación. Nuestro ejemplo incluye los tipos de relación DRIVES, OWNS, LIVES_WITH y MARRIED_TO.

Las propiedades son sinónimos de atributos y pueden contener diversos tipos de datos, desde números y cadenas hasta datos espaciales y temporales. En la Figura 2-1, asignamos las propiedades como pares nombre-valor, donde el nombre de la propiedad va primero y luego su valor. Por ejemplo, el nodo Persona de la izquierda tiene la propiedad "Dan" y la relación CASADO_CON tiene la propiedad el: 1 de enero de 2013.

Un subgrafo es un grafo dentro de un grafo mayor. Los subgrafos son útiles como filtros, por ejemplo, cuando necesitamos un subconjunto con características específicas para un análisis específico.

Una ruta es un grupo de nodos y sus relaciones de conexión. Un ejemplo de una ruta simple, basada en la Figura 2-1, podría contener los nodos Dan, Ann y Car, y las relaciones DRIVES y OWNS.

Los gráficos varían en tipo, forma y tamaño, así como en los atributos que se pueden usar para el análisis. A continuación, describiremos los tipos de gráficos más adecuados para los algoritmos de grafos. Tenga en cuenta que estas explicaciones se aplican tanto a gráficos como a subgráficos.

Tipos y estructuras de gráficos

En la teoría clásica de grafos, el término grafo se equipara a un grafo simple (o estricto) donde los nodos solo tienen una relación entre sí, como se muestra en el lado izquierdo de la Figura 2-2. Sin embargo, la mayoría de los grafos del mundo real tienen múltiples relaciones entre nodos.

e incluso relaciones autorreferenciales. Hoy en día, este término se usa comúnmente para los tres tipos de gráficos de la Figura 2-2, por lo que también lo usamos de forma inclusiva.

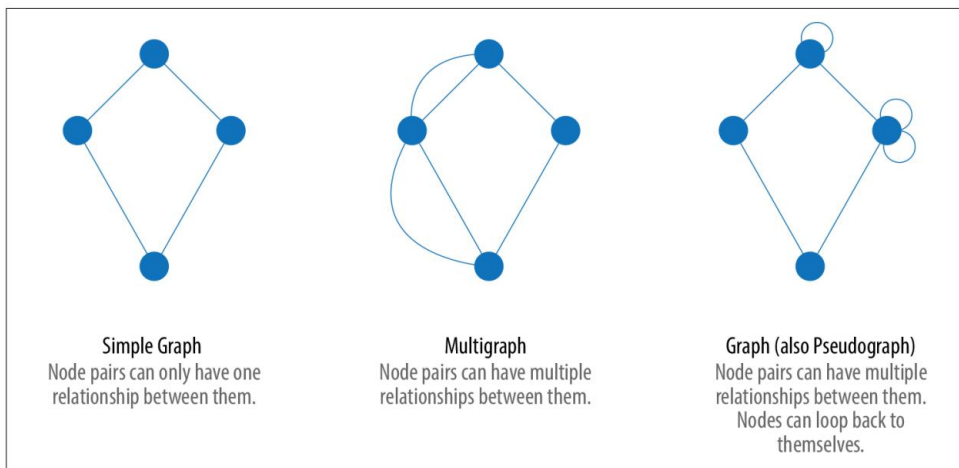


Figura 2-2. En este libro, utilizamos el término gráfico para incluir cualquiera de estos tipos clásicos de gráficos.

Estructuras aleatorias, de mundo pequeño y sin escala

Los gráficos adoptan diversas formas. La Figura 2-3 muestra tres tipos representativos de redes:

- Redes aleatorias
- Redes de mundo pequeño
- Redes sin escala

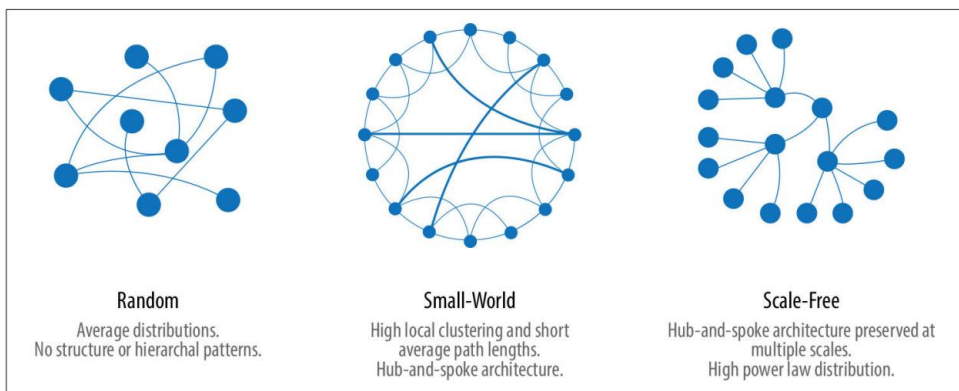


Figura 2-3. Tres estructuras de red con gráficos y comportamientos distintivos.

En una distribución de conexiones completamente promedio, se forma una red aleatoria sin jerarquías. Este tipo de grafo sin forma es plano, sin patrones discernibles. Todos los nodos tienen la misma probabilidad de estar conectados a cualquier otro. Una red de mundo

pequeño es extremadamente común en las redes sociales; presenta conexiones localizadas y un patrón radial. El juego "Seis Grados de Kevin Bacon" podría ser el ejemplo más conocido del efecto de mundo pequeño.

Aunque te relacionas principalmente con un grupo pequeño de amigos, nunca estás a muchos pasos de distancia de cualquier otra persona, incluso si se trata de un actor famoso o está al otro lado del planeta.

- Una red sin escala se produce cuando hay distribuciones de ley de potencia y se conserva una arquitectura de concentrador y radios independientemente de la escala, como en la World Wide Web.

Estos tipos de redes producen grafos con estructuras, distribuciones y comportamientos distintivos. A medida que trabajamos con algoritmos de grafos, reconoceremos patrones similares en nuestros resultados.

Sabores de gráficos

Para aprovechar al máximo los algoritmos de grafos, es importante familiarizarse con los grafos más característicos que encontraremos. La Tabla 2-1 resume los atributos comunes de los grafos. En las siguientes secciones, analizaremos las diferentes variantes con más detalle.

Tabla 2-1. Atributos comunes de los gráficos

Atributo de gráfico	Factor clave	Consideración del algoritmo
Conectado versus desconectado	Si existe una ruta entre dos nodos cualesquiera en el gráfico, independientemente de la distancia	islas de nodos pueden provocar un comportamiento inesperado, como quedarse atascadas o no poder procesar componentes desconectados.
Ponderado versus no ponderado	Si hay valores (específicos del dominio) en las relaciones o nodos	Muchos algoritmos esperan pesos, y veremos diferencias significativas en el rendimiento y los resultados cuando se ignoran.
Dirigido versus sin dirección	Si las relaciones definen o no explícitamente un nodo de inicio y un nodo final	Esto añade contexto enriquecido para inferir significado adicional. En algunos algoritmos, se puede establecer explícitamente el uso de una dirección, ambas o ninguna.
Cíclico versus acíclico	Si las rutas comienzan y terminan en el mismo nodo	Los grafos cíclicos son comunes, pero los algoritmos deben ser cuidadosos (normalmente almacenando el estado del recorrido), ya que los ciclos pueden impedir la terminación. Los grafos acíclicos (o árboles de expansión) son la base de muchos algoritmos de grafos.
Relación entre escaso y denso con la relación de nodos		Los grafos extremadamente densos o escasamente conectados pueden generar resultados divergentes. El modelado de datos puede ser útil, suponiendo que el dominio no sea inherentemente denso ni disperso.

Atributo de gráfico	Factor clave	Consideración del algoritmo
Monopartito, bipartito y k-partido	Si los nodos se conectan a solo otro tipo de nodo (por ejemplo, usuarios a los que les gustan las películas) o a muchos otros tipos de nodos (por ejemplo, usuarios a los que les gustan las películas)	Útil para crear relaciones para analizar y proyectar gráficos más útiles.

Grafos Conexos vs. Grafos Desconectados. Un grafo es conexo si existe una ruta entre todos sus nodos. Si el grafo tiene islas, es desconectado. Si los nodos de esas islas están conectados, se denominan componentes (o, en ocasiones, clústeres), como se muestra en la [Figura 2-4](#).

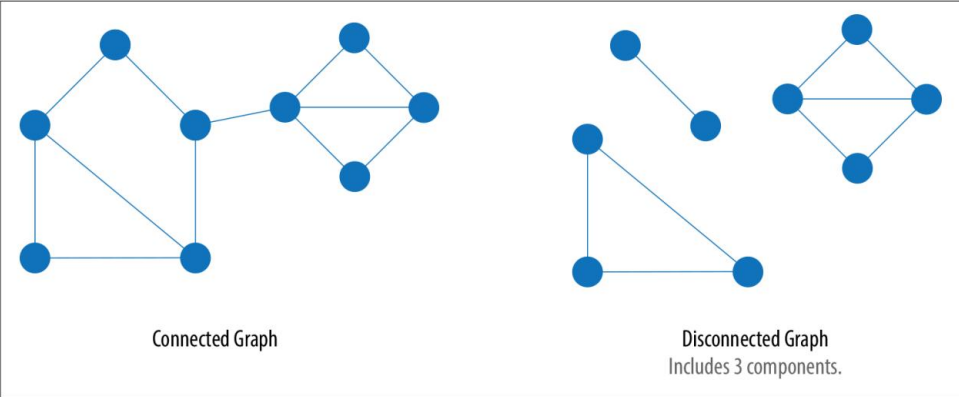


Figura 2-4. Si tenemos islas en nuestro gráfico, es un gráfico desconectado.

Algunos algoritmos presentan dificultades con grafos desconectados y pueden producir resultados engañosos. Si obtenemos resultados inesperados, un buen primer paso es comprobar la estructura de nuestro grafo.

Gráficos no ponderados versus gráficos ponderados

Los gráficos no ponderados no tienen valores de peso asignados a sus nodos o relaciones. En los gráficos ponderados, estos valores pueden representar diversas medidas, como el coste, el tiempo, la distancia, la capacidad o incluso una priorización específica del dominio. [La Figura 2-5](#) visualiza la diferencia.

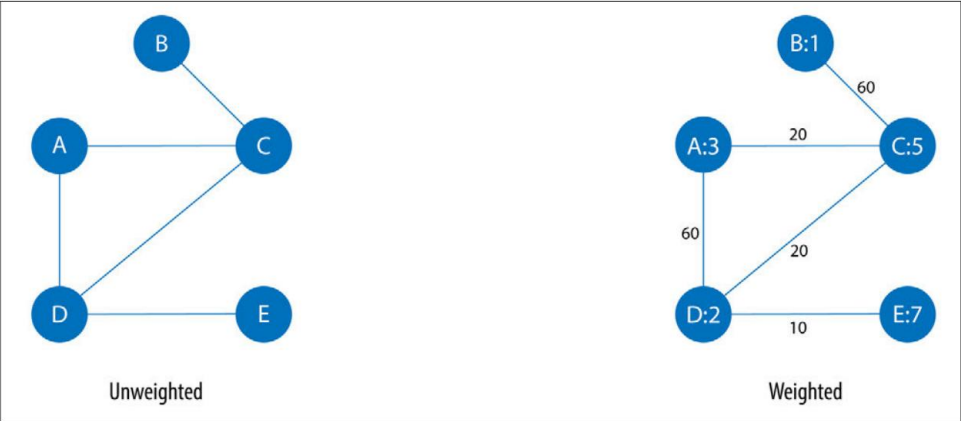


Figura 2-5. Los gráficos ponderados pueden contener valores en relaciones o nodos.

Los algoritmos básicos de grafos pueden usar pesos para el procesamiento como representación de la fuerza o el valor de las relaciones. Muchos algoritmos calculan métricas que luego pueden usarse como pesos para el procesamiento posterior. Algunos algoritmos actualizan los valores de los pesos a medida que avanzan para encontrar totales acumulados, valores mínimos u óptimos.

Un uso clásico de los grafos ponderados es en los algoritmos de búsqueda de rutas. Estos algoritmos sustentan las aplicaciones de mapas en nuestros teléfonos y calculan las rutas de transporte más cortas, económicas y rápidas entre ubicaciones. Por ejemplo, [la Figura 2-6](#) utiliza dos métodos diferentes para calcular la ruta más corta.

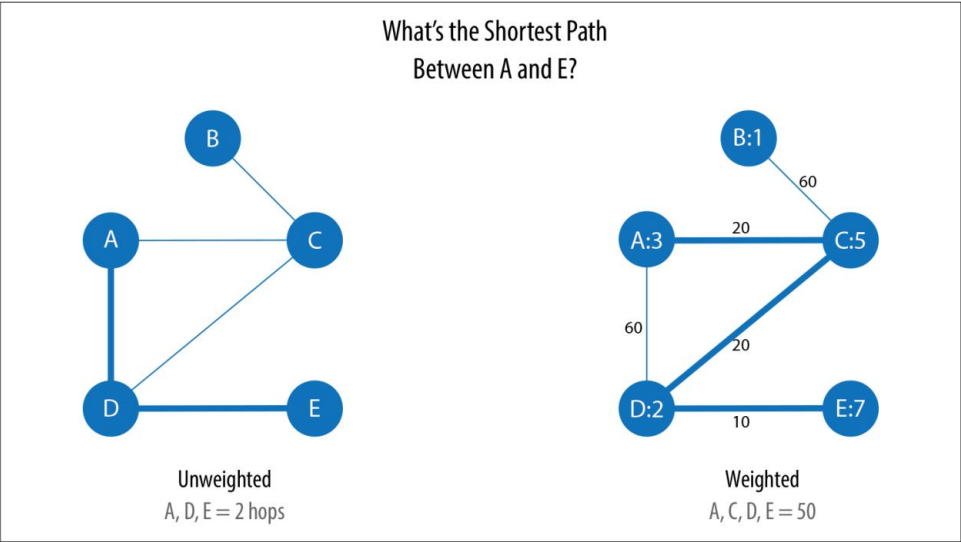


Figura 2-6. Las rutas más cortas pueden variar para gráficos ponderados y no ponderados que, por lo demás, serían idénticos.

Sin ponderaciones, nuestra ruta más corta se calcula en términos del número de relaciones (comúnmente llamadas saltos). A y E tienen una ruta más corta de dos saltos, lo que indica que solo hay un nodo (D) entre ellos. Sin embargo, la ruta ponderada más corta de A a E nos lleva de A a C, de D a E. Si las ponderaciones representan una distancia física en kilómetros, la distancia total sería de 50 km. En este caso, la ruta más corta en términos de número de saltos equivaldría a una ruta física más larga de 70 km.

Grafos no dirigidos vs. grafos dirigidos . En un grafo no

dirigido, las relaciones se consideran bidireccionales (por ejemplo, las amistades). En un grafo dirigido, las relaciones tienen una dirección específica. Las relaciones que apuntan a un nodo se denominan enlaces entrantes y, como era de esperar, los enlaces salientes son aquellos que se originan en un nodo.

La dirección añade otra dimensión de información. Las relaciones del mismo tipo, pero en direcciones opuestas, tienen un significado semántico diferente, expresando dependencia o indicando fluidez. Esto puede utilizarse como indicador de credibilidad o fortaleza grupal. Las preferencias personales y las relaciones sociales se expresan muy bien con la dirección.

Por ejemplo, si asumimos en la [Figura 2-7](#) que el gráfico dirigido era una red de estudiantes y las relaciones eran “me gusta”, entonces calcularíamos que A y C son más populares.

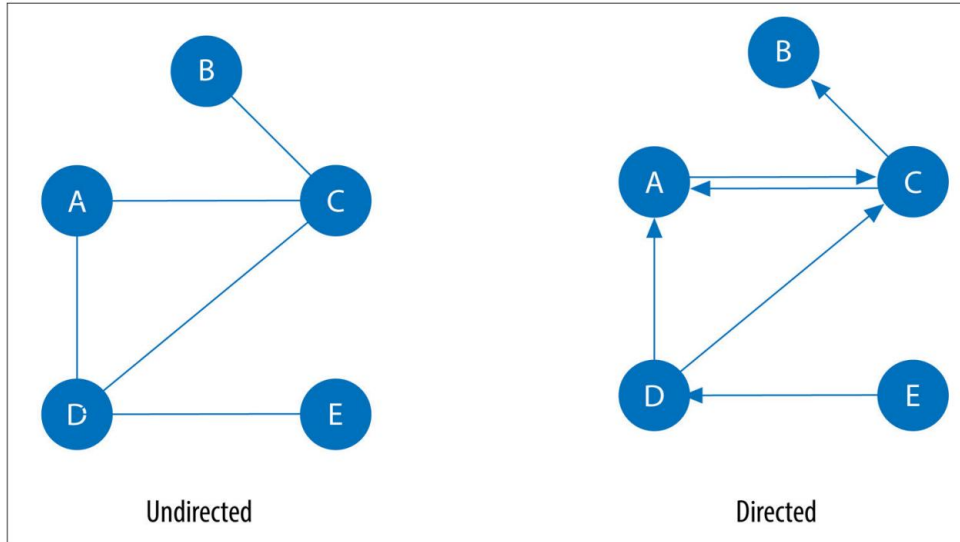


Figura 2-7. Muchos algoritmos nos permiten realizar cálculos basándose únicamente en conexiones entrantes o salientes, en ambas direcciones o sin dirección.

Las redes de carreteras ilustran por qué podríamos querer usar ambos tipos de gráficos. Por ejemplo, las autopistas entre ciudades suelen transitarse en ambas direcciones. Sin embargo, dentro de las ciudades, algunas carreteras son de un solo sentido. (¡Lo mismo ocurre con algunos flujos de información!)

Obtenemos resultados diferentes al ejecutar algoritmos de forma indirecta en comparación con los dirigidos. En un grafo indirecto, por ejemplo, para autopistas o amistades, asumiríamos que todas las relaciones siempre son bidireccionales.

Si reimaginamos [la Figura 2-7](#) como una red vial dirigida, se puede llegar a A desde C y D, pero solo se puede salir por C. Además, si no existieran relaciones entre A y C, esto indicaría un callejón sin salida. Quizás esto sea menos probable en una red vial unidireccional, pero no en un proceso o una página web.

Grafos Acíclicos vs. Grafos Cíclicos. En teoría de

grafos, los ciclos son caminos a través de relaciones y nodos que comienzan y terminan en el mismo nodo. Un grafo acíclico no tiene tales ciclos. Como se muestra en [la Figura 2-8](#), tanto los grafos dirigidos como los no dirigidos pueden tener ciclos, pero cuando son dirigidos, los caminos siguen la dirección de la relación. Un grafo acíclico dirigido (DAG), mostrado en el Gráfico 1, por definición siempre tendrá callejones sin salida (también llamados nodos hoja).

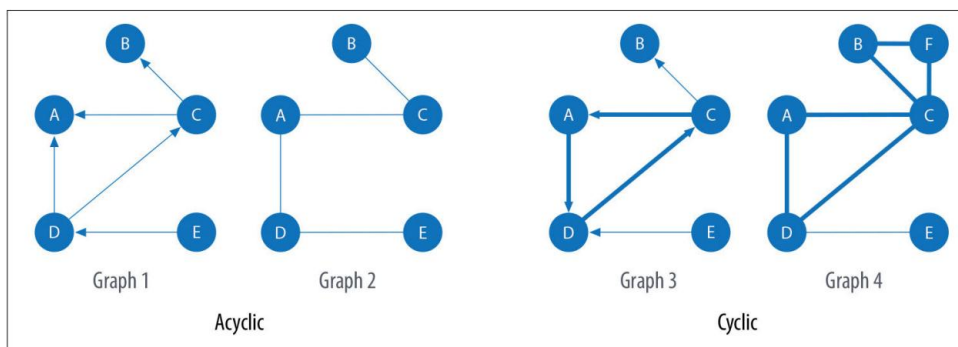


Figura 2-8. En los grafos acíclicos, es imposible comenzar y terminar en el mismo nodo sin retroceder.

Los grafos 1 y 2 no tienen ciclos, ya que no es posible comenzar y terminar en el mismo nodo sin repetir una relación. Quizás recuerdes del [Capítulo 1](#) que la no repetición de relaciones fue el problema de los puentes de Königsberg que dio origen a la teoría de grafos. El grafo 3 de [la Figura 2-8](#) muestra un ciclo simple después del ADCA sin nodos repetidos. En el grafo 4, el grafo cíclico no dirigido se ha vuelto más interesante al añadir un nodo y una relación. Ahora hay un ciclo cerrado con un nodo repetido (C), después del BFCDACB. De hecho, hay múltiples ciclos en el grafo 4.

Los ciclos son comunes, y a veces es necesario convertir grafos cíclicos en grafos acíclicos (cortando relaciones) para eliminar problemas de procesamiento. Los grafos acíclicos dirigidos surgen naturalmente en la programación, la genealogía y los historiales de versiones.

Árboles

En la teoría clásica de grafos, un grafo acíclico sin dirección se denomina árbol. En informática, los árboles también pueden ser dirigidos. Una definición más inclusiva sería un grafo donde dos nodos cualesquiera están conectados por una sola ruta. Los árboles son importantes para comprender las estructuras de grafos y muchos algoritmos. Desempeñan un papel clave en el diseño de redes, estructuras de datos y optimizaciones de búsqueda para mejorar la categorización o las jerarquías organizativas.

Se ha escrito mucho sobre los árboles y sus variaciones. La **Figura 2-9** ilustra los árboles comunes que probablemente encontremos.

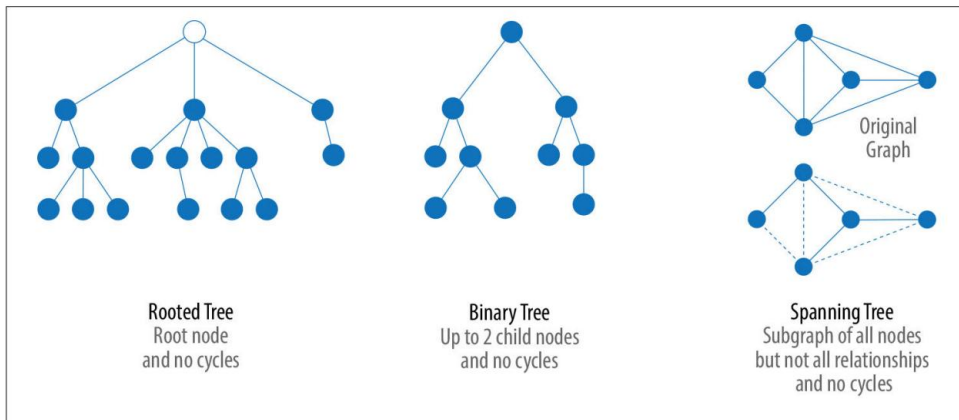


Figura 2-9. De estos gráficos de árbol prototípicos, los árboles de expansión son los más utilizados para algoritmos de gráficos.

De estas variaciones, los árboles de expansión son los más relevantes para este libro. Un árbol de expansión es un subgrafo que incluye todos los nodos de un grafo acíclico mayor, pero no todas las relaciones. Un árbol de expansión mínimo conecta todos los nodos de un grafo con el menor número de saltos o con las rutas de menor ponderación.

Grafos dispersos versus grafos densos. La escasez de

un grafo se basa en el número de relaciones que posee en comparación con el número máximo posible de relaciones, lo que ocurriría si existiera una relación entre cada par de nodos. Un grafo donde cada nodo tiene una relación con todos los demás se denomina grafo completo o grupo para los componentes. Por ejemplo, si todos mis amigos se conocieran, se trataría de un grupo.

La densidad máxima de un grafo es el número de relaciones posibles en un grafo completo. Se calcula con la fórmula $\text{MaxD} = \frac{N(N-1)}{2}$ donde N es el número de nodos. Para medir la densidad real, utilizamos la fórmula $D = \frac{2(R)}{N(N-1)}$ donde R es el Número de relaciones. En la Figura 2-10, podemos ver tres medidas de densidad real para grafos no dirigidos.

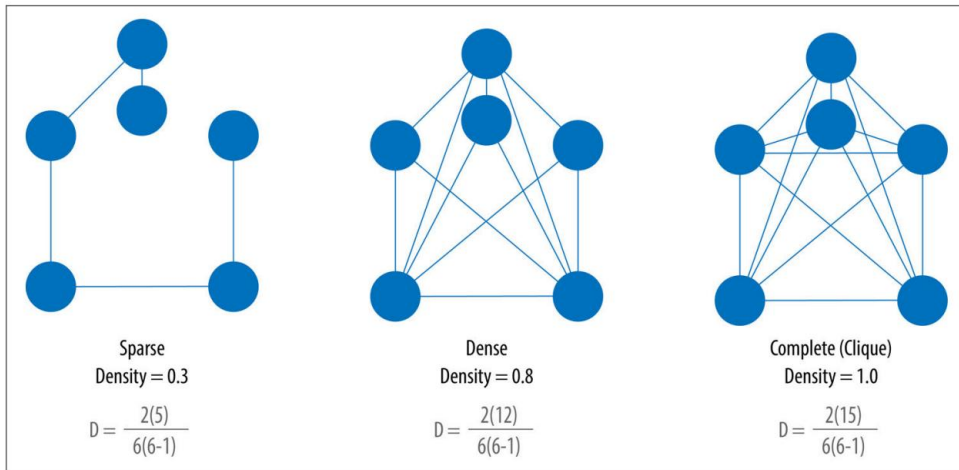


Figura 2-10. Verificar la densidad de un gráfico puede ayudarle a evaluar resultados inesperados.

Aunque no existe una línea divisoria estricta, cualquier grafo con una densidad real cercana a la máxima se considera denso. La mayoría de los grafos basados en redes reales tienden a la dispersión, con una correlación aproximadamente lineal entre el número total de nodos y el total de relaciones. Esto se aplica especialmente cuando entran en juego elementos físicos, como las limitaciones prácticas en cuanto al número de cables, tuberías, carreteras o amistades que se pueden conectar en un punto.

Algunos algoritmos arrojarán resultados sin sentido al ejecutarse en grafos extremadamente dispersos o densos. Si un grafo es demasiado disperso, es posible que no existan suficientes relaciones para que los algoritmos calculen resultados útiles. Por otro lado, los nodos con conexiones muy densas no aportan mucha información adicional debido a su alta conectividad. Las densidades altas también pueden sesgar algunos resultados o añadir complejidad computacional. En estas situaciones, filtrar el subgrafo relevante es una estrategia práctica.

Grafos monpartitos, bipartitos y k-partitos. La mayoría de las

redes contienen datos con múltiples tipos de nodos y relaciones. Sin embargo, los algoritmos de grafos con frecuencia consideran solo un tipo de nodo y un tipo de relación.

Los gráficos con un tipo de nodo y un tipo de relación a veces se denominan monpartitos.

Un grafo bipartito es un grafo cuyos nodos se pueden dividir en dos conjuntos, de modo que las relaciones solo conectan un nodo de un conjunto con uno de otro. La Figura 2-11 muestra un ejemplo de este tipo de grafo. Tiene dos conjuntos de nodos: un conjunto de espectadores y un conjunto de programas de televisión. Solo existen relaciones entre los dos conjuntos y no existen conexiones intraconjunto. En otras palabras, en el Gráfico 1, los programas de televisión solo se relacionan con los espectadores, no con otros programas, y los espectadores tampoco están directamente vinculados entre sí.

A partir de nuestro gráfico bipartito de espectadores y programas de televisión, creamos dos proyecciones monopartitas: el Gráfico 2 de conexiones de espectadores basadas en programas en común y el Gráfico 3 de programas de televisión basados en espectadores en común. También podemos filtrar por tipo de relación, como visto, valorado o reseñado.

Proyectar grafos monopartitos con conexiones inferidas es una parte importante del análisis de grafos. Este tipo de proyecciones ayuda a descubrir relaciones y cualidades indirectas. Por ejemplo, en el Gráfico 2 de la Figura 2-11, Bev y Ann solo han visto un programa de televisión en común, mientras que Bev y Evan tienen dos. En el Gráfico 3, ponderamos las relaciones entre los programas de televisión según las visualizaciones agregadas de los espectadores en común. Esta, u otras métricas como la similitud, pueden utilizarse para inferir significado entre actividades como ver Battlestar Galactica y Firefly. Esto puede fundamentar nuestra recomendación para alguien similar a Evan, quien, en la Figura 2-11, acaba de ver el último episodio de Firefly.

Los grafos k-partitos hacen referencia al número de tipos de nodos que tienen nuestros datos (k). Por ejemplo, si tenemos tres tipos de nodos, tendríamos un grafo tripartito. Esto simplemente extiende los conceptos bipartitos y monopartitos para dar cuenta de más tipos de nodos. Muchos grafos del mundo real, especialmente los grafos de conocimiento, tienen un valor grande para k, ya que combinan muchos conceptos y tipos de información diferentes. Un ejemplo del uso de un mayor número de tipos de nodos es crear nuevas recetas asignando un conjunto de recetas a un conjunto de ingredientes a un compuesto químico y luego deduciendo nuevas mezclas que conecten las preferencias populares. También podríamos reducir el número de tipos de nodos por generalización, como tratar muchas formas de un nodo, como la espinaca o la berza, como simplemente "verduras de hoja verde".

Ahora que hemos revisado los tipos de gráficos con los que es más probable que trabajemos, aprendamos sobre los tipos de algoritmos gráficos que ejecutaremos en esos gráficos.

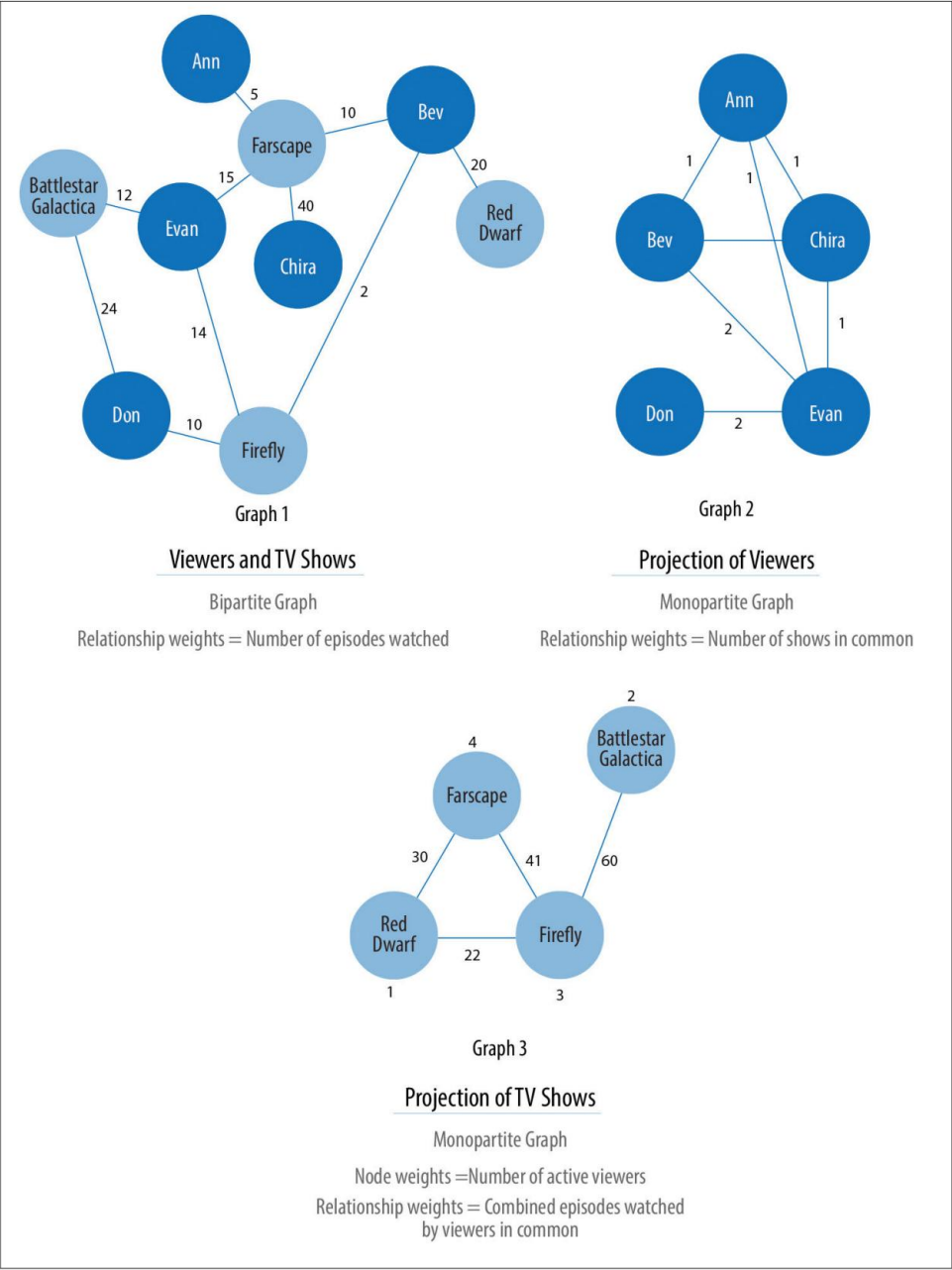


Figura 2-11. Los gráficos bipartitos a menudo se proyectan en gráficos monopartitos para un análisis más específico.

Tipos de algoritmos gráficos

Analicemos las tres áreas de análisis que están en el corazón de los algoritmos gráficos.

Estas categorías corresponden a los capítulos sobre algoritmos de búsqueda y localización de rutas, cálculo de centralidad y detección de comunidades.

Las rutas son

fundamentales para el análisis y los algoritmos de grafos, por lo que aquí es donde comenzaremos nuestros capítulos con ejemplos específicos de algoritmos. Encontrar las rutas más cortas es probablemente la tarea más frecuente con los algoritmos de grafos y es precursora de varios tipos de análisis. La ruta más corta es la ruta transversal con menos saltos o menor peso. Si el grafo es dirigido, entonces es la ruta más corta entre dos nodos, según lo permitan las direcciones de la relación.

Tipos de ruta

La ruta más corta promedio se utiliza para considerar la eficiencia y la resiliencia general de las redes, como por ejemplo para comprender la distancia promedio entre estaciones de metro. A veces, también podríamos querer comprender la ruta optimizada más larga para situaciones como determinar qué estaciones de metro están más alejadas o tienen más paradas entre sí, incluso cuando se elige la mejor ruta. En este caso, usaríamos el diámetro de un grafo para encontrar la ruta más corta y más larga entre todos los pares de nodos.

Centralidad La

centralidad consiste en comprender qué nodos son más importantes en una red.

Pero ¿qué entendemos por importancia? Existen diferentes tipos de algoritmos de centralidad creados para medir diferentes aspectos, como la capacidad de difundir información rápidamente frente a la de conectar grupos distintos. En este libro, nos centraremos en cómo se estructuran los nodos y las relaciones.

La conectividad para la detección

de comunidades es un concepto fundamental de la teoría de grafos que permite un análisis de redes sofisticado, como la búsqueda de comunidades. La mayoría de las redes del mundo real presentan subestructuras (a menudo cuasifractales) de subgrafos más o menos independientes.

La conectividad se utiliza para identificar comunidades y cuantificar la calidad de las agrupaciones. La evaluación de diferentes tipos de comunidades dentro de un grafo permite descubrir estructuras, como núcleos y jerarquías, y la tendencia de los grupos a atraer o rechazar a otros. Estas técnicas se emplean para estudiar fenómenos emergentes, como los que dan lugar a cámaras de eco y efectos de burbuja de filtro.

Resumen

Los grafos son intuitivos. Se alinean con nuestra forma de pensar y dibujar sistemas. Los principios básicos del trabajo con grafos se pueden asimilar rápidamente una vez que hemos desentrañado parte de la terminología y las capas. En este capítulo, hemos explicado las ideas y expresiones que se utilizan más adelante en este libro y descrito los tipos de grafos que encontrará.

Referencias de teoría de grafos

Si estás entusiasmado por aprender más sobre la teoría de grafos en sí, hay algunos textos introductorios que recomendamos:

- Introducción a la teoría de grafos, de Richard J. Trudeau (Dover), es una introducción muy bien escrita y agradable.
- Introducción a la teoría de grafos, quinta edición, de Robin J. Wilson (Pearson), es una introducción sólida con buenas ilustraciones.
- Teoría de grafos y sus aplicaciones, tercera edición, de Jonathan L. Gross, Jay Yellen y Mark Anderson (Chapman y Hall), asume más conocimientos de matemáticas y proporciona más detalles y ejercicios.

A continuación, veremos el procesamiento de gráficos y los tipos de análisis antes de profundizar en cómo usar algoritmos de gráficos en Apache Spark y Neo4j.

Plataformas gráficas y procesamiento

En este capítulo, abordaremos rápidamente diferentes métodos de procesamiento de gráficos y las plataformas más comunes. Analizaremos con más detalle las dos plataformas utilizadas en este libro, Apache Spark y Neo4j, y cuándo pueden ser adecuadas para diferentes requisitos. Se incluyen pautas de instalación para prepararle para los siguientes capítulos.

Consideraciones sobre la plataforma gráfica y el procesamiento

El procesamiento analítico de grafos posee cualidades únicas, como un cálculo basado en la estructura, con enfoque global y difícil de analizar. En esta sección, analizaremos las consideraciones generales para las plataformas y el procesamiento de grafos.

Consideraciones sobre la plataforma

Existe un debate sobre si es mejor ampliar o reducir el procesamiento de gráficos.

¿Debería utilizar potentes máquinas multinúcleo con gran memoria y centrarse en estructuras de datos eficientes y algoritmos multihilo? ¿O merece la pena invertir en plataformas de procesamiento distribuido y algoritmos relacionados?

Un enfoque de evaluación útil es la Configuración que Supera el Rendimiento de un Solo Hilo (COST), como se describe en el artículo de investigación "[¡Escalabilidad! ¿Pero a qué COST?](#)" de F. McSherry, M. Isard y D. Murray. COST nos permite comparar la escalabilidad de un sistema con la sobrecarga que este introduce. El concepto central es que un sistema bien configurado, que utiliza un algoritmo y una estructura de datos optimizados, puede superar las soluciones actuales de escalabilidad horizontal de propósito general. Es un método para medir las mejoras de rendimiento sin recompensar los sistemas que enmascaran ineficiencias mediante la paralelización. Separar los conceptos de escalabilidad y uso eficiente de recursos nos ayudará a construir una plataforma configurada específicamente para nuestras necesidades.

Algunos enfoques para las plataformas gráficas incluyen soluciones altamente integradas que optimizan los algoritmos, el procesamiento y la recuperación de memoria para trabajar en una coordinación más estrecha.

Consideraciones de procesamiento.

Existen diferentes enfoques para expresar el procesamiento de datos; por ejemplo, el procesamiento por flujos o por lotes, o el paradigma map-reduce para datos basados en registros. Sin embargo, para datos de grafos, también existen enfoques que incorporan las dependencias de datos inherentes a las estructuras de grafos en su procesamiento.

Centrado en nodos

Este enfoque utiliza nodos como unidades de procesamiento, permitiéndoles acumular y calcular el estado, y comunicar los cambios de estado mediante mensajes a sus vecinos. Este modelo utiliza las funciones de transformación proporcionadas para implementar cada algoritmo de forma más sencilla.

Centrado en las

relaciones Este enfoque tiene similitudes con el modelo centrado en los nodos, pero puede funcionar mejor para el análisis de subgrafos y secuenciales.

Centrado en

grafos Estos modelos procesan nodos dentro de un subgrafo independientemente de otros subgrafos mientras que la comunicación (mínima) con otros subgrafos ocurre a través de mensajes.

Centrado en el

recorrido Estos modelos utilizan la acumulación de datos por parte del recorrido mientras navega por el gráfico como medio de cálculo.

Centrado en algoritmos:

Estos enfoques utilizan diversos métodos para optimizar las implementaciones por algoritmo. Es un híbrido de los modelos anteriores.



Pregel es un marco de procesamiento paralelo tolerante a fallas y centrado en nodos creado por Google para el análisis de alto rendimiento de gráficos grandes. Pregel se basa en el modelo de Paralelo Sincrónico en Masa (BSP). BSP simplifica la programación paralela al contar con fases de computación y comunicación diferenciadas.

Pregel añade una abstracción centrada en los nodos sobre BSP, mediante la cual los algoritmos calculan valores a partir de los mensajes entrantes de los vecinos de cada nodo. Estos cálculos se ejecutan una vez por iteración y pueden actualizar los valores de los nodos y enviar mensajes a otros nodos. Los nodos también pueden combinar mensajes para su transmisión durante la fase de comunicación, lo que reduce significativamente la cantidad de interferencias en la red. El algoritmo se completa cuando no se envían nuevos mensajes o se alcanza un límite establecido.

La mayoría de estos enfoques específicos para grafos requieren la presencia del grafo completo para realizar operaciones topológicas cruzadas eficientes. Esto se debe a que la separación y distribución de los datos del grafo conlleva extensas transferencias de datos y reorganización entre instancias de trabajador. Esto puede resultar difícil para los numerosos algoritmos que necesitan procesar iterativamente la estructura del gráfico global.

Plataformas representativas

Para abordar los requisitos del procesamiento de gráficos, han surgido varias plataformas.

Tradicionalmente existía una separación entre los motores de cálculo de gráficos y las bases de datos de gráficos, lo que requería que los usuarios movieran sus datos dependiendo de las necesidades de sus procesos:

Motores de cómputo de

grafos. Son motores de solo lectura, no transaccionales, que se centran en la ejecución eficiente de análisis iterativos de grafos y consultas de todo el grafo. Los motores de cómputo de grafos admiten diferentes paradigmas de definición y procesamiento para algoritmos de grafos, como enfoques centrados en nodos (p. ej., Pregel, Gather-Apply-Scatter) o basados en MapReduce (p. ej., PACT). Ejemplos de estos motores son Giraph, GraphLab, Graph-Engine y Apache Spark.

Bases de datos de

grafos. Con un enfoque transaccional, se centran en escrituras y lecturas rápidas mediante consultas más pequeñas que generalmente afectan a una pequeña fracción del grafo. Sus fortalezas residen en la robustez operativa y la alta escalabilidad concurrente para numerosos usuarios.

Selección de nuestra plataforma.

Elegir una plataforma de producción implica muchas consideraciones, como el tipo de análisis a ejecutar, las necesidades de rendimiento, el entorno existente y las preferencias del equipo. En este libro, utilizamos Apache Spark y Neo4j para mostrar algoritmos de grafos porque ambos ofrecen ventajas únicas.

Spark es un ejemplo de motor de cómputo de grafos escalable y centrado en nodos. Su popular marco de cómputo y sus bibliotecas son compatibles con diversos flujos de trabajo de ciencia de datos.

Spark puede ser la plataforma adecuada cuando:

- Los algoritmos son fundamentalmente paralelizables o particionables.
- Los flujos de trabajo de algoritmos necesitan operaciones "multilingües" en múltiples herramientas y los indicadores.
- El análisis se puede ejecutar sin conexión por lotes.
- El análisis gráfico se basa en datos no transformados a formato gráfico.
- El equipo necesita y cuenta con la experiencia para codificar e implementar sus propios algoritmos.
- El equipo utiliza algoritmos gráficos con poca frecuencia.

- El equipo prefiere mantener todos los datos y análisis dentro del ecosistema Hadoop.

La plataforma de grafos Neo4j es un ejemplo de una base de datos de grafos estrechamente integrada y un procesamiento centrado en algoritmos, optimizado para grafos. Es popular para crear aplicaciones basadas en grafos e incluye una biblioteca de algoritmos de grafos optimizada para su base de datos de grafos nativa. Neo4j puede ser la plataforma ideal cuando:

- Los algoritmos son más iterativos y requieren una buena localización de memoria. •

Los algoritmos y los resultados son sensibles al rendimiento. • El

análisis de grafos se basa en datos de grafos complejos o requiere un recorrido profundo de rutas. •

El análisis y los resultados se integran con las cargas de trabajo transaccionales.

- Los resultados se utilizan para enriquecer un grafo

existente. • El equipo necesita integrarse con herramientas de visualización

basadas en grafos. • El equipo prefiere algoritmos preconfigurados y con soporte.

Finalmente, algunas organizaciones utilizan tanto Neo4j como Spark para el procesamiento de gráficos: Spark para el filtrado y preprocesamiento de alto nivel de conjuntos de datos masivos y la integración de datos, y Neo4j para el procesamiento y la integración más específicos con aplicaciones basadas en gráficos.

Apache Spark (en

adelante, Spark) es un motor de análisis para el procesamiento de datos a gran escala. Utiliza una abstracción de tabla denominada DataFrame para representar y procesar datos en filas de columnas con nombre y tipo. La plataforma integra diversas fuentes de datos y es compatible con lenguajes como Scala, Python y R. Spark admite diversas bibliotecas de análisis, como se muestra en [la Figura 3-1](#). Su sistema basado en memoria funciona mediante grafos de cómputo distribuidos eficientemente.

GraphFrames es una biblioteca de procesamiento de gráficos para Spark que sucedió a GraphX en 2016, aunque es independiente del núcleo de Apache Spark. GraphFrames se basa en GraphX, pero utiliza DataFrames como estructura de datos subyacente. GraphFrames es compatible con los lenguajes de programación Java, Scala y Python. En la primavera de 2019, se aceptó la propuesta "Spark Graph: Property Graphs, Cypher Queries, and Algorithms" (véase ["Spark Graph Evolution" en la página 33](#)). Esperamos que esto incorpore varias funciones de gráficos que utilizan el marco DataFrame y el lenguaje de consulta Cypher al proyecto principal de Spark. Sin embargo, en este libro, nuestros ejemplos se basarán en la API de Python (PySpark) debido a su popularidad actual entre los científicos de datos de Spark.

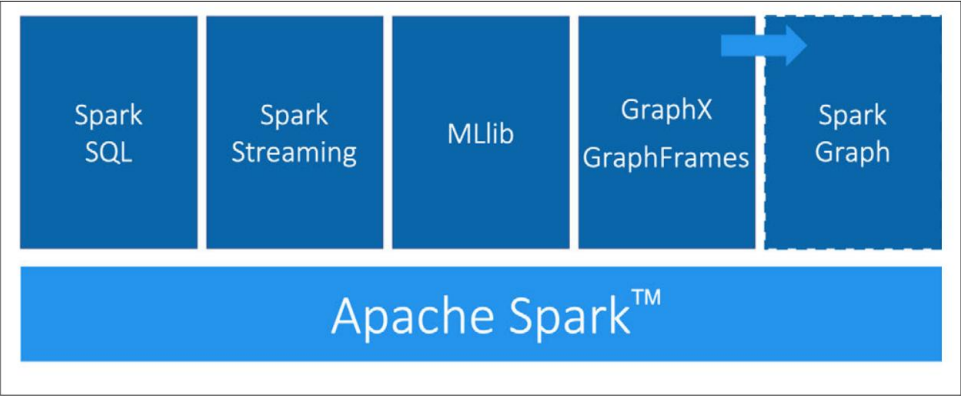


Figura 3-1. Spark es un framework de computación en clúster, distribuido y de código abierto, de propósito general. Incluye varios módulos para diversas cargas de trabajo.

Evolución de Spark Graph **El**

proyecto Spark Graph es una iniciativa conjunta de los colaboradores del proyecto Apache en Databricks y Neo4j para brindar soporte para DataFrames, Cypher y algoritmos basados en DataFrames en el proyecto principal Apache Spark como parte del lanzamiento 3.0.

Cypher comenzó como un lenguaje de consulta de gráficos declarativo implementado en Neo4j, pero a través del proyecto openCypher ahora lo utilizan varios proveedores de bases de datos y un proyecto de código abierto, **Cypher para Apache Spark (CAPS)**.

Próximamente, esperamos usar CAPS para cargar y proyectar datos de gráficos como parte integral de la plataforma Spark. Publicaremos ejemplos de Cypher una vez implementado el proyecto Spark Graph.

Este desarrollo no afecta los algoritmos tratados en este libro, pero podría añadir nuevas opciones a la forma de llamar a los procedimientos. El modelo de datos, los conceptos y el cálculo subyacentes de los algoritmos de grafos se mantendrán sin cambios.

Los nodos y las relaciones se representan como DataFrames con un ID único para cada nodo y un nodo de origen y uno de destino para cada relación. Podemos ver un ejemplo de un DataFrame de nodos en **la Tabla 3-1** y un DataFrame de relaciones en **la Tabla 3-2**. Un GraphFrame basado en estos DataFrames tendría dos nodos, JFK y SEA, y una relación, de JFK a SEA.

Tabla 3-1. Nodos DataFrame

—	ciudad	estado
JFK	Nueva York	NY
MAR	de Seattle,	WA

Tabla 3-2. Relaciones del DataFrame

origen	dst	retardo	tripId
JFK	SEA	45	1058923

El DataFrame de nodos debe tener una columna id ; el valor de esta columna se utiliza para identificar de forma única cada nodo. El DataFrame de relaciones debe tener las columnas src y dst ; los valores de estas columnas describen qué nodos están conectados y deben hacer referencia a las entradas que aparecen en la columna id del DataFrame de nodos.

Los nodos y relaciones de DataFrames se pueden cargar utilizando cualquiera de las **fuentes de datos de DataFrame**, incluyendo Parquet, JSON y CSV. Las consultas se describen mediante una combinación de la API de PySpark y Spark SQL.

GraphFrames también proporciona a los usuarios **un punto de extensión** para implementar algoritmos que no están disponibles de fábrica.

Instalación de

Spark. Puedes descargar Spark desde el **sitio web de Apache Spark**. Una vez descargado, necesitas instalar las siguientes bibliotecas para ejecutar trabajos de Spark desde Python:

```
pip install pyspark graphframes
```

Luego puedes iniciar el REPL de pyspark ejecutando el siguiente comando:

```
exportar SPARK_VERSION="spark-2.4.0-bin-hadoop2.7"
./${SPARK_VERSION}/bin/pyspark \ --
  driver-memory 2g \ --
  executor-memory 6g \ --
  packages graphframes:graphframes:0.7.0-spark2.4-s_2.11
```

Al momento de escribir esto, la última versión de Spark es spark-2.4.0-bin-hadoop2.7, pero es posible que haya cambiado para cuando lea esto. De ser así, asegúrese de modificar la variable de entorno SPARK_VERSION como corresponda.



Aunque los trabajos de Spark deben ejecutarse en un clúster de máquinas, para fines de demostración, solo los ejecutaremos en una sola máquina. Puede obtener más información sobre la ejecución de Spark en entornos de producción en **Spark: La Guía Definitiva**, de Bill Chambers y Matei Zaharia (O'Reilly).

Ahora está listo para aprender a ejecutar algoritmos gráficos en Spark.

Plataforma de grafos Neo4j

La plataforma de grafos Neo4j admite el procesamiento transaccional y analítico de datos de grafos. Incluye almacenamiento y computación de grafos con gestión de datos y

Herramientas de análisis. El conjunto de herramientas integradas se basa en un protocolo, una API y un lenguaje de consulta comunes (Cypher) para proporcionar un acceso eficaz para diferentes usos, como se muestra en la Figura 3-2.

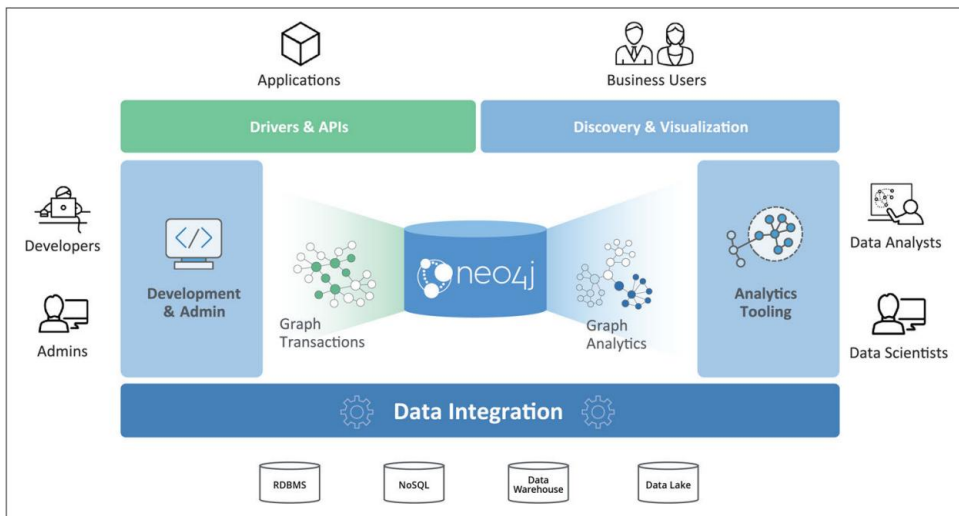


Figura 3-2. La plataforma de gráficos Neo4j está construida alrededor de una base de datos de gráficos nativa que admite aplicaciones transaccionales y análisis de gráficos.

En este libro, utilizaremos la **biblioteca de algoritmos de grafos Neo4j**. Esta biblioteca se instala como complemento junto con la base de datos y proporciona un conjunto de **procedimientos definidos por el usuario** que pueden ejecutarse mediante el lenguaje de consulta Cypher.

La biblioteca de algoritmos de grafos incluye versiones paralelas de algoritmos que facilitan el análisis de grafos y los flujos de trabajo de aprendizaje automático. Los algoritmos se ejecutan sobre un marco de computación paralela basado en tareas y están optimizados para la plataforma Neo4j. Para diferentes tamaños de grafos, existen implementaciones internas que escalan hasta decenas de miles de millones de nodos y relaciones.

Los resultados se pueden transmitir al cliente como un flujo de tuplas y los resultados tabulares se pueden usar como tabla de referencia para su posterior procesamiento. Opcionalmente, los resultados también se pueden reescribir en la base de datos de forma eficiente como propiedades de nodo o tipos de relación.



En este libro, también utilizaremos la **biblioteca Neo4j Awesome Procedures on Cypher (APOC)**. APOC consta de más de 450 procedimientos y funciones para facilitar tareas comunes como la integración de datos, la conversión de datos y la refactorización de modelos.

Instalación de

Neo4j Neo4j Desktop es una forma conveniente para que los desarrolladores trabajen con bases de datos locales de Neo4j. Se puede descargar desde el [sitio web de Neo4j](#). Los algoritmos gráficos y las bibliotecas APOC se pueden instalar como complementos una vez instalado y abierto Neo4j Desktop. En el menú de la izquierda, cree un proyecto y selecciónelo. A continuación, haga clic en "Administrar" en la base de datos donde desea instalar los complementos. En la pestaña "Complementos", verá opciones para varios complementos. Haga clic en el botón "Instalar" para los algoritmos gráficos y APOC. Consulte las figuras 3-3 y 3-4.

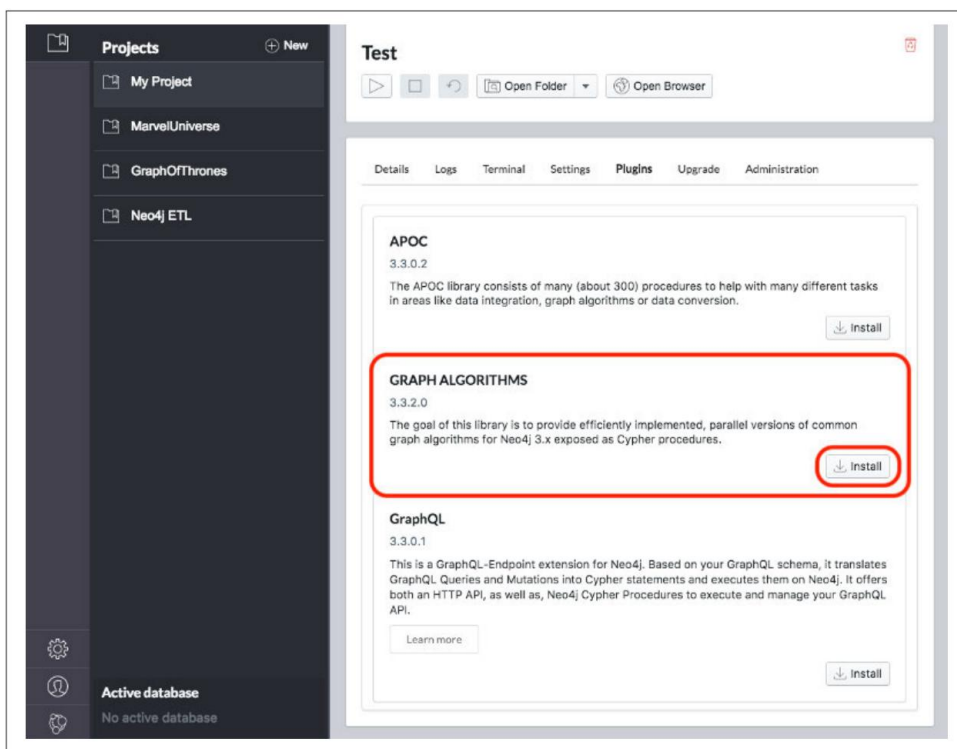


Figura 3-3. Instalación de la biblioteca de algoritmos gráficos

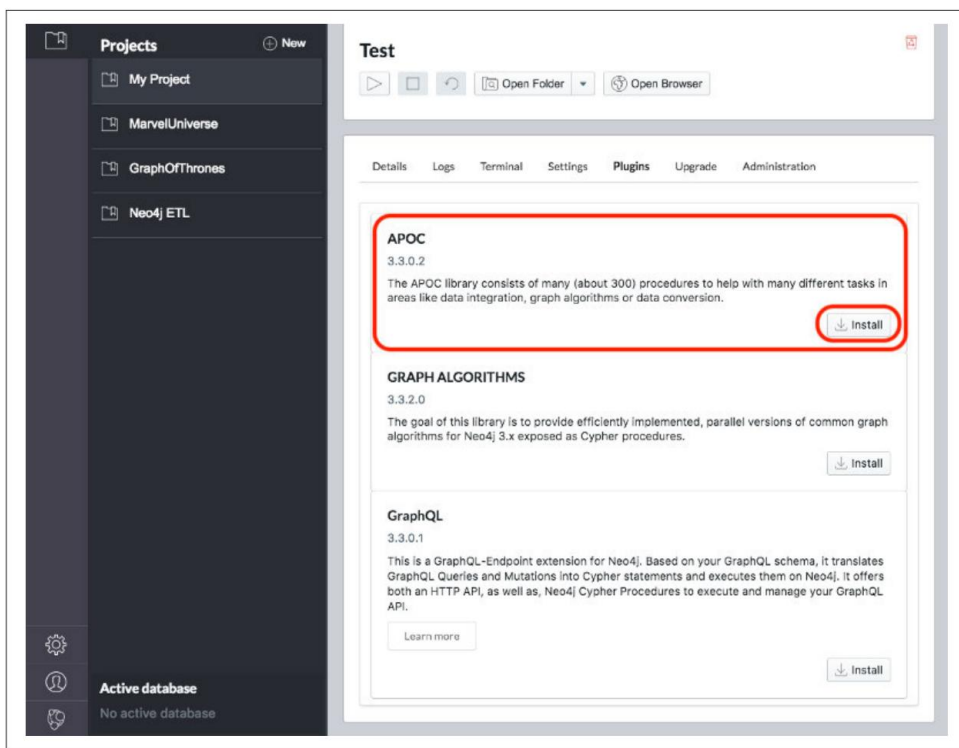


Figura 3-4. Instalación de la biblioteca APOC

Jennifer Reif explica el proceso de instalación con más detalle en su entrada de blog "[Explora nuevos mundos: cómo añadir plugins a Neo4j](#)". Ya estás listo para aprender a ejecutar algoritmos de grafos en Neo4j.

Resumen

En los capítulos anteriores, describimos la importancia del análisis de grafos para el estudio de redes del mundo real y analizamos los conceptos fundamentales, el análisis y el procesamiento de grafos. Esto nos proporciona una base sólida para comprender cómo aplicar algoritmos de grafos. En los próximos capítulos, descubriremos cómo ejecutar algoritmos de grafos con ejemplos en Spark y Neo4j.

Algoritmos de búsqueda de rutas y de grafos

Los algoritmos de búsqueda de grafos exploran un grafo, ya sea para descubrimiento general o búsqueda explícita. Estos algoritmos trazan rutas a través del grafo, pero no se espera que sean computacionalmente óptimas. Abordaremos la búsqueda en amplitud y la búsqueda en profundidad, ya que son fundamentales para recorrer un grafo y suelen ser un primer paso necesario para muchos otros tipos de análisis.

Los algoritmos de búsqueda de rutas se basan en algoritmos de búsqueda de grafos y exploran rutas entre nodos, comenzando en un nodo y recorriendo las relaciones hasta llegar al destino. Estos algoritmos se utilizan para identificar rutas óptimas a través de un grafo para usos como la planificación logística, el enrutamiento de llamadas de menor coste o IP, y la simulación de juegos.

Específicamente, los algoritmos de búsqueda de rutas que cubriremos son:

- Camino más corto, con dos variaciones útiles (A^* y Yen): encontrar el camino o los caminos más cortos entre dos nodos elegidos
- Ruta más corta de todos los pares y ruta más corta de una sola fuente: para encontrar la ruta más corta rutas entre todos los pares o desde un nodo elegido a todos los demás
- Árbol de expansión mínimo: para encontrar una estructura de árbol conectada con el árbol de expansión mínimo. costo por visitar todos los nodos desde un nodo elegido
- Paseo aleatorio: porque es un paso de preprocesamiento/muestreo útil para la máquina flujos de trabajo de aprendizaje y otros algoritmos gráficos

En este capítulo, explicaremos cómo funcionan estos algoritmos y mostraremos ejemplos en Spark y Neo4j. Si un algoritmo solo está disponible en una plataforma, proporcionaremos solo ese ejemplo o ilustraremos cómo personalizar nuestra implementación.

La Figura 4-1 muestra las diferencias clave entre estos tipos de algoritmos, y la Tabla 4-1 es una referencia rápida de lo que calcula cada algoritmo con un ejemplo de uso.

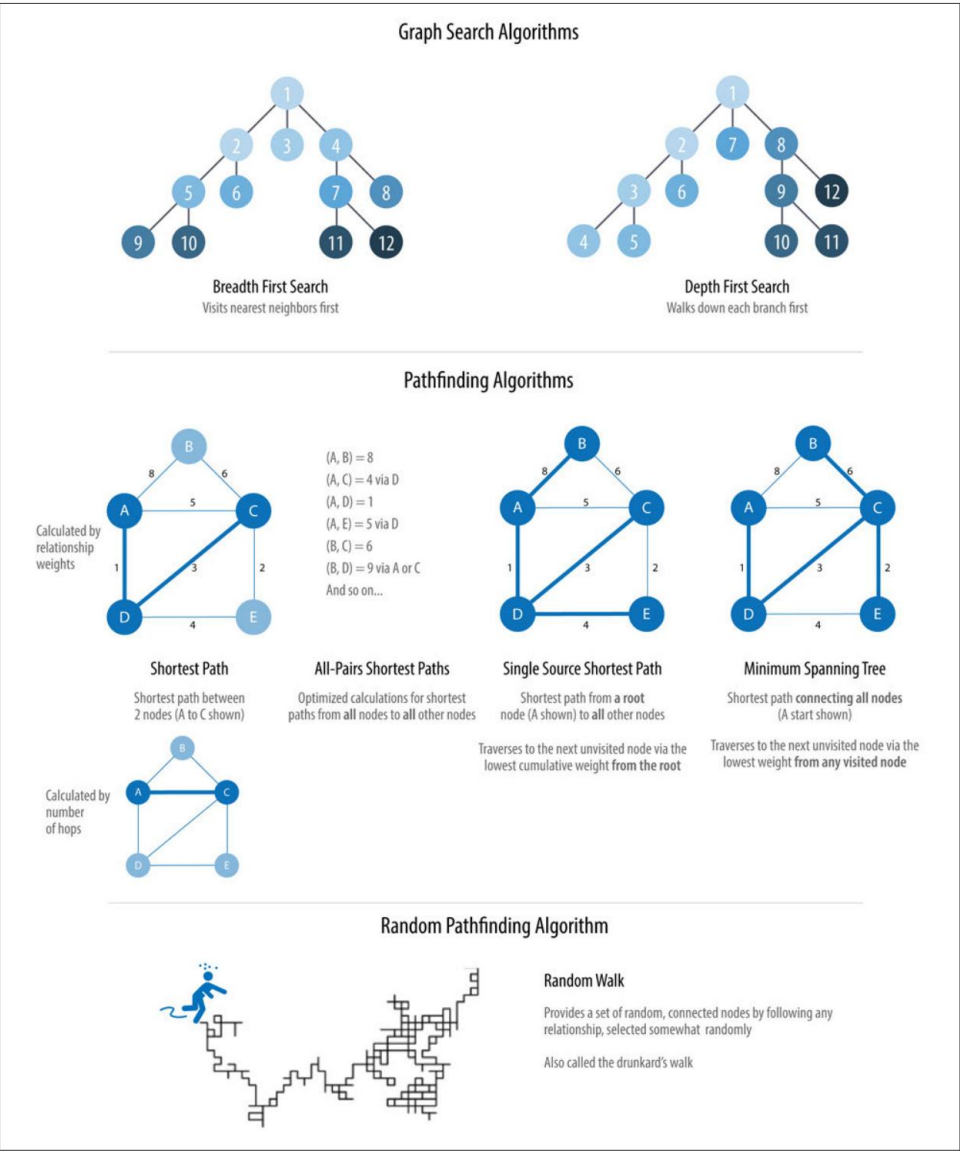


Tabla 4-1. Descripción general de los algoritmos de búsqueda de rutas y de grafos

Tipo de algoritmo	Qué hace	Ejemplo de uso	Chispa ejemplo	Neo4j ejemplo
Búsqueda en amplitud	Recorre una estructura de árbol en abanico. Salir a explorar los vecinos más cercanos y luego sus vecinos de subnivel	Localización de nodos vecinos en Sistemas GPS para identificar lugares de interés cercanos	Sí	No
Búsqueda en profundidad	Recorre una estructura de árbol explorando lo más lejos posible por cada rama antes de dar marcha atrás	Descubriendo un óptimo ruta de solución en los juegos simulaciones con opciones jerárquicas	No	No
Camino más corto Variaciones: A*, Yen's	Calcula la ruta más corta entre un par de nodos	Encontrar indicaciones para llegar en coche entre dos ubicaciones	Sí	Sí
El camino más corto de todos los pares	Calcula la ruta más corta entre todos los pares de nodos en el gráfico	Evaluación de rutas alternativas alrededor de un atasco de tráfico	Sí	Sí
Ruta más corta de una sola fuente	Calcula la ruta más costera entre una nodo raíz único y todos los demás nodos	Enrutamiento de teléfono de menor costo llamadas	Sí	Sí
Árbol de expansión mínimo	Calcula la ruta en un árbol conectado Estructura con el menor costo para visitando todos los nodos	Optimización conectada enrutamiento, como el tendido de cables o recolección de basura	No	Sí
Paseo aleatorio	Devuelve una lista de nodos a lo largo de una ruta de tamaño especificado eligiendo al azar relaciones a recorrer.	Aumentar la formación para aprendizaje automático o datos para algoritmos gráficos.	No	Sí

Primero, echaremos un vistazo al conjunto de datos para nuestros ejemplos y explicaremos cómo importarlos. los datos en Apache Spark y Neo4j. Para cada algoritmo, comenzaremos con un breve Descripción del algoritmo y cualquier información pertinente sobre su funcionamiento. La mayoría Las secciones también incluyen orientación sobre cuándo utilizar algoritmos relacionados. Finalmente, proporcionamos Código de muestra de trabajo que utiliza el conjunto de datos de muestra al final de cada sección del algoritmo.

¡Comencemos!

Datos de ejemplo: El gráfico de transporte

Todos los datos conectados contienen rutas entre nodos, por lo que la búsqueda y la búsqueda de rutas son esenciales. Los datos de transporte son los puntos de partida para el análisis gráfico. Los conjuntos de datos de transporte ilustran esto. relaciones de una manera intuitiva y accesible. Los ejemplos de este capítulo se ejecutan Contra un [gráfico que contiene un subconjunto de la red de carreteras europea](#). Puede descargar los archivos de nodos y relaciones del [repositorio de GitHub del libro](#).

Tabla 4-2. transport-nodes.csv

id	latitud	población de longitud
Ámsterdam	52.379189	4.899431 821752

ciudad	latitud	longitud	población
Utrech	52.092876	5.104480	334176
La Haya	52.078663	4.288788	514861
Immingham	53.61239	-0.22219	9642
Doncaster	53.52285	-1.13116	302400
Hoek van Holland	51.9775	4.13333	9382
Felixstowe	51.96375	1.3511	23689
Ipswich	52.05917	1.15545	133384
Colchester	51.88921	0.90421	104390
Londres	51.509865	-0.118092	8787892
Róterdam	51.9225	4.47917	623652
Gouda	52.01667	4.70833	70939

Tabla 4-3. relaciones-de-transporte.csv

origen	destino	modo	costo de la relación
Ámsterdam	Utrech	EROAD	46
Ámsterdam	La Haya	EROAD	59
La Haya	Róterdam	EROAD	26
Ámsterdam	Immingham	EROAD	369
Immingham	Doncaster	EROAD	74
Doncaster	Londres	EROAD	277
Hoek van Holland	Den Haag	EROAD	27
Felixstowe	Hoek van Holland	EROAD	207
Ipswich	Felixstowe	EROAD	22
Colchester	Ipswich	EROAD	32
Londres	Colchester	EROAD	106
Gouda	Róterdam	EROAD	25
Gouda	Utrech	EROAD	35
La Haya	Gouda	EROAD	32
Hoek van Holland	Róterdam	EROAD	33

La figura 4-2 muestra el gráfico objetivo que queremos construir.

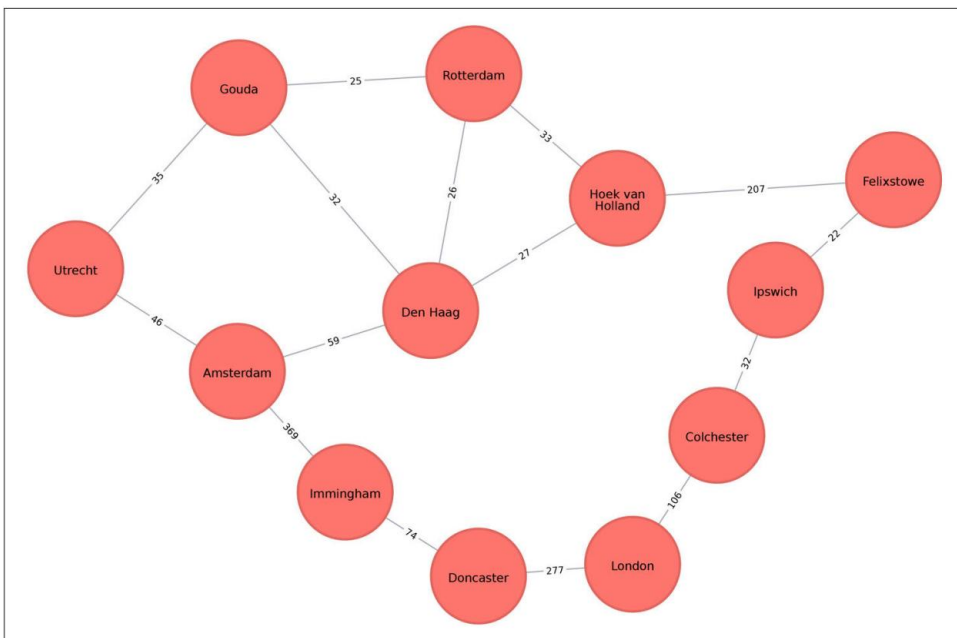


Figura 4-2. El gráfico de transporte

Para simplificar, consideramos que el grafo de la Figura 4-2 no es dirigido, ya que la mayoría de las carreteras entre ciudades son bidireccionales. Obtendríamos resultados ligeramente diferentes si evaluáramos el grafo como dirigido debido al pequeño número de calles de un solo sentido, pero el enfoque general sigue siendo similar. Sin embargo, tanto Spark como Neo4j operan con grafos dirigidos. En casos como este, donde deseamos trabajar con grafos no dirigidos (por ejemplo, carreteras bidireccionales), existe una manera sencilla de lograrlo:

Para Spark, crearemos dos relaciones para cada fila en `transport-relationships.csv`: una que va de `dst` a `src` y otra de `src` a `dst`. Para Neo4j, crearemos una sola relación y luego ignoraremos la dirección de la relación al ejecutar los algoritmos.

Una vez comprendidos esas pequeñas soluciones alternativas de modelado, ahora podemos continuar con la carga de gráficos en Spark y Neo4j desde los archivos CSV de ejemplo.

Importación de datos a Apache Spark

Comenzando con Spark, primero importaremos los paquetes que necesitamos de Spark y el Paquete GraphFrames:

```

desde pyspark.sql.types importar *
desde graphframes importar *

```

La siguiente función crea un GraphFrame a partir de los archivos CSV de ejemplo:

```
def create_transport_graph(): campos_de_nodo
    = [
        Campo de estructura("id", Tipo de cadena(), Verdadero),
        StructField("latitud", FloatType(), Verdadero),
        StructField("longitud", FloatType(), True),
        StructField("población", IntegerType(), True)

    ] nodos = spark.read.csv("data/transport-nodes.csv", encabezado=True,
                             esquema=StructType(node_fields))

    rels = spark.read.csv("data/transport-relationships.csv", header=True) reversed_rels = (rels.withColumn("newSrc",
    rels.dst)

                                                .withColumn("newDst", rels.src) .drop("dst",

                                                "src") .withColumnRenamed("newSrc",
                                                "src") .withColumnRenamed("newDst",
                                                "dst") .select("src", "dst", "relación", "costo"))

    relaciones = rels.union(reversed_rels)

    devuelve GraphFrame(nodos, relaciones)
```

Cargar los nodos es fácil, pero para las relaciones necesitamos hacer un pequeño preprocesamiento para poder crear cada relación dos veces.

Ahora llamemos a esa función:

```
g = crear_gráfico_de_transporte()
```

Importando los datos a Neo4j . Ahora,

para Neo4j, comenzaremos cargando los nodos:

```
CON "https://github.com/neo4j-graph-analytics/book/raw/master/data" COMO base CON base + "transport-nodes.csv"
COMO uri CARGAR CSV CON ENCABEZADOS DESDE uri
COMO fila FUSIONAR (place:Place {id:row.id})

ESTABLECER lugar.latitud = toFloat(fila.latitud), lugar.longitud =
toFloat(fila.latitud), lugar.población = toInteger(fila.población)
```

Y ahora las relaciones:

```
CON "https://github.com/neo4j-graph-analytics/book/raw/master/data/" COMO base CON base + "transport-
relationships.csv" COMO uri CARGAR CSV CON ENCABEZADOS DESDE
uri COMO fila COINCIDIR (origen: Lugar {id: fila.src})

COINCIDIR (destino:Lugar {id: row.dst})
MERGE (origen)-[:EROAD {distancia: toInteger(fila.costo)}]->(destino)
```

Aunque almacenamos relaciones dirigidas, ignoraremos la dirección cuando ejecutemos algoritmos más adelante en el capítulo.

Búsqueda en amplitud

La búsqueda en amplitud (BFS) es uno de los algoritmos fundamentales para el recorrido de grafos. Parte de un nodo seleccionado y explora todos sus vecinos a un salto de distancia antes de visitar todos los vecinos a dos saltos de distancia, y así sucesivamente.

El algoritmo fue publicado por primera vez en 1959 por Edward F. Moore, quien lo utilizó para encontrar el camino más corto para salir de un laberinto. Posteriormente, CY Lee lo desarrolló como un algoritmo de enrutamiento de cables en 1961, como se describe en "[Un algoritmo para conexiones de caminos y sus aplicaciones](#)".

BFS se utiliza comúnmente como base para otros algoritmos más orientados a objetivos. Por ejemplo, [la Ruta más corta](#), [los Componentes conectados](#) y [la Centralidad de cercanía](#) utilizan el algoritmo BFS. También se puede utilizar para encontrar la ruta más corta entre nodos.

[La Figura 4-3](#) muestra el orden en que visitaríamos los nodos de nuestro grafo de transporte si realizáramos una búsqueda en amplitud con origen en la ciudad neerlandesa de Den Haag (en español, La Haya). Los números junto al nombre de la ciudad indican el orden de visita de cada nodo.

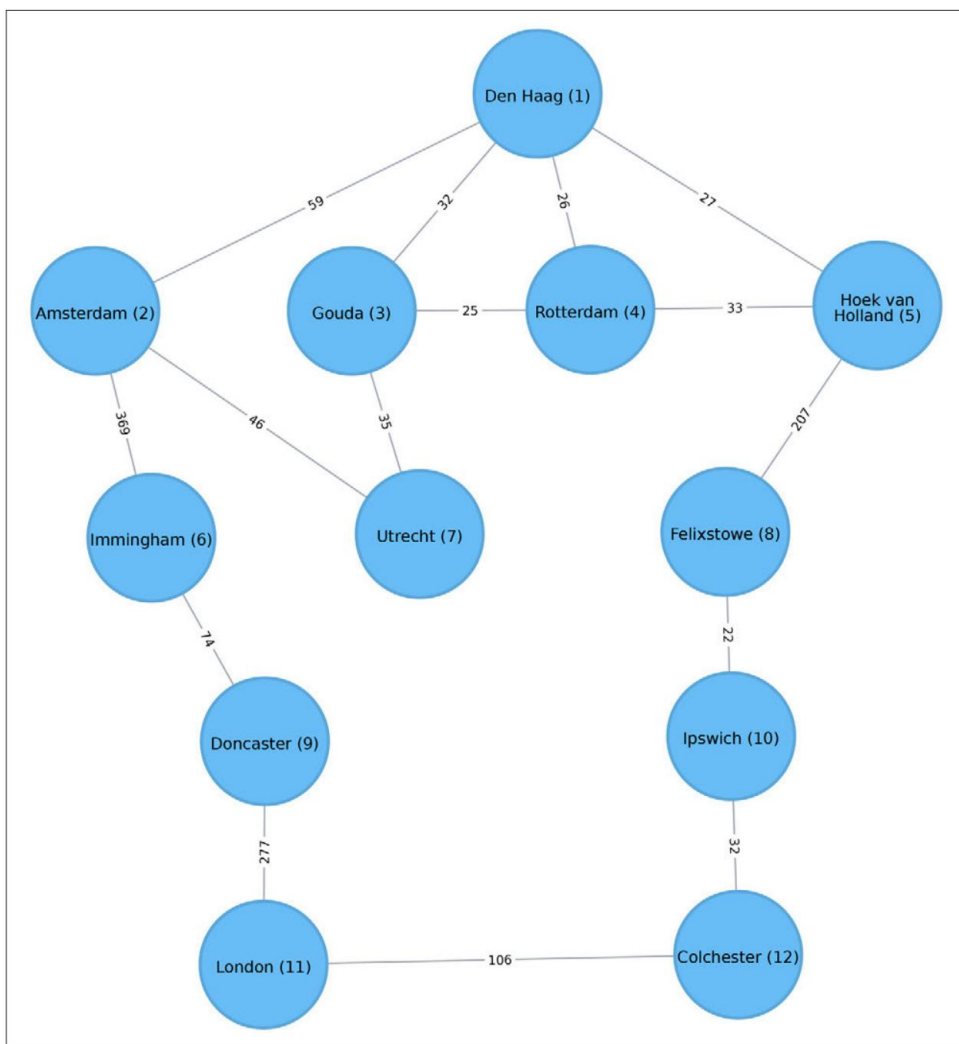


Figura 4-3. Búsqueda en amplitud desde La Haya. Los números de nodo indican el orden de recorrido.

Primero visitamos a todos los vecinos directos de La Haya, antes de visitar a sus vecinos y a los vecinos de sus vecinos, hasta que nos quedamos sin relaciones para recorrer.

Búsqueda en amplitud con Apache Spark. La

implementación del algoritmo de búsqueda en amplitud de Spark encuentra la ruta más corta entre dos nodos según el número de relaciones (es decir, saltos) entre ellos. Puede asignar un nombre explícito al nodo de destino o añadir criterios que deban cumplirse.

Por ejemplo, podemos utilizar la función `bfs` para encontrar la primera ciudad de tamaño mediano (según los estándares europeos) que tenga una población de entre 100.000 y 300.000 personas.

Primero, veamos qué lugares tienen una población que coincide con esos criterios:

```
(g.vértices
 .filter("población > 100000 y población <
300000") .sort("población") .show())
```

Este es el resultado que veremos:

	latitud	longitud	población
Colchester	51.88921	0.90421	104390
Ipswich	52.05917	1.15545	133384

Solo hay dos lugares que coinciden con nuestros criterios y esperamos llegar primero a Ipswich mediante una búsqueda en amplitud.

El siguiente código encuentra la ruta más corta desde La Haya a una ciudad de tamaño mediano:

```
from_expr = "id='Den Haag'"
to_expr = "población > 100 000 y población < 300 000 e id <> 'Den Haag'"
result = g.bfs(from_expr, to_expr)
```

El resultado contiene columnas que describen los nodos y las relaciones entre las dos ciudades. Podemos ejecutar el siguiente código para ver la lista de columnas devuelta:

```
print(resultado.columnas)
```

Este es el resultado que veremos:

```
['desde', 'e0', 'v1', 'e1', 'v2', 'e2', 'hasta']
```

Las columnas que empiezan por `e` representan relaciones (aristas) y las columnas que empiezan por `v` representan nodos (vértices). Solo nos interesan los nodos, así que filtremos las columnas que empiezan por `e` del DataFrame resultante:

```
columnas = [columna para columna en resultado.columnas si no columna.startswith("e")]
resultado.select(columnas).show()
```

Si ejecutamos el código en `pyspark` veremos este resultado:

de			a
[Den Haag, 52.078...	[Hoek van Holland...	[Felixstowe, 51.9...	[Ipswich, 52.0591...

Como era de esperar, el algoritmo `bfs` devuelve Ipswich. Recuerde que esta función se cumple al encontrar la primera coincidencia y, como puede observarse en la [Figura 4-3](#), Ipswich se evalúa antes que Colchester.

Búsqueda en profundidad

La búsqueda en profundidad (DFS) es el otro algoritmo fundamental para recorrer grafos. Parte de un nodo seleccionado, elige uno de sus vecinos y recorre esa ruta hasta donde puede antes de retroceder.

El DFS fue inventado originalmente por el matemático francés Charles Pierre Trémaux como una estrategia para resolver laberintos. Proporciona una herramienta útil para simular posibles rutas para el modelado de escenarios.

La **Figura 4-4** muestra el orden en el que visitaríamos los nodos de nuestro gráfico de transporte si estuviéramos realizando un DFS que comenzara desde Den Haag.

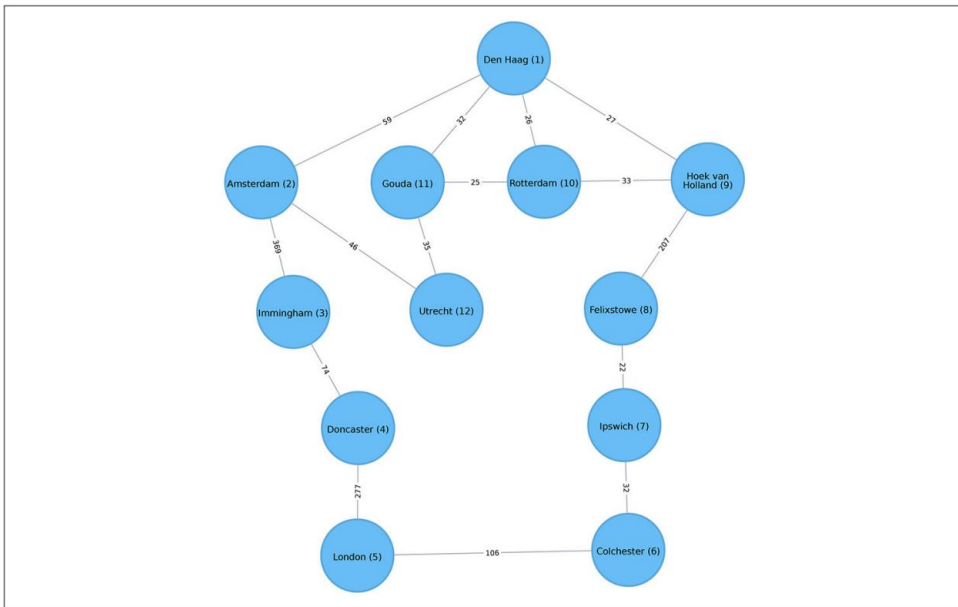


Figura 4-4. Búsqueda en profundidad desde La Haya. Los números de nodo indican el orden de recorrido.

Observe la diferencia en el orden de los nodos en comparación con el BFS. Para este DFS, comenzamos recorriendo de La Haya a Ámsterdam y, a partir de ahí, podemos llegar a todos los demás nodos del grafo sin necesidad de retroceder.

Podemos ver cómo los algoritmos de búsqueda sientan las bases para moverse a través de gráficos. Ahora veamos los algoritmos de búsqueda de rutas que encuentran la ruta más económica en términos de número de saltos o peso. Los pesos pueden ser cualquier medida, como tiempo, distancia, capacidad o costo.

Dos caminos/ciclos especiales.

Hay dos caminos especiales en el análisis de grafos que vale la pena mencionar. Primero, un camino euleriano es aquel en el que cada relación se visita exactamente una vez. Segundo, un camino hamiltoniano es aquel en el que cada nodo se visita exactamente una vez. Un camino puede ser tanto euleriano como hamiltoniano, y si comienza y termina en el mismo nodo, se considera un ciclo o recorrido. La Figura 4-5 muestra una comparación visual .

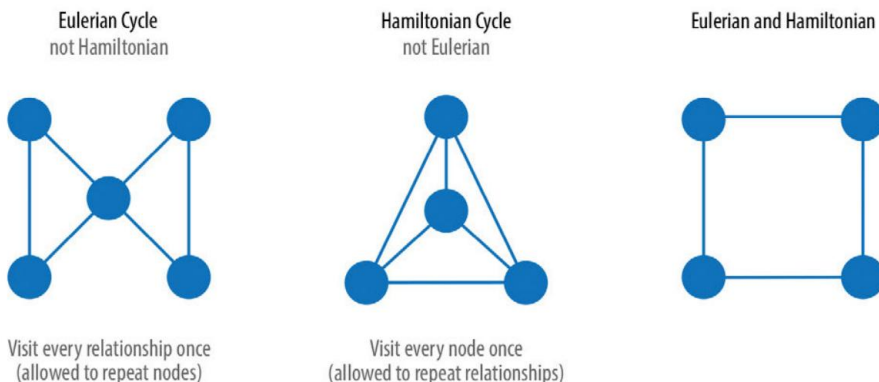


Figura 4-5. Los ciclos eulerianos y hamiltonianos tienen un significado histórico especial.

El problema de los puentes de Königsberg del [Capítulo 1](#) buscaba un ciclo euleriano. Es fácil ver cómo esto se aplica a escenarios de enrutamiento como la dirección de quitanieves y la entrega de correo. Sin embargo, otros algoritmos también utilizan rutas eulerianas al procesar datos en estructuras de árbol y su estudio matemático es más sencillo que el de otros ciclos.

El ciclo hamiltoniano se conoce mejor por su relación con el problema del viajante (PVI), que plantea la pregunta: "¿Cuál es la ruta más corta posible para que un viajante visite cada una de sus ciudades asignadas y regrese a la ciudad de origen?". Aunque aparentemente similar a un recorrido euleriano, el PVI requiere un mayor esfuerzo computacional con alternativas de aproximación. Se utiliza en una amplia variedad de problemas de planificación, logística y optimización.

Camino más corto

El algoritmo de la Ruta Más Corta calcula la ruta más corta (ponderada) entre un par de nodos. Resulta útil para interacciones de usuario y flujos de trabajo dinámicos, ya que funciona en tiempo real.

La búsqueda de rutas se remonta al siglo XIX y se considera un problema clásico de grafos. Cobró relevancia a principios de la década de 1950 en el contexto del enrutamiento alternativo; es decir, encontrar la segunda ruta más corta si la más corta está bloqueada. En 1956, Edsger Dijkstra creó el más conocido de estos algoritmos.

El algoritmo de la Ruta Más Corta de Dijkstra funciona encontrando primero la relación de menor ponderación desde el nodo inicial hasta los nodos directamente conectados. Registra dichas ponderaciones y se desplaza al nodo más cercano. A continuación, realiza el mismo cálculo, pero ahora como un total acumulado desde el nodo inicial. El algoritmo continúa haciendo esto, evaluando una oleada de ponderaciones acumuladas y eligiendo siempre la ruta acumulada de menor ponderación para avanzar hasta llegar al nodo de destino.



Notarás en el análisis de gráficos el uso de los términos peso, costo, distancia y salto al describir relaciones y rutas.

El "peso" es el valor numérico de una propiedad específica de una relación. El "costo" se usa de forma similar, pero lo veremos con más frecuencia al considerar el peso total de una ruta.

"Distancia" se usa a menudo en un algoritmo como el nombre de la propiedad de relación que indica el coste de atravesar un par de nodos. No es necesario que sea una medida física real de distancia. "Salto" se usa comúnmente para expresar el número de relaciones entre dos nodos. Es posible que vea algunos de estos términos combinados, como en "Está a cinco saltos de distancia de Londres" o "Ese es el coste más bajo para la distancia".

¿Cuándo debo utilizar la ruta más corta?

Utilice la Ruta más corta para encontrar rutas óptimas entre un par de nodos, basándose en el número de saltos o en cualquier valor de relación ponderada. Por ejemplo, puede proporcionar respuestas en tiempo real sobre los grados de separación, la distancia más corta entre puntos o la ruta más económica. También puede usar este algoritmo para explorar las conexiones entre nodos específicos.

Algunos ejemplos de casos de uso incluyen:

- Encontrar direcciones entre ubicaciones. Las herramientas de mapas web como Google Maps utilizan el algoritmo de la ruta más corta, o una variante similar, para proporcionar indicaciones de conducción.
- Encontrar los grados de separación entre personas en redes sociales. Por ejemplo, al ver el perfil de alguien en LinkedIn, se indicará cuántas personas lo separan en el gráfico, además de enumerar sus conexiones mutuas.
- Encontrar el número de grados de separación entre un actor y Kevin Bacon según las películas en las que han aparecido (el Número de Bacon). Un ejemplo de esto se puede ver en el [sitio web del Oráculo de Bacon](#). El Proyecto del Número de Erdős proporciona

un análisis gráfico similar basado en la colaboración con Paul Erdős, uno de los matemáticos más prolíficos del siglo XX.



El algoritmo de Dijkstra no admite ponderaciones negativas. El algoritmo asume que añadir una relación a una ruta nunca puede acortarla, una invariante que se violaría con ponderaciones negativas.

Ruta más corta con Neo4j. La

biblioteca de algoritmos de grafos de Neo4j incluye un procedimiento integrado que permite calcular rutas más cortas ponderadas y no ponderadas. Primero, aprendamos a calcular rutas más cortas ponderadas.



Todos los algoritmos de la Ruta Más Corta de Neo4j asumen que el grafo subyacente no tiene dirección. Puede anular esto pasando el parámetro dirección: "SALIENTE" o dirección: "ENTRANTE".

Para que el algoritmo de la Ruta Más Corta de Neo4j ignore los pesos, debemos pasar null como tercer parámetro al procedimiento, lo que indica que no queremos considerar la propiedad de peso al ejecutar el algoritmo. El algoritmo asumirá entonces un peso predeterminado de 1.0 para cada relación:

```
PARTIDO (fuente:Lugar {id: "Ámsterdam"}),
    (destino:Lugar {id: "Londres"})
LLAMA a algo.shortestPath.stream(origen, destino, nulo)
YIELD nodeid, costo
RETURN algoritmo.getNodeById(nodeid).id COMO lugar, costo
```

Esta consulta devuelve el siguiente resultado:

lugar	costo
Amsterdam	0.0
Immingham	1.0
Doncaster	2.0
Londres	3.0

Aquí, el costo es el total acumulado de las relaciones (o saltos). Esta es la misma ruta que vemos al usar la búsqueda en amplitud en Spark.

Incluso podríamos calcular la distancia total que se necesita para seguir esta ruta con un poco de posprocesamiento de Cypher. El siguiente procedimiento calcula la ruta más corta sin ponderar y luego calcula su costo real:

```
PARTIDO (fuente:Lugar {id: "Ámsterdam"}),
        (destino:Lugar {id: "Londres"})
LLAMA a algo.shortestPath.stream(origen, destino, nulo)
YIELD nodeId, costo

CON collect(algo.getNodeById(nodeId)) COMO ruta DESENGROLLAR
rango(0, tamaño(ruta)-1) COMO índice CON ruta[índice]
COMO actual, ruta[índice+1] COMO siguiente CON actual, siguiente,
[(actual)-[r:EROAD]-(siguiente) | r.distancia][0] COMO distancia

CON recopilar({actual: actual, siguiente: siguiente, distancia: distancia}) COMO paradas DESENGROLLAR
rango(0, tamaño(paradas)-1) COMO índice CON
paradas[índice] COMO ubicación, paradas, índice RETORNO
ubicación.actual.id COMO lugar, reducir(acc=0.0,
    distancia en [parada
        en paradas[0..índice] | parada.distancia] | acc + distancia) COMO costo
```

Si el código anterior te parece un poco complejo, ten en cuenta que la parte complicada es encontrar la manera de adaptar los datos para incluir el coste de todo el recorrido. Es útil tener esto en cuenta cuando necesitamos el coste acumulado del trayecto.

La consulta devuelve el siguiente resultado:

lugar	costo
Ámsterdam	0.0
Immingham	369.0
Doncaster	443.0
Londres	720.0

La Figura 4-6 muestra la ruta más corta sin ponderar de Ámsterdam a Londres, que pasa por el menor número de ciudades. Tiene un coste total de 720 km.

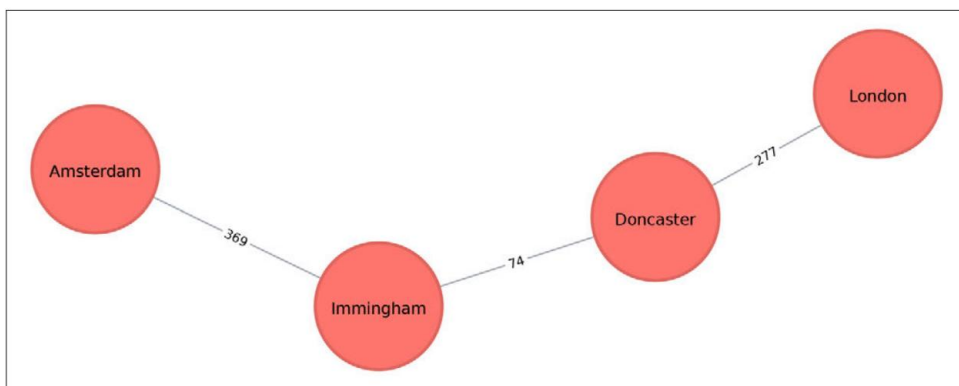


Figura 4-6. La ruta más corta no ponderada entre Ámsterdam y Londres

Elegir una ruta con el menor número de nodos visitados puede ser muy útil en situaciones como los sistemas de metro, donde es muy deseable tener menos paradas. Sin embargo, en un escenario de conducción, probablemente nos interese más el coste total utilizando la ruta ponderada más corta.

Ruta más corta (ponderada) con Neo4j

Podemos ejecutar el algoritmo de ruta más corta ponderada para encontrar la ruta más corta entre Ámsterdam y Londres de la siguiente manera:

```

PARTIDO (fuente:Lugar {id: "Ámsterdam"}),
  (destino:Lugar {id: "Londres"})
LLAMA a algo.shortestPath.stream(origen, destino, "distancia")
YIELD nodeId, costo
RETURN algoritmo.getNodeById(nodeId).id COMO lugar, costo
  
```

Los parámetros que se pasan a este algoritmo son:

fuente

El nodo donde comienza nuestra búsqueda del camino más corto

destino

El nodo donde termina nuestro camino más corto

distancia

El nombre de la propiedad de relación que indica el costo de atravesar un par de nodos

El coste es el número de kilómetros entre dos ubicaciones. La consulta devuelve el siguiente resultado:

lugar	costo
Amsterdam	0.0

lugar	costo
La Haya	59.0
Hoek van Holland	86.0
Felixstowe	293.0
Ipswich	315.0
Colchester	347.0
Londres	453.0

La ruta más rápida nos lleva por La Haya, Hoek van Holland, Felixstowe, Ipswich y Colchester. El coste mostrado es el total acumulado a medida que avanzamos por las ciudades. Primero vamos de Ámsterdam a La Haya, con un coste de 59. Luego, de La Haya a Hoek van Holland, con un coste acumulado de 86, y así sucesivamente. Finalmente, llegamos a Londres, desde Colchester, con un coste total de 453 km.

Recuerde que la ruta más corta no ponderada tuvo un costo total de 720 km, por lo que pudimos ahorrar 267 km al tener en cuenta los pesos al calcular la ruta más corta.

Ruta más corta (ponderada) con Apache Spark. En la sección

"**Búsqueda en amplitud con Apache Spark**", aprendimos a encontrar la ruta más corta entre dos nodos. Esta ruta se basaba en saltos y, por lo tanto, no es la misma que la ruta ponderada más corta, que nos indicaría la distancia total más corta entre ciudades.

Si queremos encontrar la ruta ponderada más corta (en este caso, la distancia), necesitamos usar la propiedad `cost`, que se utiliza para varios tipos de ponderación. Esta opción no está disponible de fábrica con `GraphFrames`, por lo que necesitamos escribir nuestra propia versión de la Ruta Más Corta Ponderada usando su **framework de mensajes agregados**. La mayoría de nuestros ejemplos de algoritmos para Spark utilizan el proceso más sencillo de invocar algoritmos de la biblioteca, pero tenemos la opción de escribir nuestras propias funciones. Puede encontrar más información sobre los mensajes agregados en la sección "**Paso de mensajes mediante mensajes agregados**" de la guía del usuario de `GraphFrames`.



Cuando estén disponibles, recomendamos aprovechar las bibliotecas preexistentes y probadas. Escribir nuestras propias funciones, especialmente para algoritmos más complejos, requiere una comprensión más profunda de nuestros datos y cálculos.

El siguiente ejemplo debe considerarse una implementación de referencia y deberá optimizarse antes de ejecutarse en un conjunto de datos más grande. Quienes no estén interesados en desarrollar sus propias funciones pueden omitir este ejemplo.

Antes de crear nuestra función, importaremos algunas bibliotecas que usaremos:

```
desde graphframes.lib importe AggregateMessages como AM
desde pyspark.sql importe funciones como F
```

El módulo Aggregate_Messages es parte de la biblioteca GraphFrames y contiene algunas funciones auxiliares útiles.

Ahora, escribamos nuestra función. Primero, creamos una función definida por el usuario que usaremos para construir las rutas entre nuestro origen y destino:

```
add_path_udf = F.udf(ruta lambda , id: ruta + [id], ArrayType(StringType()))
```

Y ahora la función principal, que calcula la ruta más corta partiendo de un origen y regresa tan pronto como se ha visitado el destino:

```
def shortest_path(g, origen, destino, nombre_columna="costo"):
    si g.vertices.filter(g.vertices.id == destino).count() == 0:
        devolver (spark.createDataFrame(sc.emptyRDD(),
            g.vertices.schema).withColumn("ruta", F.array()))

    vértices = (g.vertices.withColumn("visitado",
        F.lit(False)).withColumn("distancia", F.when(g.vertices["id"] == origen,
            0).otherwise(float("inf"))).withColumn("ruta",
        F.array())) cached_vertices = AM.getCachedDataFrame(vertices)
    g2 = GraphFrame(cached_vertices, g.edges)

    mientras g2.vertices.filter('visitado == False').first():
        id_del_nodo_actual = g2.vertices.filter('visitado == Falso').sort
            ("distancia").first().id

        msg_distance = AM.edge[nombre_de_columna] + AM.src['distancia']
        msg_path = add_path_udf(AM.src["ruta"], AM.src["id"]) msg_for_dst
        = F.when(AM.src["id"] == id_de_nodo_actual,
            F.struct(distancia=msg, ruta=msg))
        nuevas_distancias = g2.aggregateMessages(F.min(AM.msg).alias("aggMess"),
            sendToDst=msg_para_dst)

        nueva_columna_visitada =
            F.cuando( g2.vértices.visitados | (g2.vértices.id == id_del_nodo_actual),
                Verdadero).de lo contrario(Falso)
        nueva_distancia_col = F.cuando(nuevas_distancias["aggMess"].isNull() &
            (nuevas_distancias.aggMess["col1"]
                < g2.vértices.distancia),
            nuevas_distancias.aggMess["col1"]).de
            lo contrario(g2.vértices.distancia)
        nueva_ruta_col = F.cuando(nuevas_distancias["aggMess"].isNotNull() &
            (nuevas_distancias.aggMess["col1"]
                < g2.vértices.distancia),
            nuevas_distancias.aggMess["col2"]).cast("matriz<cadena>")).de lo contrario(g2.vértices.ruta)

        nuevos_vértices = (g2.vértices.join(nuevas_distancias, on="id",
```



```

        how="left_outer") .drop(nuevas_distancias["id"]) .withColumn("visitado",
        nueva_columna_visitada) .withColumn("nuevaDistancia",
        nueva_columna_distancia) .withColumn("nuevaRuta",
        nueva_columna_ruta) .drop("aggMess", "distancia",
        "ruta") .withColumnRenamed('nuevaDistancia', 'distancia') .withColumnRenamed('nuevaRuta', 'ruta'))

vértices_nuevos_en_caché = AM.getCachedDataFrame(nuevos_vértices) g2 =
GraphFrame(vértices_nuevos_en_caché, g2.edges) si
g2.vertices.filter(g2.vertices.id == destino).first().visited:
    return (g2.vertices.filter(g2.vertices.id == destino) .withColumn("nuevaRuta",
        add_path_udf("ruta", "id"))) .drop("visitado",

        "ruta") .withColumnRenamed("nuevaRuta", "ruta"))

return (spark.createDataFrame(sc.emptyRDD(), g.vertices.schema) .withColumn("ruta",
        F.array()))

```



Si almacenamos referencias a cualquier DataFrames en nuestras funciones, debemos almacenarlas en caché mediante la función `AM.getCachedDataFrame` ; de lo contrario, se producirá una fuga de memoria durante la ejecución. En la función `shortest_path` , la usamos para almacenar en caché los vértices y los nuevos DataFrames.

Si quisiéramos encontrar el camino más corto entre Amsterdam y Colchester podríamos llamar a esa función así:

```

resultado = ruta_más_corta(g, "Ámsterdam", "Colchester", "costo") resultado.select("id",
"distancia", "ruta").show(truncar=False)

```

Lo que devolvería el siguiente resultado:

camino de distancia	
Colchester 347.0	[Ámsterdam, La Haya, Hoek van Holland, Felixstowe, Ipswich, Colchester]

La distancia total del camino más corto entre Ámsterdam y Colchester es de 347 km y pasa por La Haya, Hoek van Holland, Felixstowe e Ipswich. En cambio, el camino más corto en términos de número de relaciones entre las ubicaciones, calculado con el [algoritmo de búsqueda en amplitud](#) (véase la [Figura 4-4](#)), nos llevaría por Immingham, Doncaster y Londres.

Variación del camino más corto: A*

El algoritmo A* de la ruta más corta mejora el de Dijkstra al encontrar las rutas más cortas con mayor rapidez. Esto se logra al permitir la inclusión de información adicional que el algoritmo puede utilizar, como parte de una función heurística, al determinar qué rutas explorar.

próximo.

El algoritmo fue inventado por Peter Hart, Nils Nilsson y Bertram Raphael y descrito en su artículo de 1968 **“Una base formal para la determinación heurística de rutas de costo mínimo”**.

El algoritmo A* opera determinando cuál de sus rutas parciales expandir en cada iteración de su bucle principal. Esto se basa en una estimación del coste (heurístico) restante para alcanzar el nodo objetivo.



Sea reflexivo en la heurística empleada para estimar los costos de ruta. Subestimar los costos de ruta puede incluir innecesariamente algunas rutas que podrían haberse eliminado, pero los resultados seguirán siendo precisos. Sin embargo, si la heurística sobreestima los costos de ruta, puede omitir rutas más cortas reales (estimadas incorrectamente como más largas) que deberían haberse evaluado, lo que puede generar resultados inexactos.

A* selecciona la ruta que minimiza la siguiente función:

$$f(n) = g(n) + h(n)$$

dónde:

- $g(n)$ es el costo de la ruta desde el punto de partida hasta el nodo n . • $h(n)$ es el costo estimado de la ruta desde el nodo n hasta el nodo de destino, calculado mediante una heurística.



En la implementación de Neo4j, la distancia geoespacial se utiliza como heurística. En nuestro conjunto de datos de transporte de ejemplo, utilizamos la latitud y la longitud de cada ubicación como parte de la función heurística.

A* con Neo4j

La siguiente consulta ejecuta el algoritmo A* para encontrar la ruta más corta entre Den La Haya y Londres:

```
PARTIDO (fuente: Lugar {id: "Den Haag"}),  
  (destino:Lugar {id: "Londres"})  
LLAMA a algo.shortestPath.astar.stream(origen,  
  destino, "distancia", "latitud", "longitud")  
YIELD nodeId, costo  
RETURN algoritmo.getNodeById(nodeId).id COMO lugar, costo
```

Los parámetros que se pasan a este algoritmo son:

fuentes

El nodo donde comienza nuestra búsqueda de ruta más corta.

destino

El nodo donde termina nuestra búsqueda del camino más corto.

distancia El

nombre de la propiedad de relación que indica el costo de atravesar un par de nodos. El costo es el número de kilómetros entre dos ubicaciones.

latitud El

nombre de la propiedad del nodo utilizada para representar la latitud de cada nodo como parte del cálculo heurístico geoespacial.

longitud El

nombre de la propiedad del nodo utilizada para representar la longitud de cada nodo como parte del cálculo heurístico geoespacial.

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

lugar	costo
La Haya	0.0
Hoek van Holland	27.0
Felixstowe	234.0
Ipswich	256.0
Colchester	288.0
Londres	394.0

Obtendríamos el mismo resultado utilizando el algoritmo de ruta más corta, pero en conjuntos de datos más complejos el algoritmo A* será más rápido ya que evalúa menos rutas.

Variación del camino más corto: los k caminos más cortos de Yen

El algoritmo k-Shortest Paths de Yen es similar al algoritmo Shortest Path, pero en lugar de encontrar solo el camino más corto entre dos pares de nodos, también calcula el segundo camino más corto, el tercer camino más corto y así sucesivamente hasta k-1 desviaciones de los caminos más cortos.

Jin Y. Yen inventó el algoritmo en 1971 y lo describió en "Encontrar las K rutas sin bucles más cortas en una red". Este algoritmo es útil para obtener rutas alternativas cuando encontrar la ruta más corta no es nuestro único objetivo. Puede ser especialmente útil cuando necesitamos más de un plan B.

Yen's con Neo4j

La siguiente consulta ejecuta el algoritmo de Yen para encontrar las rutas más cortas entre Gouda y Felixstowe:

```
PARTIDO (inicio:Lugar {id:"Gouda"},
        (fin:Lugar {id:"Felixstowe"}))
LLAMA a algo.kShortestPaths.stream(inicio, fin, 5, "distancia")
Índice YIELD, nodeIds, ruta, costos
Índice de retorno ,
    [nodo en algo.getNodesById(nodeIds[1..-1]) | nodo.id] AS vía,
    reduce(acc=0.0, costo en costos | acc + costo) AS costoTotal
```

Los parámetros que se pasan a este algoritmo son:

- comenzar
 - El nodo donde comienza nuestra búsqueda de ruta más corta.
- fin
 - El nodo donde termina nuestra búsqueda del camino más corto.
- 5
 - El número máximo de rutas más cortas a encontrar.
- distancia
 - El nombre de la propiedad de relación que indica el costo de atravesar entre un par de nodos. El coste es el número de kilómetros entre dos ubicaciones.

Después de recuperar las rutas más cortas, buscamos el nodo asociado para cada ID de nodo, y luego filtramos los nodos de inicio y final de la colección.

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

índice vía		costo total
0	[Róterdam, Hoek van Holland]	265.0
1	[La Haya, Hoek van Holland]	266.0
2	[Rotterdam, La Haya, Hoek van Holland]	285.0
3	[La Haya, Róterdam, Hoek van Holland]	298.0
4	[Utrecht, Ámsterdam, La Haya, Hoek van Holland]	374.0

La figura 4-7 muestra la ruta más corta entre Gouda y Felixstowe.

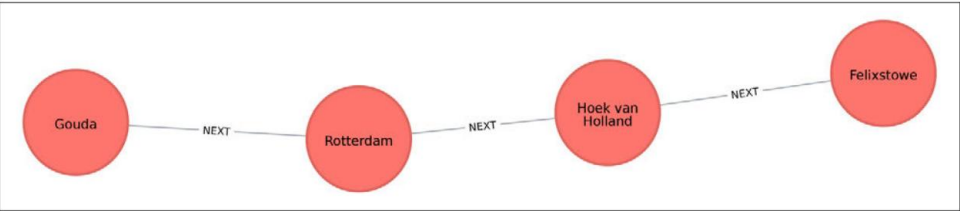


Figura 4-7. El camino más corto entre Gouda y Felixstowe

La ruta más corta de **la Figura 4-7** es interesante en comparación con los resultados ordenados por coste total. Esto ilustra que, a veces, conviene considerar varias rutas más cortas u otros parámetros. En este ejemplo, la segunda ruta más corta es solo 1 km más larga que la más corta. Si preferimos el paisaje, podríamos elegir la ruta ligeramente más larga.

El camino más corto de todos los pares

El algoritmo de la Ruta Más Corta de Todos los Pares (APSP) calcula la ruta más corta (ponderada) entre todos los pares de nodos. Es más eficiente que ejecutar el algoritmo de la Ruta Más Corta de una Sola Fuente para cada par de nodos del grafo.

APSP optimiza las operaciones registrando las distancias calculadas hasta el momento y ejecutándose en nodos en paralelo. Estas distancias conocidas pueden reutilizarse al calcular la ruta más corta a un nodo no detectado. Puede seguir el ejemplo de la siguiente sección para comprender mejor el funcionamiento del algoritmo.



Es posible que algunos pares de nodos no sean accesibles entre sí, lo que significa que no existe una ruta más corta entre ellos. El algoritmo no devuelve las distancias de estos pares de nodos.

Una mirada más de cerca al camino más corto de todos los pares

El cálculo de APSP se comprende mejor si se sigue una secuencia de operaciones. El diagrama de **la Figura 4-8** muestra los pasos para el nodo A.

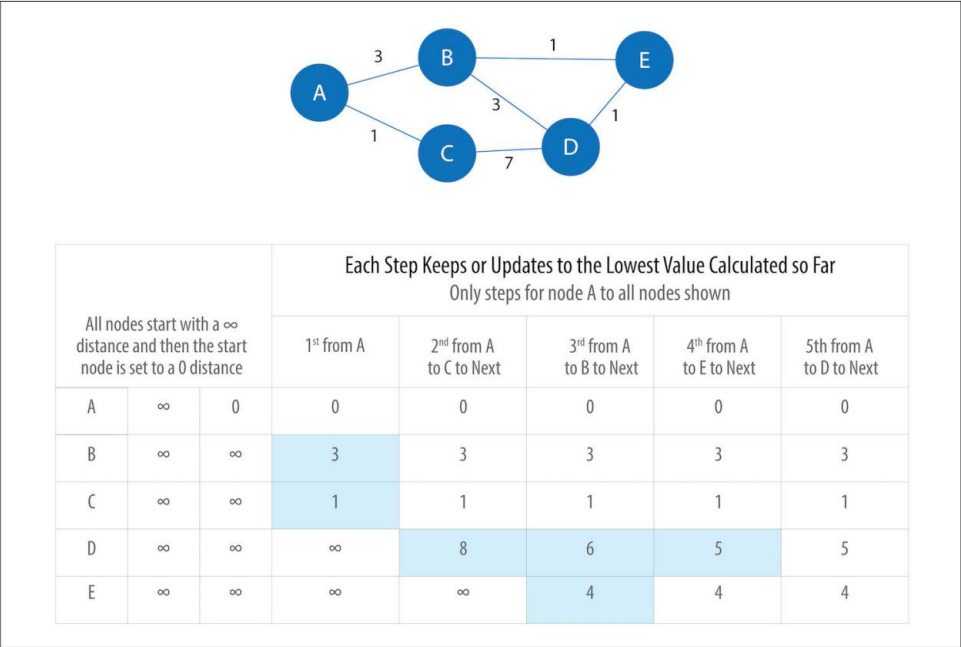


Figura 4-8. Pasos para calcular la ruta más corta desde el nodo A hasta todos los demás nodos, con las actualizaciones sombreadas.

Inicialmente, el algoritmo asume una distancia infinita a todos los nodos. Al seleccionar un nodo de inicio, la distancia a dicho nodo se establece en 0. El cálculo se realiza de la siguiente manera:

- Desde el nodo inicial A, evaluamos el coste de movernos a los nodos alcanzables y actualizamos esos valores. Buscando el valor más bajo, podemos elegir entre B (costo de 3) o C (costo de 1). Se selecciona C para la siguiente fase de la travesía.
- Ahora, desde el nodo C, el algoritmo actualiza las distancias acumuladas desde A hasta los nodos a los que se puede llegar directamente desde C. Los valores solo se actualizan cuando se encuentra un coste menor:

$A=0, B=3, C=1, D=8, E=\infty$

- Luego, se selecciona B como el siguiente nodo más cercano que aún no se ha visitado. Tiene relación con los nodos A, D y E. El algoritmo calcula la distancia a estos nodos sumando la distancia de A a B con la distancia de B a cada uno de ellos. Tenga en cuenta que el costo más bajo es desde el nodo inicial A hasta el nodo actual.

Siempre se conserva como un costo hundido. El resultado del cálculo de la distancia (d):

$d(A,A) = d(A,B) + d(B,A) = 3 + 3 = 6$
 $d(A,D) = d(A,B) + d(B,D) = 3 + 3 = 6$
 $d(A,E) = d(A,B) + d(B,E) = 3 + 1 = 4$

- En este paso, la distancia del nodo A al B y de regreso al A, que se muestra como $d(A,A) = 6$, es mayor que la distancia más corta ya calculada (0), por lo que su valor no se actualiza.
 - Las distancias de los nodos D (6) y E (4) son menores que las distancias calculadas previamente, por lo que sus valores se actualizan.
4. A continuación, se selecciona E. Solo el total acumulado para alcanzar D (5) es ahora menor y, por lo tanto, es el único actualizado.
 5. Cuando finalmente se evalúa D, no hay nuevos pesos de ruta mínimos; no se actualiza nada y el algoritmo finaliza.



Aunque el algoritmo de la Ruta Más Corta de Todos los Pares está optimizado para ejecutar cálculos en paralelo para cada nodo, esto puede resultar en un grafo muy grande. Considere usar un subgrafo si solo necesita evaluar rutas entre una subcategoría de nodos.

¿Cuándo debo utilizar la ruta más corta de todos los pares?

El algoritmo de la Ruta Más Corta de Todos los Pares se utiliza comúnmente para comprender el enrutamiento alternativo cuando la ruta más corta está bloqueada o deja de ser óptima. Por ejemplo, este algoritmo se utiliza en la planificación lógica de rutas para garantizar las mejores rutas múltiples para el enrutamiento por diversidad. Utilice la ruta más corta de todos los pares cuando necesite considerar todas las rutas posibles entre todos o la mayoría de sus nodos.

Algunos ejemplos de casos de uso incluyen:

Optimizar la ubicación de las instalaciones urbanas y la distribución de mercancías. Un ejemplo de ello es determinar la carga de tráfico prevista en los diferentes segmentos de una red de transporte. Para más información, véase el libro de RC Larson y AR Odoni, *Urban Operations Research* (Prentice-Hall).

- Encontrar una red con el máximo ancho de banda y la mínima latencia como parte de un algoritmo de diseño de centros de datos. Encontrará más detalles sobre este enfoque en el artículo [«REWIRE: Un marco basado en la optimización para el diseño de redes de centros de datos»](#), de AR Curtis et al.

Ruta más corta de todos los pares con Apache Spark

La función `shortestPaths` de Spark está diseñada para encontrar las rutas más cortas desde todos los nodos hasta un conjunto de nodos llamados puntos de referencia. Si quisiéramos encontrar la ruta más corta desde cada ubicación hasta Colchester, Immingham y Hoek van Holland, escribiríamos la siguiente consulta:

```
resultado = g.shortestPaths(["Colchester", "Immingham", "Hoek van Holland"])
resultado.sort(["id"]).select("id", "distancias").show(truncate=False)
```

Si ejecutamos ese código en pyspark veremos este resultado:

---	distancias
Amsterdam	[Immingham → 1, Hoek van Holland → 2, Colchester → 4]
Colchester	[Colchester → 0, Hoek van Holland → 3, Immingham → 3]
La Haya	[Hoek van Holland → 1, Immingham → 2, Colchester → 4]
Doncaster	[Immingham → 1, Colchester → 2, Hoek van Holland → 4]
Felixstowe	[Hoek van Holland → 1, Colchester → 2, Immingham → 4]
Gouda	[Hoek van Holland → 2, Immingham → 3, Colchester → 5]
Hoek van Holland	[Hoek van Holland → 0, Immingham → 3, Colchester → 3]
Immingham	[Immingham → 0, Colchester → 3, Hoek van Holland → 3]
Ipswich	[Colchester → 1, Hoek van Holland → 2, Immingham → 4]
Londres	[Colchester → 1, Immingham → 2, Hoek van Holland → 4]
Róterdam	[Hoek van Holland → 1, Immingham → 3, Colchester → 4]
Utrecht	[Immingham → 2, Hoek van Holland → 3, Colchester → 5]

El número que aparece junto a cada ubicación en la columna de distancias es el número de relaciones (caminos) entre ciudades que necesitamos recorrer para llegar allí desde la fuente. En nuestro ejemplo, Colchester es una de nuestras ciudades de destino y puedes verlo tiene 0 nodos que atravesar para llegar a sí mismo, pero 3 saltos que hacer desde Immingham y Hoek van Holland. Si estuviéramos planeando un viaje, podríamos usar esta información para ayudarnos a maximizar... minimizar nuestro tiempo en nuestros destinos elegidos.

Ruta más corta de todos los pares con Neo4j

Neo4j tiene una implementación paralela del algoritmo All Pairs Shortest Path, que devuelve la distancia entre cada par de nodos.

El primer parámetro de este procedimiento es la propiedad que se utilizará para calcular el valor más corto. Ruta ponderada. Si lo establecemos como nulo , el algoritmo calculará la ruta no ponderada. caminos más cortos entre todos los pares de nodos.

La siguiente consulta hace esto:

```
LLAMA a algo.allShortestPaths.stream(null)
YIELD sourceNodeId, targetNodeId, distancia
DONDE sourceNodeId < targetNodeId
DEVUELVE algo.getNodeById(sourceNodeId).id COMO fuente,
         algo.getNodeById(targetNodeId).id COMO objetivo,
         distancia
ORDENAR POR distancia DESC
LÍMITE 10
```


Este algoritmo devuelve la ruta más corta entre cada par de nodos dos veces: una con cada uno de los nodos como nodo de origen. Esto sería útil si estuviera evaluando... graficando un grafo dirigido de calles unidireccionales. Sin embargo, no necesitamos ver cada camino dos veces, por lo que filtramos los resultados para mantener solo uno de ellos usando `sourceNodeId < Predicado targetNodeId` .

La consulta devuelve los siguientes resultados:

fuelle	objetivo	distancia
Colchester	Utrecht	5.0
Londres	Róterdam	5.0
Londres	Gouda	5.0
Ipswich	Utrech	5.0
Colchester	Gouda	5.0
Colchester	Den Haag	4.0
Londres	Utrech	4.0
Londres	La Haya	4.0
Colchester	Ámsterdam	4.0
Ipswich	Gouda	4.0

Esta salida muestra los 10 pares de ubicaciones que tienen más relaciones entre sí. ellos porque pedimos los resultados en orden descendente (DESC).

Si queremos calcular las rutas ponderadas más cortas, en lugar de pasar nulo como Primer parámetro, podemos pasar el nombre de la propiedad que contiene el costo a utilizar en El cálculo de la ruta más corta. Esta propiedad se evaluará para determinar la ruta ponderada más corta entre cada par de nodos.

La siguiente consulta hace esto:

```
LLAMA a algo.allShortestPaths.stream("distancia")
YIELD sourceNodeId, targetNodeId, distancia
DONDE sourceNodeId < targetNodeId
DEVUELVE algo.getNodeById(sourceNodeId).id COMO fuente,
         algo.getNodeById(targetNodeId).id COMO objetivo,
         distancia
ORDENAR POR distancia DESC
LÍMITE 10
```

La consulta devuelve el siguiente resultado:

fuelle	objetivo	distancia
Doncaster	Hoek van Holland	529.0
Róterdam	Doncaster	528.0
Gouda	Doncaster	524.0

fuelle	objetivo	distancia
Felixstowe	Immingham	511.0
La Haya	Doncaster	502.0
Ipswich	Immingham	489.0
Utrecht	Doncaster	489.0
Londres	Utrecht	460.0
Colchester	Immingham	457.0
Immingham	Hoek van Holland	455.0

Ahora estamos viendo los 10 pares de ubicaciones más alejadas entre sí en términos de distancia total entre ellos. Tenga en cuenta que Doncaster aparece con frecuencia junto con Varias ciudades de los Países Bajos. Parece que sería un viaje largo si quisiéramos... Realice un viaje por carretera entre esas áreas.

Ruta más corta de una sola fuente

El algoritmo de ruta más corta de fuente única (SSSP), que adquirió relevancia en Casi al mismo tiempo que el algoritmo de ruta más corta de Dijkstra, actúa como una implementación para ambos problemas.

El algoritmo SSSP calcula la ruta más corta (ponderada) desde un nodo raíz a todos otros nodos en el gráfico, como se muestra en [la Figura 4-9](#).

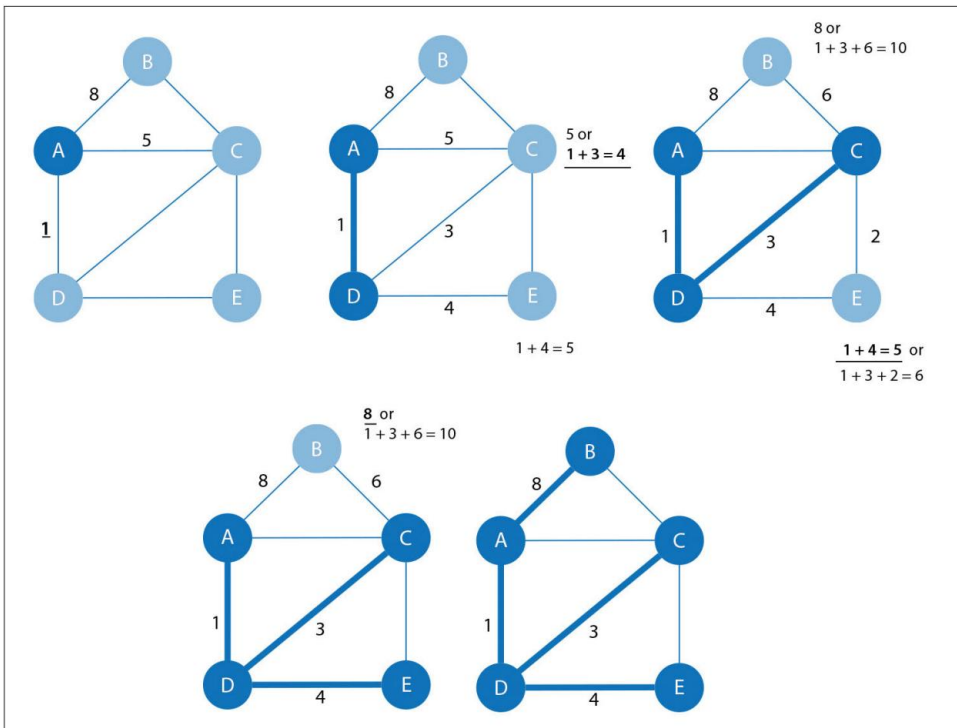


Figura 4-9. Pasos del algoritmo de ruta más corta de una sola fuente

Procede de la siguiente manera:

1. Comienza con un nodo raíz desde el cual se medirán todas las rutas. En la Figura 4-9 Hemos seleccionado el nodo A como raíz.
2. Se selecciona la relación con el menor peso proveniente de ese nodo raíz y se añade al árbol, junto con su nodo conectado. En este caso, $d(A,D)=1$.
3. Se selecciona la siguiente relación con el menor peso acumulado desde nuestro nodo raíz hasta cualquier nodo no visitado y se añade al árbol de la misma manera. Nuestras opciones en la Figura 4-9 son $d(A,B)=8$, $d(A,C)=5$ directamente o 4 mediante ADC, y $d(A,E)=5$. Por lo tanto, se elige la ruta mediante ADC y se añade C a nuestro árbol.
4. El proceso continúa hasta que no haya más nodos para agregar y tengamos nuestra ruta más corta de fuente única.

¿Cuándo debo utilizar la ruta más corta de una sola fuente?

Utilice la ruta más corta de una sola fuente cuando necesite evaluar la ruta óptima desde un punto de inicio fijo hasta todos los demás nodos. Dado que la ruta se elige en función del peso total de la ruta desde la raíz, es útil para encontrar la mejor ruta a cada nodo, pero no necesariamente cuando es necesario visitar todos los nodos en un solo viaje.

Por ejemplo, SSSP es útil para identificar las rutas principales de los servicios de emergencia, donde no se visitan todas las ubicaciones en cada incidente, pero no para encontrar una ruta única para la recolección de basura, donde se necesita visitar cada casa en un solo viaje. (En este último caso, se usaría el algoritmo de Árbol de Expansión Mínimo, que se explica más adelante).

Algunos ejemplos de casos de uso incluyen:

- Detectar cambios en la topología, como fallas de enlace, y **sugerir una nueva estructura de enrutamiento en segundos**
- Usar Dijkstra

como protocolo de enrutamiento IP para su uso en sistemas autónomos como una **red de área local (LAN)**

Ruta más corta de una sola fuente con Apache Spark. Podemos

adaptar la función `shortest_path` que escribimos para **calcular la ruta más corta entre dos ubicaciones** para que, en su lugar, nos devuelva la ruta más corta de una ubicación a todas las demás. Tenga en cuenta que estamos usando de nuevo el framework `addedMessages` de Spark para personalizar nuestra función.

Primero importaremos las mismas bibliotecas que antes:

```
desde graphframes.lib importe AggregateMessages como AM
desde pyspark.sql importe funciones como F
```

Y usaremos la misma función definida por el usuario para construir rutas:

```
add_path_udf = F.udf(ruta lambda , id: ruta + [id], ArrayType(StringType()))
```

Ahora, para la función principal, que calcula la ruta más corta a partir de un origen:

```
def sssp(g, origen, nombre_columna="costo"):
    vértices = g.vertices
    \ .withColumn("visitado", F.lit(False))
    \ .withColumn("distancia",
        F.cuando(g.vertices["id"] == origen, 0).de lo contrario(float("inf"))) \
    .withColumn("ruta", F.array())
    vértices_en_caché = AM.getCachedDataFrame(vértices) g2
    = GraphFrame(vértices_en_caché, g.aristas)

    mientras g2.vertices.filter('visitado == False').first():
        id_del_nodo_actual = g2.vertices.filter( 'visitado ==
            Falso') .sort("distancia").first().id
```

```

msg_distancia = AM.edge[nombre_columna] + AM.src["distancia"] msg_ruta =
add_path_udf(AM.src["ruta"], AM.src["id"]) msg_para_dst = F.when(AM.src["id"]
== id_nodo_actual, F.struct(msg_distancia, msg_ruta))

nuevas_distancias =
    g2.mensajesAgregados( F.min(AM.msg).alias("mensajeAgregado"), enviarAlDst=msg_para_elDst)

nueva_columna_visitada =
    F.cuando( g2.vértices.visitados | (g2.vértices.id == id_nodo_actual), Verdadero).de
    lo contrario(Falso)

nueva_columna_distancia = F.cuando(nuevas_distancias["aggMess"].isNotNull() &
    (nuevas_distancias.aggMess["col1"] <
    g2.vértices.distancia),
    nuevas_distancias.aggMess["col1"]) \ .de lo
    contrario(g2.vértices.distancia)

nueva_ruta_col = F.cuando(nuevas_distancias["aggMess"].isNotNull() &
    (nuevas_distancias.aggMess["col1"] <
    g2.vértices.distancia),

    nuevas_distancias.aggMess["col2"] \ .cast("matriz<cadena>")) \ .de lo contrario(g2.vértices.ruta)

nuevos_vértices = g2.vertices.join(nuevas_distancias, on="id", how="left_outer")
    \ .drop(nuevas_distancias["id"])

    \ .withColumn("visitado",
    nueva_columna_visitada) \ .withColumn("nuevaDistancia",
    nueva_columna_distancia) \ .withColumn("nuevaRuta",
    nueva_columna_ruta) \ .drop("aggMess", "distancia",
    "ruta") \ .withColumnRenamed('nuevaDistancia',
    'distancia') \ .withColumnRenamed('nuevaRuta', 'ruta')
nuevos_vértices_en_cache =
AM.getCachedDataFrame(nuevos_vértices) g2 =
GraphFrame(nuevos_vértices_en_cache, g2.aristas)

devolver g2.vertices \
    .withColumn("nuevaRuta", add_path_udf("ruta", "id")) \ .drop("visitado",
    "ruta")
    \ .withColumnRenamed("nuevaRuta", "ruta")

```

Si queremos encontrar la ruta más corta desde Ámsterdam a todas las demás ubicaciones, podemos llamar a la función de esta manera:

```

via_udf = F.udf( ruta lambda : ruta[1:-1], ArrayType(StringType()))

resultado = sssp(g, "Ámsterdam", "costo")

(resultado .withColumn("via", via_udf("ruta"))) .select("id",
    "distancia",

    "via") .sort("distancia") .show(truncar=Falso))

```

Definimos otra función definida por el usuario para filtrar los nodos de inicio y fin de La ruta resultante. Si ejecutamos ese código, veremos el siguiente resultado:

...	distancia	vía
Ámsterdam	0.0	[]
Utrech	46.0	[]
La Haya	59.0	[]
Gouda	81.0	[Utrech]
Róterdam	85.0	[La Haya]
Hoek van Holland	86.0	[La Haya]
Felixstowe	293.0	[La Haya, Hoek van Holland]
Ipswich	315.0	[La Haya, Hoek van Holland, Felixstowe]
Colchester	347.0	[La Haya, Hoek van Holland, Felixstowe, Ipswich]
Immingham	369.0	[]
Doncaster	443.0	[Immingham]
Londres	453.0	[La Haya, Hoek van Holland, Felixstowe, Ipswich, Colchester]

En estos resultados vemos las distancias físicas en kilómetros desde el nodo raíz, Ámsterdam, a todas las demás ciudades del gráfico, ordenadas por distancia más corta.

Ruta más corta de una sola fuente con Neo4j

Neo4j implementa una variación de SSSP, llamada **algoritmo Delta-Stepping** que divide el algoritmo de Dijkstra en varias fases que pueden ejecutarse en paralelo.

La siguiente consulta ejecuta el algoritmo Delta-Stepping:

```
PARTIDO (n:Lugar {id:"Londres"})
LLAME a algo.shortestPath.deltaStepping.stream(n, "distancia", 1.0)
YIELD nodeld, distancia
DONDE algo.isFinite(distancia)
DEVUELVE algo.getNodeById(nodeld).id COMO destino, distancia
ORDENAR POR distancia
```

La consulta devuelve el siguiente resultado:

destino	distancia
Londres	0.0
Colchester	106.0
Ipswich	138.0
Felixstowe	160.0
Doncaster	277.0
Immingham	351.0

destino	distancia
Hoek van Holland	367.0
La Haya	394.0
Róterdam	400.0
Gouda	425.0
Ámsterdam	453.0
Utrech	460.0

En estos resultados vemos las distancias físicas en kilómetros desde el nodo raíz, Londres, a todas las demás ciudades del gráfico, ordenadas por la distancia más corta.

Árbol de expansión mínimo

El algoritmo de árbol de expansión de peso mínimo parte de un nodo dado y encuentra todos sus nodos alcanzables y el conjunto de relaciones que los conectan con el peso mínimo posible. Atraviesa el siguiente nodo no visitado con el menor peso de todos los nodos visitados, evitando ciclos.

El primer algoritmo de árbol de expansión de peso mínimo conocido fue desarrollado por el científico checo Otakar Borůvka en 1926. El algoritmo de Prim, inventado en 1957, es el más simple y conocido.

El algoritmo de Prim es similar al algoritmo de la ruta más corta de Dijkstra, pero en lugar de minimizar la longitud total de una ruta que termina en cada relación, minimiza la longitud de cada relación individualmente. A diferencia del algoritmo de Dijkstra, tolera relaciones de ponderación negativa.

El algoritmo de árbol de expansión mínimo funciona como se muestra en la **Figura 4-10**.

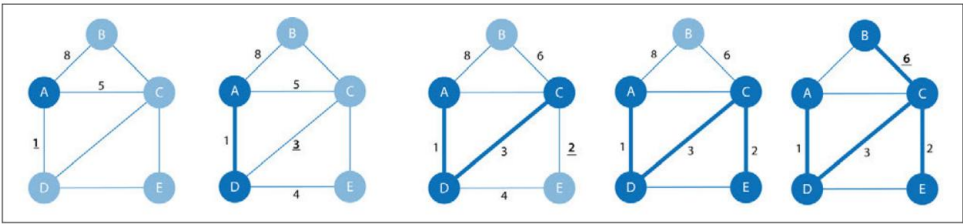


Figura 4-10. Pasos del algoritmo de árbol de expansión mínimo

Los pasos son los siguientes:

1. Comienza con un árbol que contiene solo un nodo. En la **Figura 4-10**, comenzamos con el nodo A.
2. Se selecciona la relación con el menor peso proveniente de ese nodo y se añade al árbol (junto con su nodo conectado). En este caso, AD.

3. Este proceso se repite, eligiendo siempre la relación de peso mínimo que une cualquier nodo que no esté ya en el árbol. a. Si compara

nuestro ejemplo con el de SSSP de la [Figura 4-9](#), observará que en el cuarto gráfico las rutas son diferentes. Esto se debe a que SSSP evalúa la ruta más corta basándose en los totales acumulados desde la raíz, mientras que el Árbol de Expansión Mínimo solo considera el coste del siguiente paso.

4. Cuando no hay más nodos para agregar, el árbol es un árbol de expansión mínimo.

También hay variantes de este algoritmo que encuentran el árbol de expansión de peso máximo (árbol de mayor costo) y el árbol de expansión k (tamaño de árbol limitado).

¿Cuándo debo utilizar el árbol de expansión mínimo?

Utilice el Árbol de Expansión Mínimo cuando necesite la mejor ruta para visitar todos los nodos. Dado que la ruta se elige en función del coste de cada paso, resulta útil cuando debe visitar todos los nodos de una sola vez. (Consulte la sección anterior sobre ["Ruta más corta de una sola fuente" en la página 65](#) si no necesita una ruta para un solo viaje).

Este algoritmo se puede utilizar para optimizar rutas en sistemas conectados, como tuberías de agua y diseño de circuitos. También se emplea para aproximar algunos problemas con tiempos de cálculo desconocidos, como el problema del viajante y ciertos tipos de problemas de redondeo. Aunque no siempre encuentra la solución óptima absoluta, este algoritmo facilita enormemente el análisis, potencialmente complejo y con un alto consumo de recursos.

Algunos ejemplos de casos de uso incluyen:

Minimizar el costo de viaje para explorar un país. ["Una aplicación de árboles de expansión mínimos a la planificación de viajes"](#) describe cómo el algoritmo analizó las conexiones aéreas y marítimas para lograrlo.

- Visualizar correlaciones entre los rendimientos de las divisas. Esto se describe en ["Mini-Aplicación del árbol de expansión mínimo en el mercado de divisas"](#).
- Rastreo del historial de transmisión de la infección en un brote. Para más información, véase ["Uso del modelo de árbol de expansión mínimo para la investigación epidemiológica molecular de un brote nosocomial de infección por el virus de la hepatitis C"](#).



El algoritmo de árbol de expansión mínimo solo ofrece resultados significativos cuando se ejecuta en un grafo donde las relaciones tienen pesos diferentes. Si el grafo no tiene pesos, o si todas las relaciones tienen el mismo peso, cualquier árbol de expansión es un árbol de expansión mínimo.

Árbol de expansión mínimo con Neo4j

Veamos el algoritmo del árbol de expansión mínimo en acción. La siguiente consulta encuentra un árbol de expansión que comienza desde Ámsterdam:

```
PARTIDO (n:Lugar {id:"Ámsterdam"})
LLAMA algo.spanningTree.minimum("Lugar", "EROAD", "distancia", id(n),
    {write:true, writeProperty:"MINST"})
YIELD loadMillis, computeMillis, writeMillis, effectiveNodeCount
DEVUELVE loadMillis, computeMillis, writeMillis, effectiveNodeCount
```

Los parámetros que se pasan a este algoritmo son:

Lugar

Las etiquetas de nodo a tener en cuenta al calcular el árbol de expansión

EROAD

Los tipos de relación a tener en cuenta al calcular el árbol de expansión

distancia

El nombre de la propiedad de relación que indica el costo de atravesar entre un par de nodos

identificación(n)

El identificador del nodo interno del nodo desde el que debe comenzar el árbol de expansión

Esta consulta almacena sus resultados en el gráfico. Si queremos devolver el peso mínimo... árbol de expansión podemos ejecutar la siguiente consulta:

```
COINCIDIR ruta = (n:Lugar {id:"Ámsterdam"})-[:MINST*]-()
CON relaciones(ruta) COMO rels
DESCONECTA relaciones AS rel
CON DISTINTO rel COMO rel
DEVUELVE startNode(rel).id COMO origen, endNode(rel).id COMO destino,
    distancia relativa AS costo
```

Y este es el resultado de la consulta:

fuelle	destino	costo
Ámsterdam	Utrech	46.0
Utrech	Gouda	35.0
Gouda	Róterdam	25.0
Róterdam	La Haya	26.0
La Haya	Hoek van Holland	27.0
Hoek van Holland	Felixstowe	207.0
Felixstowe	Ipswich	22.0
Ipswich	Colchester	32.0
Colchester	Londres	106.0

fuelle	destino	costo
Londres	Doncaster	277.0
Doncaster	Immingham	74.0

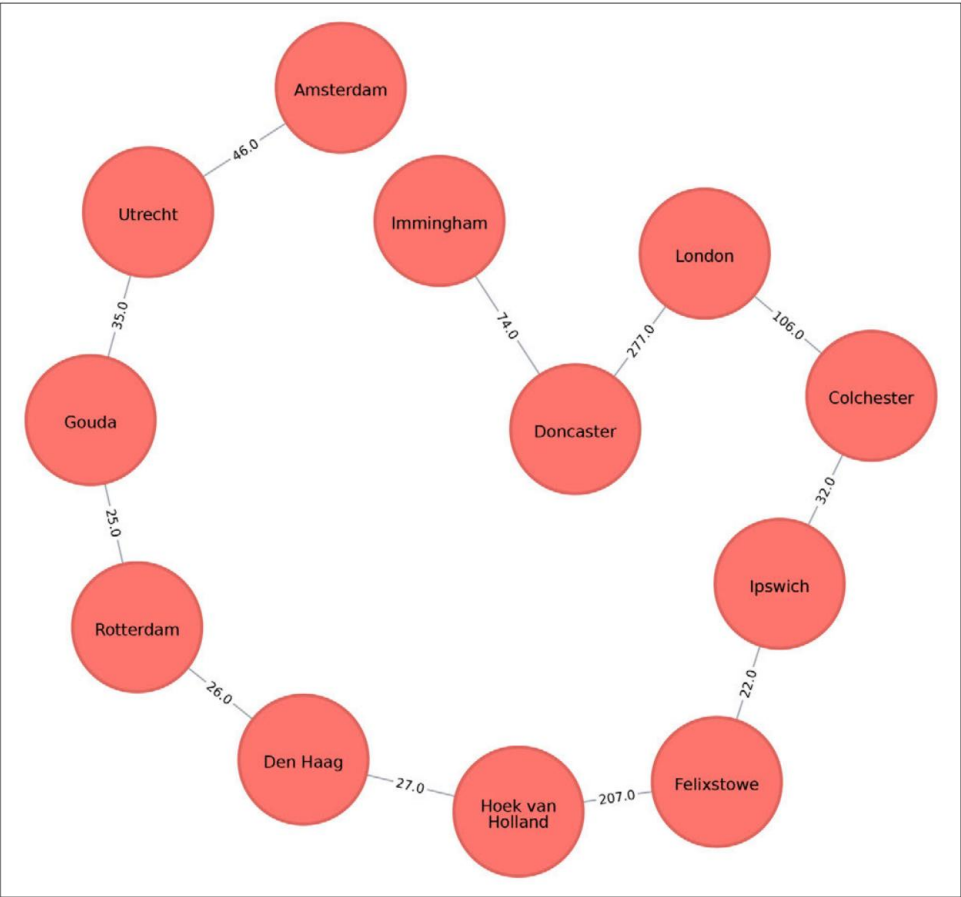


Figura 4-11. Un árbol de expansión de peso mínimo de Ámsterdam

Si estuviéramos en Ámsterdam y quisiéramos visitar todos los demás lugares de nuestro conjunto de datos durante el mismo viaje, la Figura 4-11 muestra la ruta continua más corta para hacerlo.

Paseo aleatorio

El algoritmo Random Walk proporciona un conjunto de nodos en una ruta aleatoria en un gráfico. El término fue mencionado por primera vez por Karl Pearson en 1905 en una carta a la revista Nature, titulado “El problema del paseo aleatorio”. Aunque el concepto se remonta incluso más atrás, Además, sólo más recientemente se han aplicado los paseos aleatorios a la ciencia de redes.

En general, una caminata aleatoria se describe a veces como similar a cómo una persona ebria recorre una ciudad. Sabe adónde quiere llegar, pero puede tomar una ruta muy tortuosa para llegar.

El algoritmo comienza en un nodo y sigue, de forma algo aleatoria, una de las relaciones hacia adelante o hacia atrás hasta un nodo vecino. Luego repite el proceso desde ese nodo y así sucesivamente hasta alcanzar la longitud de ruta establecida. (Decimos, de forma algo aleatoria, porque el número de relaciones que tiene un nodo y sus vecinos influye en la probabilidad de que un nodo sea recorrido).

¿Cuándo debo utilizar el paseo aleatorio?

Utilice el algoritmo Random Walk como parte de otros algoritmos o canales de datos cuando necesite generar un conjunto mayoritariamente aleatorio de nodos conectados.

Algunos ejemplos de casos de uso incluyen:

- Como parte de los algoritmos node2vec y graph2vec, que crean incrustaciones de nodos. Estas incrustaciones de nodos podrían utilizarse como entrada para una red neuronal.
- Como parte de la detección de comunidades de Walktrap e Infomap. Si un recorrido aleatorio devuelve repetidamente un pequeño conjunto de nodos, indica que dicho conjunto de nodos podría tener una estructura de comunidad.
- Como parte del proceso de entrenamiento de modelos de aprendizaje automático. Esto se describe con más detalle en el artículo de David Mack [«Revisión de la predicción con Neo4j y TensorFlow»](#).

Puede leer sobre más casos de uso en un artículo de N. Masuda, MA Porter y R. Lambiotte, [“Paseos aleatorios y difusión en redes”](#).

Paseo aleatorio con Neo4j. Neo4j

implementa el algoritmo Paseo aleatorio. Admite dos modos para elegir la siguiente relación en cada etapa del algoritmo:

aleatorio

Elige al azar una relación para seguir

nodo2vec

Elige la relación a seguir basándose en el cálculo de una distribución de probabilidad de los vecinos anteriores

La siguiente consulta hace esto:

```
PARTIDO (fuente: Lugar {id: "Londres"})
LLAMA algo.randomWalk.stream(id(fuente), 5, 1)
YIELD nodeIds
UNWIND algo.getNodesById(nodeIds) COMO lugar RETURN
lugar.id COMO lugar
```

Los parámetros que se pasan a este algoritmo son:

id(fuente)

El identificador del nodo interno del punto de inicio de nuestra caminata aleatoria

5

El número de saltos que debe dar nuestro paseo aleatorio

1

El número de paseos aleatorios que queremos calcular

Devuelve el siguiente resultado:

lugar
Londres
Doncaster
Immingham
Ámsterdam
Utrecht
Ámsterdam

En cada etapa del recorrido aleatorio, la siguiente relación se elige aleatoriamente. Esto significa que si volvemos a ejecutar el algoritmo, incluso con los mismos parámetros, probablemente no obtengamos el mismo resultado. También es posible que un recorrido retroceda sobre sí mismo, como se observa en la [Figura 4-12](#), donde vamos de Ámsterdam a La Haya y de vuelta.

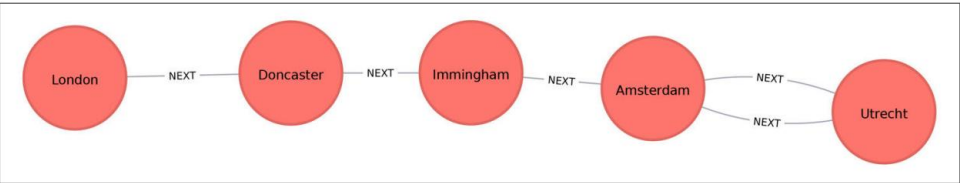


Figura 4-12. Un paseo aleatorio que comienza en Londres.

Resumen

Los algoritmos de búsqueda de rutas son útiles para comprender cómo se conectan nuestros datos. En este capítulo, comenzamos con los algoritmos fundamentales de amplitud y profundidad, antes de pasar a Dijkstra y otros algoritmos de ruta más corta. También analizamos variantes de los algoritmos de ruta más corta optimizadas para encontrar la ruta más corta de un nodo a todos los demás nodos o entre todos los pares de nodos en un grafo. Finalizamos con el algoritmo de paseo aleatorio, que permite encontrar conjuntos arbitrarios de rutas.

A continuación, aprenderemos sobre los algoritmos de centralidad que se pueden utilizar para encontrar nodos influyentes en un gráfico.

Recursos de algoritmos.

Hay muchos libros sobre algoritmos, pero uno destaca por su cobertura de conceptos fundamentales y algoritmos de grafos: El Manual de Diseño de Algoritmos, de Steven S. Skiena (Springer). Recomendamos ampliamente este libro de texto a quienes buscan un recurso completo sobre algoritmos clásicos y técnicas de diseño, o simplemente desean profundizar en el funcionamiento de diversos algoritmos.

Algoritmos de centralidad

Los algoritmos de centralidad se utilizan para comprender las funciones de nodos específicos en un grafo y su impacto en dicha red. Son útiles porque identifican los nodos más importantes y nos ayudan a comprender dinámicas de grupo como la credibilidad, la accesibilidad, la velocidad de propagación y los puentes entre grupos. Aunque muchos de estos algoritmos se inventaron para el análisis de redes sociales, desde entonces se han utilizado en diversos sectores e industrias.

Cubriremos los siguientes algoritmos:

- Centralidad de grado como métrica de referencia de conectividad
- Centralidad de cercanía para medir qué tan central es un nodo con respecto al grupo, incluidas dos variaciones para grupos desconectados
- Centralidad de intermediación para encontrar puntos de control, incluida una alternativa para aproximación
- PageRank para comprender la influencia general, incluida una opción popular para personalización



Distintos algoritmos de centralidad pueden producir resultados significativamente distintos según su propósito de medición. Si observa resultados subóptimos, es recomendable comprobar que el algoritmo utilizado se ajusta a su propósito.

Explicaremos cómo funcionan estos algoritmos y mostraremos ejemplos en Spark y Neo4j. Cuando un algoritmo no está disponible en una plataforma o las diferencias no son importantes, proporcionaremos solo un ejemplo de plataforma.

La Figura 5-1 muestra las diferencias entre los tipos de preguntas que los algoritmos de centralidad pueden responder, y la Tabla 5-1 es una referencia rápida de lo que calcula cada algoritmo con un ejemplo de uso.

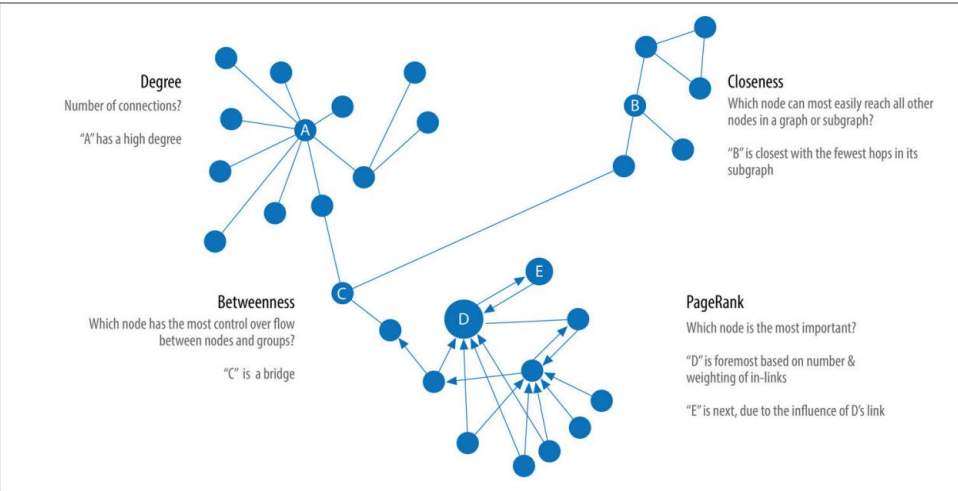


Figura 5-1. Algoritmos de centralidad representativos y los tipos de preguntas que responden

Tabla 5-1. Resumen de los algoritmos de centralidad

Tipo de algoritmo	Qué hace	Ejemplo de uso	Ejemplo de Spark	Ejemplo de Neo4j
Centralidad de grados	Mide el número de relaciones que tiene un nodo	Estimar la popularidad de una persona observando su grado de entrada y usando su grado de salida para estimar su gregarismo.	Sí	No
Centralidad de cercanía Variaciones: Wasserman y Fausto, Centralidad Armónica	Calcula qué nodos tienen las rutas más cortas a todos los demás nodos	Encontrar la ubicación óptima de los nuevos servicios públicos para lograr la máxima accesibilidad	Sí	Sí
Centralidad de intermediación Variación: Aleatorizada-Brandes aproximado	Mide el número de rutas más cortas que pasan por un nodo	Mejorar la focalización de los fármacos mediante el descubrimiento de los genes de control para enfermedades específicas	No	Sí
PageRank Variación: Personalizada PageRank	Estima la importancia de un nodo actual a partir de sus vecinos vinculados y sus vecinos (popularizado por Google)	Encontrar las características más influyentes para la extracción en el aprendizaje automático y clasificar el texto por relevancia de la entidad en el procesamiento del lenguaje natural.	Sí	Sí



Varios algoritmos de centralidad calculan las rutas más cortas entre cada par de nodos. Esto funciona bien para grafos pequeños y medianos, pero para grafos grandes puede resultar computacionalmente prohibitivo. Para evitar tiempos de ejecución prolongados en grafos grandes, algunos algoritmos (por ejemplo, Betweenness Centrality) tienen versiones de aproximación.

Primero, describiremos el conjunto de datos para nuestros ejemplos y explicaremos cómo importar los datos. en Apache Spark y Neo4j. Cada algoritmo se describe en el orden indicado en **Tabla 5-1**. Comenzaremos con una breve descripción del algoritmo y, cuando sea necesario, Información sobre su funcionamiento. Las variaciones de los algoritmos ya abordados incluirán Menos detalles. La mayoría de las secciones también incluyen orientación sobre cuándo usar el algoritmo relacionado. Demostramos un código de ejemplo utilizando un conjunto de datos de muestra al final de cada sección.

¡Comencemos!

Ejemplo de datos gráficos: El gráfico social

Los algoritmos de centralidad son relevantes para todos los gráficos, pero las redes sociales proporcionan una Una forma comprensible de pensar en la influencia dinámica y el flujo de información. Los ejemplos de este capítulo se comparan con un pequeño gráfico similar al de Twitter. Puedes descargar Los archivos de nodos y relaciones que usaremos para crear nuestro gráfico están en el [GitHub del libro](#). [repositorio](#).

Tabla 5-2. social-nodes.csv

id
Alicia
Brigida
Carlos
Doug
Marca
Miguel
David
Amy
Jaime

Tabla 5-3. relaciones-sociales.csv

origen	destino	relación
Alicia	Bridget	SIGUE
Alicia	Charles	SIGUE
Marca	Doug	SIGUE
Bridget	Michael	SIGUE
Doug	Marca	SIGUE
Michael	Alice	SIGUE
Alicia	Michael	SIGUE
Bridget	Alice	SIGUE
Michael	Bridget	SIGUE

origen		relación
Charles Doug		SIGUE
Bridget Doug		SIGUE
Michael Doug		SIGUE
Alicia	Doug	SIGUE
Marca	Alicia	SIGUE
David Amy		SIGUE
James David		SIGUE

La figura 5-2 ilustra el gráfico que queremos construir.

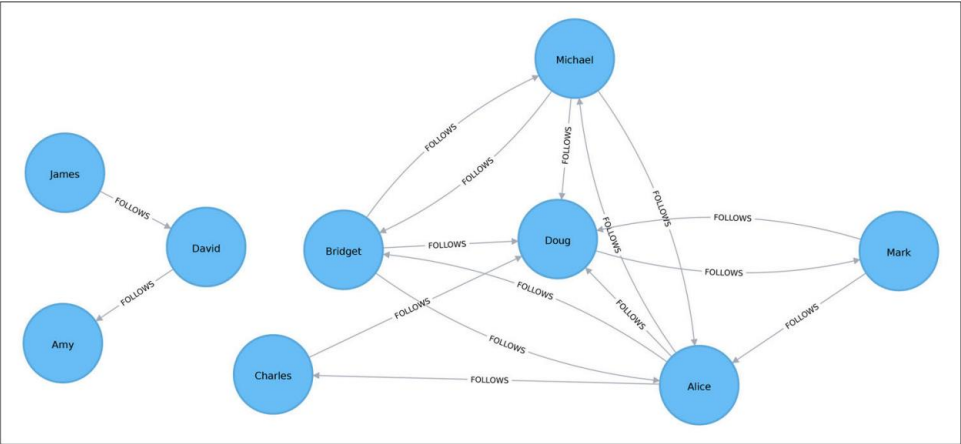


Figura 5-2. El modelo gráfico

Tenemos un conjunto más grande de usuarios con conexiones entre ellos y un conjunto más pequeño Sin conexiones con ese grupo más grande.

Creemos gráficos en Spark y Neo4j basados en el contenido de esos archivos CSV.

Importación de datos a Apache Spark

Primero, importaremos los paquetes necesarios de Spark y el paquete GraphFrames:

```
desde graphframes importar *
desde pyspark importar SparkContext
```

Podemos escribir el siguiente código para crear un GraphFrame basado en el contenido de la Archivos CSV:

```
v = spark.read.csv("datos/social-nodes.csv", encabezado=Verdadero)
e = spark.read.csv("datos/relaciones-sociales.csv", header=True)
g = GraphFrame(v, e)
```

Importando los datos a Neo4j. A continuación,

cargaremos los datos para Neo4j. La siguiente consulta importa nodos:

```
CON "https://github.com/neo4j-graph-analytics/book/raw/master/data/" COMO base CON base + "social-nodes.csv"
COMO uri CARGAR CSV CON ENCABEZADOS DESDE
uri COMO fila FUSIONAR (:User {id: row.id})
```

Y esta consulta importa relaciones:

```
CON "https://github.com/neo4j-graph-analytics/book/raw/master/data/" COMO base CON base + "social-relationships.csv"
COMO uri CARGAR CSV CON ENCABEZADOS DE uri COMO fila

COINCIDIR (fuente: Usuario {id: row.src})
COINCIDIR (destino: Usuario {id: row.dst})
FUSIONAR (origen)-[:SIGUE]->(destino)
```

Ahora que nuestros gráficos están cargados, ¡pasemos a los algoritmos!

Centralidad de grados

La centralidad de grado es el algoritmo más simple que abordaremos en este libro. Contabiliza el número de relaciones entrantes y salientes de un nodo y se utiliza para encontrar nodos populares en un grafo. La centralidad de grado fue propuesta por Linton C. Freeman en su artículo de 1979 "**Centralidad en redes sociales: Aclaración conceptual**".

Alcanzar

Comprender el alcance de un nodo es una medida justa de su importancia. ¿Cuántos otros nodos puede alcanzar actualmente? El grado de un nodo es el número de relaciones directas que tiene, calculadas para el grado de entrada y el grado de salida. Se puede considerar esto como el alcance inmediato del nodo. Por ejemplo, una persona con un alto grado en una red social activa tendría muchos contactos inmediatos y sería más propensa a contraer un resfriado circulando en su red.

El grado promedio de una red es simplemente el número total de relaciones dividido entre el número total de nodos; puede estar muy sesgado por nodos de alto grado. La distribución de grado es la probabilidad de que un nodo seleccionado aleatoriamente tenga un cierto número de relaciones.

La Figura 5-3 ilustra la diferencia al observar la distribución real de conexiones entre los temas del subreddit. Si simplemente se tomara el promedio, se asumiría que la mayoría de los temas tienen 10 conexiones, cuando en realidad la mayoría solo tiene 2.

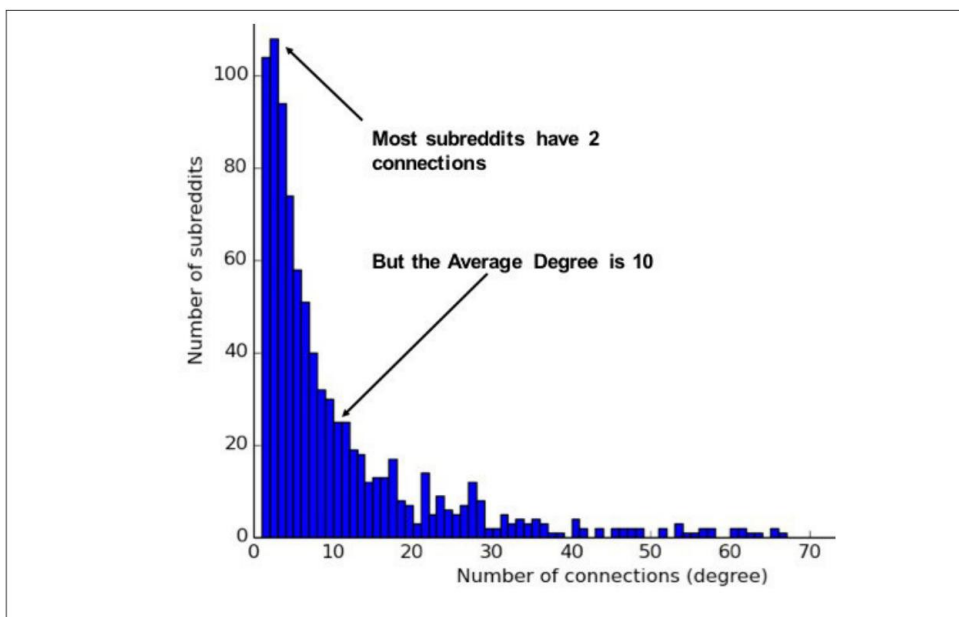


Figura 5-3. Este mapeo de la distribución de grados en subreddit, realizado por [Jacob Silterrapa](#), ofrece un ejemplo de cómo el promedio no suele reflejar la distribución real en las redes. CC BY-SA 3.0.

Estas medidas se utilizan para categorizar tipos de redes como las redes sin escala o de mundo pequeño que se analizaron en el [Capítulo 2](#). También proporcionan una medida rápida para ayudar a estimar el potencial de que las cosas se propaguen o se extiendan a lo largo de una red.

¿Cuándo debo utilizar la centralidad de grados?

Utilice la centralidad de grado si intenta analizar la influencia observando el número de relaciones entrantes y salientes, o si busca la popularidad de nodos individuales. Funciona bien cuando se preocupa por la conectividad inmediata o las probabilidades a corto plazo. Sin embargo, la centralidad de grado también se aplica al análisis global cuando desea evaluar el grado mínimo, el grado máximo, el grado medio y la desviación estándar en todo el gráfico.

Algunos ejemplos de casos de uso incluyen:

- Identificar a personas influyentes a través de sus relaciones, como las conexiones de personas en una red social. Por ejemplo, en el informe ["Hombres y mujeres más influyentes en Twitter 2017"](#) de [BrandWatch](#), las 5 personas más influyentes de cada categoría tienen más de 40 millones de seguidores cada una.
- Separar a los estafadores de los usuarios legítimos de un sitio de subastas en línea. La centralidad ponderada de los estafadores tiende a ser significativamente mayor debido a la colusión dirigida

con precios que aumentan artificialmente. Lea más en el artículo de P. Bangcharoensap et al. et al., “[Aprendizaje semisupervisado basado en gráficos de dos pasos para el fraude en subastas en línea](#)” [Detección](#)”.

Centralidad de grados con Apache Spark

Ahora ejecutaremos el algoritmo de centralidad de grado con el siguiente código:

```
grado_total = g.grados
in_degree = g.inDegrees
grado_de_salida = g.grados_de_salida

(grado_total.join(grado_dentro, "id", cómo="izquierda")
 .join(grado_de_salida, "id", cómo="izquierda")
 .fillna(0)
 .sort("inDegree", ascendente=Falso)
 .espectáculo())
```

Primero calculamos los grados totales de entrada y salida. Luego, unimos esos DataFrames. juntos, utilizando una unión izquierda para retener cualquier nodo que no tenga entrada o salida relaciones. Si los nodos no tienen relaciones, establecemos ese valor en 0 usando el relleno. función.

Aquí está el resultado de ejecutar el código en pyspark:

	Grado de entrada	Grado de salida	Grado
Doug	6	5	1
Alicia	7	3	4
Miguel	5	2	3
Brigida	5	2	3
Carlos	2	1	1
Marca	3	1	2
David	2	1	1
Amy	1	1	0
Santiago	1	0	1

Podemos ver en [la Figura 5-4](#) que Doug es el usuario más popular en nuestro gráfico de Twitter, con cinco seguidores (enlaces internos). Todos los demás usuarios en esa parte del gráfico lo siguen y Solo sigue a una persona. En la red real de Twitter, las celebridades tienen un alto... cuenta de seguidores, pero tiende a seguir a pocas personas. Por lo tanto, podríamos considerar a Doug un ¡celebridad!

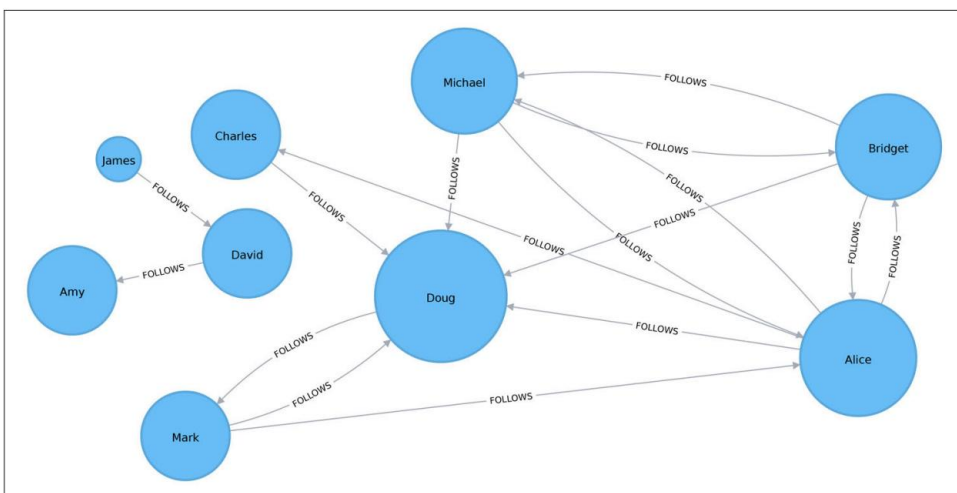


Figura 5-4. Visualización de la centralidad de grado

Si estuviéramos creando una página que mostrara los usuarios más seguidos o quisiéramos sugerir personas para seguir, podríamos usar este algoritmo para identificar a esas personas.



Algunos datos pueden contener nodos muy densos con muchas relaciones. Estos no aportan mucha información adicional y pueden distorsionar algunos resultados o añadir complejidad computacional. Se recomienda filtrar estas notas densas mediante un subgrafo o una proyección para resumir las relaciones como ponderaciones.

Centralidad de cercanía

La centralidad de cercanía es una forma de detectar nodos que pueden difundir información de manera eficiente a través de un subgrafo.

La medida de la centralidad de un nodo es su distancia media (distancia inversa) a todos los demás nodos. Los nodos con una puntuación alta de cercanía tienen las distancias más cortas a todos los demás nodos.

Para cada nodo, el algoritmo de centralidad de cercanía calcula la suma de sus distancias a todos los demás nodos, basándose en el cálculo de las rutas más cortas entre todos los pares de nodos.

Luego, la suma resultante se invierte para determinar la puntuación de centralidad de cercanía para ese nodo.

La centralidad de cercanía de un nodo se calcula mediante la fórmula:

$$C(u) = \frac{1}{\sum_{v \neq u} d(u, v)}$$

dónde:

- u es un nodo.
- n es el número de nodos en el gráfico.
- $d(u,v)$ es la distancia más corta entre otro nodo v y u .

Es más común normalizar esta puntuación para que represente la longitud promedio de las rutas más cortas, en lugar de su suma. Este ajuste permite comparar la centralidad de proximidad de nodos de grafos de diferentes tamaños.

La fórmula para la centralidad de cercanía normalizada es la siguiente:

$$C_{\text{norm}}(u) = \frac{n-1}{\sum_{v=1}^n d(u,v)}$$

¿Cuándo debo utilizar la centralidad de cercanía?

Aplique la Centralidad de Cercanía cuando necesite saber qué nodos difunden información con mayor rapidez. El uso de relaciones ponderadas puede ser especialmente útil para evaluar la velocidad de interacción en los análisis de comunicación y comportamiento.

Algunos ejemplos de casos de uso incluyen:

- Descubrir a individuos en posiciones muy ventajosas para controlar y adquirir información y recursos vitales dentro de una organización. Un estudio de este tipo es "[Mapeo de redes de células terroristas](#)", de VE Krebs.

Como heurística para estimar el tiempo de llegada en telecomunicaciones y entrega de paquetes, donde el contenido fluye por las rutas más cortas hacia un objetivo predefinido. También se utiliza para analizar la propagación por todas las rutas más cortas simultáneamente, como la propagación de infecciones en una comunidad local. Encuentre más detalles en «[Centralidad y Flujo de Red](#)», de SP Borgatti.

- Evaluación de la importancia de las palabras en un documento, con base en un proceso de extracción de frases clave basado en grafos. Este proceso se describe en «[Comparación de medidas de centralidad para la extracción de frases clave basada en grafos](#)» de F. Boudin.



La centralidad de proximidad funciona mejor en grafos conexos. Al aplicar la fórmula original a un grafo no conexo, se obtiene una distancia infinita entre dos nodos sin una ruta entre ellos. Esto significa que obtendremos una puntuación de centralidad de proximidad infinita al sumar todas las distancias desde ese nodo. Para evitar este problema, se mostrará una variación de la fórmula original después del siguiente ejemplo.

Centralidad de cercanía con Apache Spark Apache Spark no tiene un

algoritmo integrado para la centralidad de cercanía, pero podemos escribir el nuestro utilizando el marco de trabajo addedMessages que presentamos en ["La ruta más corta \(ponderada\) con Apache Spark" en la página 54](#) del capítulo anterior.

Antes de crear nuestra función, importaremos algunas bibliotecas que usaremos:

```
desde graphframes.lib importar AggregateMessages como AM
desde pyspark.sql importar funciones como F
desde pyspark.sql.types importar *
desde operador importar itemgetter
```

También crearemos algunas funciones definidas por el usuario que necesitaremos más adelante:

```
def collect_paths(rutas):
    devuelve F.collect_set(rutas)

recopilar_rutas_udf = F.udf(recopilar_rutas, ArrayType(StringType()))

tipo_de_rutas = ArrayType(
    Tipo de estructura ([Campo de estructura ("id", Tipo de cadena ()), Campo de estructura ("distancia",

def flatten(ids):
    flat_list = [elemento para la sublista en ids para el elemento en la
    sublista] return list(dict(sorted(flat_list, key=itemgetter(0))).items())

aplanar_udf = F.udf(aplanar, tipo_de_ruta)

def new_paths(rutas, id):
    rutas = [{"id": col1, "distancia": col2 + 1} para col1, col2 en rutas
    si col1 != id] rutas.append({"id": id,
    "distancia": 1}) devuelve rutas

nuevos_caminos_udf = F.udf(nuevos_caminos, tipo_de_caminos)

def merge_paths(ids, nuevos_ids, id):
    ids_unidos = ids + (ids_nuevos si ids_nuevos de lo
    contrario []) ids_combinados = [(col1, col2) para col1, col2 en ids_unidos si col1 != id]
    ids_mejores = dict(ordenado(ids_combinados, clave=itemgetter(1), reverso=True))
    devuelve [{"id": col1, "distancia": col2} para col1, col2 en ids_mejores.items()]

fusionar_rutas_udf = F.udf(fusionar_rutas, tipo_de_rutas)
```

```
def calcular_cercanía(ids):
    nodos = len(ids)
    distancia_total = suma([col2 para col1, col2 en ids])
    devuelve 0 si distancia_total == 0 de lo contrario nodos * 1.0 / distancia_total
```

```
cercanía_udf = F.udf(calcular_cercanía, DoubleType())
```

Y ahora el cuerpo principal que calcula la centralidad de cercanía para cada nodo:

```
vértices = g.vertices.withColumn("ids", F.array())
vértices_en_cache = AM.getCachedDataFrame(vértices)
g2 = GraphFrame(vértices_en_caché, g.aristas)

para i en rango(0, g2.vertices.count()):
    msg_dst = nuevas_rutas_udf(AM.src["ids"], AM.src["id"])
    msg_src = nuevas_rutas_udf(AM.dst["ids"], AM.dst["id"])
    agg = g2.agregarMessages(F.collect_set(AM.msg).alias("agg"),
                             enviarAOrigen=msg_origen, enviarAFecha=msg_fecha)

    res = agg.withColumn("nuevosIds", flatten_udf("agg")).drop("agg")
    nuevos_vértices = (g2.vértices.join(res, on="id", how="left_outer")
                       .withColumn("identificadores fusionados", merge_paths_udf("identificadores", "identificadores nuevos",
                                         "id"))).drop("ids", "nuevosIds")
                       .withColumnRenamed("IDs fusionados", "id"))
    vértices_nuevos_en_cache = AM.getCachedDataFrame(nuevos_vértices)
    g2 = GraphFrame(nuevos_vértices_en_caché, g2.aristas)

(g2.vértices
 .withColumn("cercanía", closeness_udf("ids"))
 .sort("cercanía", ascendente=Falso)
 .show(truncar=Falso))
```

Si ejecutamos esto veremos el siguiente resultado:

	cercanía
Doug	[[Charles, 1], [Mark, 1], [Alice, 1], [Bridget, 1], [Michael, 1]] 1.0
Alicia	[[Charles, 1], [Mark, 1], [Bridget, 1], [Doug, 1], [Michael, 1]] 1.0
David	[[James, 1], [Amy, 1]] 1.0
Bridget	[[Charles, 2], [Mark, 2], [Alice, 1], [Doug, 1], [Michael, 1]] 0.7142857142857143
Michael	[[Charles, 2], [Mark, 2], [Alice, 1], [Doug, 1], [Bridget, 1]] 0.7142857142857143
James	[[Amy, 2], [David, 1]] 0.6666666666666666
Amy	[[James, 2], [David, 1]] 0.6666666666666666
Marca	[[Bridget, 2], [Charles, 2], [Michael, 2], [Doug, 1], [Alice, 1]] 0.625
Charles	[[Bridget, 2], [Mark, 2], [Michael, 2], [Doug, 1], [Alice, 1]] 0.625

Alice, Doug y David son los nodos más estrechamente conectados en el gráfico con un 1.0 puntuación, lo que significa que cada uno se conecta directamente a todos los nodos en su parte del gráfico. La figura 5-5 ilustra que, aunque David solo tiene unas pocas conexiones, dentro de su

Grupo de amigos significativo. En otras palabras, esta puntuación representa la cercanía de cada usuario con otros dentro de su subgráfico, pero no con todo el gráfico.

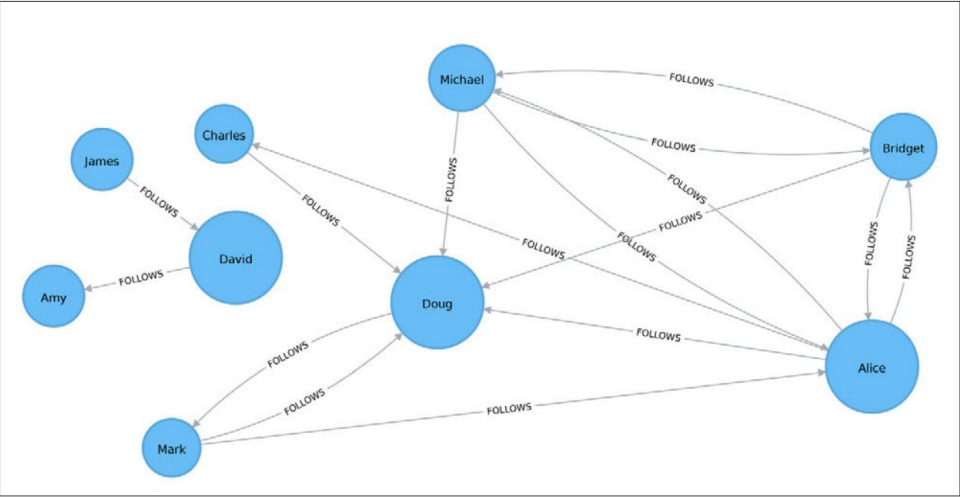


Figura 5-5. Visualización de la centralidad de cercanía

Centralidad de cercanía con Neo4j

La implementación de Centralidad de Cercanía de Neo4j utiliza la siguiente fórmula:

$$C(u) = \frac{n-1}{\sum_{v=1}^{n-1} d(u,v)}$$

dónde:

- u es un nodo.
- n es el número de nodos en el mismo componente (subgrafo o grupo) que u.
- d(u,v) es la distancia de camino más corto entre otro nodo v y u.

Una llamada al siguiente procedimiento calculará la centralidad de cercanía para cada uno de los nodos de nuestro gráfico:

```
LLAMA algo.closeness.stream("Usuario", "SIGUE")
YIELD nodeId, centralidad
RETURN algo.getNodeById(nodeId).id, centralidad
ORDER BY centralidad DESC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

usuario	centralidad
Alicia	1.0
Doug	1.0

usuario	centralidad
David	1.0
Brigida	0.7142857142857143
Miguel	0.7142857142857143
Amy	0.6666666666666666
James	0.6666666666666666
Carlos	0.625
Marca	0.625

Obtenemos los mismos resultados que con el algoritmo Spark, pero, como antes, la puntuación representa su cercanía a otros dentro de su subgráfico, pero no a todo el gráfico.



En la interpretación estricta del algoritmo de Centralidad de Cercanía, todos los nodos de nuestro grafo tendrían una puntuación de ∞ porque cada nodo tiene al menos otro nodo al que no puede acceder. Sin embargo, suele ser más útil implementar la puntuación por componente.

Lo ideal sería obtener una indicación de la cercanía a lo largo de todo el gráfico, y en las próximas dos secciones aprenderemos sobre algunas variaciones del algoritmo de centralidad de cercanía que hacen esto.

Variación de la centralidad de cercanía: Wasserman y Faust. Stanley

Wasserman y Katherine Faust idearon una fórmula mejorada para calcular la cercanía de grafos con múltiples subgrafos sin conexiones entre dichos grupos. Los detalles de su fórmula se encuentran en su libro, *Análisis de Redes Sociales: Métodos y Aplicaciones*. El resultado de esta fórmula es una relación entre la fracción de nodos del grupo alcanzables y la distancia promedio desde los nodos alcanzables.

La fórmula es la siguiente:

$$CWF(u) = \frac{n-1}{N-1} \left(\frac{n-1}{\sum_{v=1}^{n-1} d(u, v)} \right)$$

dónde:

- u es un nodo.
- N es el recuento total de nodos.
- n es el número de nodos en el mismo componente que u .
- $d(u, v)$ es la distancia de ruta más corta entre otro nodo v y u .

Podemos indicarle al procedimiento de Centralidad de Cercanía que utilice esta fórmula pasando el parámetro improved: true.

La siguiente consulta ejecuta la centralidad de cercanía utilizando la fórmula de Wasserman y Faust:

```
LLAMA algo.closeness.stream("Usuario", "SIGUE", {mejorado: verdadero})
YIELD nodeId, centralidad
RETURN algo.getNodeById(nodeId).id AS usuario, centralidad
ORDER BY centralidad DESC
```

El procedimiento da el siguiente resultado:

usuario	centralidad
Alicia	0.5
Doug	0.5
Bridget	0.35714285714285715
Miguel	0.35714285714285715
Carlos	0.3125
Marca	0.3125
David	0,125
Amy	0.08333333333333333
James	0.08333333333333333

Como muestra **la Figura 5-6** , los resultados ahora son más representativos de la proximidad de los nodos a todo el grafo. Las puntuaciones de los miembros del subgrafo más pequeño (David, Amy y James) se han atenuado, y ahora tienen las puntuaciones más bajas de todos los usuarios. Esto tiene sentido, ya que son los nodos más aislados. Esta fórmula es más útil para detectar la importancia de un nodo en todo el grafo que dentro de su propio subgrafo.

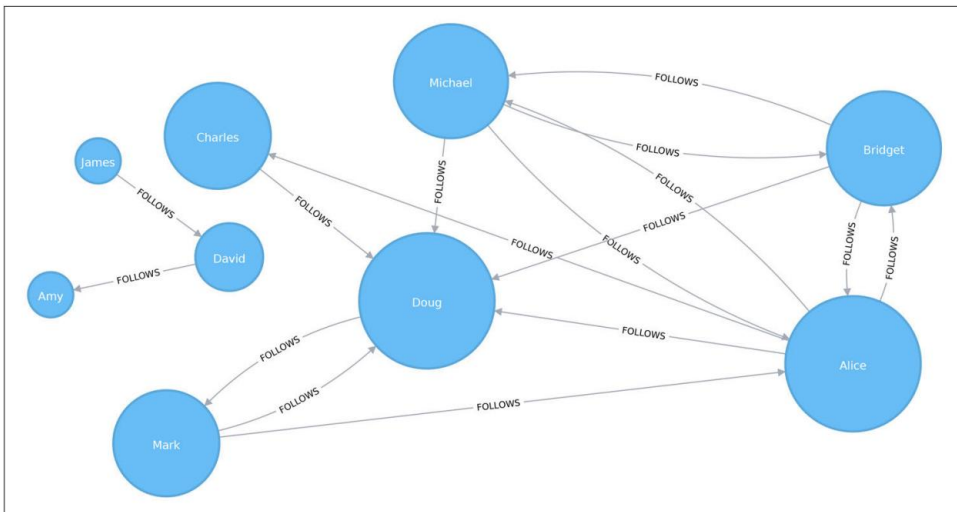


Figura 5-6. Visualización de la centralidad de cercanía

En la siguiente sección aprenderemos sobre el algoritmo de centralidad armónica, que logra resultados similares utilizando otra fórmula para calcular la cercanía.

Variación de la centralidad de proximidad: Centralidad armónica. La

centralidad armónica (también conocida como centralidad valorada) es una variante de la centralidad de proximidad, ideada para resolver el problema original con grafos inconexos. En **"Armonía en un mundo pequeño"**, M. Marchiori y V. Latora propusieron este concepto como representación práctica de una ruta más corta promedio.

Al calcular la puntuación de proximidad de cada nodo, en lugar de sumar las distancias de un nodo a todos los demás, se suma el inverso de dichas distancias. Esto significa que los valores infinitos se vuelven irrelevantes.

La centralidad armónica bruta de un nodo se calcula utilizando la siguiente fórmula:

$$H(u) = \sum_{v=1}^{n-1} \frac{1}{d(u,v)}$$

dónde:

- u es un nodo.
- n es el número de nodos en el gráfico.
- $d(u,v)$ es la distancia más corta entre otro nodo v y u .

Al igual que con la centralidad de cercanía, también podemos calcular una centralidad armónica normalizada con la siguiente fórmula:

$$H_{norm}(u) = \frac{\sum_{v=1}^{n-1} \frac{1}{d(u,v)}}{n-1}$$

esta fórmula, los valores ∞ se manejan limpiamente.

Centralidad armónica con Neo4j

La siguiente consulta ejecuta el algoritmo de centralidad armónica:

```
LLAMA algo.closeness.harmonic.stream("Usuario", "SIGUE")
YIELD nodeId, centralidad
RETURN algo.getNodeById(nodeId).id AS usuario, centralidad
ORDER BY centralidad DESC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

usuario	centralidad
Alicia	0.625
Doug	0.625
Bridget	0.5
Miguel	0.5
Carlos	0.4375
Marca	0.4375
David	0.25
Amy	0.1875
James	0.1875

Los resultados de este algoritmo difieren de los del algoritmo de centralidad de cercanía original, pero son similares a los de la mejora de Wasserman y Faust.

Se puede utilizar cualquiera de los dos algoritmos cuando se trabaja con gráficos con más de un componente conectado.

Centralidad de intermediación

A veces, el engranaje más importante del sistema no es el que ostenta el mayor poder manifiesto o el estatus más alto. A veces, son los intermediarios que conectan grupos o los intermediarios quienes tienen el mayor control sobre los recursos o el flujo de información. La centralidad de intermediación es una forma de detectar el grado de influencia que un nodo tiene sobre el flujo de información o recursos en un grafo. Se utiliza típicamente para encontrar nodos que sirven de puente entre una parte del grafo y otra.

El algoritmo de Centralidad de Intermediación calcula primero la ruta más corta (ponderada) entre cada par de nodos de un grafo conexo. Cada nodo recibe una puntuación basada en el número de estas rutas más cortas que pasan por él. Cuantos más caminos más cortos recorra un nodo, mayor será su puntuación.

La centralidad de intermediación se consideró una de las “tres concepciones intuitivas distintas de centralidad” cuando fue introducida por Linton C. Freeman en su artículo de 1971, “[Un conjunto de medidas de centralidad basadas en la intermediación](#)”.

Puentes y puntos de control.

Un puente en una red puede ser un nodo o una relación. En un grafo muy simple, se pueden encontrar buscando el nodo o la relación que, si se eliminara, causaría la desconexión de una sección del grafo. Sin embargo, como esto no es práctico en un grafo típico, utilizamos un algoritmo de centralidad de intermediación. También podemos medir la intermediación de un clúster tratando el grupo como un nodo.

Un nodo se considera fundamental para otros dos nodos si se encuentra en cada camino más corto entre esos nodos, como se muestra en [la Figura 5-7](#).

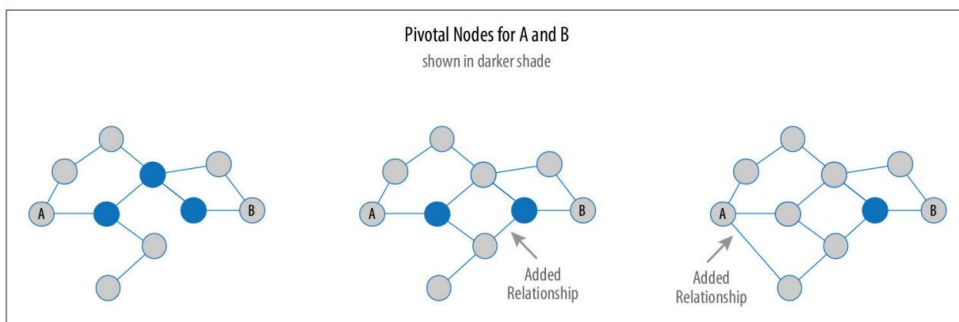


Figura 5-7. Los nodos pivotaes se encuentran en cada ruta más corta entre dos nodos. Crear más rutas más cortas puede reducir el número de nodos pivotaes para usos como la mitigación de riesgos.

Los nodos pivotaes desempeñan un papel importante en la conexión de otros nodos: si se elimina un nodo pivotal, la nueva ruta más corta para los pares de nodos originales será más larga o más costosa. Esto puede ser un factor a considerar al evaluar puntos individuales de vulnerabilidad.

Cálculo de la centralidad de intermediación

La centralidad de intermediación de un nodo se calcula sumando los resultados de la siguiente fórmula para todas las rutas más cortas:

$$B(u) = \sum_{s \neq u \neq t} \frac{1}{p_{st}}$$

dónde:

- u es un nodo.
- p es el número total de caminos más cortos entre los nodos s y t.

- $p(u)$ es el número de caminos más cortos entre los nodos s y t que pasan por nodo u .

La figura 5-8 ilustra los pasos para calcular la centralidad de intermediación.

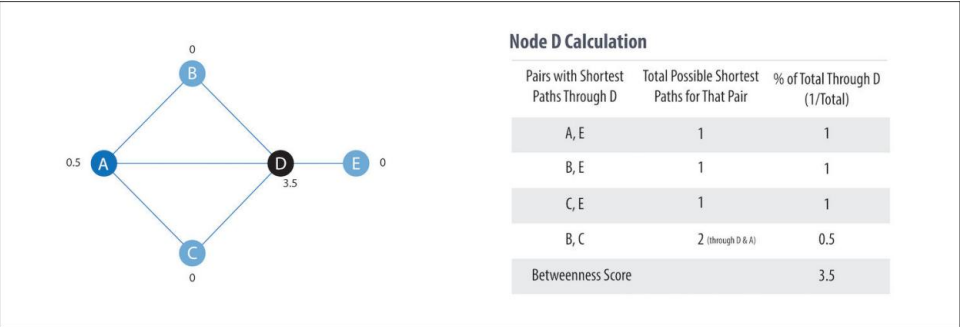


Figura 5-8. Conceptos básicos para el cálculo de la centralidad de intermediación

Aquí está el procedimiento:

1. Para cada nodo, encuentre los caminos más cortos que lo atraviesan.
 - a. B, C, E no tienen caminos más cortos y se les asigna un valor de 0.
2. Para cada camino más corto en el paso 1, calcule su porcentaje del total posible caminos más cortos para ese par.
3. Suma todos los valores del paso 2 para encontrar la puntuación de centralidad de intermediación de un nodo.

La tabla de la Figura 5-8 ilustra los pasos 2 y 3 para el nodo D.
4. Repita el proceso para cada nodo.

¿Cuándo debo utilizar la centralidad de intermediación?

La centralidad de intermediación se aplica a una amplia gama de problemas en redes del mundo real. Lo usamos para encontrar cuellos de botella, puntos de control y vulnerabilidades.

Algunos ejemplos de casos de uso incluyen:

- Identificación de personas influyentes en diversas organizaciones. Las personas con poder no necesariamente ocupan puestos directivos, pero pueden encontrarse en puestos de intermediación mediante la Centralidad de Intermediación. La eliminación de estas personas influyentes puede desestabilizar gravemente la organización. Esto podría ser considerado una interrupción bienvenida por las fuerzas del orden si la organización es delictiva, o podría ser un desastre si una empresa pierde personal clave que subestimó. Encontrará más detalles en «Calificaciones de Intermediación en Operaciones de Ringing», de C. Morselli y J. Roy.

- Descubrimiento de puntos de transferencia clave en redes como las eléctricas. Contrariamente a la intuición, la eliminación de puentes específicos puede mejorar la robustez general al aislar las perturbaciones. Los detalles de la investigación se incluyen en «Robustez de las redes eléctricas europeas bajo ataque intencional», de R. Solé, et al.
- Ayudar a los microblogueros a ampliar su alcance en Twitter con un motor de recomendaciones para identificar influencers. Este enfoque se describe en el artículo de S. Wu et al., «Hacer recomendaciones en un microblog para mejorar el impacto de un usuario objetivo».



La centralidad de intermediación asume que toda la comunicación entre nodos se produce por la ruta más corta y con la misma frecuencia, lo cual no siempre ocurre en la vida real. Por lo tanto, no ofrece una visión precisa de los nodos más influyentes en un grafo, sino una buena representación. Mark Newman explica esto con más detalle en *Networks: An Introduction* (Oxford University Press, p. 186).

Centralidad de intermediación con Neo4j. Spark no

cuenta con un algoritmo integrado para la centralidad de intermediación, por lo que demostraremos este algoritmo con Neo4j. Una llamada al siguiente procedimiento calculará la centralidad de intermediación para cada nodo de nuestro grafo:

```
LLAMA algo.betweenness.stream("Usuario", "SIGUE")
YIELD nodeId, centralidad
RETURN algo.getNodeById(nodeId).id AS usuario, centralidad
ORDER BY centralidad DESC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

usuario	centralidad
Alicia	10.0
Doug	7.0
Marca	7.0
David	1.0
Bridget	0.0
Charles	0.0
Miguel	0.0
Amy	0.0
Jaime	0.0

Como podemos ver en la [Figura 5-9](#), Alice es el intermediario principal de esta red, pero Mark y Doug no se quedan atrás. En el subgrafo más pequeño, todos los caminos más cortos pasan por David, por lo que es importante para el flujo de información entre esos nodos.

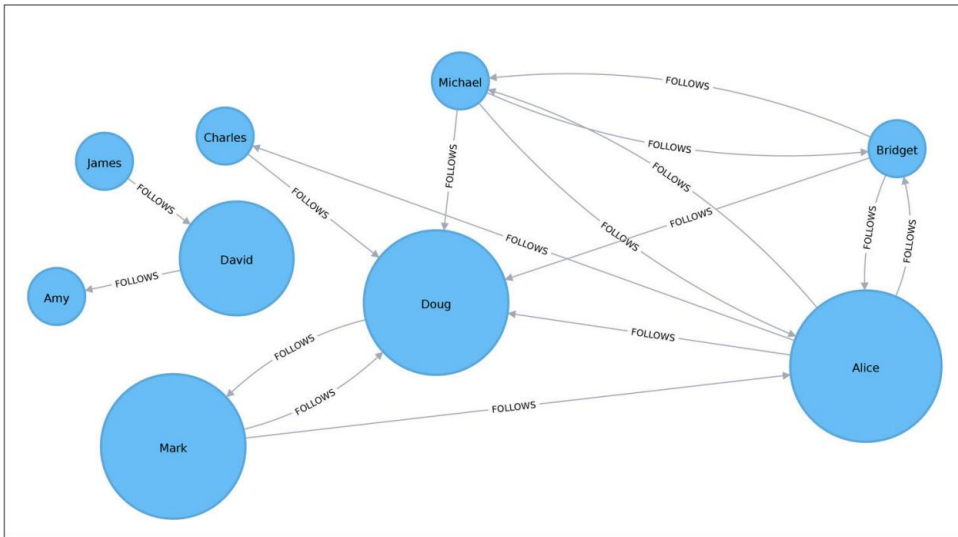


Figura 5-9. Visualización de la centralidad de intermediación



Para grafos grandes, el cálculo exacto de la centralidad no es práctico. El algoritmo más rápido conocido para calcular con precisión la intermediación de todos los nodos tiene un tiempo de ejecución proporcional al producto del número de nodos por el número de relaciones.

Es posible que queramos filtrar primero hasta un subgrafo o utilizar (descrito en la siguiente sección) que funciona con un subconjunto de nodos.

Podemos unir nuestros dos componentes desconectados introduciendo un nuevo usuario llamado Jason, que sigue y es seguido por personas de ambos grupos de usuarios:

CON ["James", "Michael", "Alice", "Doug", "Amy"] COMO usuarios existentes

COINCIDIR (existente:Usuario) DONDE existente.id EN usuariosexistentes

FUSIONAR (nuevoUsuario:Usuario {id: "Jason"})

FUSIONAR (nuevoUsuario)-[:SIGUE]-(existente)

FUSIONAR (nuevoUsuario)-[:SIGUE]->(existente)

Si volvemos a ejecutar el algoritmo veremos este resultado:

usuario	centralidad
Jason	44.33333333333333
Doug	18.333333333333332
Alicia	16.666666666666664
Amy	8.0
Santiago	8.0
Miguel	4.0
Marca	2.1666666666666665
David	0.5
Brigida	0.0
Carlos	0.0

Jason tiene la puntuación más alta porque la comunicación entre los dos grupos de usuarios será pasar a través de él. Se puede decir que Jason actúa como un puente local entre los dos conjuntos de usuarios, como se ilustra en la **Figura 5-10**.

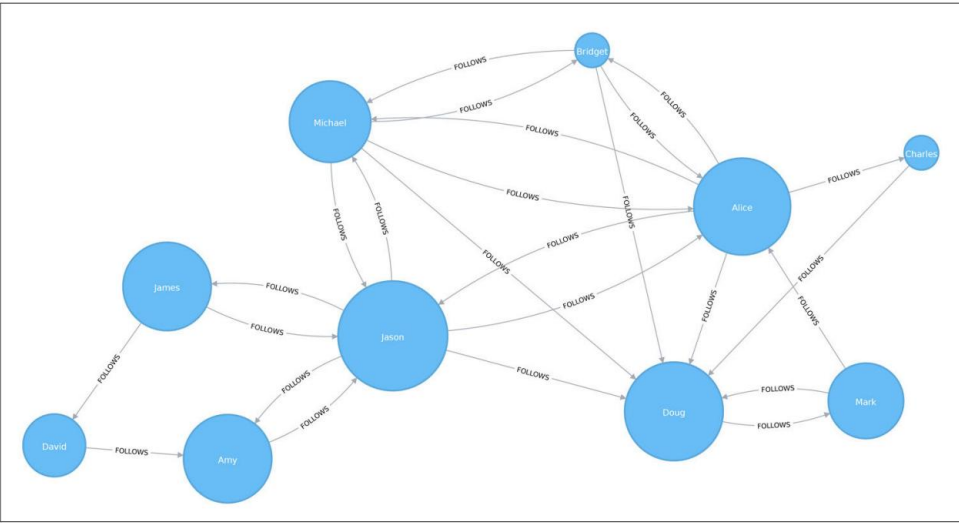


Figura 5-10. Visualización de la centralidad de intermediación con Jason

Antes de pasar a la siguiente sección, restablezcamos nuestro gráfico eliminando a Jason y su relaciones:

```
COINCIDIR (usuario:Usuario {id: "Jason"})
DESCONECTAR ELIMINAR usuario
```

Variación de la centralidad de intermediación: aleatoria-aproximada Brandes

Recuerde que calcular la centralidad de intermediación exacta en grafos grandes puede ser muy costoso. Por lo tanto, podríamos optar por un algoritmo de aproximación que se ejecute mucho más rápido, pero que aun así proporcione información útil (aunque imprecisa).

El algoritmo de Brandes Aleatorizado-Aproximado (RA-Brandes) es el más conocido para calcular una puntuación aproximada de centralidad de intermediación. En lugar de calcular la ruta más corta entre cada par de nodos, el algoritmo RA-Brandes considera solo un subconjunto de nodos. Dos estrategias comunes para seleccionar el subconjunto de nodos son:

Aleatorio

Los nodos se seleccionan de forma uniforme, aleatoria, con una probabilidad de selección definida. La probabilidad predeterminada es $\frac{\log_{10}(N)}{e^2}$. Si la probabilidad es 1, el algoritmo funciona de la misma manera. de la misma manera que el algoritmo de centralidad de intermediación normal, donde se cargan todos los nodos.

Los

nodos de grado se seleccionan aleatoriamente, pero aquellos cuyo grado es inferior a la media se excluyen automáticamente (es decir, solo los nodos con muchas relaciones tienen la posibilidad de ser visitados).

Como optimización adicional, podría limitar la profundidad utilizada por el algoritmo de ruta más corta, que luego proporcionará un subconjunto de todas las rutas más cortas.

Aproximación de la centralidad de intermediación con Neo4j

La siguiente consulta ejecuta el algoritmo RA-Brandes utilizando el método de selección aleatoria:

```
LLAMA algo.betweenness.sampled.stream("Usuario", "SIGUE", {estrategia:"grado"})
YIELD nodeId, centralidad
RETURN algo.getNodeById(nodeId).id AS usuario, centralidad
ORDER BY centralidad DESC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

usuario	centralidad
Alicia	9.0
Marca	9.0
Doug	4.5

usuario	centralidad
David	2.25
Brígida	0.0
Carlos	0.0
Miguel	0.0
Amy	0.0
James	0.0

Nuestros principales influencers son similares a los de antes, aunque Mark ahora tiene una clasificación más alta que Doug.

Debido a la naturaleza aleatoria de este algoritmo, es posible que obtengamos resultados diferentes cada vez que lo ejecutemos. En gráficos más grandes, esta aleatoriedad tendrá un impacto menor que en nuestro gráfico de muestra pequeño.

PageRank

PageRank es el algoritmo de centralidad más conocido. Mide la influencia transitiva (o direccional) de los nodos. Todos los demás algoritmos de centralidad que analizamos miden la influencia directa de un nodo, mientras que PageRank considera la influencia de sus vecinos y los vecinos de estos. Por ejemplo, tener pocos amigos muy poderosos puede aumentar la influencia de una persona que tiene muchos menos. PageRank se calcula distribuyendo iterativamente el rango de un nodo entre sus vecinos o recorriendo aleatoriamente el grafo y contando la frecuencia con la que cada nodo es alcanzado durante estos recorridos.

PageRank recibe su nombre del cofundador de Google, Larry Page, quien lo creó para posicionar sitios web en los resultados de búsqueda de Google. Se parte del supuesto de que una página con más enlaces entrantes e influyentes es probablemente una fuente más fiable. PageRank mide la cantidad y la calidad de las relaciones entrantes con un nodo para determinar su importancia. Se presume que los nodos con mayor influencia en una red tienen más relaciones entrantes de otros nodos influyentes.

Influencia

La intuición detrás de la influencia es que las relaciones con nodos más importantes contribuyen más a la influencia del nodo en cuestión que las conexiones equivalentes con nodos menos importantes. Medir la influencia suele implicar puntuar los nodos, a menudo con relaciones ponderadas, y luego actualizar las puntuaciones a lo largo de varias iteraciones. En ocasiones, se puntúan todos los nodos y, en otras, se utiliza una selección aleatoria como distribución representativa.



Tenga en cuenta que las medidas de centralidad representan la importancia de un nodo en comparación con otros. La centralidad es una clasificación del impacto potencial de los nodos, no una medida del impacto real. Por ejemplo, podría identificar a las dos personas con mayor centralidad en una red, pero quizás existan políticas o normas culturales que realmente transfieran la influencia a otros. Cuantificar el impacto real es un área de investigación activa para desarrollar métricas de influencia adicionales.

La fórmula de PageRank

PageRank se define en el documento original de Google de la siguiente manera:

$$PR(u) = \frac{1-d}{C(u)} + d \left(\frac{PR(T_1)}{C(T_1)} + \dots + \frac{PR(T_n)}{C(T_n)} \right)$$

dónde:

- Suponemos que una página u tiene citas de las páginas T_1 a T_n .
- d es un factor de amortiguamiento que se establece entre 0 y 1. Normalmente se establece en 0,85. Puede considerarse como la probabilidad de que un usuario siga haciendo clic. Esto ayuda a minimizar la pérdida de posicionamiento, como se explica en la siguiente sección.
- $1-d$ es la probabilidad de que se llegue a un nodo directamente sin seguir ninguna relación.
- relaciones.

$C(T_n)$ se define como el grado de salida de un nodo T .

La Figura 5-11 muestra un pequeño ejemplo de cómo PageRank continuará actualizando el rango de un nodo hasta que converja o alcance la cantidad establecida de iteraciones.

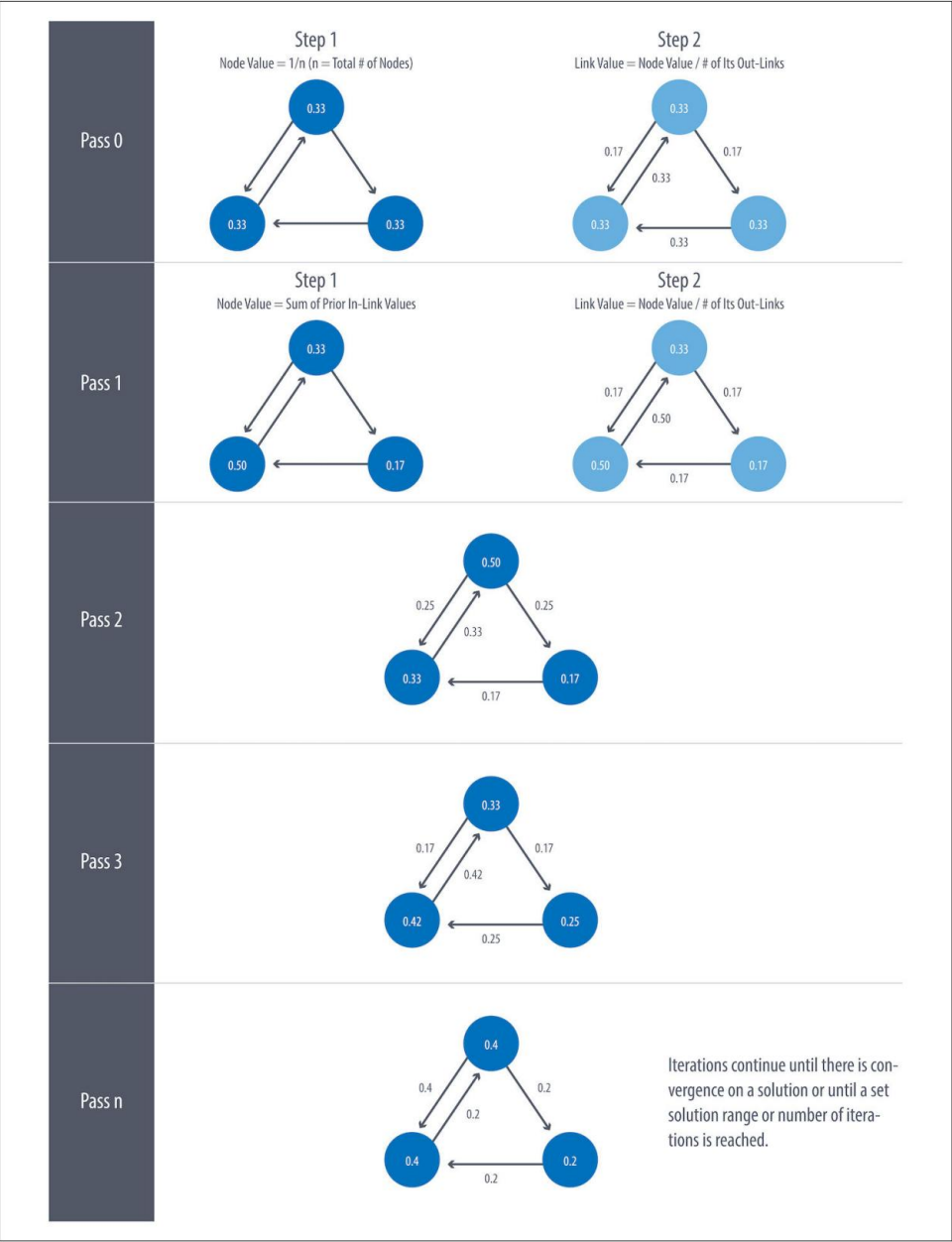


Figura 5-11. Cada iteración de PageRank consta de dos pasos de cálculo: uno para actualizar los valores de los nodos y otro para actualizar los valores de los enlaces.

Iteración, navegantes aleatorios y sumideros de rango

PageRank es un algoritmo iterativo que se ejecuta hasta que los puntajes convergen o hasta que se alcanza un número determinado de iteraciones.

Conceptualmente, PageRank asume que un usuario visita páginas web siguiendo enlaces o usando una URL aleatoria. Un factor de amortiguamiento d define la probabilidad de que el siguiente clic se realice a través de un enlace. Se puede considerar como la probabilidad de que un usuario se aburra y cambie a otra página aleatoriamente. Una puntuación de PageRank representa la probabilidad de que una página se visite a través de un enlace entrante y no aleatoriamente.

Un nodo o grupo de nodos sin relaciones salientes (también llamado nodo colgante) puede monopolizar la puntuación de PageRank al negarse a compartir. Esto se conoce como un sumidero de rango. Imagine esto como un usuario que se queda atascado en una página o en un subconjunto de páginas sin salida. Otra dificultad la crean los nodos que solo se apuntan entre sí dentro de un grupo. Las referencias circulares provocan un aumento en sus rangos a medida que el usuario rebota entre los nodos. Estas situaciones se representan en la [Figura 5-12](#).

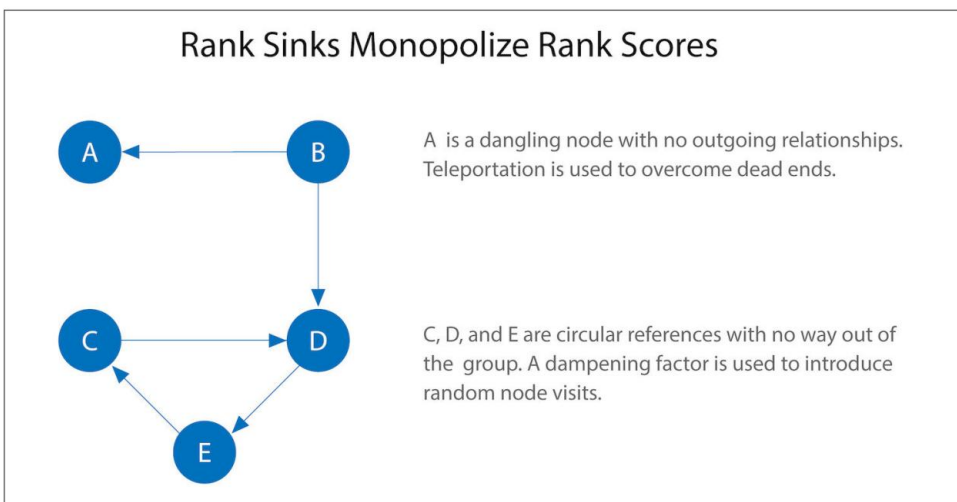


Figura 5-12. La caída de rango es causada por un nodo, o grupo de nodos, sin relaciones salientes.

Existen dos estrategias para evitar caídas de rango. Primero, cuando se llega a un nodo sin relaciones salientes, PageRank asume relaciones salientes con todos los nodos.

Atravesar estos enlaces invisibles a veces se denomina teletransportación. En segundo lugar, el factor de amortiguamiento ofrece otra oportunidad para evitar sumideros al introducir una probabilidad de enlace directo frente a la visita aleatoria a un nodo. Al establecer d en 0,85, se visita un nodo completamente aleatorio el 15 % del tiempo.

Aunque la fórmula original recomienda un factor de amortiguamiento de 0,85, su uso inicial se dio en la World Wide Web con una distribución de enlaces de ley de potencia (la mayoría de las páginas tienen muy pocos enlaces y algunas tienen muchos). Reducir el factor de amortiguamiento disminuye la probabilidad de seguir largas trayectorias de relación antes de dar un salto aleatorio. A su vez, esto aumenta la contribución de los predecesores inmediatos de un nodo a su puntuación y clasificación.

Si observa resultados inesperados de PageRank, vale la pena realizar un análisis exploratorio del gráfico para determinar si alguno de estos problemas es la causa. Lea el artículo de Ian Rogers, ["El algoritmo de PageRank de Google y cómo funciona"](#) para obtener más información.

¿Cuándo debo utilizar PageRank?

PageRank se utiliza actualmente en muchos ámbitos más allá de la indexación web. Utilice este algoritmo siempre que busque una amplia influencia en una red. Por ejemplo, si busca un gen con el mayor impacto general en una función biológica, puede que no sea el más conectado. De hecho, podría ser el gen con más relaciones con otras funciones más significativas.

Algunos ejemplos de casos de uso incluyen:

Presentar a los usuarios recomendaciones de otras cuentas que podrían querer seguir (Twitter utiliza PageRank personalizado para ello). El algoritmo se ejecuta sobre un gráfico que contiene intereses y conexiones comunes. El enfoque se describe con más detalle en el artículo [«WTF: El servicio a quién seguir en Twitter»](#), de P. Gupta et al.

Predicción del flujo vehicular y el movimiento humano en espacios públicos o calles. El algoritmo se ejecuta sobre un gráfico de intersecciones viales, donde la puntuación de PageRank refleja la tendencia de las personas a aparcar o finalizar su viaje en cada calle. Esto se describe con más detalle en ["Carreteras naturales autoorganizadas para la predicción del flujo vehicular: Un estudio de sensibilidad"](#), un artículo de B. Jiang, S. Zhao y J. Yin.

- Como parte de los sistemas de detección de anomalías y fraudes en los sectores de la salud y los seguros, PageRank ayuda a identificar médicos o proveedores con comportamientos inusuales, y las puntuaciones se incorporan a un algoritmo de aprendizaje automático.

David Gleich describe muchos más usos del algoritmo en su artículo ["PageRank Beyond the Web"](#).

PageRank con Apache Spark. Ahora

estamos listos para ejecutar el algoritmo PageRank. GraphFrames admite dos implementaciones de PageRank:

- La primera implementación ejecuta PageRank durante un número fijo de iteraciones. Esto se puede ejecutar configurando el parámetro `maxIter`.
- La segunda implementación ejecuta PageRank hasta la convergencia. Esto se puede ejecutar estableciendo el parámetro `tol`.

PageRank con un número fijo de iteraciones

Veamos un ejemplo del enfoque de iteraciones fijas:

```
resultados = g.pageRank(resetProbability=0.15, maxIter=20)
resultados.vertices.sort("pagerank", ascendente=Falso).show()
```



Observe que en Spark el factor de amortiguamiento se denomina, de forma más intuitiva, probabilidad de reinicio, con el valor inverso. En otras palabras, una probabilidad de reinicio de 0,15 en este ejemplo equivale a un factor de amortiguamiento de 0,85 en Neo4j.

Si ejecutamos ese código en pyspark veremos este resultado:

	PageRank
Doug	2.2865372087512252
Marca	2.1424484186137263
Alicia	1.520330830262095
Miguel	0.7274429252585624
Brígida	0.7274429252585624
Carlos	0.5213852310709753
Amy	0.5097143486157744
David	0.36655842368870073
Jaime	0.1981396884803788

Como era de esperar, Doug tiene el PageRank más alto porque todos los demás usuarios de su subgráfico lo siguen. Aunque Mark solo tiene un seguidor, este es Doug, por lo que Mark también se considera importante en este gráfico. No solo importa el número de seguidores, sino también la importancia de esos seguidores.



Las relaciones en el gráfico en el que ejecutamos el algoritmo PageRank no tienen pesos, por lo que cada relación se considera igual. Los pesos de relación se agregan especificando una columna de peso en el DataFrame de relaciones.

PageRank hasta la convergencia

Y ahora probemos la implementación de convergencia que ejecutará PageRank hasta que se acerque a una solución dentro de la tolerancia establecida:

```
resultados = g.pageRank(resetProbability=0.15, tol=0.01)
resultados.vertices.sort("pagerank", ascendente=Falso).show()
```

Si ejecutamos ese código en pyspark veremos este resultado:

	PageRank
Doug	2.223318885989745
Marca	2.090451188336932
Alicia	1.5056291439101062
Miguel	0.733738785109624
Bridget	0.733738785109624
Amy	0.559446807245026
Carlos	0.5338811076334145
David	0.40232326274180685
Jaime	0.21747203391449021

Las puntuaciones de PageRank de cada persona son ligeramente diferentes que con la variante con número fijo de iteraciones, pero como era de esperar, su orden sigue siendo el mismo.



Aunque la convergencia hacia una solución perfecta pueda parecer ideal, en algunos casos, PageRank no puede converger matemáticamente. Para gráficos grandes, la ejecución de PageRank puede ser prohibitivamente larga. Un límite de tolerancia ayuda a establecer un rango aceptable para un resultado convergente, pero muchos optan por usar (o combinar este enfoque con) la opción de iteración máxima. La configuración de iteración máxima generalmente proporciona una mayor consistencia en el rendimiento. Independientemente de la opción que elija, es posible que deba probar varios límites diferentes para encontrar el que funcione para su conjunto de datos. Los gráficos grandes suelen requerir más iteraciones o una tolerancia menor que los gráficos medianos para una mayor precisión.

PageRank con Neo4j. También

podemos ejecutar PageRank en Neo4j. Una llamada al siguiente procedimiento calculará el PageRank de cada nodo de nuestro gráfico:

```
LLAMA algo.pageRank.stream('Usuario', 'SIGUE', {iteraciones:20, factor de amortiguamiento:0.85})
YIELD nodeId, puntuación
RETURN algo.getNodeById(nodeId).id AS página, puntuación
ORDENAR POR puntuación DESC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

puntuación de la página	
Doug	1.6704119999999998
Marca	1.5610085
Alicia	1.1106700000000003
Bridget	0.535373
Michael	0.535373
Amy	0.385875
Charles	0.3844895
David	0.2775
Jaime	0.15000000000000002

Al igual que en el ejemplo de Spark, Doug es el usuario más influyente, seguido de cerca por Mark, el único usuario al que Doug sigue. La importancia relativa de los nodos se observa en la [Figura 5-13](#).



Las implementaciones de PageRank varían, por lo que pueden generar puntuaciones diferentes incluso con el mismo orden. Neo4j inicializa los nodos con un valor de 1 menos el factor de amortiguación, mientras que Spark usa un valor de 1. En este caso, las clasificaciones relativas (el objetivo de PageRank) son idénticas, pero los valores de puntuación subyacentes utilizados para alcanzar dichos resultados son diferentes.

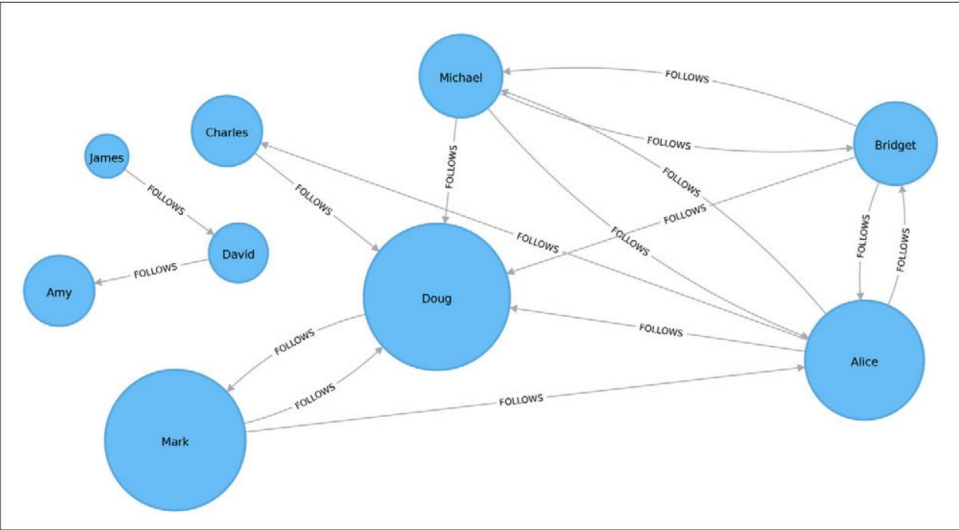


Figura 5-13. Visualización del PageRank



Al igual que en nuestro ejemplo de Spark, las relaciones en el grafo donde ejecutamos el algoritmo PageRank no tienen pesos, por lo que cada relación se considera igual. Los pesos de las relaciones se pueden considerar incluyendo la propiedad `weightProperty` en la configuración pasada al procedimiento PageRank. Por ejemplo, si las relaciones tienen una propiedad `weight` que contiene pesos, pasaríamos la siguiente configuración al procedimiento: `weightProperty: "weight"`.

Variación de PageRank: PageRank personalizado. El

PageRank personalizado (PPR) es una variante del algoritmo PageRank que calcula la importancia de los nodos en un grafo desde la perspectiva de un nodo específico. Para el PPR, los saltos aleatorios se refieren a un conjunto dado de nodos iniciales. Esto sesga los resultados hacia el nodo inicial o los personaliza. Este sesgo y la localización hacen que el PPR sea útil para recomendaciones muy específicas.

PageRank personalizado con Apache Spark.

Podemos calcular la puntuación de PageRank personalizada para un nodo determinado pasando el parámetro `sourceId`. El siguiente código calcula el PPR de Doug:

```
yo = "Doug"
resultados = g.pageRank(resetProbability=0.15, maxIter=20, sourceId=yo)
personas_a_seguir = resultados.vertices.sort("pagerank", ascendentemente=False)

ya_sigue = lista(g.edges.filter(f"src = '{me}'").toPandas()["dst"]) personas_a_excluir =
ya_sigue + [me]

personas_a_seguir[~personas_a_seguir.id.isin(personas_a_excluir)].mostrar()
```

Los resultados de esta consulta podrían usarse para recomendar personas a las que Doug debería seguir. Tenga en cuenta que también nos aseguramos de excluir del resultado final a las personas que Doug ya sigue, así como a él mismo.

Si ejecutamos ese código en pyspark veremos este resultado:

PageRank	
Alicia	0.1650183746272782
Miguel	0.048842467744891996
Brígida	0.048842467744891996
Carlos	0.03497796119878669
David	0.0
Jaime	0.0
Amy	0.0

Alice es la mejor sugerencia que Doug debería seguir, pero también podríamos sugerirle a Michael y a Bridget.

Resumen

Los algoritmos de centralidad son una herramienta excelente para identificar influenciadores en una red. En este capítulo, hemos aprendido sobre los algoritmos de centralidad prototípicos: Centralidad de Grado, Centralidad de Cercanía, Centralidad de Intermediación y PageRank. También hemos abordado diversas variaciones para abordar problemas como tiempos de ejecución prolongados y componentes aislados, así como opciones para usos alternativos.

Los algoritmos de centralidad tienen una amplia gama de usos, y recomendamos explorarlos para diversos análisis. Puede aplicar lo aprendido para identificar los puntos de contacto óptimos para la difusión de información, identificar a los intermediarios ocultos que controlan el flujo de recursos y descubrir a los actores de poder indirectos que se esconden en las sombras.

A continuación, nos centraremos en los algoritmos de detección de comunidades que analizan grupos y particiones.

Algoritmos de detección de comunidades

La formación de comunidades es común en todo tipo de redes, e identificarlas es esencial para evaluar el comportamiento grupal y los fenómenos emergentes. El principio general para encontrar comunidades es que sus miembros tendrán más relaciones dentro del grupo que con nodos externos. La identificación de estos conjuntos relacionados revela grupos de nodos, grupos aislados y la estructura de la red. Esta información ayuda a inferir comportamientos o preferencias similares de grupos de pares, estimar la resiliencia, encontrar relaciones anidadas y preparar datos para otros análisis. Los algoritmos de detección de comunidades también se utilizan comúnmente para generar visualizaciones de redes para una inspección general.

Proporcionaremos detalles sobre los algoritmos de detección de comunidades más representativos:

- Recuento de triángulos y coeficiente de agrupamiento para la densidad general de relaciones •

Componentes fuertemente conectados y componentes conectados para encontrar con
clústeres conectados

- Propagación de etiquetas para inferir rápidamente grupos basados en etiquetas de nodos •

Modularidad de Lovain para observar la calidad y las jerarquías de agrupamiento

Explicaremos cómo funcionan los algoritmos y mostraremos ejemplos en Apache Spark y Neo4j. Si un algoritmo solo está disponible en una plataforma, proporcionaremos un solo ejemplo. Utilizamos relaciones ponderadas para estos algoritmos porque suelen utilizarse para capturar la importancia de diferentes relaciones.

La [Figura 6-1](#) brinda una descripción general de las diferencias entre los algoritmos de detección de la comunidad tratados aquí, y la [Tabla 6-1](#) proporciona una referencia rápida de lo que calcula cada algoritmo con ejemplos de uso.

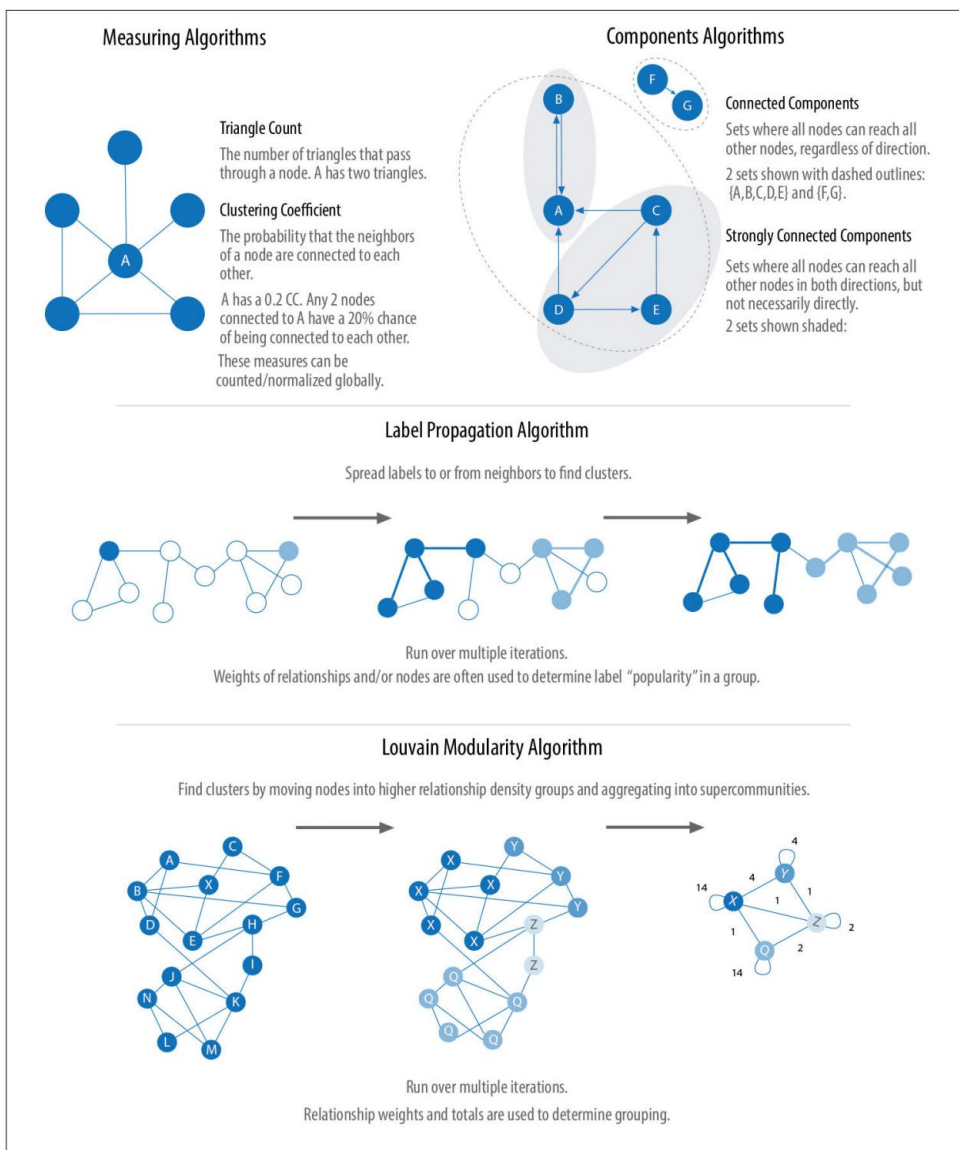


Figura 6-1. Algoritmos representativos de detección de comunidades



Usamos los términos conjunto, partición, clúster, grupo y comunidad indistintamente. Estos términos indican que se pueden agrupar nodos similares. Los algoritmos de detección de comunidades también se denominan algoritmos de agrupamiento y partición. En cada sección, utilizamos los términos más comunes en la literatura para cada algoritmo en particular.

Tabla 6-1. Resumen de los algoritmos de detección de comunidades

Tipo de algoritmo	Qué hace	Ejemplo de uso	Ejemplo de Spark	Ejemplo de Neo4j
Conteo de triángulos y Coeficiente de agrupamiento	Mide cuántos nodos forman triángulos y el grado en que los nodos tienden a agruparse.	Estimación de la estabilidad del grupo y si la red podría exhibir comportamientos de "mundo pequeño" observados en gráficos con grupos muy unidos	Sí	Sí
Fuertemente conectado Componentes	Encuentra grupos en los que cada nodo es accesible desde cualquier otro nodo del mismo grupo siguiendo la dirección de las relaciones.	Realizar recomendaciones de productos en función de la afiliación a un grupo o artículos similares	Sí	Sí
Componentes conectados	Encuentra grupos donde cada nodo es accesible desde cualquier otro nodo del mismo grupo, independientemente de la dirección de las relaciones.	Realizar agrupaciones rápidas para otros algoritmos e identificar islas	Sí	Sí
Propagación de etiquetas	Infiere grupos al difundir etiquetas basadas en las mayorías del vecindario	Comprender el consenso en las comunidades sociales o encontrar combinaciones peligrosas de posibles fármacos recetados conjuntamente	Sí	Sí
Modularidad de Lovaina	Maximiza la precisión presunta de las agrupaciones al comparar los pesos y densidades de las relaciones con una estimación o promedio definido.	En el análisis de fraude, se evalúa si un grupo tiene solo unos pocos malos comportamientos discretos o está actuando como una red de fraude.	No	Sí

Primero, describiremos los datos de nuestros ejemplos y explicaremos cómo importarlos a Spark y Neo4j. Los algoritmos se describen en el orden indicado en la **Tabla 6-1**. Para cada uno, encontrará una breve descripción y consejos sobre cuándo usarlo. La mayoría de las secciones también incluyen instrucciones sobre cuándo usar algoritmos relacionados. Al final de cada sección de algoritmo, mostramos código de ejemplo con datos de muestra.



Al utilizar algoritmos de detección de comunidades, tenga en cuenta la densidad de las relaciones.

Si el grafo es muy denso, es posible que todos los nodos se congreguen en uno o solo unos pocos clústeres. Puedes contrarrestar esto filtrando por grado, ponderaciones de relación o métricas de similitud.

Por otro lado, si el grafo es demasiado disperso, con pocos nodos conectados, podría terminar con cada nodo en su propio clúster. En este caso, intente incorporar tipos de relación adicionales que contengan información más relevante.

Ejemplo de datos gráficos: El gráfico de dependencia del software

Los gráficos de dependencia son particularmente adecuados para demostrar que, a veces, Existen diferencias sutiles entre los algoritmos de detección de comunidades porque tienden a ser más conectados y jerárquicos. Los ejemplos de este capítulo se comparan con un gráfico que contiene dependencias entre bibliotecas de Python, aunque la dependencia Los gráficos se utilizan en diversos campos, desde el software hasta las redes energéticas. Este tipo de software... Los desarrolladores utilizan el gráfico de dependencia para realizar un seguimiento de las interdependencias transitivas. y conflictos en proyectos de software. Puede descargar los nodos y archivos desde [repositorio de GitHub del libro](#).

Tabla 6-2. sw-nodes.csv

nombre
seis
pandas
Numpy
python-dateutil
pytz
chispa py
matplotlib
espacioso
py4j
Jupyter
consola jpy
nbconvert
ipykernel
cliente jpy
jpy-core

Tabla 6-3. sw-relationships.csv

origen	destino	relación
pandas	Numpy	DEPENDE DE
pandas	pytz	DEPENDE DE
pandas	python-dateutil	DEPENDE_DE
python-dateutil	seis	DEPENDE DE
pyspark	py4j matplotlib	DEPENDE DE
matplotlib	Numpy	DEPENDE DE
matplotlib	python-dateutil	DEPENDE_DE
	seis	DEPENDE DE

origen	destino	relación
matplotlib	pytz	DEPENDS DE
espaciosos	seis	DEPENDS DE
espaciosos	conversión	DEPENDS DE
Jupyter	nb de numpy	DEPENDS DE
Jupyter	ipykernel	DEPENDS DE
Jupyter	consola jpy	DEPENDS DE
consola jpy	cliente jpy	DEPENDS DE
consola jpy	ipykernel	DEPENDS DE
cliente jpy	núcleo	DEPENDS DE
nbconvert	jpy núcleo jpy	DEPENDS DE

La Figura 6-2 muestra el gráfico que queremos construir. Al observar este gráfico, vemos Hay tres grupos de bibliotecas. Podemos usar visualizaciones en conjuntos de datos más pequeños. como herramienta para ayudar a validar los clústeres derivados por los algoritmos de detección de comunidades.

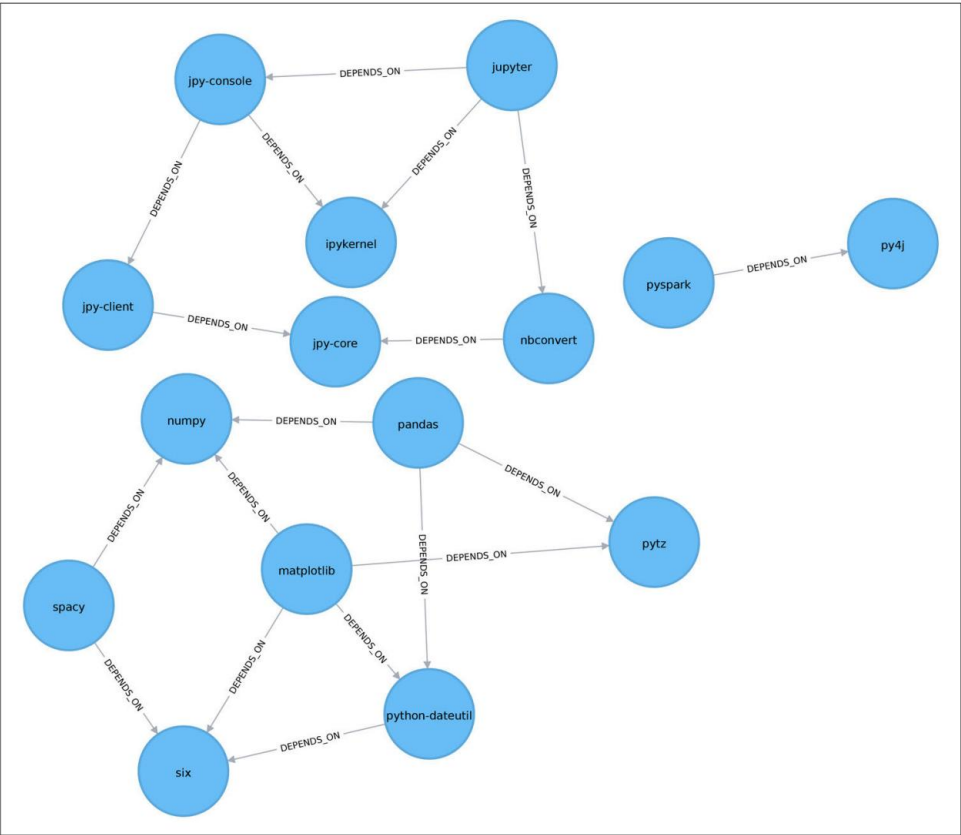


Figura 6-2. El modelo gráfico

Creemos gráficos en Spark y Neo4j a partir de los archivos CSV de ejemplo.

Importación de datos a Apache Spark

Primero importaremos los paquetes que necesitamos de Apache Spark y el paquete GraphFrames:

```
desde graphframes importar *
```

La siguiente función crea un GraphFrame a partir de los archivos CSV de ejemplo:

```
def crear_software_graph():
    nodos = spark.read.csv("data/sw-nodes.csv", header=True) relaciones =
    spark.read.csv("data/sw-relationships.csv", header=True) return GraphFrame(nodos, relaciones)
```

Ahora llamemos a esa función:

```
g = crear_software_graph()
```

Importando los datos a Neo4j. A continuación,

haremos lo mismo para Neo4j. La siguiente consulta importa los nodos:

```
CON "https://github.com/neo4j-graph-analytics/book/raw/master/data/" COMO base CON base + "sw-nodes.csv" COMO
uri CARGAR CSV CON ENCABEZADOS DESDE
uri COMO fila FUSIONAR (:Library {id: row.id})
```

Y esto importa las relaciones:

```
CON "https://github.com/neo4j-graph-analytics/book/raw/master/data/" COMO base CON base + "sw-relationships.csv"
COMO uri CARGAR CSV CON ENCABEZADOS DE uri COMO
fila COINCIDIR (fuente:Library {id: row.src})

COINCIDIR (destino:Biblioteca {id: row.dst})
FUSIONAR (origen)-[:DEPENDS_ON]->(destino)
```

¡Ahora que tenemos nuestros gráficos cargados, pasemos a los algoritmos!

Recuento de triángulos y coeficiente de agrupamiento

Los algoritmos de Conteo de Triángulos y Coeficiente de Agrupamiento se presentan juntos debido a su frecuente uso conjunto. El Conteo de Triángulos determina el número de triángulos que pasan por cada nodo del grafo. Un triángulo es un conjunto de tres nodos, donde cada nodo tiene una relación con los demás. El Conteo de Triángulos también puede ejecutarse globalmente para evaluar el conjunto de datos.



Las redes con un gran número de triángulos tienen más probabilidades de exhibir estructuras y comportamientos de mundo pequeño.

El objetivo del algoritmo de Coeficiente de Agrupamiento es medir la afinidad de un grupo en comparación con su posible agrupamiento. El algoritmo utiliza el Conteo de Triángulos en sus cálculos, que proporciona una relación entre los triángulos existentes y las posibles relaciones. Un valor máximo de 1 indica una camarilla donde cada nodo está conectado con todos los demás.

Hay dos tipos de coeficientes de agrupamiento: agrupamiento local y agrupamiento global.

Coeficiente de agrupamiento local. El

coeficiente de agrupamiento local de un nodo indica la probabilidad de que sus vecinos también estén conectados. El cálculo de esta puntuación implica el conteo de triángulos.

El coeficiente de agrupamiento de un nodo se puede encontrar multiplicando el número de triángulos que pasan por el nodo por dos y luego dividiendo ese resultado por el número máximo de relaciones en el grupo, que siempre es el grado de ese nodo, menos uno.

En la figura 6-3 se muestran ejemplos de diferentes triángulos y coeficientes de agrupamiento para un nodo con cinco relaciones.

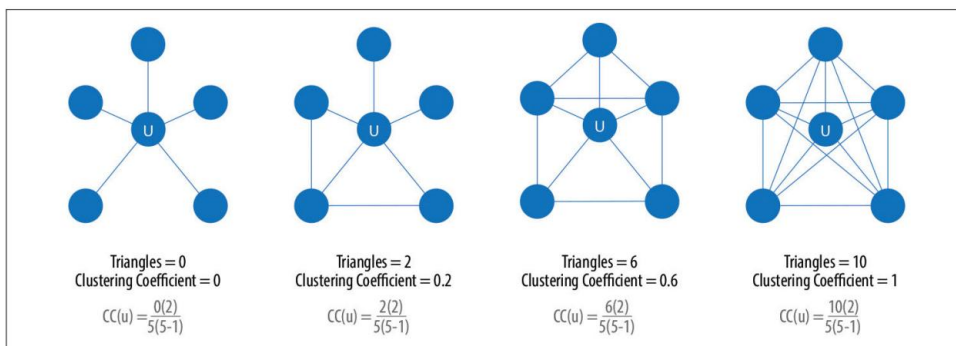


Figura 6-3. Recuentos de triángulos y coeficientes de agrupamiento para el nodo u

Observe que en la Figura 6-3 usamos un nodo con cinco relaciones, lo que indica que el coeficiente de agrupamiento siempre será igual al 10 % del número de triángulos. Podemos observar que esto no es así al modificar el número de relaciones. Si modificamos el segundo ejemplo para tener cuatro relaciones (y los mismos dos triángulos), el coeficiente es 0,33.

El coeficiente de agrupamiento de un nodo utiliza la fórmula:

$$CC(u) = \frac{2R_{tu}}{k_u(k_u - 1)}$$

dónde:

- u es un nodo.
- $R(u)$ es el número de relaciones a través de los vecinos de u (esto se puede obtener utilizando el número de triángulos que pasan por u).
- $k(u)$ es el grado de u .

Coefficiente de agrupamiento global E_l

coeficiente de agrupamiento global es la suma normalizada de los coeficientes de agrupamiento locales.

Los coeficientes de agrupamiento nos brindan un medio efectivo para encontrar grupos obvios como camarillas, donde cada nodo tiene una relación con todos los demás nodos, pero también podemos especificar umbrales para establecer niveles (por ejemplo, donde los nodos están conectados en un 40%).

¿Cuándo debo utilizar el recuento de triángulos y el coeficiente de agrupamiento?

Utilice el conteo de triángulos cuando necesite determinar la estabilidad de un grupo o como parte del cálculo de otras medidas de red, como el coeficiente de agrupamiento. El conteo de triángulos es popular en el análisis de redes sociales, donde se utiliza para detectar comunidades.

El coeficiente de agrupamiento puede proporcionar la probabilidad de que nodos seleccionados aleatoriamente estén conectados. También puede utilizarse para evaluar rápidamente la cohesión de un grupo específico o de la red en general. Juntos, estos algoritmos se utilizan para estimar la resiliencia y buscar estructuras de red.

Algunos ejemplos de casos de uso incluyen:

- Identificación de características para clasificar un sitio web como spam. Esto se describe en el artículo «Algoritmos eficientes de semitransmisión para el conteo de triángulos locales en grafos masivos», de L. Becchetti et al.
- Investigación de la estructura comunitaria del grafo social de Facebook, donde los investigadores encontraron densas comunidades de usuarios en un grafo global por lo demás disperso. Encuentre este estudio en el artículo "La anatomía del grafo social de Facebook", de J. Ugander et al.
- Exploración de la estructura temática de la web y

detección de comunidades de páginas con temas comunes, basándose en los vínculos recíprocos entre ellas.

Para más información, consulte "La curvatura de los coenlaces revela capas temáticas ocultas en la World Wide Web", de J.-P. Eckmann y E. Moses.

Conteo de triángulos con Apache Spark

Ahora estamos listos para ejecutar el algoritmo de conteo de triángulos. Podemos usar lo siguiente: Código para hacer esto:

```
resultado = g.triangleCount()  
(resultado.sort("count", ascendente=False)  
  .filter('count > 0')  
  .espectáculo())
```

Si ejecutamos ese código en pyspark veremos este resultado:

Identificación del conteo	
1	Jupyter
1	python-dateutil
1	seis
1	ipykernel
1	matplotlib
1	consola jpy

Un triángulo en este gráfico indicaría que dos de los vecinos de un nodo también son vecinos. Bors. Seis de nuestras bibliotecas participan en estos triángulos.

¿Y si queremos saber qué nodos hay en esos triángulos? Ahí es donde un triángulo... Entra el stream. Para esto, necesitamos Neo4j.

Triángulos con Neo4j

Obtener un flujo de triángulos no está disponible usando Spark, pero podemos devolverlo usando Neo4j:

```
LLAMA a algo.triangle.stream("Biblioteca","DEPENDENCIA")  
YIELD nodoA, nodoB, nodoC  
DEVOLVER algo.getNodeById(nodoA).id COMO nodoA,  
  algo.getNodeById(nodoB).id COMO nodoB,  
  algo.getNodeById(nodoC).id COMO nodoC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

nodoA	nodoB	nodoC
matplotlib	seis	python-dateutil
Jupyter	consola jpy	ipykernel

Vemos las mismas seis bibliotecas que antes, pero ahora sabemos cómo están conectadas. ted. matplotlib, six y python-dateutil forman un triángulo. jupyter, jpy-console y ipykernel forma el otro.

Podemos ver estos triángulos visualmente en la Figura 6-4.

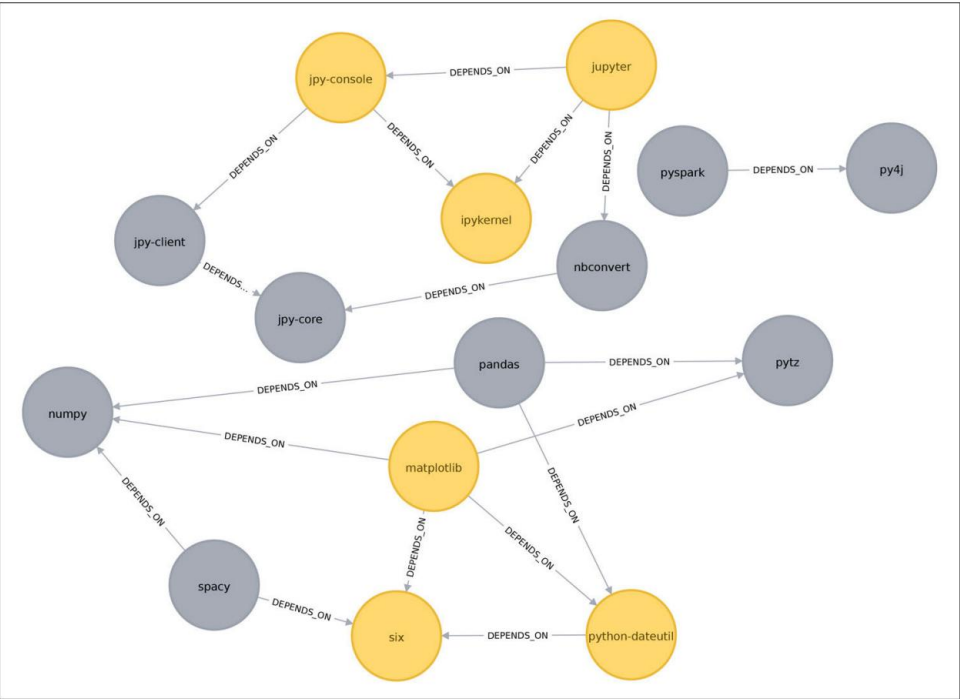


Figura 6-4. Triángulos en el gráfico de dependencia del software

Coeficiente de agrupamiento local con Neo4j

También podemos calcular el coeficiente de agrupamiento local. La siguiente consulta calculará...
Atrasa esto para cada nodo:

```
LLAMA algo.triangleCount.stream('Biblioteca', 'DEPENDS_ON')
YIELD nodeld, triángulos, coeficiente
DONDE coeficiente > 0
RETORNA algo.getNodeById(nodeld).id AS biblioteca, coeficiente
ORDENAR POR coeficiente DESC
```

Al ejecutar este procedimiento se obtendrá el siguiente resultado:

biblioteca	coeficiente
ipykernel	1.0
Jupyter	0.3333333333333333
consola jpy	0.3333333333333333
seis	0.3333333333333333
python-dateutil	0.3333333333333333

```
coeficiente
matplotlib 0.16666666666666666
```

ipykernel tiene una puntuación de 1, lo que significa que todos sus vecinos son vecinos entre sí. Esto se puede observar claramente en la [Figura 6-4](#). Esto indica que la comunidad directamente relacionada con ipykernel es muy cohesiva.

En este ejemplo de código, hemos filtrado los nodos con una puntuación de coeficiente de 0, pero los nodos con coeficientes bajos también pueden ser interesantes. Una puntuación baja puede indicar que un nodo es un **agujero estructural**: un nodo bien conectado con nodos de diferentes comunidades que, de otro modo, no estarían conectados entre sí. Este es un método para encontrar posibles puentes que analizamos en el [capítulo 5](#).

Componentes fuertemente conectados

El algoritmo de Componentes Fuertemente Conectados (SCC) es uno de los primeros algoritmos de grafos. SCC encuentra conjuntos de nodos conectados en un grafo dirigido, donde cada nodo es accesible en ambas direcciones desde cualquier otro nodo del mismo conjunto. Sus operaciones en tiempo de ejecución escalan adecuadamente, proporcionalmente al número de nodos. En la [Figura 6-5](#) se puede observar que los nodos de un grupo SCC no necesitan ser vecinos inmediatos, pero deben existir rutas direccionales entre todos los nodos del conjunto.

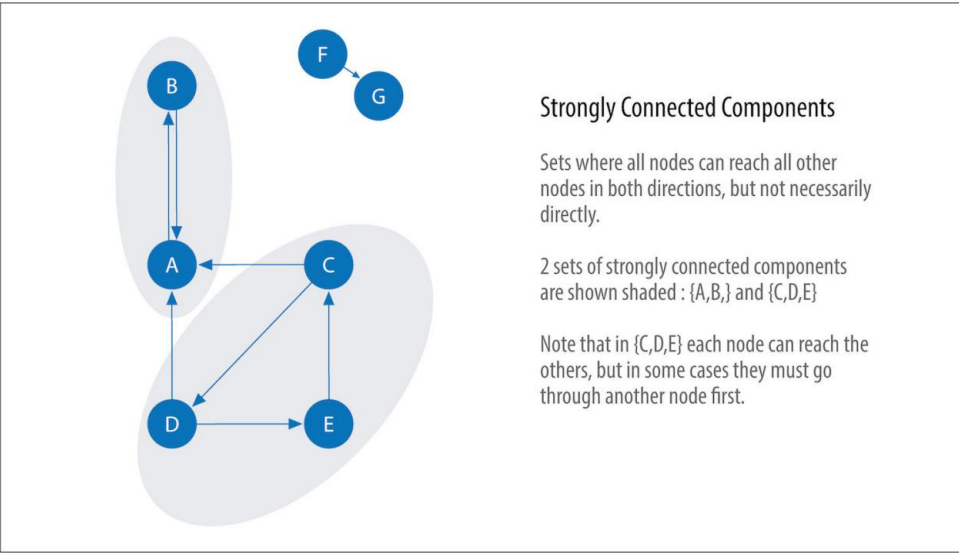


Figura 6-5. Componentes fuertemente conectados



La descomposición de un gráfico dirigido en sus componentes fuertemente conectados es una aplicación clásica del **algoritmo de búsqueda en profundidad**. Neo4j utiliza DFS bajo el capó como parte de su implementación del algoritmo SCC.

¿Cuándo debo utilizar componentes fuertemente conectados?

Utilice los Componentes Fuertemente Conectados como un paso inicial en el análisis de grafos para ver cómo está estructurado un grafo o para identificar grupos compactos que puedan justificar una investigación independiente. Un componente fuertemente conectado puede utilizarse para perfilar comportamientos o inclinaciones similares en un grupo para aplicaciones como los motores de recomendación.

Muchos algoritmos de detección de comunidades, como SCC, se utilizan para encontrar y agrupar clústeres en nodos individuales para un posterior análisis interclúster. También se puede usar SCC para visualizar ciclos en análisis, como la detección de procesos que podrían bloquearse porque cada subproceso espera la intervención de otro miembro.

Algunos ejemplos de casos de uso incluyen:

- Encontrar el conjunto de empresas en las que cada miembro posee directa y/o indirectamente acciones de todos los demás miembros, como en "**La red de control corporativo global**", un análisis de poderosas corporaciones transnacionales realizado por S. Vitali, JB Glattfelder y S. Battiston.
- Cálculo de la conectividad de diferentes configuraciones de red al medir el rendimiento del enrutamiento en redes inalámbricas multisalto. Lea más en "**Rendimiento del enrutamiento en presencia de enlaces unidireccionales en redes inalámbricas multisalto**", de MK Marina y SR Das.
- Actúa como primer paso en muchos algoritmos

de grafos que solo funcionan en grafos fuertemente conectados. En las redes sociales encontramos muchos grupos fuertemente conectados. En estos grupos, las personas suelen tener preferencias similares, y el algoritmo SCC se utiliza para encontrar dichos grupos y sugerir páginas que les gusten o productos que comprar a quienes aún no lo han hecho.



Algunos algoritmos tienen estrategias para escapar de bucles infinitos, pero si estamos escribiendo nuestros propios algoritmos o encontrando procesos no terminantes, podemos usar SCC para verificar ciclos.

Componentes fuertemente conectados con Apache Spark

Para comenzar con Apache Spark, primero importaremos los paquetes que necesitamos de Spark y el paquete GraphFrames:

desde `graphframes` importar *

desde `pyspark.sql` importar funciones como `F`

Ahora estamos listos para ejecutar el algoritmo de Componentes Fuertemente Conexas. Lo usaremos para determinar si existen dependencias circulares en nuestro grafo.



Dos nodos solo pueden estar en el mismo componente fuertemente conectado si hay caminos entre ellos en ambas direcciones.

Para ello escribimos el siguiente código:

```
resultado = g.stronglyConnectedComponents(maxIter=10)
```

```
(resultado.sort("componente") .groupby("componente") .agg(F.collect_list("id").alias("bibliotecas"))) .show(truncar=False))
```

Si ejecutamos ese código en pyspark veremos este resultado:

componente	bibliotecas
180388626432	[núcleo jpy]
223338299392	[espacioso]
498216206336	[numpy]
523986010112	[seis]
549755813888	[pandas]
558345748480	[nbconvert]
661424963584	[ipykernel]
721554505728	[jupyter]
764504178688	[ipy-client]
833223655424	[pytz]
910533066752	[python-dateutil]
936302870528	[pyspark]
944892805120	[matplotlib]
1099511627776	[jpy-console]
1279900254208	[py4j]

Podrás observar que cada nodo de la biblioteca está asignado a un componente único. Este es la partición o subgrupo al que pertenece, y como esperábamos, cada nodo está en su propia partición. Esto significa que nuestro proyecto de software no tiene dependencias circulares entre estas bibliotecas.

Componentes fuertemente conectados con Neo4j

Ejecutemos el mismo algoritmo con Neo4j. Ejecutemos la siguiente consulta para ejecutar el algoritmo:

```
LLAMA algo.scc.stream("Biblioteca", "DEPENDS_ON")
YIELD nodeId, partición
DEVUELVE partición, recopila(algo.getNodeById(nodeId)) COMO bibliotecas
ORDENAR POR tamaño(bibliotecas) DESC
```

Los parámetros que se pasan a este algoritmo son:

Biblioteca

La etiqueta del nodo que se cargará desde el gráfico

DEPENDS DE

El tipo de relación que se cargará desde el gráfico

Este es el resultado que veremos cuando ejecutemos la consulta:

bibliotecas de particiones	
8	[ipykernel]
11	[seis]
2	[matplotlib]
5	[jupyter]
14	[python-dateutil]
13	[numpy]
4	[py4j]
7	[nbconvert]
1	[chispa py]
10	(nócleo del yen japonés)
9	[cliente jpy]
3	[espacioso]
12	[pandas]
6	[jpy-consola]
0	[pytz]

Al igual que en el ejemplo de Spark, cada nodo está en su propia partición.

Hasta ahora el algoritmo sólo ha revelado que nuestras bibliotecas de Python se comportan muy bien, pero creemos una dependencia circular en el gráfico para hacer las cosas más interesantes. Esto debería significar que terminaremos con algunos nodos en la misma partición.

La siguiente consulta agrega una biblioteca adicional que crea una dependencia circular entre py4j y pyspark:

```
COINCIDIR (py4j:Library {id: "py4j"})
COINCIDIR (pyspark:Library {id: "pyspark"})
FUSIONAR (extra:Biblioteca {id: "extra"})
FUSIONAR (py4j)-[:DEPENDENCE_DE]->(extra)
FUSIONAR (extra)-[:DEPENDENCE_DE]->(pyspark)
```

Podemos ver claramente la dependencia circular que se creó en la Figura 6-6.

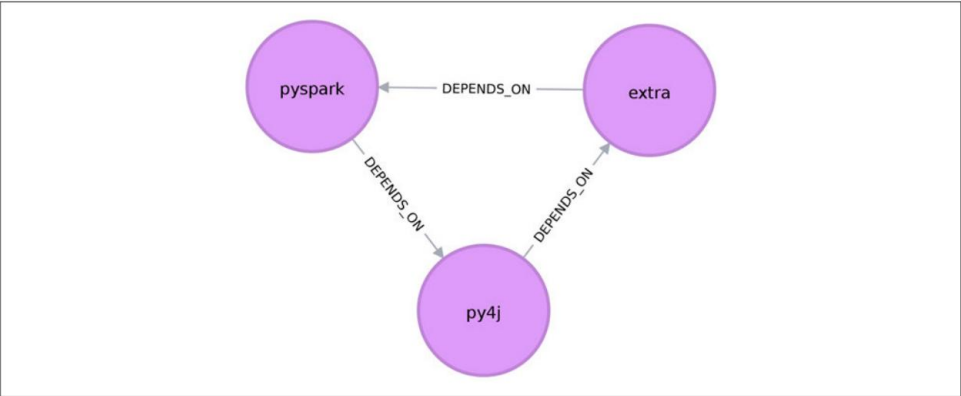


Figura 6-6. Una dependencia circular entre pyspark, py4j y extra

Ahora, si ejecutamos nuevamente el algoritmo SCC, veremos un resultado ligeramente diferente:

bibliotecas de particiones	
1	[pyspark, py4j, extra]
8	[ipykernel]
11	[seis]
2	[matplotlib]
5	[jupyter]
14	[numpy]
13	[pandas]
7	[nbconvert]
10	[núcleo del yen japonés]
9	[cliente jpy]
3	[espacioso]
15	[python-dateutil]
6	[jpy-consola]
0	[pytz]

Pyspark, Py4j y Extra son todos parte de la misma partición, y los SCC nos ayudaron a encontrarlos.
¡La dependencia circular!

Antes de pasar al siguiente algoritmo, eliminaremos la biblioteca adicional y sus relaciones del gráfico:

```
COINCIDIR (extra:Biblioteca {id: "extra"})
DESPRENDA ELIMINAR extra
```

Componentes conectados

El algoritmo de Componentes Conexos (a veces llamado Búsqueda de Unión o Componentes Débilmente Conexos) encuentra conjuntos de nodos conectados en un grafo no dirigido, donde cada nodo es accesible desde cualquier otro nodo del mismo conjunto. Se diferencia del algoritmo SCC porque solo requiere que exista una ruta entre pares de nodos en una dirección, mientras que el SCC requiere que exista una ruta en ambas direcciones. Bernard A. Galler y Michael J. Fischer describieron este algoritmo por primera vez en su artículo de 1964, «[Un Algoritmo de Equivalencia Mejorado](#)».

¿Cuándo debo utilizar componentes conectados?

Al igual que con SCC, los Componentes Conectados se suelen usar al principio del análisis para comprender la estructura de un grafo. Dado su escalabilidad eficiente, considere este algoritmo para grafos que requieren actualizaciones frecuentes. Puede mostrar rápidamente nuevos nodos en común entre grupos, lo cual resulta útil para análisis como la detección de fraudes.

Acostúmbrese a ejecutar Componentes Conectados para comprobar si un grafo está conectado, como paso preparatorio para el análisis general de grafos. Realizar esta prueba rápida puede evitar ejecutar algoritmos accidentalmente en un solo componente desconectado de un grafo y obtener resultados incorrectos.

Algunos ejemplos de casos de uso incluyen:

- Realizar un seguimiento de clústeres de registros de bases de datos, como parte del proceso de deduplicación. La deduplicación es una tarea importante en las aplicaciones de gestión de datos maestros; el enfoque se describe con más detalle en «[Un algoritmo eficiente e independiente del dominio para detectar registros de bases de datos aproximadamente duplicados](#)», de A. Monge y C. Elkan.
- Analizar redes de citas.

Un estudio utiliza componentes conectados para determinar qué tan bien conectada está una red y luego ver si la conectividad se mantiene si los nodos “centrales” o “de autoridad” se mueven del grafo. Este caso de uso se explica con más detalle en «[Caracterización y minería de grafos de citas de la literatura informática](#)», un artículo de Y. An, JCM Janssen y EE Milios.

Componentes conectados con Apache Spark

Para comenzar con Apache Spark, primero importaremos los paquetes que necesitamos de Spark y el paquete GraphFrames:

desde `pyspark.sql` importar funciones como `F`

Ahora estamos listos para ejecutar el algoritmo de componentes conectados.



Dos nodos pueden estar en el mismo componente conectado si hay una ruta entre ellos en cualquier dirección.

Para ello escribimos el siguiente código:

```
resultado = g.connectedComponents()
```

```
(resultado.sort("componente") .groupby("componente") .agg(F.collect_list("id").alias("bibliotecas"))) .show(truncar=False))
```

Si ejecutamos ese código en pyspark veremos este resultado:

bibliotecas de componentes

180388626432 [jpy-core, nbconvert, ipykernel, jupyter, jpy-client, jpy-console]

223338299392 [spacy, numpy, six, pandas, pytz, python-dateutil, matplotlib]

936302870528 [pyspark, py4j]

Los resultados muestran tres grupos de nodos, que también pueden verse en [la Figura 6-7](#).

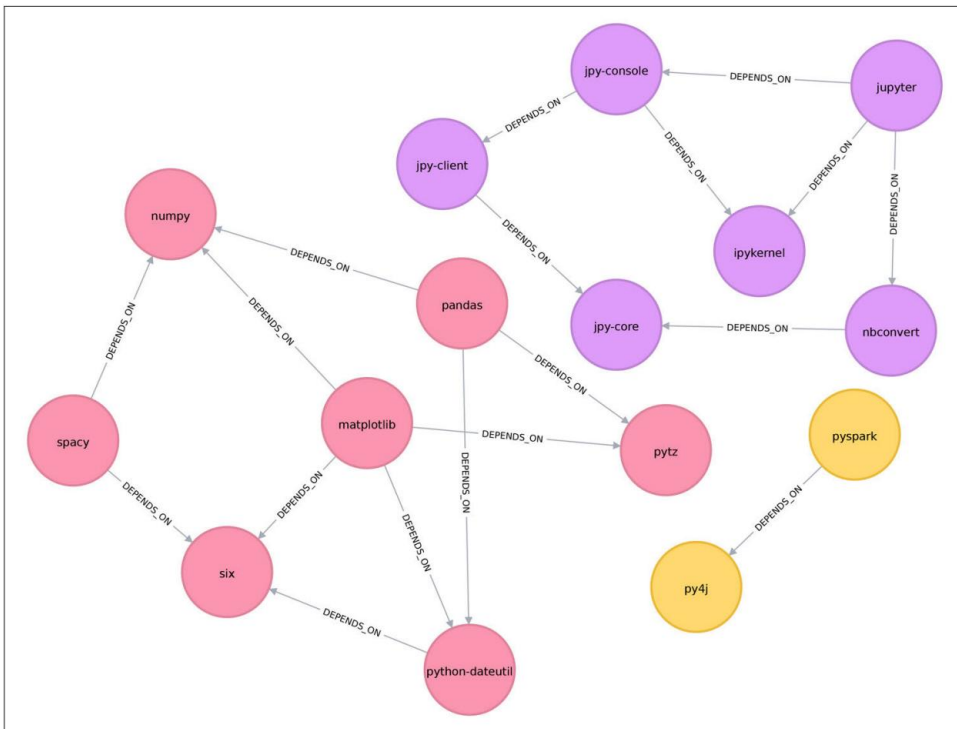


Figura 6-7. Clústeres encontrados por el algoritmo de Componentes Conectados

En este ejemplo, es muy fácil ver que hay tres componentes con solo una inspección visual. Este algoritmo muestra su valor en gráficos más grandes, donde la inspección visual no es posible o requiere mucho tiempo.

Componentes conectados con Neo4j

También podemos ejecutar este algoritmo en Neo4j ejecutando la siguiente consulta:

```

LLAMA a algo.unionFind.stream("Biblioteca", "DEPENDS_ON")
YIELD nodeId,setId
RETURN setId, collect(algo.getNodeById(nodeId)) AS bibliotecas ORDENAR
POR tamaño(bibliotecas) DESC

```

Los parámetros que se pasan a este algoritmo son:

Biblioteca

La etiqueta del nodo que se cargará desde el gráfico

DEPENDS DE

El tipo de relación que se cargará desde el gráfico

Aquí está el resultado:

bibliotecas setId

2	[Pytz, matplotlib, spacy, six, pandas, numpy, python-dateutil]
5	[Jupyter, jpy-console, nbconvert, ipykernel, jpy-client, jpy-core]
1	[Pyspark, py4j]

Como era de esperar, obtenemos exactamente los mismos resultados que con Spark.

Los dos algoritmos de detección de comunidades que hemos analizado hasta ahora son deterministas: arrojan los mismos resultados cada vez que los ejecutamos. Los dos siguientes algoritmos son ejemplos de algoritmos no deterministas, en los que podemos obtener resultados diferentes si los ejecutamos varias veces, incluso con los mismos datos.

Propagación de etiquetas

El algoritmo de Propagación de Etiquetas (LPA) es un algoritmo rápido para encontrar comunidades en un grafo. En LPA, los nodos seleccionan su grupo basándose en sus vecinos directos. Este proceso es ideal para redes donde las agrupaciones son menos claras y las ponderaciones pueden utilizarse para ayudar a un nodo a determinar en qué comunidad ubicarse. También es ideal para el aprendizaje semisupervisado, ya que se puede inicializar el proceso con etiquetas de nodo indicativas preasignadas.

La intuición detrás de este algoritmo es que una sola etiqueta puede volverse dominante rápidamente en un grupo de nodos densamente conectados, pero tendrá dificultades para cruzar una región escasamente conectada. Las etiquetas quedan atrapadas dentro de un grupo de nodos densamente conectados, y los nodos que terminan con la misma etiqueta al finalizar el algoritmo se consideran parte de la misma comunidad. El algoritmo resuelve las superposiciones, donde los nodos potencialmente forman parte de múltiples clústeres, asignando la pertenencia al vecindario de etiquetas con la relación combinada más alta y el mayor peso del nodo.

LPA es un algoritmo relativamente nuevo propuesto en 2007 por UN Raghavan, R. Albert y S. Kumara, en un artículo titulado **“Algoritmo de tiempo casi lineal para detectar estructuras comunitarias en redes a gran escala”**.

La Figura 6-8 muestra dos variantes de la Propagación de Etiquetas: un método de inserción simple y el método de extracción más común, que se basa en pesos de relación. El método de extracción se presta bien a la paralelización.

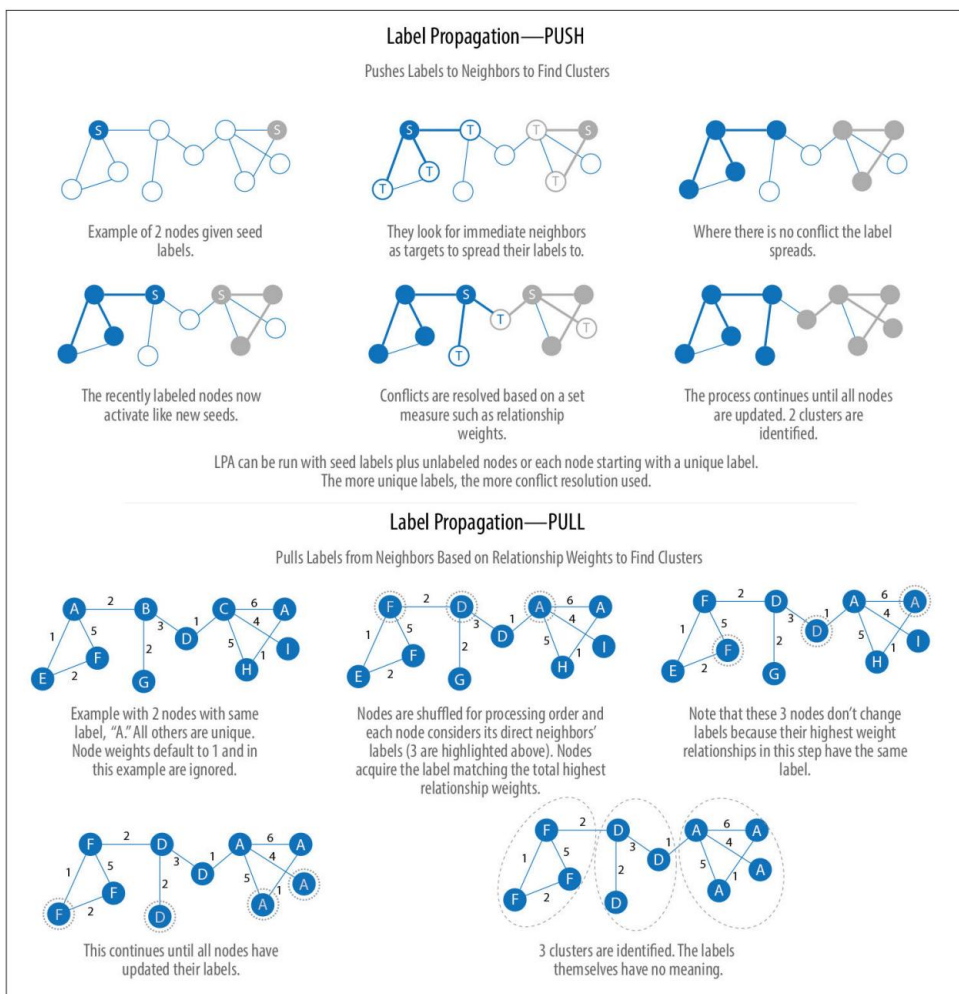


Figura 6-8. Dos variantes de la propagación de etiquetas

Los pasos que se utilizan a menudo para el método de extracción de propagación de etiquetas son:

1. Cada nodo se inicializa con una etiqueta única (un identificador) y, opcionalmente, se pueden utilizar etiquetas "semilla" preliminares.
2. Estas etiquetas se propagan a través de la red.
3. En cada iteración de propagación, cada nodo actualiza su etiqueta para que coincida con la que tiene el peso máximo, el cual se calcula con base en los pesos de los nodos vecinos y sus relaciones. Los empates se resuelven de forma uniforme y aleatoria.
4. LPA alcanza la convergencia cuando cada nodo tiene la etiqueta mayoritaria de sus vecinos.

A medida que las etiquetas se propagan, grupos de nodos densamente conectados alcanzan rápidamente un consenso sobre una etiqueta única. Al final de la propagación, solo quedan unas pocas etiquetas, y los nodos que comparten la misma etiqueta pertenecen a la misma comunidad.

Aprendizaje Semi-Supervisado y Etiquetas Semilla. A diferencia

de otros algoritmos, la Propagación de Etiquetas puede devolver diferentes estructuras de comunidad al ejecutarse varias veces en el mismo grafo. El orden en que LPA evalúa los nodos puede influir en las comunidades finales que devuelve.

El rango de soluciones se reduce cuando algunos nodos reciben etiquetas preliminares (es decir, etiquetas semilla), mientras que otros no las tienen. Los nodos sin etiquetas tienen mayor probabilidad de adoptar las etiquetas preliminares.

Este uso de la Propagación de Etiquetas puede considerarse un método de aprendizaje semisupervisado para encontrar comunidades. El aprendizaje semisupervisado es un tipo de tareas y técnicas de aprendizaje automático que operan con una pequeña cantidad de datos etiquetados, junto con una mayor cantidad de datos sin etiquetar. También podemos ejecutar el algoritmo repetidamente en grafos a medida que evolucionan.

Finalmente, LPA a veces no converge en una única solución. En esta situación, los resultados de nuestra comunidad cambiarán continuamente entre unas pocas comunidades notablemente similares y el algoritmo nunca se completará. Las etiquetas de semilla ayudan a guiarlo hacia una solución. Spark y Neo4j utilizan un número máximo establecido de iteraciones para evitar una ejecución interminable. Debe probar la configuración de iteración de sus datos para equilibrar la precisión y tiempo de ejecución.

¿Cuándo debo utilizar la propagación de etiquetas?

Utilice la Propagación de Etiquetas en redes a gran escala para la detección inicial de comunidades, especialmente cuando se dispone de ponderaciones. Este algoritmo se puede paralelizar y, por lo tanto, es extremadamente rápido en la partición de grafos.

Algunos ejemplos de casos de uso incluyen:

- Asignación de polaridad a tuits como parte del análisis semántico. En este escenario, se utilizan etiquetas de semilla positivas y negativas de un clasificador en combinación con el grafo de seguidores de Twitter. Para más información, consulte [«Clasificación de polaridad de Twitter con propagación de etiquetas sobre enlaces léxicos y el grafo de seguidores»](#), de M. Speriosu et al.
- Detección de combinaciones potencialmente peligrosas de posibles fármacos coprescritos, basándose en la similitud química y los perfiles de efectos secundarios. Véase [“Predicción de interacciones fármaco-fármaco mediante propagación de etiquetas basada en efectos secundarios clínicos”](#), un artículo de P. Zhang et al.

- Inferencia de características de diálogo e intención del usuario para un modelo de aprendizaje automático.
Para más información, consulte [«Inferencia de características basada en la propagación de etiquetas en grafos de datos wiki para DST»](#), un artículo de Y. Murase et al.

Propagación de etiquetas con Apache Spark

Para comenzar con Apache Spark, primero importaremos los paquetes que necesitamos de Spark y el paquete GraphFrames:

```
desde pyspark.sql importar funciones como F
```

Ahora estamos listos para ejecutar el algoritmo de propagación de etiquetas. Para ello, escribimos el siguiente código:

```
resultado = g.labelPropagation(maxIter=10)

(resultado .sort("etiqueta") .groupby("etiqueta") .agg(F.collect_list("id"))) .show(truncar=False))
```

Si ejecutamos ese código en pyspark veremos este resultado:

etiqueta	lista_de_colección(id)
180388626432 [jpy-core, jpy-console, jupyter]	223338299392
[matplotlib, spacy]	498216206336 [python-dateutil,
numpy, six, pytz]	549755813888 [pandas] 558345748480 [nbconvert,
ipykernel, jpy-client]	936302870528
[pyspark]	1279900254208 [py4j]

En comparación con **los Componentes Conectados**, en este ejemplo tenemos más clústeres de bibliotecas. LPA es menos estricto que los Componentes Conectados en cuanto a la determinación de clústeres. Es posible que dos vecinos (nodos conectados directamente) se encuentren en clústeres diferentes mediante la Propagación de Etiquetas. Sin embargo, al usar Componentes Conectados, un nodo siempre estaría en el mismo clúster que sus vecinos, ya que este algoritmo basa la agrupación estrictamente en relaciones.

En nuestro ejemplo, la diferencia más obvia es que las bibliotecas Jupyter se han dividido en dos comunidades: una que contiene las partes principales de la biblioteca y la otra las herramientas orientadas al cliente.

Propagación de etiquetas con Neo4j.

Probemos ahora el mismo algoritmo con Neo4j. Podemos ejecutar LPA ejecutando la siguiente consulta:

```
LLAMA algo.labelPropagation.stream("Biblioteca", "DEPENDS_ON",
  { iteraciones: 10 })
YIELD nodeld, etiqueta
RETURN
      etiqueta, recopilar(algo.getNodeById(nodeld).id) COMO
bibliotecas ORDENAR POR tamaño(bibliotecas) DESC
```

Los parámetros que se pasan a este algoritmo son:

Biblioteca

La etiqueta del nodo que se cargará desde el gráfico

DEPENDE DE

El tipo de relación que se cargará desde el gráfico

iteraciones: 10

El número máximo de iteraciones a ejecutar

Estos son los resultados que veríamos:

bibliotecas de etiquetas	
11	[matplotlib, spacy, six, pandas, python-dateutil]
10	[jupyter, jpy-console, nbconvert, jpy-client, jpy-core]
4	[pyspark, py4j]
8	[ipykernel]
13	[numpy]
0	[pytz]

Los resultados, que también se pueden ver visualmente en la [Figura 6-9](#), son bastante similares a los que obtuvimos con Apache Spark.

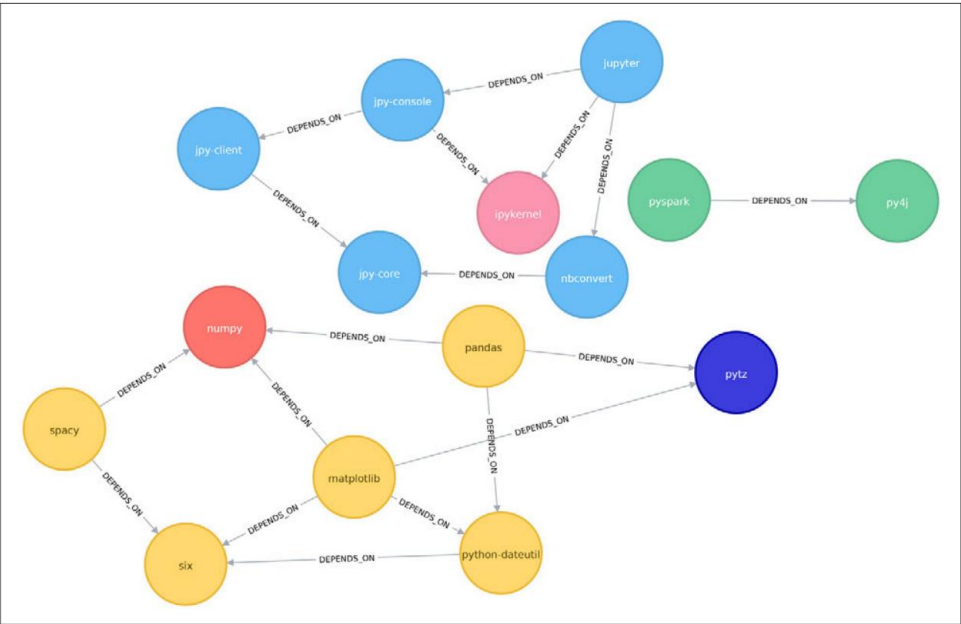


Figura 6-9. Clústeres encontrados por el algoritmo de propagación de etiquetas

También podemos ejecutar el algoritmo asumiendo que el gráfico no está dirigido, lo que significa que los nodos intentarán adoptar etiquetas de las bibliotecas de las que dependen, así como de aquellas que dependen de ellos.

Para ello, pasamos el parámetro DIRECCIÓN:BOTH al algoritmo:

```
LLAMA algo.labelPropagation.stream("Biblioteca", "DEPENDS_ON",
    { iteraciones: 10, dirección: "AMBOS" })
YIELD nodeId, etiqueta
RETURN
    etiqueta, recopilar(algo.getNodeById(nodeId).id) COMO
    bibliotecas ORDENAR POR tamaño(bibliotecas) DESC
```

Si ejecutamos esto, obtendremos el siguiente resultado:

bibliotecas de etiquetas	
11	[pytz, matplotlib, spacy, six, pandas, numpy, python-dateutil]
10	[nbconvert, jpy-client, jpy-core]
6	[jupyter, jpy-console, ipykernel]
4	[pyspark, py4j]

El número de clústeres se ha reducido de seis a cuatro, y todos los nodos de la sección matplotlib-lib del gráfico están ahora agrupados. Esto se puede apreciar con mayor claridad en la [Figura 6-10](#).

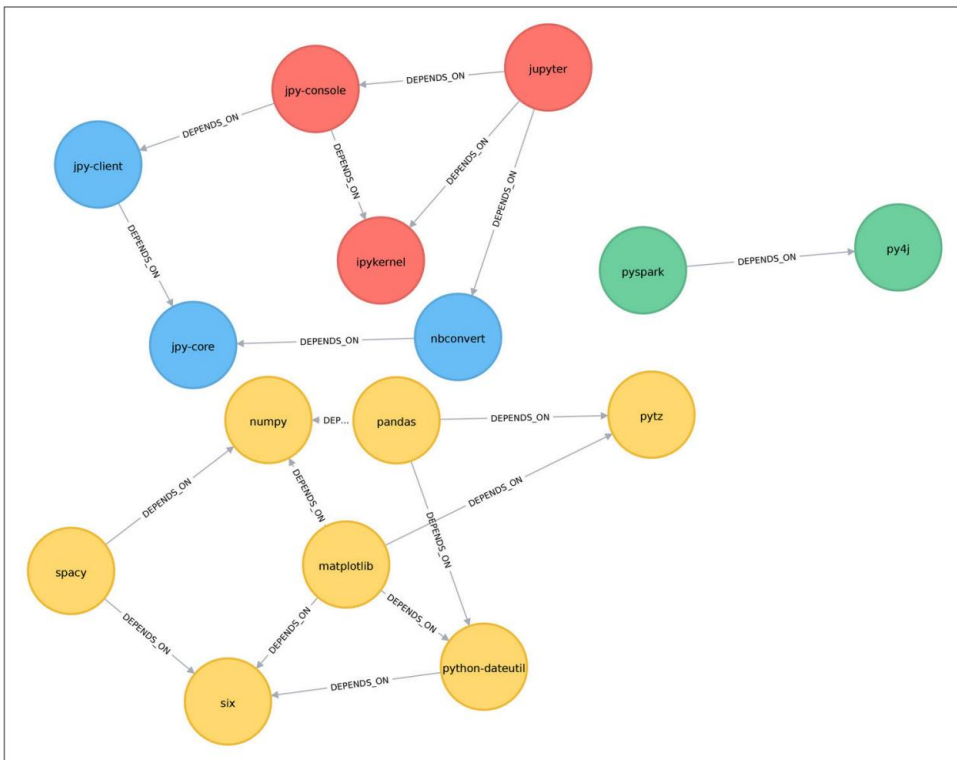


Figura 6-10. Clústeres encontrados por el algoritmo de propagación de etiquetas, al ignorar la dirección de la relación

Aunque los resultados de la propagación de etiquetas en estos datos son similares para cálculos dirigidos y no dirigidos, en gráficos complejos se observarán diferencias más significativas. Esto se debe a que ignorar la dirección provoca que los nodos intenten adoptar más etiquetas, independientemente del origen de la relación.

Modularidad de Lovaina

El algoritmo de Modularidad de Louvain encuentra clústeres comparando la densidad de la comunidad al asignar nodos a diferentes grupos. Puede considerarse un análisis hipotético para probar diversas agrupaciones con el objetivo de alcanzar un óptimo global.

Propuesto en 2008, el **algoritmo de Louvain** es uno de los algoritmos modulares más rápidos. Además de detectar comunidades, también revela una jerarquía de comunidades.

a diferentes escalas. Esto es útil para comprender la estructura de una red en diferentes niveles de granularidad.

Louvain cuantifica la eficacia de la asignación de un nodo a un grupo analizando la densidad de conexiones dentro de un clúster en comparación con una muestra promedio o aleatoria. Esta medida de asignación a la comunidad se denomina modularidad.

Agrupación basada en la calidad

mediante modularidad. La modularidad es una técnica para descubrir comunidades mediante la partición de un grafo en módulos (o clústeres) de mayor detalle y la posterior medición de la solidez de las agrupaciones. En lugar de simplemente observar la concentración de conexiones dentro de un clúster, este método compara las densidades de relaciones en clústeres dados con las densidades entre clústeres. La medida de la calidad de estas agrupaciones se denomina modularidad.

Los algoritmos de modularidad optimizan las comunidades localmente y luego globalmente, utilizando múltiples iteraciones para probar diferentes agrupaciones y aumentando la aspereza. Esta estrategia identifica las jerarquías de la comunidad y proporciona una comprensión amplia de la estructura general. Sin embargo, todos los algoritmos de modularidad presentan dos desventajas:

- Fusionan comunidades más pequeñas en comunidades más grandes.
- Puede producirse una meseta donde existen varias opciones de partición con una modularidad similar, formando máximos locales e impidiendo el progreso.

Para obtener más información, consulte el artículo [“El rendimiento de la maximización de la modularidad en contextos prácticos”](#), de BH Good, Y.-A. de Montjoye y A. Clauset.

Cálculo de la modularidad.

Un cálculo simple de la modularidad se basa en la fracción de relaciones dentro de los grupos dados menos la fracción esperada si las relaciones se distribuyeran aleatoriamente entre todos los nodos. El valor siempre está entre 1 y -1; los valores positivos indican una mayor densidad de relaciones de la esperada por azar, mientras que los valores negativos indican una menor densidad. [La Figura 6-11](#) ilustra diferentes puntuaciones de modularidad basadas en la agrupación de nodos.

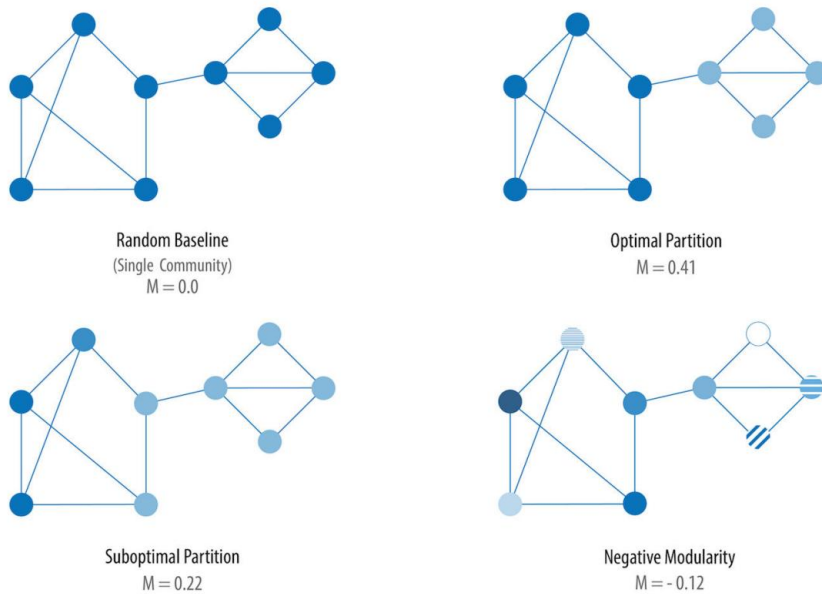


Figura 6-11. Cuatro puntuaciones de modularidad basadas en diferentes opciones de partición.

La fórmula para la modularidad de un grupo es:

$$M = \sum_c \left[\frac{L_c}{Y_o} - \left(\frac{k_c}{2L} \right)^2 \right]$$

dónde:

- L es el número de relaciones en todo el grupo. • L_c es el número de relaciones en una partición.
- k_{do} es el grado total de nodos en una partición.

El cálculo para la partición óptima en la parte superior de la **Figura 6-11** es el siguiente:

• La partición oscura es $\left(\frac{7}{13} - \left(\frac{15}{2(13)} \right)^2 \right) = 0.205$

• La partición de luz es $\left(\frac{5}{13} - \left(\frac{11}{2(13)} \right)^2 \right) = 0.206$

• Estos se suman para $M = 0.205 + 0.206 = 0.411$

41

Inicialmente, el algoritmo de modularidad de Lovaina optimiza la modularidad localmente en todos los nodos, lo que encuentra pequeñas comunidades; luego, cada pequeña comunidad se agrupa en un nodo de conglomerado más grande y se repite el primer paso hasta alcanzar un óptimo global.

El algoritmo consiste en la aplicación repetida de dos pasos, como se ilustra en la [Figura 6-12](#).

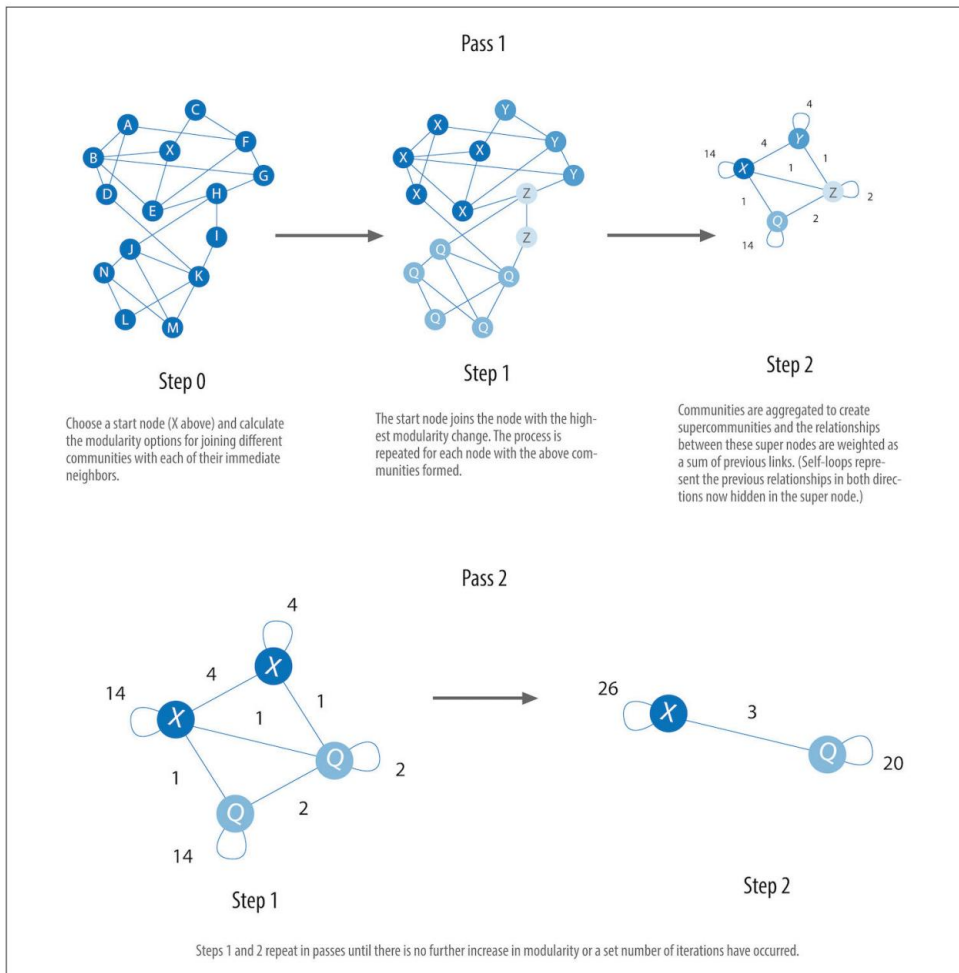


Figura 6-12. El proceso del algoritmo de Louvain

Los pasos del algoritmo de Lovaina incluyen:

1. Una asignación "codiciosa" de nodos a las comunidades, favoreciendo las optimizaciones locales de modularidad.

2. La definición de una red de grano grueso basada en las comunidades encontradas en el primer paso.
Esta red de grano grueso se utilizará en la siguiente iteración del algoritmo.

Estos dos pasos se repiten hasta que no sea posible realizar más reasignaciones de comunidades que aumenten la modularidad.

Parte del primer paso de optimización consiste en evaluar la modularidad de un grupo. Louvain utiliza la siguiente fórmula para lograrlo:

$$Q = \frac{1}{2m} \sum_{u,v} \left[A_{uv} - \frac{k_u k_v}{2m} \right] \delta_{c_u, c_v}$$

dónde:

- u y v son nodos.
- m es el peso total de la relación en todo el gráfico ($2m$ es una norma común).
valor de malización en fórmulas de modularidad).
- $A_{uv} - \frac{k_u k_v}{2m}$ es la fuerza de la relación entre u y v en comparación con lo que esperaríamos con una asignación aleatoria (tiende hacia promedios) de esos nodos en la red.

— A_{uv} es el peso de la relación entre u y v . — k_u es la suma de los

pesos de la relación para u . — k_v es la suma de los

pesos de la relación para v . es igual a 1 si u y v están

- δ_{c_u, c_v} asignados a la misma comunidad, y 0 si están asignados a la misma comunidad.
no lo son.

Otra parte de ese primer paso evalúa el cambio en la modularidad si un nodo se traslada a otro grupo.

Louvain utiliza una variante más compleja de esta fórmula y luego determina la mejor asignación de grupo.

¿Cuándo debo utilizar Louvain?

Utilice la modularidad de Louvain para encontrar comunidades en redes extensas. Este algoritmo aplica una modularidad heurística, a diferencia de la modularidad exacta, lo cual requiere un alto coste computacional. Por lo tanto, Louvain puede utilizarse en grafos grandes donde los algoritmos de modularidad estándar pueden presentar dificultades.

Louvain también es muy útil para evaluar la estructura de redes complejas, en particular para descubrir múltiples niveles de jerarquía, como los que se pueden encontrar en una organización criminal. El algoritmo puede proporcionar resultados que permiten ampliar diferentes niveles de granularidad y encontrar subcomunidades dentro de subcomunidades dentro de subcomunidades.

Algunos ejemplos de casos de uso incluyen:

- Detección de ciberataques. El algoritmo de Louvain se utilizó en [un estudio de 2016 realizado por SV. Shanbhaq](#) de detección rápida de comunidades en ciberredes a gran escala para aplicaciones de ciberseguridad. Una vez detectadas, estas comunidades pueden utilizarse para detectar ciberataques.
- Extracción de temas de plataformas sociales en línea, como Twitter y YouTube, basándose en la coincidencia de términos en documentos como parte del proceso de modelado de temas. Este enfoque se describe en un artículo de GS Kido, RA Igawa y S. Barbon Jr., ["Modelado de temas basado en el método de Louvain en redes sociales en línea"](#). • Encontrar estructuras comunitarias jerárquicas dentro de la red funcional del cerebro, como se describe en ["Modularidad jerárquica en redes funcionales del cerebro humano"](#) de D. Meunier et al.



Los algoritmos de optimización de modularidad, incluyendo Louvain, presentan dos problemas. Primero, pueden pasar por alto comunidades pequeñas dentro de redes grandes. Este problema se puede solucionar revisando los pasos intermedios de consolidación. Segundo, en grafos grandes con comunidades superpuestas, los optimizadores de modularidad podrían no determinar correctamente los máximos globales. En este último caso, recomendamos usar cualquier algoritmo de modularidad como guía para una estimación general, pero no para una precisión completa.

Louvain con Neo4j. Veamos el

algoritmo de Louvain en acción. Podemos ejecutar la siguiente consulta para ejecutar el algoritmo en nuestro grafo:

```
LLAMA algo.louvain.stream("Biblioteca", "DEPENDS_ON")
YIELD nodeId, comunidades
RETURN algo.getNodeById(nodeId).id AS bibliotecas, comunidades
```

Los parámetros que se pasan a este algoritmo son:

Biblioteca

La etiqueta del nodo que se cargará desde el gráfico

DEPENDS DE

El tipo de relación que se cargará desde el gráfico

Estos son los resultados:

bibliotecas	comunidades
pytz	[0, 0]

bibliotecas	comunidades
pyspark	[1, 1]
matplotlib	[2, 0]
espacioso	[2, 0]
py4j	[1, 1]
jupyter	[3, 2]
jpy-console	[3, 2]
nbconvert	[4, 2]
ipykernel	[3, 2]
cliente jpy	[4, 2]
núcleo jpy	[4, 2]
seis	[2, 0]
pandas	[0, 0]
numpy	[2, 0]
python-dateutil	[2, 0]

La columna de comunidades describe la comunidad en la que se dividen los nodos en dos niveles.

El último valor de la matriz es la comunidad final y el otro es un intermedio.

Comió la comunidad.

Los números asignados a las comunidades intermedias y finales son simplemente etiquetas sin significado medible. Trátenlos como etiquetas que indican qué comunidad

Los nodos pertenecen a, por ejemplo, "pertenece a una comunidad etiquetada como 0", "una comunidad etiquetada como 4", y así sucesivamente.

Por ejemplo, matplotlib tiene un resultado de [2,0]. Esto significa que la coma final de matplotlib

La comunidad está etiquetada como 0 y su comunidad intermedia está etiquetada como 2.

Es más fácil ver cómo funciona esto si almacenamos estas comunidades usando la versión de escritura.

del algoritmo y luego consultarlo posteriormente. La siguiente consulta ejecutará el Lou-

Algoritmo vano y almacena el resultado en la propiedad de comunidades en cada nodo:

```
LLAMA a algo.louvain("Biblioteca", "DEPENDEN_DE")
```

También podríamos almacenar las comunidades resultantes utilizando la versión de streaming del

algoritmo, seguido de la llamada a la cláusula SET para almacenar el resultado. La siguiente consulta muestra cómo podríamos hacer esto:

```
LLAMA algo.louvain.stream("Biblioteca", "DEPENDS_ON")
YIELD nodeId, comunidades
CON algo.getNodeById(nodeId) COMO nodo, comunidades
SET node.comunities = comunidades
```

Una vez que hayamos ejecutado cualquiera de esas consultas, podemos escribir la siguiente consulta para encontrar la grupos finales:

```
PARTIDO (!:Biblioteca)
DEVOLVER l.communities[-1] COMO comunidad, recopilar(l.id) COMO bibliotecas
ORDENAR POR tamaño(bibliotecas) DESC
```

`l.communities[-1]` devuelve el último elemento de la matriz subyacente que contiene esta propiedad. `viveres`.

Al ejecutar la consulta se obtiene este resultado:

bibliotecas comunitarias	
0	[pytz, matplotlib, spacy, seis, pandas, numpy, python-dateutil]
2	[jupyter, jpy-console, nbconvert, ipykernel, jpy-client, jpy-core]
1	[pyspark, py4j]

Esta agrupación es la misma que vimos con el algoritmo de componentes conectados.

Matplotlib forma parte de una comunidad con Pytz, Spacy, Six, Pandas, Numpy y Python-DateUtil. Esto se puede apreciar con mayor claridad en [la Figura 6-13](#).

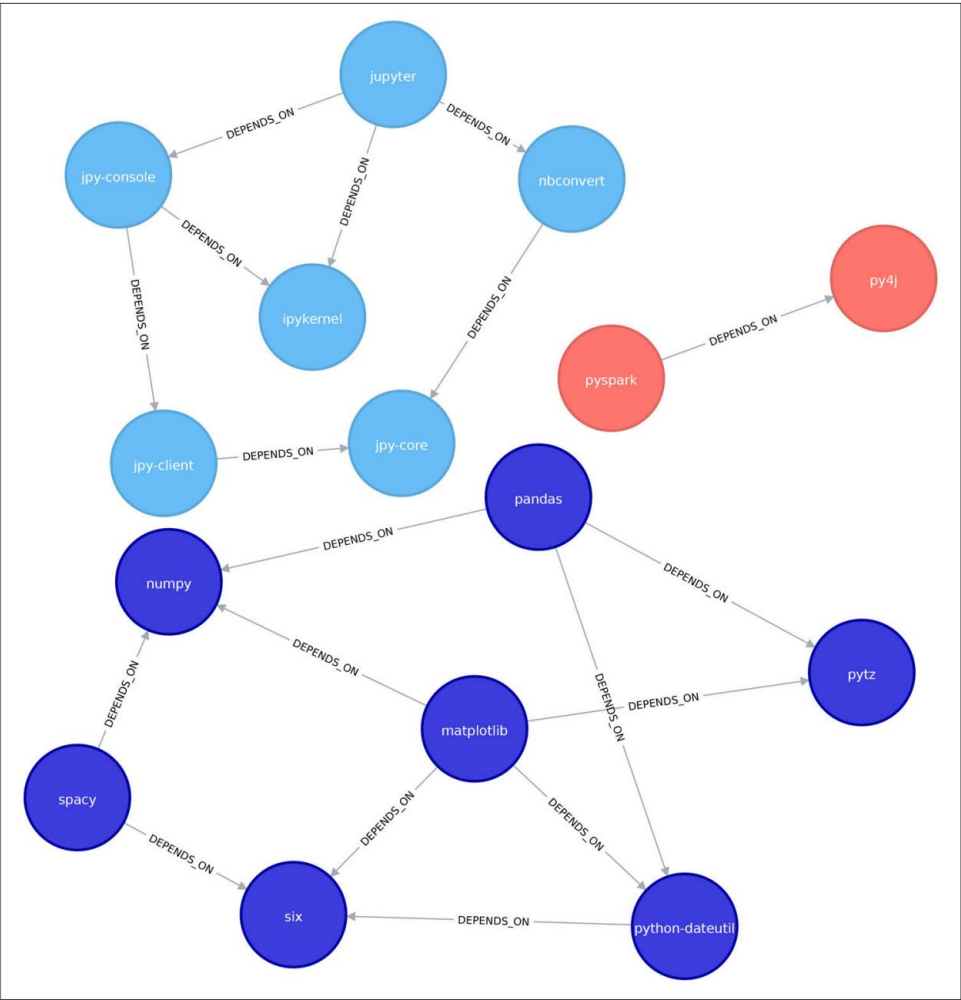


Figura 6-13. Clústeres encontrados por el algoritmo de Louvain

Una característica adicional del algoritmo de Louvain es que también podemos ver la agrupación intermedia. Esto nos mostrará agrupaciones más detalladas que la capa final:

```
PARTIDO (l:Biblioteca)
DEVOLVER l.communities[0] COMO comunidad, recopilar(l.id) COMO bibliotecas
ORDENAR POR tamaño(bibliotecas) DESC
```

Al ejecutar esa consulta se obtiene este resultado:

bibliotecas comunitarias	
2	[matplotlib, spacy, six, python-dateutil]
4	[nbconvert, jpy-client, jpy-core]

bibliotecas comunitarias	
3	[jupyter, jpy-consola, ipykernel]
1	[pyspark, py4j]
0	[Pytz, pandas]
5	[numpy]

Las bibliotecas de la comunidad matplotlib ahora se han dividido en tres bibliotecas más pequeñas. comunidades:

- matplotlib, spacy, six y python-dateutil
- pytz y pandas
- numpy

Podemos ver este desglose visualmente en [la Figura 6-14](#).

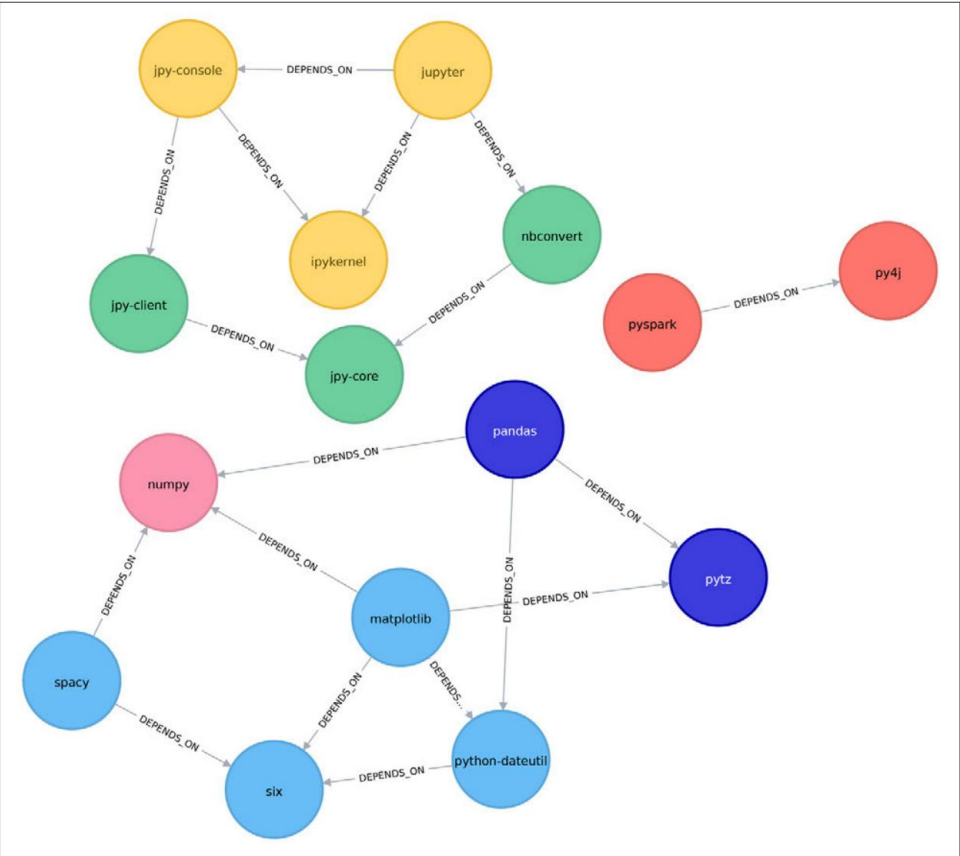


Figura 6-14. Clústeres intermedios encontrados por el algoritmo de Louvain

Aunque este gráfico solo mostró dos capas de jerarquía, si ejecutáramos este algoritmo en un gráfico más grande, observaríamos una jerarquía más compleja. Los clústeres intermedios que revela Louvain pueden ser muy útiles para detectar comunidades de grano fino que otros algoritmos de detección de comunidades podrían no detectar.

Validando comunidades

Los algoritmos de detección de comunidades generalmente tienen el mismo objetivo: identificar grupos. Sin embargo, dado que diferentes algoritmos parten de diferentes suposiciones, pueden descubrir comunidades diferentes. Esto hace que elegir el algoritmo adecuado para un problema particular sea más difícil y requiera cierta exploración.

La mayoría de los algoritmos de detección de comunidades funcionan razonablemente bien cuando la densidad de relaciones dentro de los grupos es alta en comparación con su entorno, pero las redes del mundo real suelen ser menos definidas. Podemos validar la precisión de las comunidades encontradas comparando nuestros resultados con un punto de referencia basado en datos de comunidades conocidas.

Dos de los algoritmos de referencia más conocidos son los algoritmos Girvan-Newman (GN) y Lancichinetti-Fortunato-Radicchi (LFR). Las redes de referencia que estos algoritmos generan son bastante diferentes: GN genera una red aleatoria más homogénea, mientras que LFR crea un grafo más heterogéneo donde los grados de los nodos y el tamaño de la comunidad se distribuyen según una ley de potencia.

Dado que la precisión de nuestras pruebas depende del parámetro de referencia utilizado, es importante que este coincida con nuestro conjunto de datos. En la medida de lo posible, busque densidades, distribuciones de relaciones, definiciones de comunidad y dominios relacionados similares.

Resumen

Los algoritmos de detección de comunidades son útiles para comprender la forma en que los nodos se agrupan en un gráfico.

En este capítulo, comenzamos aprendiendo sobre los algoritmos de conteo de triángulos y coeficiente de agrupamiento. Posteriormente, pasamos a dos algoritmos deterministas de detección de comunidades: componentes fuertemente conectados y componentes conectados. Estos algoritmos tienen definiciones estrictas de lo que constituye una comunidad y son muy útiles para comprender la estructura del grafo en las primeras etapas del proceso de análisis de grafos.

Terminamos con Propagación de Etiquetas y Louvain, dos algoritmos no deterministas que detectan mejor comunidades de grano fino. Louvain también nos mostró una jerarquía de comunidades a diferentes escalas.

En el próximo capítulo, tomaremos un conjunto de datos mucho más grande y aprenderemos cómo combinar los algoritmos para obtener aún más información sobre nuestros datos conectados.

Algoritmos gráficos en la práctica

El enfoque que adoptamos para el análisis de gráficos evoluciona a medida que nos familiarizamos con el comportamiento de diferentes algoritmos en conjuntos de datos específicos. En este capítulo, repasaremos varios ejemplos para que comprenda mejor cómo abordar el análisis de datos de gráficos a gran escala utilizando conjuntos de datos de Yelp y el Departamento de Transporte de EE. UU. Analizaremos el análisis de datos de Yelp en Neo4j, que incluye una descripción general de los datos, la combinación de algoritmos para realizar recomendaciones de viajes y la extracción de datos de usuarios y empresas para consultoría. En Spark, analizaremos datos de aerolíneas estadounidenses para comprender los patrones de tráfico y los retrasos, así como la conexión entre aeropuertos entre diferentes aerolíneas.

Debido a que los algoritmos de búsqueda de rutas son sencillos, nuestros ejemplos utilizarán estos algoritmos de centralidad y detección de comunidad:

- PageRank para encontrar revisores influyentes de Yelp y luego correlacionar sus calificaciones para hoteles específicos
- Centralidad de intermediación para descubrir revisores conectados a múltiples grupos y luego extrae sus preferencias
- Propagación de etiquetas con una proyección para crear supercategorías de Yelp similares negocios
- Centralidad de grado para identificar rápidamente los centros aeroportuarios en el conjunto de datos de transporte de EE. UU. • Componentes fuertemente conectados para analizar grupos de rutas aeroportuarias en EE. UU.

Análisis de datos de Yelp con Neo4j

Yelp ayuda a las personas a encontrar negocios locales basándose en reseñas, preferencias y recomendaciones. A finales de 2018, se habían escrito más de 180 millones de reseñas en la plataforma.

2018. Desde 2013, Yelp ha realizado el **desafío Yelp Dataset**, una competencia que alienta a las personas a explorar e investigar el conjunto de datos abiertos de Yelp.

A partir de la ronda 12 (realizada en 2018) del desafío, el conjunto de datos abiertos contenía:

- Más de 7 millones de reseñas y consejos
- Más de 1,5 millones de usuarios y 280 000 imágenes •
- Más de 188 000 empresas con 1,4 millones de atributos
- 10 áreas metropolitanas

Desde su lanzamiento, el conjunto de datos se ha popularizado, con **cientos de artículos académicos** escritos con este material. El conjunto de datos de Yelp representa datos reales muy bien estructurados y altamente interconectados. Es un excelente ejemplo de algoritmos de grafos que también puedes descargar y explorar.

Red social Yelp

Además de escribir y leer reseñas de negocios, los usuarios de Yelp forman una red social. Pueden enviar solicitudes de amistad a otros usuarios que hayan conocido mientras navegan por Yelp.com, o conectar sus libretas de direcciones o gráficos de Facebook.

El conjunto de datos de Yelp también incluye una red social. La **Figura 7-1** muestra una captura de pantalla de la sección "Amigos" del perfil de Yelp de Mark.

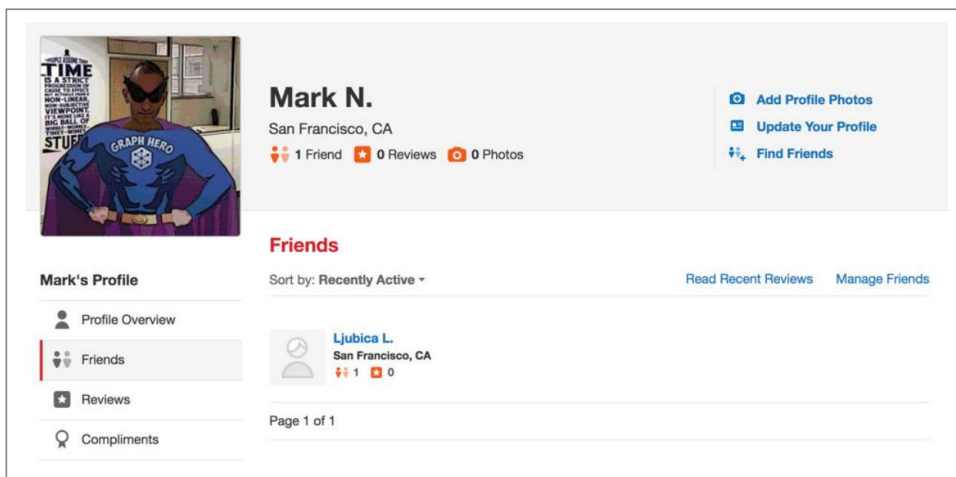


Figura 7-1. Perfil de Yelp de Mark

Aparte de que Mark necesita algunos amigos más, estamos listos para empezar. Para ilustrar cómo podríamos analizar los datos de Yelp en Neo4j, usaremos un escenario en el que trabajamos para una empresa de información turística. Primero exploraremos los datos de Yelp y luego veremos cómo...

Para ayudar a las personas a planificar sus viajes con nuestra aplicación, les guiaremos en la búsqueda de buenas recomendaciones de alojamiento y actividades en ciudades importantes como Las Vegas.

Otra parte de nuestro escenario empresarial implicará la consultoría para empresas de destinos turísticos. Por ejemplo, ayudaremos a los hoteles a identificar visitantes influyentes y, posteriormente, a las empresas a las que deberían dirigirse para programas de promoción cruzada.

Importación de datos

Hay muchos métodos diferentes para importar datos a Neo4j, incluida la **herramienta Importar**, el **comando LOAD CSV** que hemos visto en capítulos anteriores y **los controladores de Neo4j**.

Para el conjunto de datos de Yelp, necesitamos realizar una importación única de una gran cantidad de datos, por lo que la herramienta Importar es la mejor opción. Consulte **"Importación masiva de datos de Neo4j y Yelp"** en la **página 225** para obtener más información.

Modelo gráfico Los

datos de Yelp se representan en un modelo gráfico como se muestra en la **Figura 7-2**.

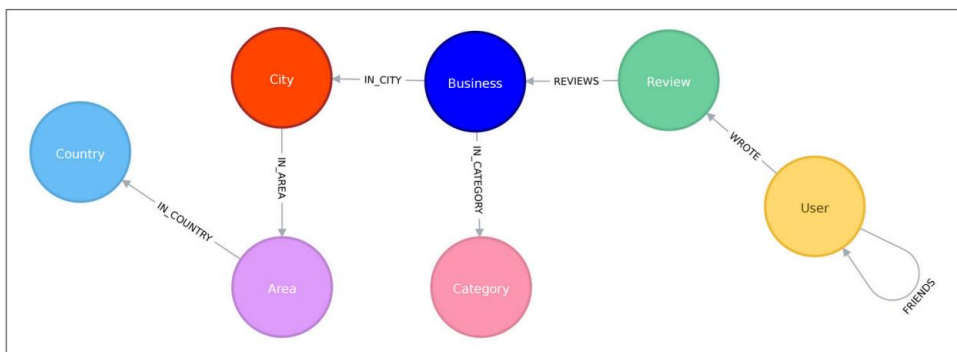


Figura 7-2. El modelo gráfico de Yelp

Nuestro grafo contiene nodos etiquetados por Usuario, que tienen relaciones de AMIGO con otros Usuarios. Los usuarios también escriben reseñas y consejos sobre negocios. Todos los metadatos se almacenan como propiedades de los nodos, excepto las categorías de negocios, que se representan mediante nodos de Categoría independientes. Para los datos de ubicación, hemos extraído los atributos de Ciudad, Área y País en el subgrafo. En otros casos de uso, podría ser conveniente extraer otros atributos de los nodos, como fechas, o agrupar los nodos en relaciones, como reseñas.

El conjunto de datos de Yelp también incluye sugerencias y fotografías de los usuarios, pero no los usaremos en nuestro ejemplo.

Una breve descripción general de los datos de

Yelp. Una vez cargados los datos en Neo4j, ejecutaremos algunas consultas exploratorias. Preguntaremos cuántos nodos hay en cada categoría o qué tipos de relaciones existen para familiarizarnos con los datos de Yelp. Anteriormente, mostramos consultas de Cypher para nuestros ejemplos de Neo4j, pero es posible que las ejecutemos desde otro lenguaje de programación. Dado que Python es el lenguaje predilecto de los científicos de datos, en esta sección usaremos el controlador Python de Neo4j para conectar los resultados con otras bibliotecas del ecosistema Python. Si solo queremos mostrar el resultado de una consulta, usaremos Cypher directamente.

También mostraremos cómo combinar Neo4j con la popular biblioteca Pandas, que resulta eficaz para la manipulación de datos fuera de la base de datos. Veremos cómo usar la biblioteca Tabulate para embellecer los resultados obtenidos con Pandas y cómo crear representaciones visuales de datos con Matplotlib.

También usaremos la biblioteca de procedimientos APOC de Neo4j para escribir consultas Cypher aún más potentes. Encontrará más información sobre APOC en [«APOC y otras herramientas de Neo4j» en la página 228](#).

Primero instalemos las bibliotecas de Python:

```
pip install neo4j-driver tabulate pandas matplotlib
```

Una vez que hayamos hecho eso, importaremos esas bibliotecas:

```
desde neo4j.v1 importar GraphDatabase
importar pandas como
pd desde tabular importar tabular
```

Importar matplotlib puede ser complicado en macOS, pero las siguientes líneas deberían resolver el problema:

```
importar matplotlib
matplotlib.use('TkAgg')
importar matplotlib.pyplot como plt
```

Si está utilizando otro sistema operativo, es posible que la línea central no sea necesaria.

Ahora, creemos una instancia del controlador Neo4j que apunte a una base de datos Neo4j local:

```
controlador = GraphDatabase.driver("bolt://localhost", auth=("neo4j", "neo"))
```



Necesitará actualizar la inicialización del controlador para usar su propio host y credenciales.

Para empezar, veamos algunos números generales para nodos y relaciones. El siguiente código calcula las cardinalidades de las etiquetas de nodo (es decir, cuenta el número de nodos para cada etiqueta) en la base de datos:

```

resultado = {"etiqueta": [], "cuenta": []} con
driver.session() como sesión:
    etiquetas = [fila["etiqueta"] para fila en sesión.run("CALL db.labels()")] para etiqueta en etiquetas:

        consulta = f"COINCIDIR (:`{etiqueta}`) DEVOLVER recuento(*) como
        recuento" recuento = sesión.ejecutar(consulta).single()
        ["recuento"]
        resultado["etiqueta"].append(etiqueta) resultado["recuento"].append(recuento)

df = pd.DataFrame(datos=resultado)
print(tabulate(df.sort_values("count"), headers='keys', tablefmt='psql',
               showindex=False))

```

Si ejecutamos ese código veremos cuántos nodos tenemos para cada etiqueta:

etiqueta	contar
País	17
Área	54
Ciudad	1093
Categoría	1293
Negocio	174567
Usuario	1326101
Revisar	5261669

También podríamos crear una representación visual de las cardinalidades, con el siguiente código:

```

plt.style.use('cinco treinta y ocho')

ax = df.plot(kind='bar', x='label', y='count', legend=None)

ax.xaxis.set_label_text("") plt.yscale("log")
plt.xticks(rotación=45)
plt.tight_layout() plt.show()

```

Podemos ver el gráfico generado por este código en la [Figura 7-3](#). Tenga en cuenta que este gráfico utiliza escala logarítmica.

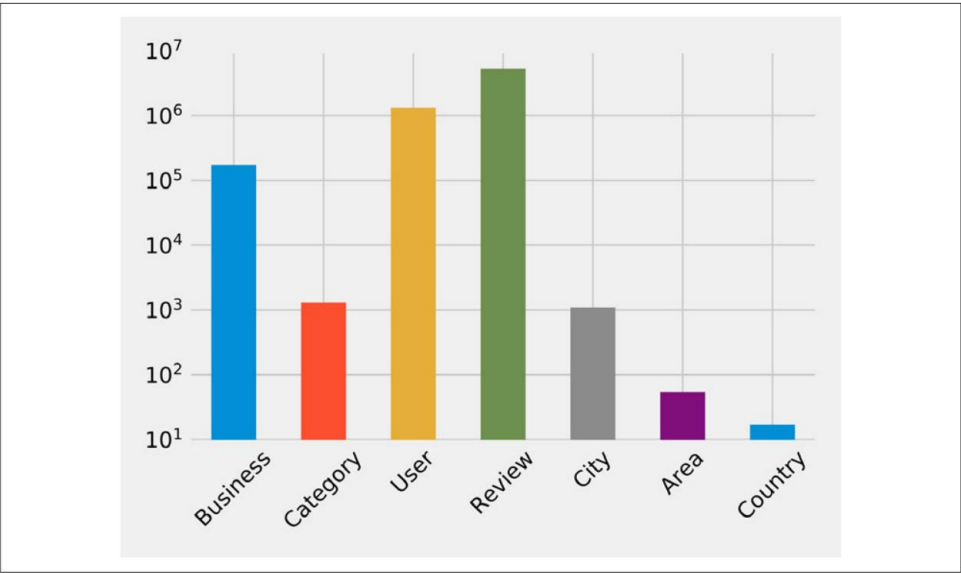


Figura 7-3. Número de nodos para cada categoría de etiqueta.

De manera similar, podemos calcular las cardinalidades de las relaciones:

```
resultado = {"relType": [], "count": []}
con driver.session() como sesión:
    rel_types = [fila["relationshipType"] para la fila en session.run
                  ("LLAMAR db.relationshipTypes()")]

    para rel_type en rel_types:
        consulta = f"COINCIDIR ()-[:{rel_type}]->() DEVOLVER count(*) como count"
        count = sesión.run(consulta).single()[0]
        resultado["relType"].append(rel_type)
        resultado["count"].append(count)

df = pd.DataFrame(datos=resultado)
imprimir(tabulación(df.sort_values("count")), encabezados='claves',
          tablefmt='psql', showindex=False))
```

Si ejecutamos ese código veremos el número de cada tipo de relación:

tipo de relación	contar
EN EL PAÍS	54
EN_ÁREA	1154
EN_CIUADAD	174566
EN CATEGORÍA	667527
ESCRIBIÓ	5261669
RESEÑAS	5261669
AMIGOS	10645356

En la Figura 7-4 se puede observar un gráfico de cardinalidades . Al igual que el gráfico de cardinalidades de los nodos, este gráfico utiliza escala logarítmica.

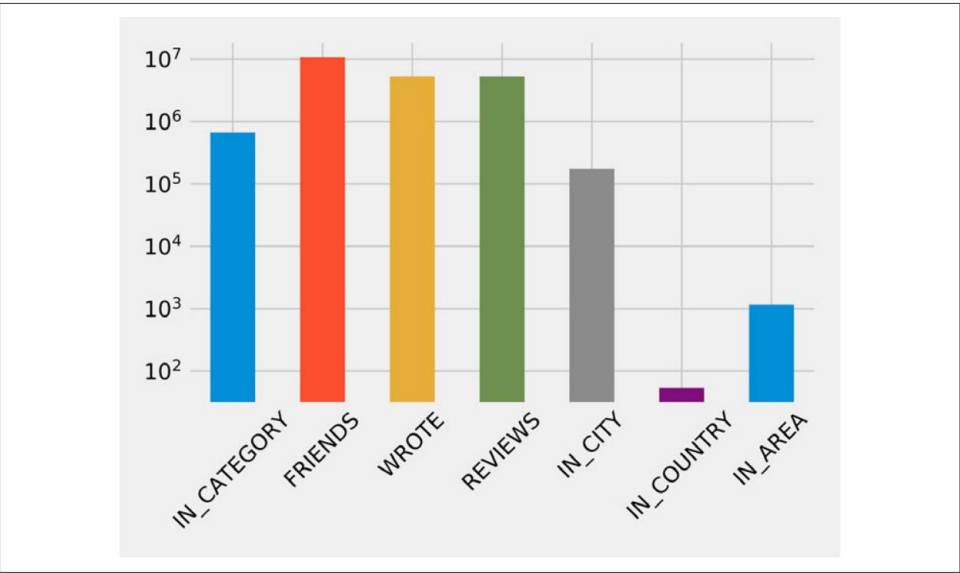


Figura 7-4. Número de relaciones por tipo de relación

Estas consultas no deberían revelar nada sorprendente, pero son útiles para comprender el contenido de los datos. También sirven como una comprobación rápida de que se importaron correctamente.

Suponemos que Yelp tiene muchas reseñas de hoteles, pero conviene comprobarlo antes de centrarnos en ese sector. Podemos averiguar cuántos hoteles aparecen en esos datos y cuántas reseñas tienen ejecutando la siguiente consulta:

```
PARTIDO (categoría:Categoría {nombre: "Hoteles"})
DEVUELVE tamaño((categoría<-[:IN_CATEGORY]-{})) COMO empresas,
tamaño(:Reseña)-[:RESEÑAS]->(:Negocio)-[:EN_CATEGORIA]-> (categoría)
AS reseñas
```

Aquí está el resultado:

reseñas de empresas	
2683	183759

Trabajamos con muchas empresas y recibimos muchísimas reseñas. En la siguiente sección, analizaremos los datos con más detalle en nuestro escenario empresarial.

Para añadir

recomendaciones populares a nuestra app, empezamos buscando los hoteles mejor valorados como una heurística para las opciones más populares de reserva. Podemos añadir su puntuación para comprender la experiencia real. Para ver los 10 hoteles con más reseñas y representar gráficamente su distribución de puntuaciones, usamos el siguiente código :

```
# Encuentra los 10 hoteles con más opiniones
"""

consulta =
COINCIDIR (reseña:Reseña)-:(RESEÑAS)->(negocio:Negocio), (negocio)-:(EN_CATEGORÍA)-
>(categoría:Categoría (nombre: $categoría)), (negocio)-:(EN_CIUDAD)->:(Ciudad {nombre: $ciudad})

DEVOLVER business.name COMO business, recopilar(review.stars) COMO allReviews ORDENAR POR tamaño(allReviews) DESC
LÍMITE 10
"""

fig = plt.figure()
fig.set_size_inches(10.5, 14.5)
fig.subplots_adjust(hspace=0.4, wspace=0.4)

con driver.session() como sesión: parámetros =
{ "ciudad": "Las Vegas", "categoría": "Hoteles"} resultado = session.run(consulta,
parámetros para índice, fila en enumerate(resultado):
negocio = fila["negocio"] estrellas =
pd.Series(fila["todasLasReseñas"])

total = estrellas.count()
promedio_estrellas = estrellas.mean().round(2)

# Calcular la distribución de estrellas
stars_histogram = stars.value_counts().sort_index() stars_histogram /=
float(stars_histogram.sum())

# Dibuje un gráfico de barras que muestre la distribución de las calificaciones de estrellas
ax = fig.add_subplot(5, 2, index+1)
stars_histogram.plot(kind="bar", legend=None, color="darkblue",
titulo=f"{negocio}\nPromedio:
{promedio_estrellas}, Total: {total}")

plt.tight_layout() plt.show()
```

Hemos restringido la búsqueda por ciudad y categoría para centrarnos en los hoteles de Las Vegas. Al ejecutar ese código, obtenemos el gráfico de [la Figura 7-5](#). Observe que el eje x representa la calificación del hotel y el eje y el porcentaje general de cada calificación.

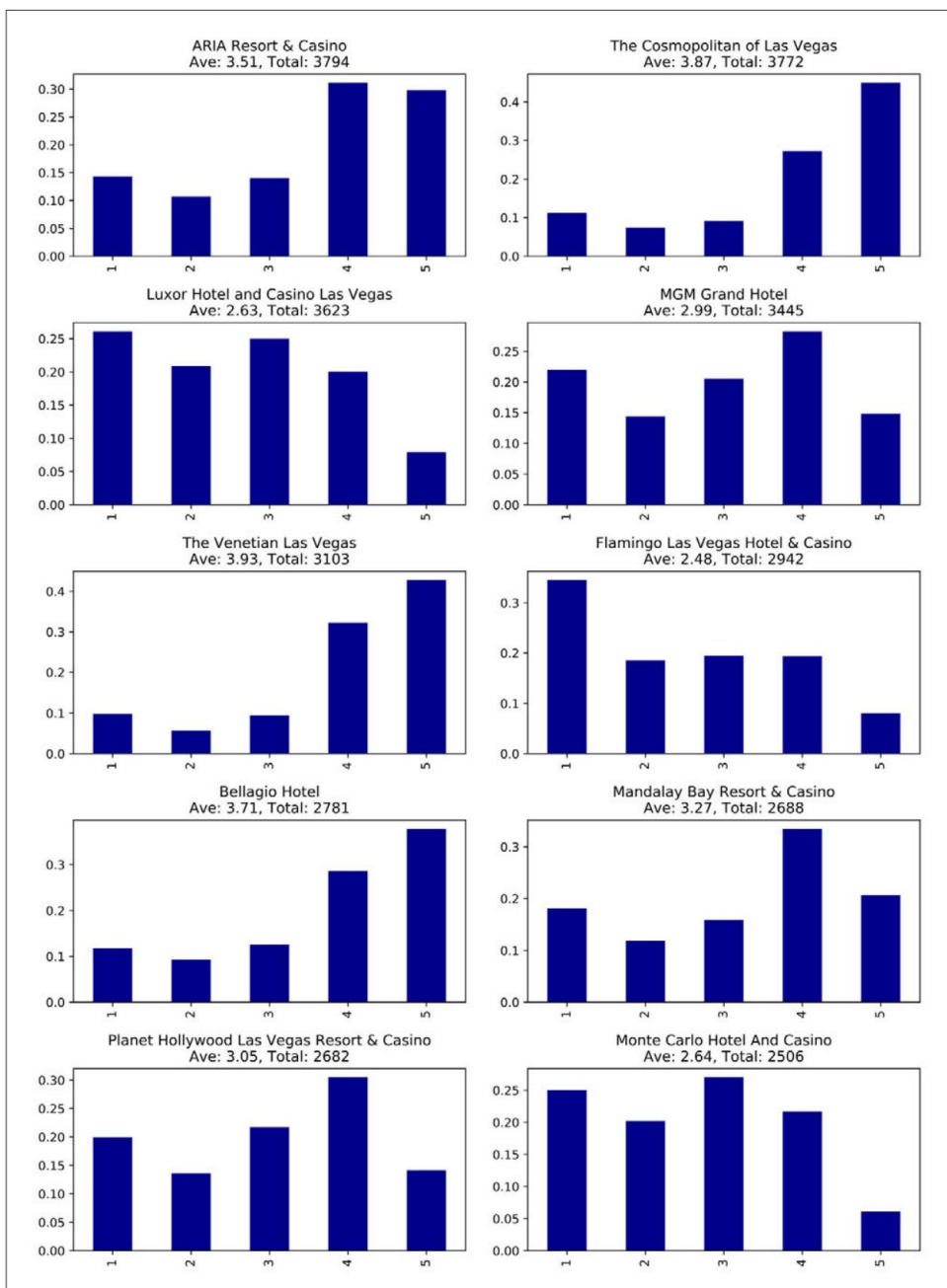


Figura 7-5. Los 10 hoteles con más reseñas, con el número de estrellas en el eje x y su porcentaje de calificación general en el eje y.

Estos hoteles tienen muchísimas reseñas, muchas más de las que cualquiera podría leer. Sería mejor mostrar a nuestros usuarios el contenido de las reseñas más relevantes y...

Hazlos más visibles en nuestra aplicación. Para realizar este análisis, pasaremos de la exploración gráfica básica al uso de algoritmos gráficos.

Encontrar reseñas influyentes de

hoteles. Una forma de decidir qué reseñas publicar es ordenarlas según la influencia del reseñador en Yelp. Ejecutaremos el algoritmo PageRank sobre el gráfico proyectado de todos los usuarios que han reseñado al menos tres hoteles. Recuerde de capítulos anteriores que una proyección puede ayudar a filtrar información no esencial, así como a añadir datos de relación (a veces inferidos). Usaremos el gráfico de amigos de Yelp (presentado en "Red social de Yelp" en la página 146) como las relaciones entre usuarios. El algoritmo PageRank descubrirá a los reseñadores con más influencia sobre más usuarios, incluso si no son amigos directos.



Si dos personas son amigas en Yelp, existen dos relaciones de AMISTAD entre ellas. Por ejemplo, si A y B son amigas, existirá una relación de AMISTAD de A hacia B y otra de B hacia A.

Necesitamos escribir una consulta que proyecte un subgráfico de usuarios con más de tres reseñas y luego ejecute el algoritmo PageRank sobre ese subgráfico proyectado.

Es más fácil entender cómo funciona la proyección de subgrafos con un pequeño ejemplo.

La Figura 7-6 muestra un gráfico de tres amigos en común: Mark, Arya y Praveena. Mark y Praveena han reseñado tres hoteles y formarán parte del gráfico proyectado.

Arya, por otro lado, sólo ha revisado un hotel y por lo tanto quedará excluido de la proyección.

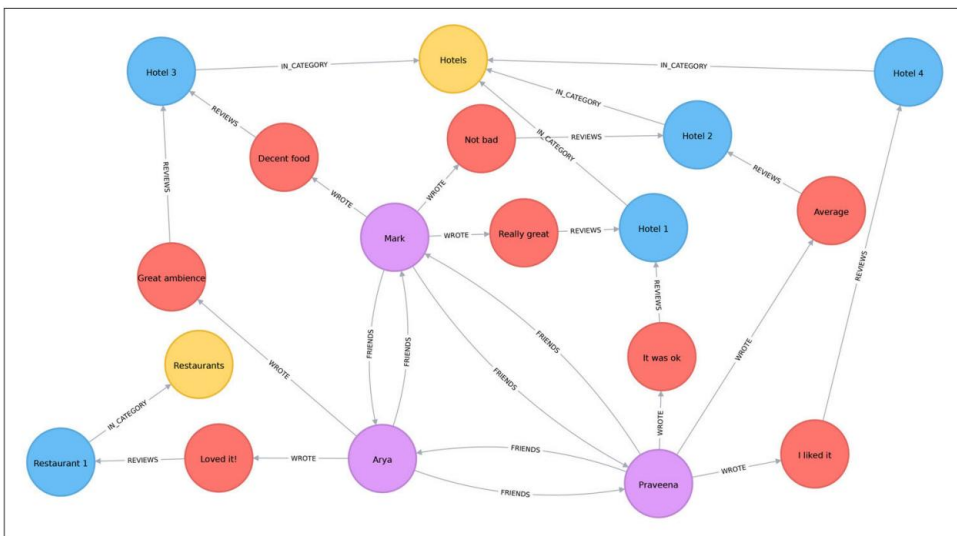


Figura 7-6. Un ejemplo de gráfico de Yelp

Nuestro gráfico proyectado solo incluirá a Mark y Praveena, como se muestra en la Figura 7-7.

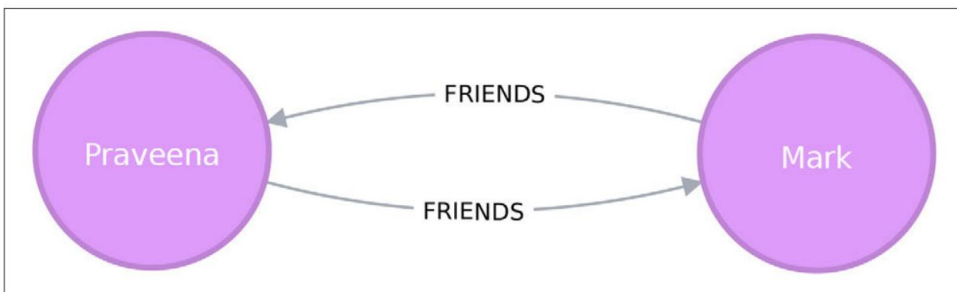


Figura 7-7. Nuestro gráfico proyectado de muestra

Ahora que hemos visto cómo funcionan las proyecciones de gráficos, avancemos. La siguiente consulta ejecuta el algoritmo PageRank sobre nuestro gráfico proyectado y almacena el resultado en la propiedad hotelPageRank de cada nodo:

LLAMAR

```
algo.pageRank( 'COINCIDIR (u:Usuario)-[:ESCRIBÍÓ]->()-[:RESEÑAS]->()-
                                     [:EN_CATEGORÍA]-> (:Categoría {nombre: $categoría})
CONTIGO, CUENTA (*) COMO reseñas
DONDE las reseñas >= $cutOff
DEVUELVE id(u) COMO id',
'COINCIDIR (u1:Usuario)-[:ESCRIBÍÓ]->()-[:RESEÑAS]->()-[:EN_CATEGORÍA]->
                                     (:Categoría {nombre: $categoría})
PARTIDO (u1)-[:AMIGOS]->(u2)
DEVUELVE id(u1) COMO origen, id(u2) COMO destino,
```

```
{graph: "cypher", write: true, writeProperty: "hotelPageRank", params: {category:
"Hoteles", cutOff: 3}}
)
```

Es posible que hayas notado que no configuramos el factor de amortiguación o el límite máximo de iteración discutido en el **Capítulo 5**. Si no se configura explícitamente, Neo4j tiene como valor predeterminado un factor de amortiguación de 0,85 con maxIterations establecido en 20`.

Ahora veamos la distribución de los valores de PageRank para que sepamos cómo filtrar nuestros datos:

```
PARTIDO (u:Usuario)
DONDE existe(u.hotelPageRank)
RETORNO cuenta(u.hotelPageRank) COMO cuenta,
promedio(u.hotelPageRank) COMO
ave, percentileDisc(u.hotelPageRank, 0.5) COMO `50%`,
percentileDisc(u.hotelPageRank, 0.75) COMO `75%`,
percentileDisc(u.hotelPageRank, 0.90) COMO `90%`,
percentileDisc(u.hotelPageRank, 0.95) COMO `95%`,
percentileDisc(u.hotelPageRank, 0.99) COMO `99%`,
percentileDisc(u.hotelPageRank, 0.999) COMO `99.9%`,
percentileDisc(u.hotelPageRank, 0.9999) COMO `99.99%`,
percentileDisc(u.hotelPageRank, 0.99999) COMO `99.999%`,
percentileDisc(u.hotelPageRank, 1) COMO `100%`
```

Si ejecutamos esa consulta obtendremos este resultado:

contar	Cra	50%	75%	90%	95%	99%	99.9%	99.99%	99.999%	100%
1326101	0,1614898	0,15	0,15	0,157497	0,181875	0,330081	1,649511	6,825738	15,27376	22,98046

Para interpretar esta tabla de percentiles, el valor del 90% de 0,157497 significa que el 90% de los usuarios tuvo una puntuación de PageRank más baja. El 99,99% refleja el rango de influencia de los mejores revisores, el 0,0001%, y el 100% es simplemente la puntuación de PageRank más alta.

Es interesante que el 90 % de nuestros usuarios tengan una puntuación inferior a 0,16, cercana al promedio general, y solo ligeramente superior al 0,15 que les asigna el algoritmo PageRank. Parece que estos datos reflejan una distribución exponencial con algunos revisores muy influyentes.

Como nos interesa encontrar solo a los usuarios más influyentes, crearemos una consulta que solo encuentre usuarios con una puntuación de PageRank dentro del 0,001 % superior. La siguiente consulta encuentra revisores con una puntuación de PageRank superior a 1,64951 (observe que este es el grupo del 99,9 %).

```
// Solo buscar usuarios que tengan una puntuación de hotelPageRank en el 0,001 % superior de usuarios
MATCH (u:User)
DONDE u.hotelPageRank > 1.64951

// Encuentra el top 10 de esos usuarios CON
u ORDENAR POR u.hotelPageRank DESC
```

LÍMITE 10

```
DEVUELVE u.nombre COMO nombre,  
u.hotelPageRank AS pageRank,  
tamaño((u)-[:ESCRIBIÓ]->()-[:RESEÑAS]->()-[:EN_CATEGORÍA]->  
(:Categoría {nombre: "Hoteles"})) AS hotelReviews,  
tamaño((u)-[:WROTE]->()) COMO totalReviews,  
tamaño((u)-[:AMIGOS]-()) COMO amigos
```

Si ejecutamos esa consulta obtendremos los resultados que se ven aquí:

nombre	pageRank	hotelOpiniones	totalOpiniones	amigos
Phil	17.361242	15	134	8154
Felipe	16.871013	21	620	9634
Villancico	12.416060999999997	6	119	6218
Misti	12.239516000000004	19	730	6230
José	12.003887499999998	5	32	6596
Miguel	11.460049	13	51	6572
...	11.431505999999997	103	1322	6498
Abby	11.376136999999998	9	82	7922
Erica	10.993773	6	15	7071
Randy	10.748785999999999	21	125	7846

Estos resultados nos muestran que Phil es el revisor más creíble, aunque no lo ha sido. Ha revisado muchos hoteles. Probablemente esté relacionado con personas muy influyentes, pero si... Si quisiera un flujo de nuevas reseñas, su perfil no sería la mejor opción. Philip tiene un Tiene una puntuación ligeramente inferior, pero tiene más amigos y ha escrito cinco veces más reseñas. que Phil. Si bien J ha escrito la mayor cantidad de reseñas de todos y tiene una cantidad razonable de Amigos, el puntaje de PageRank de J no es el más alto, pero aún está entre los 10 primeros. Para nuestra aplicación, Elija resaltar las reseñas de hoteles de Phil, Philip y J para brindarnos la combinación adecuada de Influencers y número de reseñas.

Ahora que hemos mejorado nuestras recomendaciones en la aplicación con reseñas relevantes, vamos a... Pasemos al otro lado de nuestro negocio: la consultoría.

Consultoría de negocios de viajes

Como parte de nuestro servicio de consultoría, los hoteles se suscriben para recibir alertas cuando un evento influyente... El visitante escribe sobre su estancia para que pueda tomar las medidas necesarias. Primero, veremos Calificaciones del Bellagio, ordenadas por los críticos más influyentes:

```
consulta = ""\nCOINCIDIR (b:Negocio {nombre: $hotel})\nPARTIDO (b)<-[:RESEÑAS]-(reseña)<-[:ESCRIBE]-(usuario)\nDONDE existe(usuario.hotelPageRank)\nDEVUELVE nombreUsuario COMO nombre,
```

```
usuario.hotelPageRank AS pageRank,
reseña.estrellas AS estrellas

con driver.session() como sesión:
    parámetros = { "hotel": "Hotel Bellagio" }
    df = pd.DataFrame([dict(registro) para registro en sesión.run(consulta, parámetros)])
    df = df.round(2)
    df = df[["nombre", "pageRank", "estrellas"]]

    mejores_reseñas = df.sort_values(by=["pageRank"], ascendente=False).head(10)
    print(tabulate(top_reviews, headers='keys', tablefmt='psql', showindex=False))
```

Si ejecutamos ese código obtendremos este resultado:

nombre	pageRank	estrellas
Misti	12.239516000000004	5
Miguel	11.460049	4
...	11.431505999999997	5
Erica	10.993773	4
Cristina	10.740770499999998	4
Jeremy	9.576763499999998	5
Connie	9.118103499999998	5
Joyce	7.621449000000001	4
Enrique	7.299146	5
Flora	6.7570075	4

Tenga en cuenta que estos resultados son diferentes de nuestra tabla anterior de las mejores reseñas de hoteles. Esto se debe a que aquí solo estamos mirando a los críticos que han calificado el Bellagio.

Las cosas se ven bien para el equipo de atención al cliente del hotel Bellagio, el mejor Diez influencers dan buenas calificaciones a sus hoteles. Quizás quieran fomentar esto. gente que vuelva a visitarnos y comparta sus experiencias.

¿Hay algún invitado influyente que no haya tenido una experiencia tan buena? Podemos... El siguiente código para encontrar los invitados con el PageRank más alto que calificaron su experiencia. Experiencia con menos de cuatro estrellas:

```
consulta = """
COINCIDIR (b:Negocio {nombre: $hotel})
PARTIDO (b)<[:RESEÑAS]-(reseña)<[:ESCRIBE]-(usuario)
DONDE existe(usuario.hotelPageRank) Y review.stars < $goodRating
DEVUELVE nombreUsuario COMO nombre,
        usuario.hotelPageRank AS pageRank,
        reseña.estrellas AS estrellas
"""
```

```
con driver.session() como sesión:
    parámetros = { "hotel": "Hotel Bellagio", "buena calificación": 4 }
    df = pd.DataFrame([dict(registro) para registro en sesión.run(consulta, parámetros)])
    df = df.round(2)
    df = df[["nombre", "pageRank", "estrellas"]]

    mejores_reseñas = df.sort_values(by=["pageRank"], ascendente=False).head(10)
    print(tabulate(top_reviews, headers='keys', tablefmt='psql', showindex=False))
```

Si ejecutamos ese código obtendremos los siguientes resultados:

nombre	páginaRango	estrellas
Chris	5.84	3
Lorrie	4.95	2
Dani	3.47	1
Vencedor	3.35	3
Francine	2.93	3
Rex	2.79	2
Jon	2.55	3
Raquel	2.47	3
Leslie	2.46	2
Benay	2.46	3

Nuestros usuarios con mayor puntuación que le dan al Bellagio calificaciones más bajas, Chris y Lorrie, son entre los 1.000 usuarios más influyentes (según los resultados de nuestra consulta anterior), Así que quizás sea necesario contactar personalmente. Además, dado que muchos revisores escriben durante... Durante su estancia, las alertas en tiempo real sobre personas influyentes pueden facilitar aún más la experiencia positiva. interacciones.

Promoción cruzada del Bellagio

Después de que les ayudamos a encontrar críticos influyentes, el Bellagio ahora nos ha pedido que ayudemos Identificar otras empresas para promoción cruzada con la ayuda de clientes bien conectados. clientes. En nuestro escenario, recomendamos que aumenten su base de clientes Atraer nuevos huéspedes de diferentes tipos de comunidades como una oportunidad única. Podemos usar el algoritmo de centralidad de intermediación que discutimos anteriormente para Descubra qué críticos de Bellagio no solo están bien conectados en todo Yelp red, pero también podría actuar como puente entre diferentes grupos.

Sólo nos interesa encontrar personas influyentes en Las Vegas, por lo que primero etiquetaremos a esos usuarios:

```
PARTIDO (u:Usuario)
DONDE existe((u)-[ESCRIBÍÓ]->()-[RESEÑAS]->()-[EN_LA_CIUADAD]->
(:Ciudad {nombre: "Las Vegas"}))

SET u:Las Vegas
```


Ejecutar el algoritmo de centralidad de intermediación con nuestros usuarios de Las Vegas tomaría mucho tiempo, por lo que, en su lugar, utilizaremos la variante RA-Brandes. Este algoritmo calcula una puntuación de intermediación muestreando nodos y explorando únicamente las rutas más cortas hasta cierta profundidad.

Tras algunos experimentos, mejoramos los resultados con algunos parámetros configurados de forma diferente a los valores predeterminados. Utilizaremos rutas más cortas de hasta 4 saltos (profundidad máxima de 4) y muestrearemos el 20 % de los nodos (probabilidad de 0,2). Tenga en cuenta que aumentar el número de saltos y nodos generalmente aumentará la precisión, pero a costa de un mayor tiempo para calcular los resultados. Para cualquier problema en particular, los parámetros óptimos suelen requerir pruebas para identificar el punto de rendimiento decreciente.

La siguiente consulta ejecutará el algoritmo y almacenará el resultado en la propiedad between :

```
LLAMA algo.betweenness.sampled('LasVegas', 'AMIGOS',
  {write: true, writeProperty: "entre", maxDepth: 4, probabilidad: 0.2} )
```

Antes de usar estos puntajes en nuestras consultas, escribamos una consulta exploratoria rápida para ver cómo se distribuyen los puntajes:

```
PARTIDO (u:Usuario)
DONDE existe(u.entre)
DEVUELVE count(u.between) COMO
count, avg(u.between)
COMO ave, toInteger(percentileDisc(u.between, 0.5)) COMO
`50%`, toInteger(percentileDisc(u.between, 0.75)) COMO `75%`,
toInteger(percentileDisc(u.between, 0.90)) COMO `90%`,
toInteger(percentileDisc(u.between, 0.95)) COMO `95%`,
toInteger(percentileDisc(u.between, 0.99)) COMO `99%`,
toInteger(percentileDisc(u.between, 0.999)) COMO `99.9%`,
toInteger(percentileDisc(u.between, 0.9999)) COMO `99.99%`,
toInteger(percentileDisc(u.entre, 0.99999)) COMO `99.999%`,
toInteger(percentileDisc(u.entre, 1)) COMO p100
```

Si ejecutamos esa consulta veremos el siguiente resultado:

conde avenida	50%	75%	90%	95%	99%	99.9%	99.99%	99.999%	100%
506028 320538.6014 0	10005	318944	1001655	4436409	34854988	214080923	621434012	1998032952	

La mitad de nuestros usuarios tienen una puntuación de 0, lo que significa que no están bien conectados. El percentil superior (columna del 99%) se encuentra en al menos 4 millones de rutas más cortas entre nuestro conjunto de 500.000 usuarios. En conjunto, sabemos que la mayoría de nuestros usuarios tienen poca conexión, pero unos pocos ejercen un gran control sobre la información; este es un comportamiento clásico de las redes de mundo pequeño.

Podemos averiguar quiénes son nuestros superconectores ejecutando la siguiente consulta:

```
COINCIDIR(u:Usuario)-[:ESCRIBIÓ]->()-[:RESEÑAS]->(:Negocio {nombre:"Hotel Bellagio"})
DONDE existe(u.entre)
DEVUELVE u.name COMO usuario,
    toInteger(u.between) COMO intermediación,
    u.hotelPageRank AS pageRank,
    tamaño((u)-[:ESCRIBIÓ]->()-[:RESEÑAS]->()-[:EN_CATEGORÍA]->
        (:Categoría {nombre:"Hoteles"}))
    Reseñas del hotel AS
ORDENAR POR u.entre DESC
LÍMITE 10
```

El resultado es el siguiente:

usuario	PageRank de intermediación	Reseñas de hoteles
Misti	841707563	12.239516000000004 19
Cristina	236269693	10.740770499999998 16
Erica	235806844	10.993773 6
Micro	215534452	NULO 2
...	192155233	11.431505999999997 103
Miguel	161335816	5.105143 31
Jeremy	160312436	9.576763499999998 6
Miguel	139960910	11.460049 13
Chris	136697785	5.838922499999999 5
Connie	133372418	9.118103499999998 7

Vemos aquí a algunas de las mismas personas que vimos antes en nuestra consulta de PageRank, con Mike es una excepción interesante. Fue excluido de ese cálculo porque...
No ha revisado suficientes hoteles (tres fue el límite), pero parece que es bastante...
Bien conectado en el mundo de los usuarios de Yelp de Las Vegas.

En un esfuerzo por llegar a una variedad más amplia de clientes, analizaremos otras preferencias. Los "conectores" se muestran para ver qué debemos promocionar. Muchos de estos usuarios también... restaurantes reseñados, por lo que escribimos la siguiente consulta para saber cuáles les gustan mejor:

```
// Encuentra los 50 mejores usuarios que han reseñado el Bellagio
COINCIDIR (u:Usuario)-[:ESCRIBIÓ]->()-[:RESEÑAS]->(:Negocio {nombre:"Bellagio Hotel"})
DONDE u.entre > 4436409
CON u PEDIDO POR u.entre DESC LÍMITE 50

// Encuentra los restaurantes que esos usuarios han reseñado en Las Vegas
PARTIDO (u)-[:ESCRIBE]->(reseña)-[:RESEÑAS]-(:negocio)
DONDE (negocio)-[:EN_CATEGORÍA]->(:Categoría {nombre:"Restaurantes"})
AND (negocio)-[:IN_CITY]->(:Ciudad {nombre:"Las Vegas"})

// Solo incluye restaurantes que tengan más de 3 reseñas de estos usuarios
CON negocio, avg(review.stars) COMO promedioReview, count(*) COMO númeroDeReviews
```

```
DONDE númeroDeReseñas >= 3

DEVOLVER nombre_empresa COMO empresa, reseña_promedio, númeroDeReseñas
ORDENAR POR promedioReseña DESC, númeroDeReseñas DESC
LÍMITE 10
```

Esta consulta encuentra nuestros 50 conectores más influyentes y encuentra los 10 mejores hoteles de Las Vegas. restaurantes donde al menos 3 de ellos han calificado el restaurante. Si lo publicamos, veremos...
Salida mostrada aquí:

negocio	número promedio de reseñas	
Restaurante de carnes Jean Georges	5.0	6
Casa de sushi Goyemon	5.0	6
El arte de los sabores	5.0	4
es de José Andrés	5.0	4
Parma del chef Marc	5.0	4
Yonaka japonés moderno	5.0	4
kabuto	5.0	4
Cosecha de Roy Ellamar	5.0	3
Portofino del Chef Michael LaPlaca 5.0		3
Restaurante Montesano	5.0	3

Ahora podemos recomendar que el Bellagio realice una promoción conjunta con estos restau
despotrica para atraer nuevos huéspedes de grupos a los que normalmente no podrían llegar. Superconec
Los turistas que califican bien al Bellagio se convierten en nuestro representante para estimar qué restaurantes
Podría captar la atención de nuevos tipos de visitantes objetivo.
Ahora que hemos ayudado al Bellagio a llegar a nuevos grupos, vamos a ver cómo podemos...
Puede utilizar la detección de la comunidad para mejorar aún más nuestra aplicación.

Encontrar categorías similares

Mientras nuestros usuarios finales usan la aplicación para encontrar hoteles, queremos mostrarles otras empresas.
empresas que podrían interesarles. El conjunto de datos de Yelp contiene más de 1000 categorías.
Gories, y parece probable que algunas de esas categorías sean similares entre sí.
Utilice esa similitud para hacer recomendaciones dentro de la aplicación para nuevos negocios que nuestra
Los usuarios probablemente lo encontrarán interesante.

Nuestro modelo gráfico no tiene ninguna relación entre categorías, pero podemos usar
las ideas descritas en “Gráficos monopartitos, bipartitos y k-partitos” en la página 24
Construir un gráfico de similitud de categorías basado en cómo las empresas se categorizan a sí mismas.
Por ejemplo, imaginemos que solo una empresa se clasifica en Hoteles y
Recorridos históricos, como se ve en la Figura 7-8.

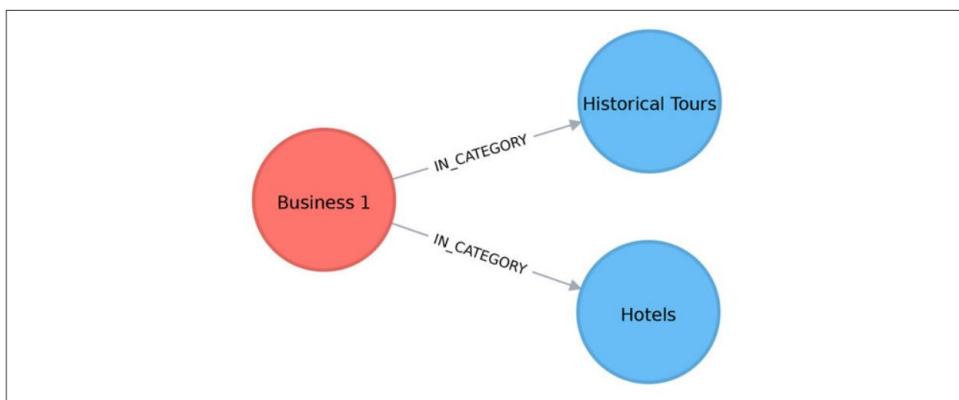


Figura 7-8. Una empresa con dos categorías

Esto daría como resultado un gráfico proyectado que tiene un vínculo entre Hoteles y Tours Históricos con un peso de 1, como se ve en la [Figura 7-9](#).

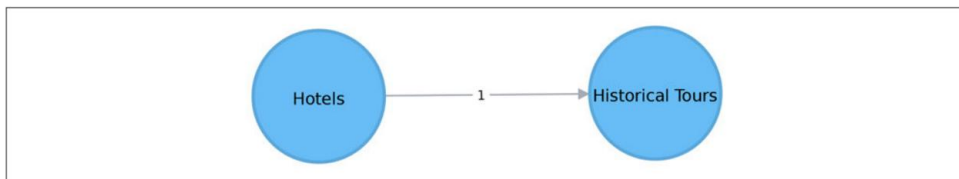


Figura 7-9. Gráfico de categorías proyectadas

En este caso, no es necesario crear el gráfico de similitud; en su lugar, podemos ejecutar un algoritmo de detección de comunidades, como la Propagación de Etiquetas, sobre un gráfico de similitud proyectado. Usar la Propagación de Etiquetas agrupará eficazmente las empresas en torno a la supercategoría con la que más tienen en común:

```

LLAMA algo.labelPropagation.stream(
  'COINCIDIR (c: Categoría) DEVOLVER id(c) COMO id',
  'COINCIDIR (c1: Categoría)-[:EN_CATEGORÍA]-[:EN_CATEGORÍA]->(c2: Categoría)
  DONDE id(c1) < id(c2)
  DEVUELVE id(c1) COMO origen, id(c2) COMO destino, count(*) COMO peso, {graph:
  "cypher"}
)
YIELD nodeld, etiqueta
COINCIDIR (c: Categoría) DONDE id(c) = nodeld
FUSIONAR (sc: SuperCategory {nombre: "SuperCategory-" + etiqueta})
FUSIONAR (c)-[:EN_SUPER_CATEGORÍA]->(sc)
  
```

Démosles a esas supercategorías un nombre más amigable: el nombre de su categoría más grande funciona bien aquí:

```

COINCIDIR (sc: SuperCategoría)-[:EN_SUPER_CATEGORÍA]->(categoría)
CON sc, categoría, tamaño((categoría)-[:IN_CATEGORY]-()) como tamaño
ORDENAR POR talla DESC
  
```

CON sc, recopilar(categoría.nombre)[0] como la categoría más grande ESTABLECER
sc.nombreamable = "SuperCat " + la categoría más grande

Podemos ver una muestra de categorías y supercategorías en la Figura 7-10.

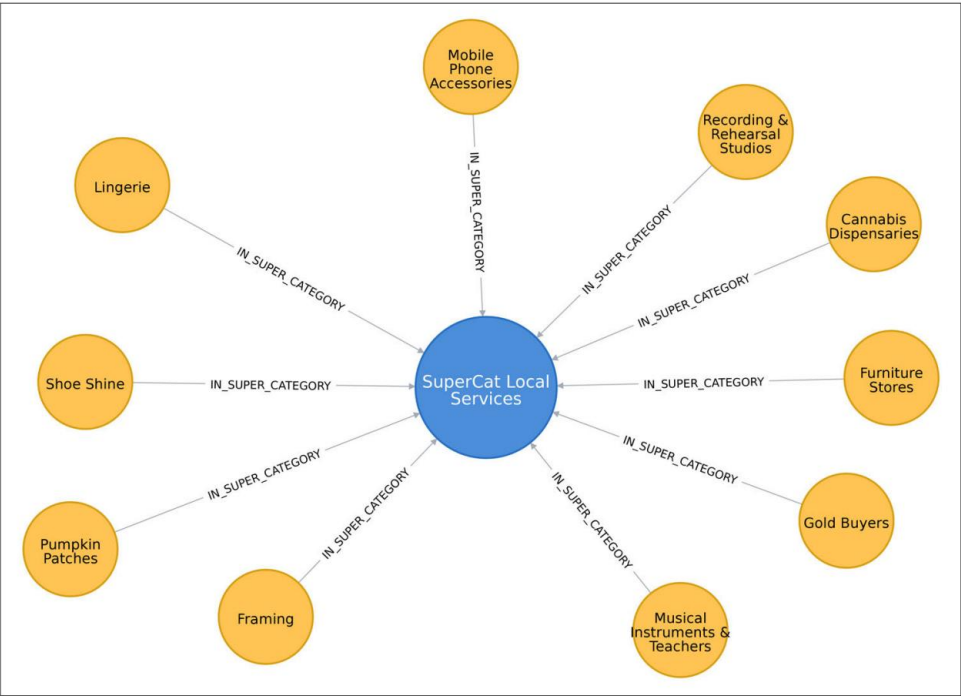


Figura 7-10. Categorías y supercategorías

La siguiente consulta encuentra las categorías similares más frecuentes a Hoteles en Las

Las Vegas:

```
COINCIDIR (hoteles:Categoría {nombre: "Hoteles"}),
    (lasVegas:Ciudad {nombre: "Las Vegas"}), (hoteles-
    [:EN_SUPER_CATEGORÍA]->())<[:EN_SUPER_CATEGORÍA]- (otraCategoría)

DEVUELVE otherCategory.name COMO otherCategory,
    tamaño((otherCategory)<[:IN_CATEGORÍA]-(:Business)- [:IN_CITY]-
    >(lasVegas)) COMO negocios

ORDENAR POR recuento DESC
LÍMITE 10
```

Si ejecutamos esa consulta veremos el siguiente resultado:

otraCategoría	negocios
Excursiones	189
Alquiler de coches	160

otraCategoría	negocios
Limusinas	84
Complains turísticos	73
Lanzaderas al aeropuerto	52
Taxis	35
Alquileres vacacionales	29
Aeropuertos	25
Aerolíneas	23
Alquiler de motocicletas	19

¿Les parecen extraños estos resultados? Obviamente, los taxis y los tours no son hoteles, pero recuerden que esto se basa en categorizaciones autoinformadas. Lo que el algoritmo de Propagación de Etiquetas realmente nos muestra en este grupo de similitud son negocios y servicios adyacentes. hielos.

Ahora busquemos algunas empresas con una calificación superior al promedio en cada una de esas categorías:

```
// Encuentra negocios en Las Vegas que tengan la misma SuperCategoría que los Hoteles
COINCIDIR (hoteles:Categoría {nombre: "Hoteles"}), (hoteles)-
[EN_SUPER_CATEGORÍA]->(<[EN_SUPER_CATEGORÍA]- (otraCategoría),
(otraCategoría)-
[EN_CATEGORÍA]->(negocios)
DÓNDE (negocio)-[IN_CITY]->(:Ciudad {nombre: "Las Vegas"})

// Seleccione 10 categorías aleatorias y calcule la calificación de estrellas del percentil 90 CON otherCategory, count(*) COMO
count, collect(business) COMO enterprises,
percentileDisc(business.averageStars, 0.9) COMO
p90Stars ORDER BY rand() DESC LIMIT 10

// Seleccione empresas de cada una de esas categorías que tengan una calificación promedio // mayor que el percentil 90 usando
una comprensión de patrones CON otherCategory, [b en empresas donde b.averageStars >= p90Stars]

Empresas AS

// Seleccione una empresa por categoría
CON otraCategoría, negocios[toInteger(rand() * tamaño(negocios))] COMO negocio

DEVUELVE otherCategory.name COMO otherCategory, business.name
COMO business, business.averageStars
COMO averageStars
```

En esta consulta, utilizamos la comprensión de patrones por primera vez. La comprensión de patrones es una construcción sintáctica para crear una lista basada en la coincidencia de patrones. Encuentra un patrón especificado mediante una cláusula MATCH con una cláusula WHERE para predicados y luego genera un

Proyección personalizada. Esta función de Cypher se agregó inspirándose en **GraphQL**, un lenguaje de consulta para API.

Si ejecutamos esa consulta veremos el siguiente resultado:

otraCategoría	negocio	Estrellas promedio
Alquiler de motocicletas	Alquiler de tirachinas Adrenaline Rush 5.0	
Snorkel	Sin City Scuba	5.0
Casas de huéspedes	Hotel Del Kacvinsky	5.0
Alquiler de coches	El equipo líder	5.0
Tours gastronómicos	Pruebe los recorridos gastronómicos BUZZ	5.0
Aeropuertos	Soporte de vuelo exclusivo	5.0
Transporte público JetSuiteX		4.6875
Estaciones de esquí	Trikke Las Vegas	4.833333333333332
Servicio de coches de ciudad MW Travel Vegas		4.866666666666665
Campamentos	Camping McWilliams	3.875

Luego podemos hacer recomendaciones en tiempo real en función de la aplicación inmediata del usuario. comportamiento. Por ejemplo, mientras los usuarios buscan hoteles en Las Vegas, ahora podemos... iluminar una variedad de negocios adyacentes de Las Vegas con buenas calificaciones. Podemos generalizar Estos enfoques se aplican a cualquier categoría de negocio, como restaurantes o teatros, en cualquier ubicación.

Ejercicios de lectura

- ¿Puedes representar gráficamente cómo varían las reseñas de los hoteles de una ciudad a lo largo del tiempo?
- ¿Qué pasa con un hotel en particular o con otro negocio?
- ¿Existen tendencias (estacionales o de otro tipo) en popularidad?
- ¿Los revisores más influyentes se conectan (hacen enlaces externos) sólo con otros revisores influyentes? ¿revisores?

Análisis de datos de vuelos de aerolíneas con Apache Spark

En esta sección, utilizaremos un escenario diferente para ilustrar el análisis del aeropuerto de EE. UU. Datos con Spark. Imagina que eres un científico de datos con una agenda de viajes considerable. y me gustaría profundizar en la información sobre vuelos y retrasos. Primero...

Explore la información del aeropuerto y del vuelo y luego profundice en los retrasos en dos puntos específicos. aeropuertos. La detección comunitaria se utilizará para analizar rutas y encontrar el mejor uso de nuestros puntos de viajero frecuente.

La Oficina de Estadísticas de Transporte de EE. UU. ofrece una **cantidad considerable de información sobre transporte**. Para nuestro análisis, utilizaremos sus datos de puntualidad de viajes aéreos de mayo de 2018, que incluyen vuelos con origen y destino en Estados Unidos durante ese mes. Para añadir más detalles sobre los aeropuertos, como la ubicación, también cargaremos datos de una fuente independiente: **OpenFlights**.

Carguemos los datos en Spark. Como en las secciones anteriores, nuestros datos están en archivos CSV disponibles en el **repositorio de Github del libro**.

```

nodos = spark.read.csv("datos/aeropuertos.csv", encabezado=Falso)

nodos_limpiados = (nodos.select("_c1", "_c3", "_c4", "_c6", "_c7").filter("_c3 = 'Estados
Unidos'").withColumnRenamed("_c1",
"nombre").withColumnRenamed("_c4",
"id").withColumnRenamed("_c6",
"latitud").withColumnRenamed("_c7",
"longitud").drop("_c3")) nodos_limpiados =

nodos_limpiados[nodos_limpiados["id"] != "\\N"]

relaciones = spark.read.csv("datos/188591317_T_ONTIME.csv", encabezado=Verdadero)

relaciones_limpiadas = (relaciones.select("ORIGEN",
"DESTINO", "FECHA_FL", "RETRASO_SALIDA",
"RETRASO_ARR", "DISTANCIA", "NÚMERO_DE_COLAS", "NÚMERO_DE_FLO",
"HORA DE SALIDA CRS", "HORA DE
LLEGADA CRS",
"TRANSPORTISTA
ÚNICO").withColumnRenamed("ORIGEN",
"origen").withColumnRenamed("DESTINO",
"horario").withColumnRenamed("RETRASO DE SALIDA", "
Retraso de depto").withColumnRenamed("RETRASO DE
LLEGADA", "Retraso de llegada").withColumnRenamed("NÚMERO
DE COLA", "NÚMERO DE COLA").withColumnRenamed
("NÚMERO DE VUELO", "NÚMERO DE
VUELO").withColumnRenamed("FECHA DE VUELO", "
fecha").withColumnRenamed("HORA DE SALIDA CRS",
"hora").withColumnRenamed("HORA DE LLEGADA CRS", "hora de
llegada").withColumnRenamed("DISTANCIA",
"distancia").withColumnRenamed("ÚNICA_OPERADORA",
"aerolínea").withColumn("RetrasoDepartamento",

F.col("RetrasoDepartamento").cast(TipoDeFlotante()))).withColumn("RetrasoLlegada", F.col("RetrasoLlegada")
)

g = GraphFrame(nodos_limpiados, relaciones_limpiadas)

```

Tenemos que hacer una limpieza en los nodos porque algunos aeropuertos no tienen códigos de aeropuerto válidos. Daremos a las columnas nombres más descriptivos y convertiremos algunos elementos.

en tipos numéricos apropiados. También debemos asegurarnos de tener columnas con los nombres id, dst y src, ya que esto es lo que espera la biblioteca GraphFrames de Spark.

También crearemos un DataFrame independiente que asigna los códigos de las aerolíneas a sus nombres. Lo usaremos más adelante en este capítulo:

```
referencia_aerolíneas = (spark.read.csv("datos/aerolíneas.csv")
    .select("_c1",
        "_c3") .withColumnRenamed("_c1",
        "nombre") .withColumnRenamed("_c3", "código"))

referencia_de_aerolíneas = referencia_de_aerolíneas[referencia_de_aerolíneas["código"] != "null"]
```

Análisis exploratorio Comencemos

con un análisis exploratorio para ver cómo se ven los datos.

Primero veamos cuántos aeropuertos tenemos:

```
g.vertices.count()

1435
```

¿Y cuántas conexiones tenemos entre estos aeropuertos?

```
g.edges.count()

616529
```

Aeropuertos populares:

¿Qué aeropuertos tienen más vuelos de salida? Podemos calcular el número de vuelos de salida de un aeropuerto utilizando el algoritmo de centralidad de grados:

```
aeropuertos_grado = g.outDegrees.withColumnRenamed("id", "old")

grado_completo_aeropuertos = (grado_aeropuertos
    .join(g.vertices, aeropuertos_grado.old == g.vertices.id) .sort("outDegree",
        ascendente=Falso) .select("id", "nombre",
        "outDegree"))

grado_de_aeropuertos_completos.show(n=10, truncar=Falso)
```

Si ejecutamos ese código veremos el siguiente resultado:

nombre de identificación	Grado de salida
Aeropuerto Internacional Hartsfield-Jackson de Atlanta (ATL)	33837
Aeropuerto Internacional O'Hare de Chicago (ORD)	28338
Aeropuerto Internacional de Dallas-Fort Worth DFW	23765
Aeropuerto Internacional Charlotte Douglas (CLT)	20251
Aeropuerto Internacional de Denver	19836

nombre de identificación	Grado de salida
Aeropuerto Internacional de Los Ángeles (LAX)	19059
Aeropuerto Internacional Phoenix Sky Harbor	15103
Aeropuerto Internacional de San Francisco SFO	14934
Aeropuerto LGA La Guardia	14709
Aeropuerto Intercontinental George Bush de Houston (IAH)	14407

La mayoría de las grandes ciudades de EE. UU. aparecen en esta lista: Chicago, Atlanta, Los Ángeles y Nueva York tienen aeropuertos populares. También podemos crear una representación visual de los vuelos de salida con el siguiente código:

```
plt.style.use('cinco treinta y ocho')
```

```
ax =
```

```
(full_airports_degree .toPandas() .head(10) .plot(kind='bar', x='id', y='outDegree', legend=Ninguno))
```

```
ax.xaxis.set_label_text("")
plt.xticks(rotación=45)
plt.tight_layout()
plt.show()
```

El gráfico resultante se puede ver en [la Figura 7-11](#).

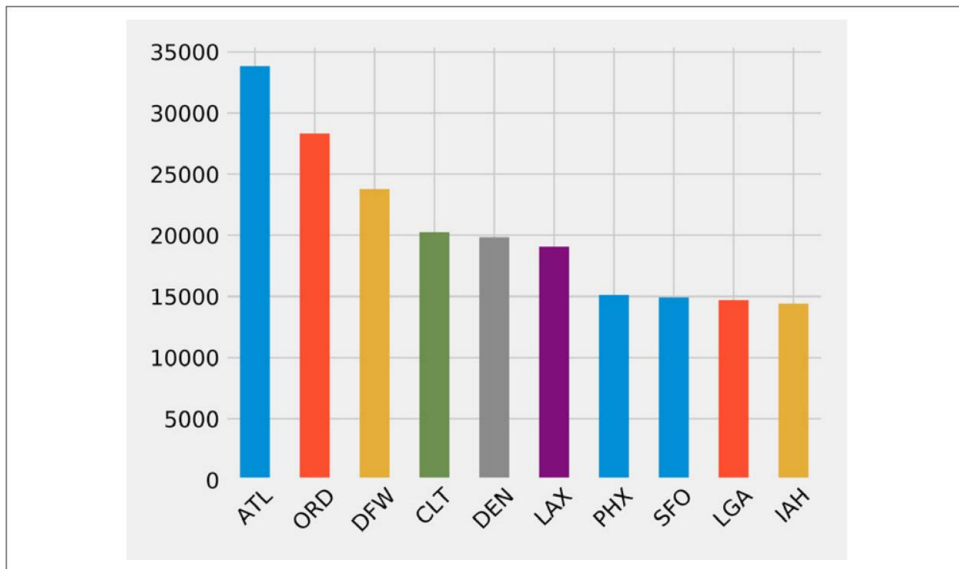


Figura 7-11. Vuelos de salida por aeropuerto

Es bastante sorprendente cómo de repente disminuye el número de vuelos. Aeropuerto Internacional de Denver El aeropuerto (DEN), el quinto aeropuerto más popular, tiene poco más de la mitad de vuelos salientes. peleas como el Aeropuerto Internacional Hartsfield Jackson de Atlanta (ATL), en primer lugar.

Retrasos de ORD

En nuestro escenario, viajamos con frecuencia entre las costas oeste y este y queremos ver retrasos en un centro de conexiones intermedio como el Aeropuerto Internacional O'Hare de Chicago (ORD).

Este conjunto de datos contiene datos sobre retrasos de vuelos, por lo que podemos profundizar en el tema.

El siguiente código encuentra el retraso promedio de los vuelos que salen de ORD agrupados por el aeropuerto de destino:

```
vuelos_retrasados = (g.bordes
    .filter("src = 'ORD' y deptDelay > 0")
    .groupBy("dst")
    .agg(F.avg("DeptDelay"), F.count("DeptDelay"))
    .withColumn("retraso promedio",
        F.round(F.col("avg(deptDelay)", 2))
    .withColumn("númeroDeRetrasos",
        F.col("count(deptDelay)"))

(vuelos_retrasados
    .join(g.vertices, vuelos_retrasados.dst == g.vertices.id)
    .sort(F.desc("retardoPromedio"))
    .select("dst", "nombre", "retrasopromedio", "númeroDeRetrasos")
    .show(n=10, truncar=Falso))
```

Una vez que hemos calculado el retraso medio agrupado por destino unimos los resultados Spark DataFrame con un DataFrame que contiene todos los vértices, de modo que podamos imprimir el nombre completo del aeropuerto de destino.

Al ejecutar este código se devuelven los 10 destinos con los peores retrasos:

nombre del horario de verano	Retraso promedio númeroDeRetrasos	
CKB Aeropuerto del Centro Norte de Virginia	145.08	12
Occidental OGG	119.67	9
Aeropuerto de Kahului MQT Aeropuerto	114.75	12
Internacional Sawyer MOB	102.2	10
Aeropuerto Regional de Mobile	101.18	17
TTN Aeropuerto de Trenton Mercer	98.5	28
AVL Aeropuerto Regional de Asheville ISP	94.08	13
Aeropuerto Mac Arthur de Long Island ANC Aeropuerto Internacional		23
Ted Stevens Anchorage 83.74 BTV Aeropuerto	83.2	25
Internacional de Burlington CMX Aeropuerto Memorial del Condado de Houghton	79.18	17

Esto es interesante, pero hay un dato que realmente destaca: ¡12 vuelos de ORD a CKB se han retrasado más de dos horas en promedio! Encuentremos los vuelos entre esos aeropuertos y veamos qué está pasando:

```
desde_expr = 'id = "ORD"'
hasta_expr = 'id = "CKB"'
ord_a_ckb = g.bfs(desde_expr, hasta_expr)

ord_to_ckb =

ord_to_ckb.select( F.col("e0.fecha"),
  F.col("e0.hora"), F.col("e0.númerodevuelo"), F.col("e0.retrasodedept"))
```

Luego podemos trazar los vuelos con el siguiente código:

```
ax =

(ord_to_ckb .sort("fecha") .toPandas() .plot(kind='bar', x='fecha', y='deptDelay', legend=Ninguno))

ax.xaxis.set_label_text("")
plt.tight_layout()
plt.show()
```

Si ejecutamos ese código obtendremos el gráfico de [la Figura 7-12](#).

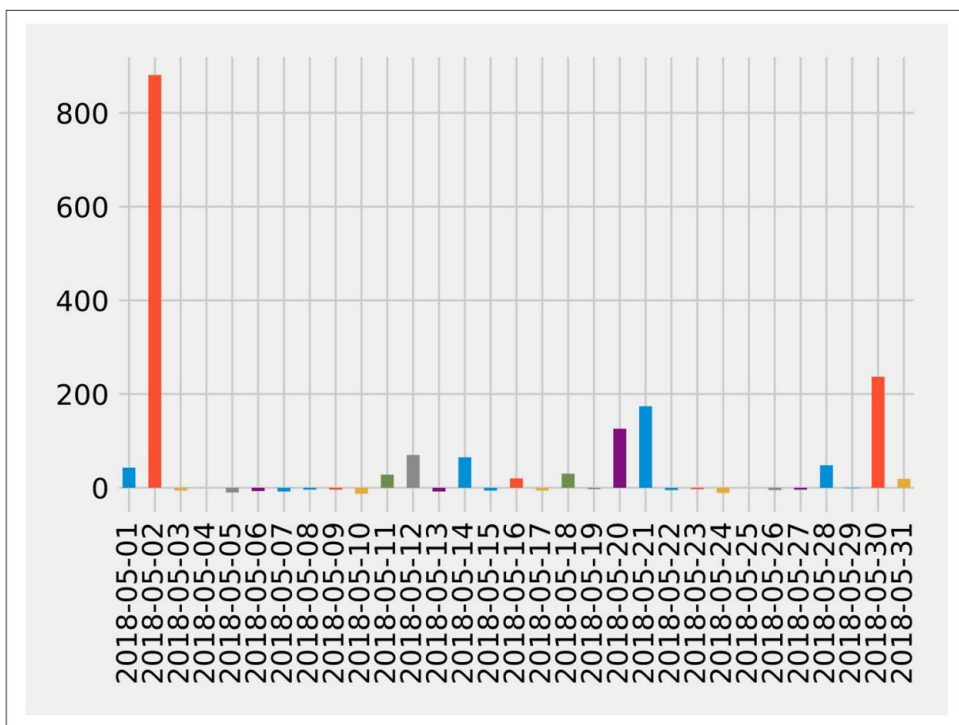


Figura 7-12. Vuelos de ORD a CKB

Aproximadamente la mitad de los vuelos se retrasaron, pero el retraso de más de 14 horas el 2 de mayo de 2018 alteró enormemente el promedio.

¿Qué sucede si queremos detectar retrasos al entrar y salir de un aeropuerto costero? Estos aeropuertos suelen verse afectados por condiciones meteorológicas adversas, por lo que podríamos encontrar retrasos interesantes.

Mal día en el SFO.

Consideremos los retrasos en un aeropuerto conocido por problemas de "techo bajo" relacionados con la niebla: el Aeropuerto Internacional de San Francisco (SFO). Un método de análisis sería observar los motivos, que son subgráficos o patrones recurrentes.



El equivalente a los motivos en Neo4j son los patrones gráficos, que se encuentran utilizando la cláusula MATCH o con expresiones de patrón en Cypher.

GraphFrames nos permite **buscar motivos**, por lo que podemos utilizar la estructura de los vuelos como parte de una consulta. Usemos motivos para encontrar los vuelos con más retrasos que llegan y salen de SFO el 11 de mayo de 2018. El siguiente código detectará estos retrasos:

```
motivos = (g.find("(a)-[ab]->(b); (b)-[bc]->(c)")
  .filter("""(b.id = 'SFO') y
    (ab.date = '2018-05-11' y bc.date = '2018-05-11') y
    (ab.arrDelay > 30 o bc.deptDelay > 30) y
    (ab.númerodevuelo = bc.númerodevuelo) y
    (ab.aerolínea = bc.aerolínea) y
    (tiempo ab < tiempo bc)"""))
```

El motivo (a)-[ab]->(b); (b)-[bc]->(c) encuentra vuelos que entran y salen de

El mismo aeropuerto. Filtramos el patrón resultante para encontrar vuelos con:

- Una secuencia donde el primer vuelo llega a SFO y el segundo vuelo sale de la OFS
- Retrasos de más de 30 minutos al llegar o salir del SFO
- El mismo número de vuelo y aerolínea

Luego podemos tomar el resultado y seleccionar las columnas que nos interesan:

```
resultado = (motivos.withColumn("delta", motivos.bc.deptDelay - motivos.ab.arrDelay)
  .select("ab", "bc", "delta")
  .sort("delta", ascendente=Falso))

resultado.seleccionar(
  F.col("ab.src").alias("a1"),
  F.col("ab.time").alias("a1DeptTime"),
  F.col("ab.arrDelay"),
  F.col("ab.dst").alias("a2"),
  F.col("bc.time").alias("a2DeptTime"),
  F.col("bc.deptDelay"),
  F.col("bc.dst").alias("a3"),
  F.col("ab.aerolínea"),
  F.col("ab.númerodevuelo"),
  F.col("delta")
).espectáculo()
```

También estamos calculando el delta entre los vuelos de llegada y salida para ver retrasos que realmente podemos atribuir a SFO.

Si ejecutamos este código obtendremos el siguiente resultado:

aerolínea	número de vuelo	a1	a1DeptTime	arRetraso	a2	a2DeptTime	deptoRetraso	a3	delta
WN	1454	PDX	1130	BUR	196.0	-18.0	OFS	1350	178.0
OO	5700	ACV	1755	-9.0	OFS	2235	64.0	RDM	73.0
UA	753	BWI	700	-3.0	OFS	1125	49.0	DIA	52.0

aerolínea número de vuelo a1 a1DeptoHora arRetraso a2 a2DeptoHora deptoRetraso a3 delta						
UA	1900	ATL 740	40.0	OFS 1110	77.0	SAN 37.0
WN	157	BUR 1405	25.0	OFS 1600	39.0	PDX 14.0
DL	745	DTW 835	34.0	OFS 1135	44.0	DTW 10.0
WN	1783	GUARNICIÓN 1830	25.0	OFS 2045	33.0	BUR 8.0
WN	5789	PDX 1855	119.0	OFS 2120	117.0	DEN -2.0
WN	1585	BUR 2025	31.0	OFS 2230	11.0	PHX -20.0

El peor infractor, WN 1454, se muestra en la fila superior; llegó temprano pero partió.

Casi tres horas de retraso. También podemos ver que hay algunos valores negativos en el columna `arrDelay`; esto significa que el vuelo a SFO fue anticipado.

Tenga en cuenta también que algunos vuelos, como el WN 5789 y el WN 1585, recuperaron tiempo mientras sobre el terreno en SFO, como se muestra con un delta negativo.

Aeropuertos interconectados por aerolínea

Ahora digamos que hemos viajado mucho y estamos decididos a acumular esos puntos de viajero frecuente.

Los métodos para ver tantos destinos de la forma más eficiente posible están a punto de expirar. Si empezamos...

Desde un aeropuerto específico de EE. UU., ¿cuántos aeropuertos diferentes podemos visitar y a los que podemos regresar?

¿El aeropuerto de partida utiliza la misma aerolínea?

Primero identifiquemos todas las aerolíneas y calculemos cuántos vuelos hay en cada una de ellas.
a ellos:

```
aerolíneas = (g.bordes
              .groupBy("aerolínea")
              .agg(F.count("aerolínea").alias("vuelos"))
              .sort("vuelos", ascendente=False))

nombre_completo_aerolíneas = (referencia_de_aerolíneas
                              .join(aerolíneas, aerolíneas.aerolínea
                                     == código_de_referencia_de_aerolíneas)
                              .select("código", "nombre", "vuelos"))
```

Y ahora vamos a crear un gráfico de barras que muestre nuestras aerolíneas:

```
ax = (nombre_completo_aerolíneas.toPandas()
      .plot(kind='bar', x='nombre', y='vuelos', legend=Ninguno))

ax.xaxis.set_label_text("")
plt.tight_layout()
plt.show()
```

Si ejecutamos esa consulta obtendremos el resultado en la [Figura 7-13](#).

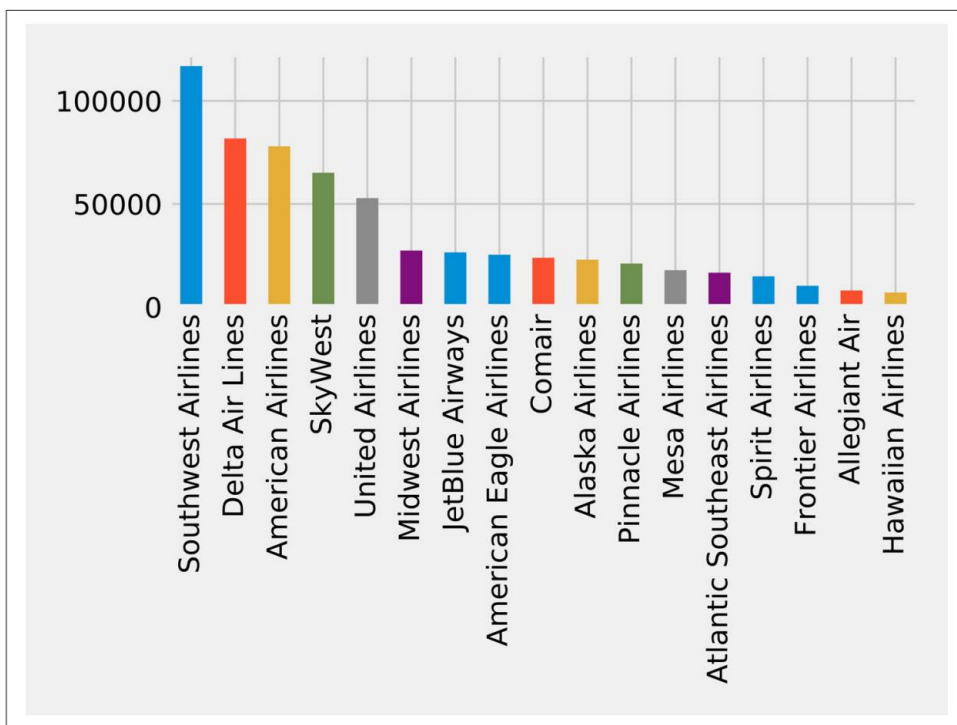


Figura 7-13. Número de vuelos por aerolínea

Ahora escribamos una función que utilice el algoritmo de componentes fuertemente conectados para encontrar agrupaciones de aeropuertos para cada aerolínea donde todos los aeropuertos tengan vuelos hacia y desde todos los demás aeropuertos en ese grupo:

```
def find_scc_components(g, aerolínea):
    # Crea un subgráfico que contenga solo vuelos de la aerolínea proporcionada
    airline_relationships = g.edges[g.edges.airline == airline]
    airline_graph = GraphFrame(g.vertices, airline_relationships)

    # Calcular los componentes fuertemente conectados
    scc = airline_graph.stronglyConnectedComponents(maxIter=10)

    # Encuentra el tamaño del componente más grande y devuelve ese
    valor

    (scc.groupBy("component").agg(F.count("id").alias("size")).sort("size", ascending=False).take(1)[0]["size"])
```

Podemos escribir el siguiente código para crear un DataFrame que contenga cada aerolínea y el número de aeropuertos de su componente fuertemente conectado más grande:


```
# Calcula el componente fuertemente conectado más grande para cada aerolínea
airline_scc = [(airline,
find_scc_components(g, airline)) for airline in airlines.toPandas()["airline"].tolist()]
airline_scc_df = spark.createDataFrame(airline_scc, ['id', 'sccCount'])

# Unir el DataFrame de SCC con el DataFrame de las aerolíneas para que podamos mostrar # la cantidad de
vuelos que tiene una aerolínea junto con la cantidad de # aeropuertos alcanzables en su
componente más grande
airline_reach = (airline_scc_df

.join(nombre_completo_aerolíneas, nombre_completo_aerolíneas.código ==
aerolínea_scc_df.id).select("código", "nombre", "vuelos",
"sccCount").sort("sccCount", ascendente=Falso))
```

Y ahora vamos a crear un gráfico de barras que muestre nuestras aerolíneas:

```
ax = (airline_reach.toPandas()).plot(kind='bar',
x='name', y='sccCount', legend=None))

ax.xaxis.set_label_text("") plt.tight_layout()
plt.show()
```

Si ejecutamos esa consulta obtendremos el resultado en la Figura 7-14.

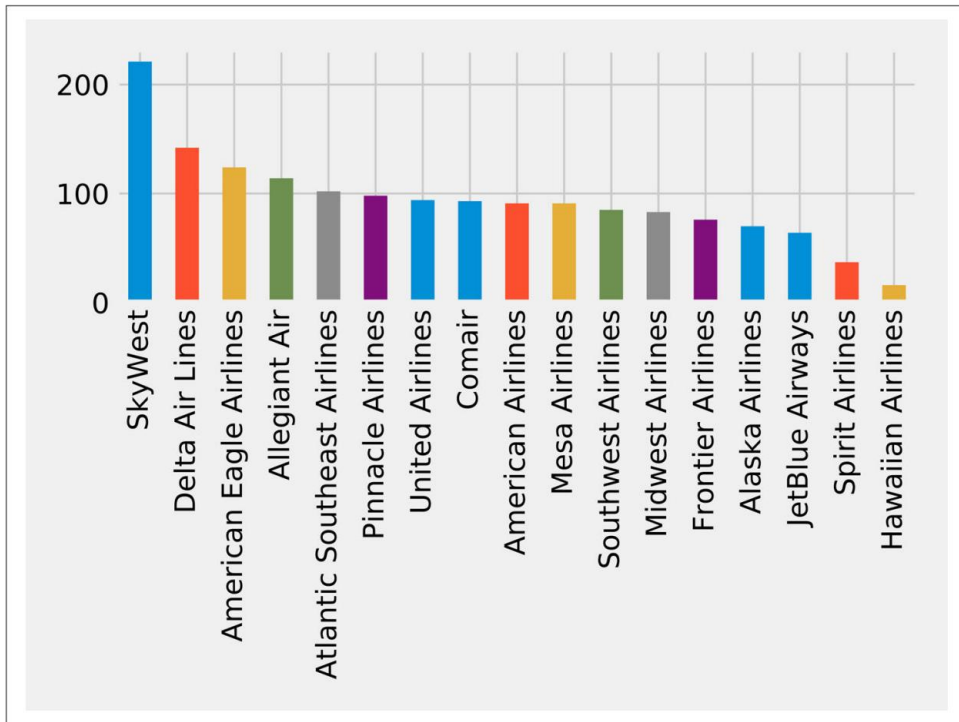


Figura 7-14. Número de aeropuertos accesibles por aerolínea

SkyWest tiene la comunidad más grande, con más de 200 aeropuertos fuertemente conectados. Esto Podría reflejar parcialmente su modelo de negocio como una aerolínea afiliada que opera aeronaves. utilizado en vuelos de aerolíneas asociadas. Southwest, por otro lado, tiene el precio más alto. Número de vuelos pero sólo conecta alrededor de 80 aeropuertos.

Ahora digamos que la mayoría de los puntos de viajero frecuente que tenemos son con Delta Airlines (DL). ¿Podemos encontrar aeropuertos que formen comunidades dentro de la red para ese destino en particular? ¿aerolínea?

```
relaciones_aerolínea = g.edges.filter("aerolínea = 'DL'")
aerolínea_gráfica = GraphFrame(g.vértices, aerolínea_relaciones)

clústeres = airline_graph.labelPropagation(maxIter=10)
(grupos
  .sort("etiqueta")
  .groupby("etiqueta")
  .agg(F.collect_list("id").alias("aeropuertos"),
      F.count("id").alias("count"))
  .sort("count", ascendente=Falso)
  .show(truncar=70, n=10))
```

Si ejecutamos esa consulta veremos el siguiente resultado:

etiqueta	contar
1606317768706 [IND, ORF, ATW, RIC, TRI, XNA, ECP, AVL, JAX, SYR, BHM, GSO, MEM, C...	89
1219770712067 [GEG, SLC, DTW, LAS, MAR, BOS, MSN, SNA, JFK, TVC, LIH, JAC, FLL, M...	53
17179869187 [RHW]	1
25769803777 [CWT]	1
25769803776 [CDW]	1
25769803782 [CONCORDMENTO]	1
25769803778 [DRT]	1
25769803779 [FOK]	1
25769803781 [HVR]	1
42949672962 [GTF]	1

La mayoría de los aeropuertos que utiliza DL se han agrupado en dos grupos; analicemos en detalle esos. Hay demasiados aeropuertos para mostrar aquí, así que solo mostraremos los aeropuertos con El mayor grado (vuelos de entrada y salida). Podemos escribir el siguiente código para Calcular el grado del aeropuerto:

```
todos_vuelos = g.degrees.withColumnRenamed("id", "ald")

(grupos
  .filter("etiqueta=1606317768706")
  .join(todos_los_vuelos, todos_los_vuelos.ald == resultado.id)
```

```
.sort("grado", ascendente=Falso)
.select("id", "nombre", "título")
.show(truncar=Falso))
```

Si ejecutamos esa consulta obtendremos este resultado:

id	nombre	grado
	Aeropuerto Internacional de Dallas-Fort Worth DFW	47514
	Aeropuerto Internacional Charlotte Douglas (CLT)	40495
IAH	Aeropuerto Intercontinental George Bush de Houston	28814
	EWR Aeropuerto Internacional Newark Liberty	25131
	Aeropuerto Internacional de Filadelfia PHL	20804
	Aeropuerto Internacional Thurgood Marshall de Baltimore/Washington (BWI) 18989	
	Aeropuerto Internacional Midway de Chicago MDW	15178
	Aeropuerto Internacional de Nashville (BNA)	12455
	DAL Dallas Love Field	12084
	Aeropuerto Internacional Washington-Dulles (IAD)	11566
STL	Aeropuerto Internacional Lambert St. Louis	11439
	Aeropuerto William P. Hobby de HOU	9742
INDIANA	Aeropuerto Internacional de Indianápolis	8543
FOXA	Aeropuerto Internacional de Pittsburgh	8410
	Aeropuerto Internacional Cleveland Hopkins	8238
	Aeropuerto Internacional CMH Port Columbus	7640
	SAT Aeropuerto Internacional de San Antonio	6532
	Aeropuerto Internacional JAX de Jacksonville	5495
	Aeropuerto Internacional Bradley BDL	4866
	Aeropuerto Internacional RSW del Suroeste de Florida	4569

En la **Figura 7-15** podemos ver que este grupo en realidad está enfocado en la Costa Este. Medio Oeste de los Estados Unidos.

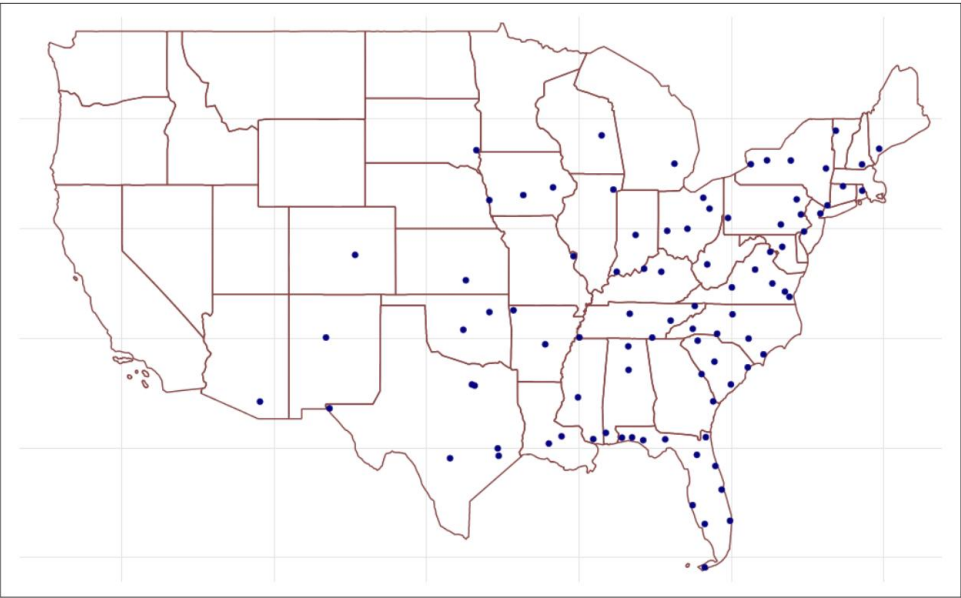


Figura 7-15. Aeropuertos del clúster 1606317768706

Y ahora hagamos lo mismo con el segundo grupo más grande:

```
(grupos
  .filter("etiqueta=1219770712067")
  .join(todos_los_vuelos, todos_los_vuelos.ald == resultado.id)
  .sort("grado", ascendente=Falso)
  .select("id", "nombre", "título")
  .show(truncar=Falso))
```

Si ejecutamos esa consulta obtenemos este resultado:

nombre de identificación	grado
Aeropuerto Internacional Hartsfield Jackson de Atlanta (ATL)	67672
Aeropuerto Internacional O'Hare de Chicago (ORD)	56681
Aeropuerto Internacional de Denver	39671
Aeropuerto Internacional de Los Ángeles (LAX)	38116
Aeropuerto Internacional Phoenix Sky Harbor	30206
Aeropuerto Internacional de San Francisco SFO	29865
Aeropuerto LGA La Guardia	29416
Aeropuerto Internacional LAS McCarran	27801
DTW Aeropuerto Metropolitano del Condado de Wayne de Detroit	27477
Aeropuerto Internacional de Minneapolis-St. Paul/Wold-Chamberlain (MSP) 27163	
Aeropuerto Internacional General Edward Lawrence Logan de BOS	26214

nombre de identificación	grado
Aeropuerto Internacional de Seattle Tacoma SEA	24098
Aeropuerto Internacional de Orlando MCO	23442
Aeropuerto Internacional JFK John F. Kennedy	22294
Aeropuerto Nacional Ronald Reagan de Washington (DCA)	22244
Aeropuerto Internacional de Salt Lake City (SLC)	18661
FLL Aeropuerto Internacional de Fort Lauderdale Hollywood	16364
Aeropuerto Internacional de San Diego	15401
Aeropuerto Internacional de Miami (MIA)	14869
Aeropuerto Internacional de Tampa TPA	12509

En la **Figura 7-16** podemos ver que este clúster está aparentemente más centrado en el centro, con algunos Paradas adicionales en el noroeste a lo largo del camino.

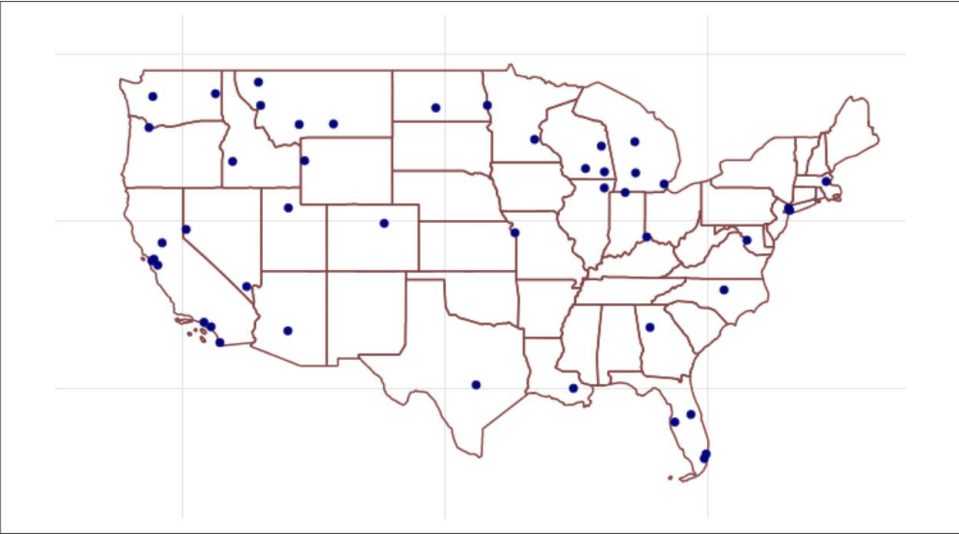


Figura 7-16. Aeropuertos del clúster 1219770712067

El código que usamos para generar estos mapas está disponible en el [repositorio de GitHub del libro](#).

Al consultar el sitio web de DL sobre programas de viajero frecuente, encontramos una promoción de "Usa dos y llévate uno gratis". Si usamos nuestros puntos para dos vuelos, recibimos otro gratis, pero...

¡Solo si volamos dentro de uno de los dos grupos! Quizás sea mejor aprovechar nuestro tiempo, y sin duda nuestros puntos, si nos quedamos en un grupo.

Ejercicios de lectura

- Utilice un algoritmo de ruta más corta para evaluar la cantidad de vuelos desde su casa al Aeropuerto Internacional Bozeman Yellowstone (BZN).
- ¿Existen diferencias si se utilizan pesos de relación?

Resumen En

los últimos capítulos, detallamos cómo funcionan los algoritmos de grafos clave para la búsqueda de rutas, la centralidad y la detección de comunidades en Apache Spark y Neo4j. En este capítulo, repasamos flujos de trabajo que incluyeron el uso de varios algoritmos en contexto con otras tareas y análisis. Utilizamos un escenario de una empresa de viajes para analizar los datos de Yelp en Neo4j y un escenario de viajes aéreos personales para evaluar los datos de aerolíneas estadounidenses en Spark.

A continuación, veremos un uso de los algoritmos de gráficos que está adquiriendo cada vez mayor importancia: el aprendizaje automático mejorado con gráficos.

Uso de algoritmos gráficos para mejorar Aprendizaje automático

Hemos cubierto varios algoritmos que aprenden y actualizan el estado en cada iteración, como la Propagación de Etiquetas; sin embargo, hasta ahora, nos hemos centrado en los algoritmos de grafos para análisis general. Dado el creciente uso de grafos en el aprendizaje automático (ML), ahora veremos cómo se pueden usar para optimizar los flujos de trabajo de ML.

En este capítulo, nos centramos en la forma más práctica de empezar a mejorar las predicciones de aprendizaje automático (ML) mediante algoritmos de grafos: la extracción de características conectadas y su uso para predecir relaciones. Primero, abordaremos algunos conceptos básicos de ML y la importancia de los datos contextuales para obtener mejores predicciones. A continuación, presentamos un breve resumen de las formas en que se aplican las características de grafos, incluyendo su uso para el fraude de spam, la detección y la predicción de enlaces.

Demostraremos cómo crear una secuencia de aprendizaje automático y, posteriormente, entrenar y evaluar un modelo para la predicción de enlaces, integrando Neo4j y Spark en nuestro flujo de trabajo. Nuestro ejemplo se basará en el conjunto de datos Citation Network, que contiene autores, artículos, relaciones entre autores y relaciones de citas. Utilizaremos varios modelos para predecir la probabilidad de colaboración entre autores de investigaciones en el futuro y mostraremos cómo los algoritmos de grafos mejoran los resultados.

Aprendizaje automático y la importancia del contexto

El aprendizaje automático no es inteligencia artificial (IA), sino un método para lograrla. El aprendizaje automático utiliza algoritmos para entrenar software mediante ejemplos específicos y mejoras progresivas basadas en los resultados esperados, sin una programación explícita sobre cómo lograr estos mejores resultados. El entrenamiento implica proporcionar una gran cantidad de datos a un modelo y permitirle aprender a procesar e incorporar esa información.

En este sentido, el aprendizaje implica que los algoritmos iteran, realizando cambios continuos para acercarse a un objetivo, como reducir los errores de clasificación en comparación con los datos de entrenamiento. El aprendizaje automático (ML) también es dinámico, con la capacidad de modificarse y optimizarse al presentar más datos. Esto puede ocurrir durante el entrenamiento previo al uso en varios lotes o como aprendizaje en línea durante el uso.

Los recientes éxitos en las predicciones de aprendizaje automático (ML), la accesibilidad a grandes conjuntos de datos y la potencia de cómputo paralelo han hecho que el ML sea más práctico para quienes desarrollan modelos probabilísticos para aplicaciones de IA. A medida que el aprendizaje automático se generaliza, es importante recordar su objetivo fundamental: tomar decisiones de forma similar a la de los humanos. Si olvidamos este objetivo, podríamos terminar con una versión más de software altamente específico y basado en reglas.

Para aumentar la precisión del aprendizaje automático y, al mismo tiempo, ampliar la aplicabilidad de las soluciones, necesitamos incorporar mucha información contextual, al igual que las personas deberían usar el contexto para tomar mejores decisiones. Los humanos usamos el contexto que nos rodea, no solo los datos directos, para determinar qué es esencial en una situación, estimar la información faltante y determinar cómo aplicar las lecciones aprendidas a nuevas situaciones. El contexto nos ayuda a mejorar las predicciones.

Gráficos, contexto y precisión. Sin información

periférica ni relacionada, las soluciones que intentan predecir el comportamiento o hacer recomendaciones para circunstancias variables requieren un entrenamiento más exhaustivo y reglas prescriptivas. Esto explica, en parte, por qué la IA es eficaz en tareas específicas y bien definidas, pero tiene dificultades con la ambigüedad. El aprendizaje automático (ML) mejorado con gráficos puede ayudar a completar esa información contextual faltante, tan importante para tomar mejores decisiones.

Sabemos, por la teoría de grafos y por la vida real, que las relaciones suelen ser los predictores más sólidos del comportamiento. Por ejemplo, si una persona vota, aumenta la probabilidad de que sus amigos, familiares e incluso compañeros de trabajo también voten. La Figura 8-1 ilustra un efecto dominó basado en la votación declarada y los amigos de Facebook del artículo de investigación de 2012 «Un experimento de 61 millones de personas sobre influencia social y movilización política», de R. Bond et al.

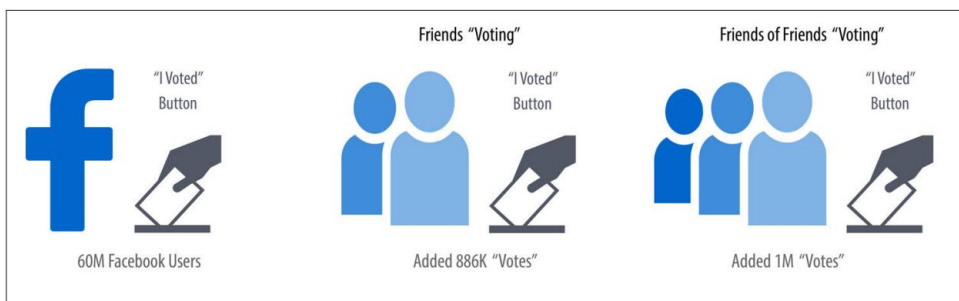


Figura 8-1. Las personas se ven influenciadas a votar por sus redes sociales. En este ejemplo, los amigos a dos pasos de distancia tuvieron un mayor impacto total que las relaciones directas.

Los autores descubrieron que los amigos que informaron haber votado influyeron en un 1,4 % adicional de los usuarios para que también afirmaran haber votado y, curiosamente, los amigos de sus amigos sumaron otro 1,7 %. Pequeños porcentajes pueden tener un impacto significativo, y como se puede observar en la Figura 8-1, las personas que se encontraban dos pasos por encima tuvieron, en total, un mayor impacto que los amigos directos por sí solos. La votación y otros ejemplos de cómo nuestras redes sociales nos impactan se tratan en el libro *Connected*, de Nicholas Christakis y James Fowler (Little, Brown and Company).

Añadir características gráficas y contexto mejora las predicciones, especialmente en situaciones donde las conexiones son importantes. Por ejemplo, las empresas minoristas personalizan las recomendaciones de productos no solo con datos históricos, sino también con datos contextuales sobre las similitudes de los clientes y su comportamiento en línea. Alexa de Amazon utiliza **varias capas de modelos contextuales** que demuestran una mayor precisión. En 2018, Amazon también introdujo la "transferencia de contexto" para incorporar referencias previas en una conversación al responder nuevas preguntas.

Desafortunadamente, muchos enfoques de aprendizaje automático actuales omiten mucha información contextual valiosa. Esto se debe a que el aprendizaje automático se basa en datos de entrada generados a partir de tuplas, lo que excluye muchas relaciones predictivas y datos de red. Además, la información contextual no siempre está fácilmente disponible o es demasiado difícil de acceder y procesar. Incluso encontrar conexiones a cuatro o más saltos de distancia puede ser un desafío a gran escala para los métodos tradicionales. Mediante el uso de grafos, podemos acceder e incorporar datos conectados con mayor facilidad.

Extracción y selección de características conectadas

La extracción y selección de características nos ayuda a tomar datos sin procesar y crear un subconjunto y un formato adecuados para entrenar nuestros modelos de aprendizaje automático. Es un paso fundamental que...

Cuando se ejecuta bien, conduce a un aprendizaje automático que produce predicciones más precisas y consistentes.

Extracción y selección de características

La extracción de características es una forma de depurar grandes volúmenes de datos y atributos para obtener un conjunto de atributos descriptivos representativos. El proceso deriva valores numéricos (características) para características o patrones distintivos en los datos de entrada, lo que permite diferenciar categorías en otros datos. Se utiliza cuando resulta difícil para un modelo analizar los datos directamente, quizás debido a su tamaño, formato o la necesidad de comparaciones incidentales.

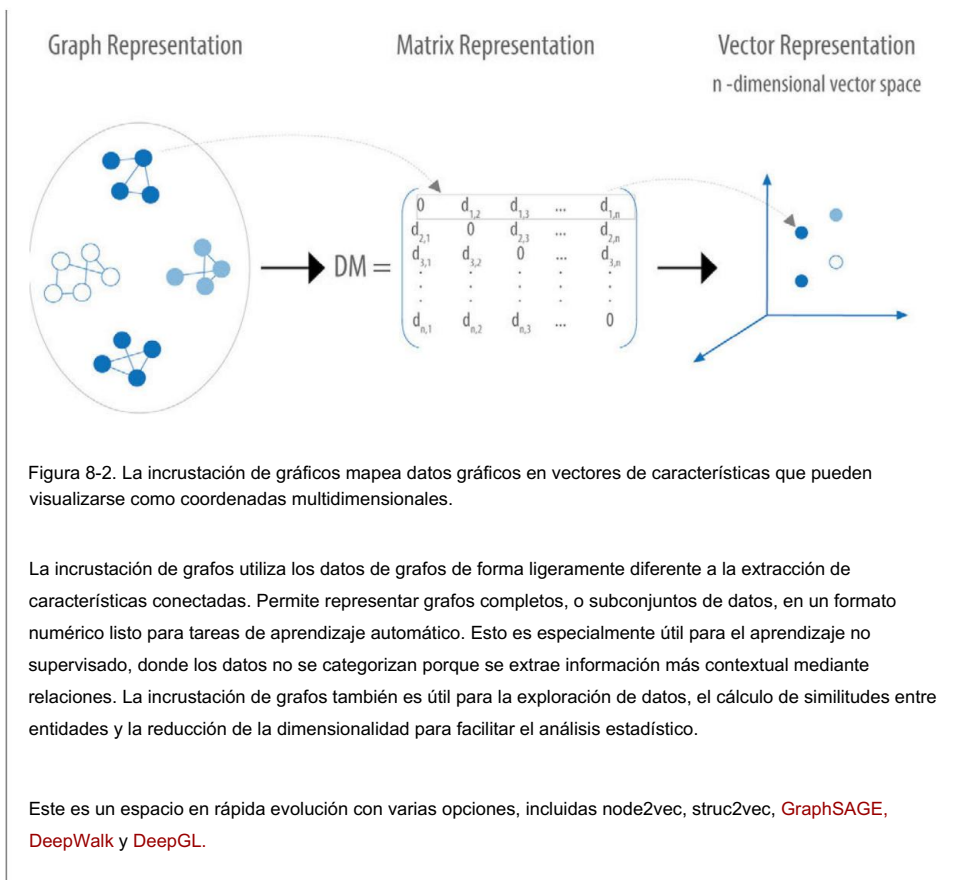
La selección de características es el proceso de determinar el subconjunto de características extraídas más importantes o influyentes para un objetivo. Se utiliza para determinar la importancia predictiva, así como para la eficiencia. Por ejemplo, si tenemos 20 características y 13 de ellas juntas explican el 92 % de lo que necesitamos predecir, podemos eliminar 7 características de nuestro modelo.

La combinación adecuada de características puede aumentar la precisión, ya que influye decisivamente en el aprendizaje de nuestros modelos. Dado que incluso mejoras modestas pueden marcar una diferencia significativa, en este capítulo nos centramos en las características conectadas. Estas características son características extraídas de la estructura de los datos. Estas características pueden derivarse de consultas locales basadas en las partes del grafo que rodean un nodo, o de consultas globales que utilizan algoritmos de grafos para identificar elementos predictivos dentro de los datos, basándose en relaciones para la extracción de características conectadas.

Y no solo es importante obtener la combinación correcta de características, sino también eliminar características innecesarias para reducir la probabilidad de que nuestros modelos sean hipersegmentados. Esto nos evita crear modelos que solo funcionan bien en nuestros datos de entrenamiento (conocido como sobreajuste) y amplía significativamente la aplicabilidad. También podemos usar algoritmos de grafos para evaluar esas características y determinar cuáles son las más influyentes en nuestro modelo para la selección de características conectadas. Por ejemplo, podemos mapear características a nodos en un grafo, crear relaciones basadas en características similares y luego calcular la centralidad de las características. Las relaciones entre características se pueden definir por la capacidad de preservar las densidades de clústeres de los puntos de datos. Este método se describe usando conjuntos de datos con alta dimensión y bajo tamaño de muestra en "[Unsupervised Graph-Based Feature Selection Via Subspace and PageRank Centrality](#)", por K. Henniab, N. Mezghani y C. Gouin-Vallerand.

Incrustaciones de grafos.

La incrustación de grafos consiste en la representación de los nodos y las relaciones en un grafo como vectores de características. Estos son simplemente conjuntos de características con mapeos dimensionales, como las coordenadas (x, y, z) que se muestran en la [Figura 8-2](#).



Ahora veamos algunos de los tipos de funciones conectadas y cómo se utilizan.

Características de

Graphy: Las características de Graphy incluyen diversas métricas relacionadas con la conexión de nuestro grafo, como el número de relaciones que entran o salen de los nodos, el número de triángulos potenciales y los vecinos en común. En nuestro ejemplo, comenzaremos con estas medidas porque son fáciles de recopilar y constituyen una buena prueba de hipótesis iniciales.

Además, cuando sabemos con precisión lo que buscamos, podemos usar la ingeniería de características. Por ejemplo, si queremos saber cuántas personas tienen una cuenta fraudulenta hasta cuatro saltos fuera de la red, este enfoque utiliza el recorrido de grafos para encontrar de forma muy eficiente rutas profundas de relaciones, analizando elementos como etiquetas, atributos, recuentos y relaciones inferidas.

También podemos automatizar fácilmente estos procesos e incorporar esas funciones de gráficos predictivos a nuestro flujo de trabajo existente. Por ejemplo, podríamos abstraer un recuento de estafadores.

relaciones y agregue ese número como un atributo de nodo para usarlo en otras tareas de aprendizaje automático.

Características de algoritmos de grafos

También podemos usar algoritmos de grafos para encontrar características donde conocemos la estructura general que buscamos pero no el patrón exacto. A modo de ejemplo, supongamos que sabemos que ciertos tipos de agrupaciones comunitarias son indicativas de fraude; quizás exista una densidad o jerarquía prototípica de relaciones. En este caso, no queremos una característica rígida de una organización exacta, sino una estructura flexible y globalmente relevante. Usaremos algoritmos de detección de comunidades para extraer características conectadas en nuestro ejemplo, pero los algoritmos de centralidad, como PageRank, también se aplican con frecuencia.

Además, los enfoques que combinan varios tipos de características conectadas parecen tener mejor rendimiento que usar un solo método. Por ejemplo, podríamos combinar características conectadas para predecir el fraude con indicadores basados en comunidades encontradas mediante el algoritmo de Louvain, nodos influyentes mediante PageRank y la medición de estafadores conocidos a tres pasos de distancia.

En la [Figura 8-3](#) se muestra un enfoque combinado, donde los autores combinan algoritmos de grafos como PageRank y Coloring con medidas gráficas como el grado de entrada y el grado de salida. Este diagrama se extrae del artículo [«Collective Spammer Detection in Evolving Multi-Relational Social Networks»](#), de S. Fakhraei et al.

La sección "Estructura del Gráfico" ilustra la extracción de características conectadas mediante varios algoritmos de grafos. Curiosamente, los autores descubrieron que extraer características conectadas de múltiples tipos de relaciones es incluso más predictivo que simplemente añadir más características. La sección "Subgrafo del Informe" muestra cómo las características del grafo se convierten en características que el modelo de aprendizaje automático (ML) puede utilizar. Al combinar varios métodos en un flujo de trabajo de ML optimizado con grafos, los autores lograron mejorar los métodos de detección anteriores y clasificar al 70 % de los spammers que anteriormente requerían etiquetado manual, con una precisión del 90 %.

Incluso una vez extraídas las características conectadas, podemos mejorar nuestro entrenamiento mediante algoritmos de grafos como PageRank para priorizar las características con mayor influencia. Esto nos permite representar adecuadamente nuestros datos, eliminando variables ruidosas que podrían degradar los resultados o ralentizar el procesamiento. Con este tipo de información, también podemos identificar características con alta coocurrencia para un mayor ajuste del modelo mediante la reducción de características. Este método se describe en el artículo de investigación [«Using PageRank in Feature Selection»](#) ([Uso de PageRank en la selección de características](#)), de D. Ienco, R. Meo y M. Botta.

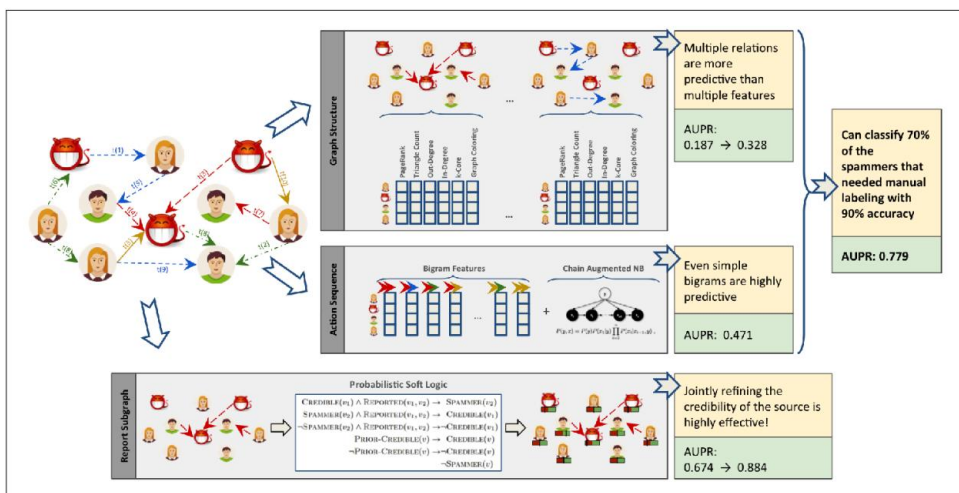


Figura 8-3. La extracción de características conectadas puede combinarse con otros métodos predictivos para mejorar los resultados. AUPR se refiere al área bajo la curva de precisión-recuperación, y se prefieren valores más altos.

Hemos analizado cómo se aplican las características conectadas a escenarios que involucran la detección de fraude y spam. En estas situaciones, las actividades suelen estar ocultas en múltiples capas de ofuscación y relaciones de red. Los métodos tradicionales de extracción y selección de características podrían no ser capaces de detectar dicho comportamiento sin la información contextual que proporcionan los gráficos.

Otra área donde las características conectadas mejoran el aprendizaje automático (y el enfoque del resto de este capítulo) es la predicción de vínculos. Esta predicción permite estimar la probabilidad de que se forme una relación en el futuro, o si ya debería estar en nuestro gráfico, pero no se encuentra debido a datos incompletos. Dado que las redes son dinámicas y pueden crecer con relativa rapidez, la capacidad de predecir vínculos que se añadirán próximamente tiene una amplia aplicabilidad, desde recomendaciones de productos hasta la reorientación de fármacos e incluso la inferencia de relaciones delictivas.

Las características conectadas de los grafos se utilizan a menudo para mejorar la predicción de enlaces mediante características básicas de grafos, así como características extraídas de algoritmos de centralidad y comunidad. La predicción de enlaces basada en la proximidad o similitud de nodos también es estándar; en el artículo «[El problema de la predicción de enlaces para redes sociales](#)», D. Liben-Nowell y J.

Kleinberg sugiere que la estructura de la red por sí sola puede contener suficiente información latente para detectar la proximidad de los nodos y superar las medidas más directas.

Ahora que hemos visto las formas en que las características conectadas pueden mejorar el aprendizaje automático, profundicemos en nuestro ejemplo de predicción de enlaces y veamos cómo podemos aplicar algoritmos gráficos para mejorar nuestras predicciones.

Gráficos y aprendizaje automático en la práctica: predicción de enlaces

El resto del capítulo mostrará un ejemplo práctico basado en el [Citation Network Dataset](#), un conjunto de datos de investigación extraído de DBLP, ACM y MAG. Este conjunto de datos se describe en el artículo [«ArnetMiner: Extracción y Minería de Redes Sociales Académicas»](#), de J. Tang et al. La última versión contiene 3.079.007 artículos, 1.766.547 autores, 9.437.718 relaciones de autor y 25.166.994 relaciones de citación.

Trabajaremos con un subconjunto centrado en artículos que aparecieron en las siguientes publicaciones:

- Apuntes de conferencias sobre informática •

Comunicaciones de la ACM • Conferencia

internacional sobre ingeniería de software • Avances en informática

y comunicaciones

Nuestro conjunto de datos resultante contiene 51.956 artículos, 80.299 autores, 140.575 relaciones entre autores y 28.706 relaciones de citas. Crearemos un gráfico de coautorías basado en los autores que han colaborado en artículos y, a continuación, prediciremos futuras colaboraciones entre pares de autores. Solo nos interesan las colaboraciones entre autores que no han colaborado previamente; no nos preocupan las colaboraciones múltiples entre pares de autores.

En el resto del capítulo, configuraremos las herramientas necesarias e importaremos los datos a Neo4j. A continuación, explicaremos cómo equilibrar correctamente los datos y dividir las muestras en Spark DataFrames para el entrenamiento y las pruebas. Después, explicaremos nuestra hipótesis y los métodos para la predicción de enlaces antes de crear una canalización de aprendizaje automático en Spark.

Finalmente, repasaremos el entrenamiento y la evaluación de varios modelos de predicción, comenzando con características gráficas básicas y agregando más características de algoritmos gráficos extraídas con Neo4j.

Herramientas y datos

Comencemos configurando nuestras herramientas y datos. Luego, exploraremos nuestro conjunto de datos y crearemos una secuencia de aprendizaje automático.

Antes de hacer cualquier otra cosa, configuremos las bibliotecas utilizadas en este capítulo:

py2neo

Una biblioteca de Python Neo4j que se integra bien con el ecosistema de ciencia de datos de Python.
tema

pandas

Una biblioteca de alto rendimiento para la gestión de datos fuera de una base de datos con estructuras de datos fáciles de usar y herramientas de análisis de datos.

Spark MLlib

Biblioteca de aprendizaje automático de Spark



Utilizamos MLlib como ejemplo de biblioteca de aprendizaje automático. El enfoque mostrado en este capítulo podría utilizarse en combinación con otras bibliotecas de aprendizaje automático, como scikit-learn.

Todo el código mostrado se ejecutará en el REPL de pyspark. Podemos iniciar el REPL ejecutando el siguiente comando:

```
exportar SPARK_VERSION="spark-2.4.0-bin-hadoop2.7"
./${SPARK_VERSION}/bin/pyspark \ --
  driver-memory 2g \ --
  executor-memory 6g \ --
  packages julioasotodv:spark-tree-plotting:0.2
```

Este comando es similar al que usamos para iniciar el REPL en el [Capítulo 3](#), pero en lugar de GraphFrames, cargamos el paquete spark-tree-plotting. Al momento de escribir esto, la última versión de Spark es spark-2.4.0-bin-hadoop2.7, pero como es posible que haya cambiado para cuando lea esto, asegúrese de modificar la variable de entorno SPARK_VERSION como corresponda.

Una vez que lo hayamos lanzado, importaremos las siguientes bibliotecas que usaremos:

```
desde py2neo importar Graph
importar pandas como
pd desde numpy.random importar randint

desde pyspark.ml importar Pipeline
desde pyspark.ml.classification importar RandomForestClassifier desde
pyspark.ml.feature importar StringIndexer, VectorAssembler desde
pyspark.ml.evaluation importar BinaryClassificationEvaluator

desde pyspark.sql.types importar *
desde pyspark.sql importar funciones como F

de sklearn.metrics importar roc_curve, auc de
colecciones importar Counter

desde cycler importar cycler
importar matplotlib
matplotlib.use('TkAgg')
importar matplotlib.pyplot como plt
```


Y ahora vamos a crear una conexión a nuestra base de datos Neo4j:

```
gráfico = Gráfico("bolt://localhost:7687", auth=("neo4j", "neo"))
```

Importando los datos a Neo4j. Ahora estamos

listos para cargar los datos en Neo4j y crear una distribución equilibrada para nuestro entrenamiento y pruebas.

Necesitamos descargar el archivo ZIP de la **versión 10** del conjunto de datos, descomprimirlo y guardar el contenido en nuestra carpeta de importación. Deberíamos tener los siguientes archivos:

- dblp-ref-0.json • dblp-
- ref-1.json • dblp-
- ref-2.json • dblp-
- ref-3.json

Una vez que tengamos esos archivos en la carpeta de importación, necesitamos agregar la siguiente propiedad a nuestro archivo de configuración de Neo4j para que podamos procesarlos usando la biblioteca APOC:

```
apoc.import.file.enabled=verdadero
apoc.import.file.use_neo4j_config=verdadero
```

Primero crearemos restricciones para garantizar que no creemos artículos o autores duplicados:

```
CREAR RESTRICCIÓN EN (artículo:Artículo)
AFIRMACIÓN article.index ES ÚNICO;
```

```
CREAR RESTRICCIÓN EN (autor:Autor)
ASSERT author.name ES ÚNICO;
```

Ahora podemos ejecutar la siguiente consulta para importar los datos de los archivos JSON:

```
LLAMA apoc.periodic.iterate( 'DESENCOLLAR
["dblp-ref-0.json","dblp-ref-1.json",
"dblp-ref-2.json","dblp-ref-3.json"] COMO archivo LLAMA
apoc.load.json("file:/// " + archivo)
Valor de rendimiento
DÓNDE valor.venue EN ["Apuntes de Ciencias de la Computación", "Comunicaciones
de la ACM", "conferencia internacional
sobre ingeniería de software", "avances en informática y comunicaciones"]

valor de retorno',
'FUSIONAR (a:Artículo {indice:valor.id})
AL CREAR ESTABLECER a += apoc.map.clean(valor,["id","autores","referencias"],[0])
CON a,valor.autores como autores
DESENCOLLAR autores como
autor FUSIONAR (b:Autor{nombre:autor})
FUSIONAR (b)<-[AUTOR]-{a} ',
{tamañoodelote: 10000, listadeiteraciones: verdadero});
```

Esto da como resultado el esquema gráfico que se ve en la **Figura 8-4**.

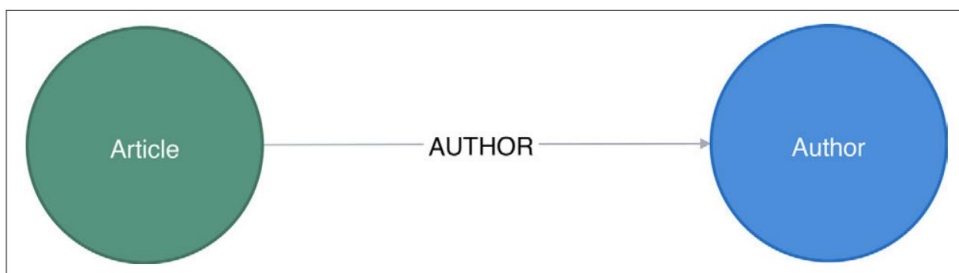


Figura 8-4. El gráfico de citas

Este es un gráfico simple que conecta artículos y autores, por lo que agregaremos más información que podamos inferir de las relaciones para ayudar con las predicciones.

El gráfico de coautoría. Queremos

predecir futuras colaboraciones entre autores, así que comenzaremos creando un gráfico de coautoría. La siguiente consulta Cypher de Neo4j creará una relación CO_AUTHOR entre cada par de autores que hayan colaborado en un artículo:

```
COINCIDIR (a1)<-[:AUTOR]-(trabajo)-[:AUTOR]->(a2:Author)
CON a1, a2, papel
ORDENAR POR a1, papel.año
CON a1, a2, recopilar(papel)[0].año COMO año, contar(*) COMO colaboraciones FUSIONAR
(a1)-[coautor:CO_AUTHOR {año: año}]- (a2)
SET coautor.colaboraciones = colaboraciones;
```

La propiedad de año que configuramos en la relación CO_AUTHOR en la consulta corresponde al primer año en que ambos autores colaboraron. Solo nos interesa la primera vez que colaboraron; las colaboraciones posteriores no son relevantes.

La Figura 8-5 muestra un ejemplo de una parte del gráfico que se crea. Ya podemos observar algunas estructuras comunitarias interesantes.

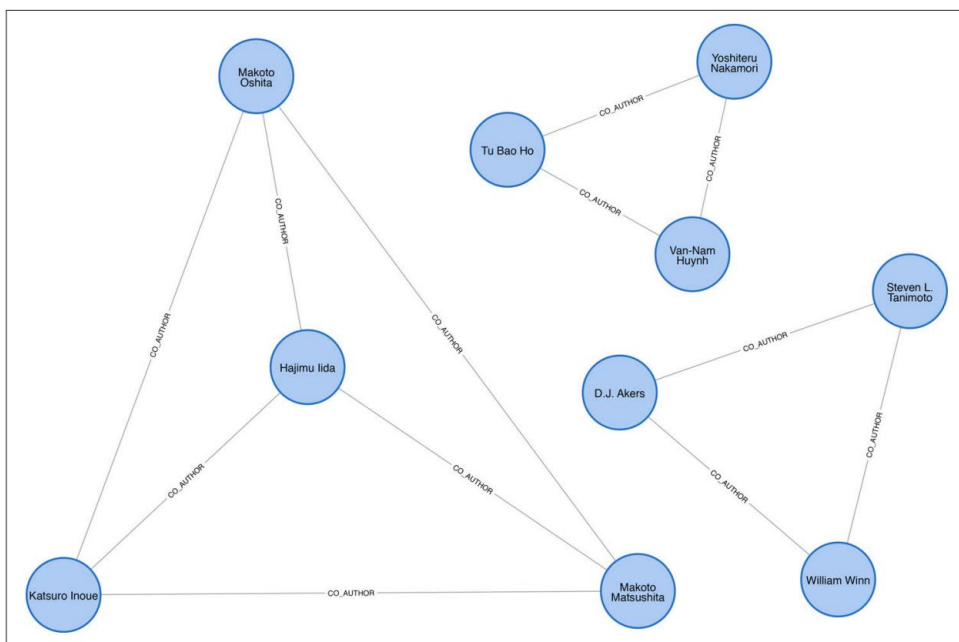


Figura 8-5. El gráfico de coautores

Cada círculo en este diagrama representa un autor y las líneas entre ellos representan relaciones `CO_AUTHOR`. Por lo tanto, a la izquierda tenemos cuatro autores que han colaborado entre sí, y a la derecha, dos ejemplos de tres autores que han colaborado. Ahora que tenemos los datos cargados y un gráfico básico, creemos los dos conjuntos de datos que necesitaremos para el entrenamiento y las pruebas.

Creación de conjuntos de datos de prueba y entrenamiento equilibrados Con

los problemas de predicción de enlaces queremos intentar predecir la creación futura de enlaces. Este conjunto de datos funciona bien para eso porque tenemos fechas en los artículos que podemos usar.

Para dividir nuestros datos, necesitamos determinar qué año usaremos para definir nuestra división de entrenamiento/prueba. Entrenaremos nuestro modelo con todo lo anterior a ese año y luego lo probaremos con los enlaces creados después de esa fecha.

Empecemos por averiguar cuándo se publicaron los artículos. Podemos escribir la siguiente consulta para obtener el recuento de artículos, agrupados por año:

```

"""
consulta =
COINCIDIR (artículo:Artículo)
DEVOLVER artículo.año COMO año, recuento (*) COMO recuento
ORDENAR POR año
"""

```

```

por_año = graph.run(consulta).to_data_frame()

```

Visualicemos esto como un gráfico de barras, con el siguiente código:

```
plt.style.use('cinco y treinta y ocho') ax =
por_año.plot(tipo='barra', x='año', y='conteo', leyenda=Ninguno, tamaño_de_la_figura=(15,8))
ax.xaxis.set_label_text("")
plt.tight_layout()
plt.show()
```

Podemos ver el gráfico generado al ejecutar este código en la Figura 8-6.

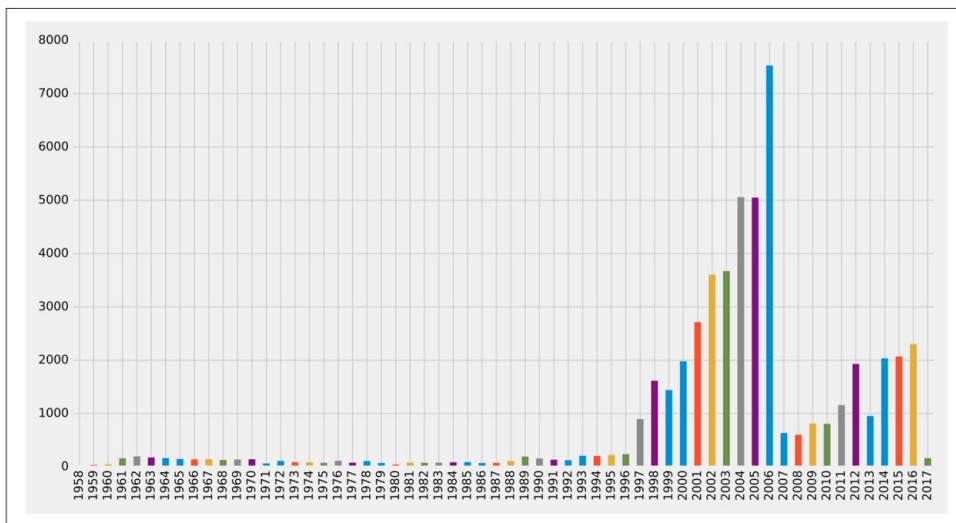


Figura 8-6. Artículos por año

Se publicaron muy pocos artículos antes de 1997, y posteriormente se publicaron muchos entre 2001 y 2006, antes de una caída y un ascenso gradual desde 2011 (excluyendo 2013). Parece que 2006 podría ser un buen año para dividir nuestros datos para entrenar nuestro modelo y realizar predicciones. Veamos cuántos artículos se publicaron antes de ese año y cuántos durante y después. Podemos escribir la siguiente consulta para calcularlo:

```
PARTIDO (artículo:Artículo)
RETORNO artículo.año < 2006 AS entrenamiento, recuento (*) AS recuento
```

El resultado de esto es el siguiente, donde verdadero significa que el artículo se publicó antes de 2006:

recuento de entrenamiento	
FALSO	21059
verdadero	30897

¡No está mal! El 60% de los artículos se publicaron antes de 2006 y el 40% durante o después de 2006.

Esta es una división bastante equilibrada de datos para nuestro entrenamiento y pruebas.

Ahora que tenemos una buena distribución de artículos, usemos la misma distribución de 2006 para la coautoría. Crearemos una relación CO_AUTOR_TEMPRANO entre pares de autores cuya primera colaboración fue antes de 2006:

```
COINCIDIR (a1)<-[:AUTOR]-(trabajo)-[:AUTOR]->(a2:Autor)
CON a1, a2, papel
ORDENAR POR a1, papel.año
CON a1, a2, recopilar(papel)[0].año COMO año, contar(*) COMO colaboraciones DONDE año < 2006
FUSIONAR (a1)-
[coautor:CO_AUTOR_ANTICIPADO {año: año}]->(a2)
SET coautor.colaboraciones = colaboraciones;
```

Y luego crearemos una relación CO_AUTHOR_LATE entre pares de autores cuya primera colaboración fue durante o después de 2006:

```
COINCIDIR (a1)<-[:AUTOR]-(trabajo)-[:AUTOR]->(a2:Autor)
CON a1, a2, papel
ORDENAR POR a1, papel.año
CON a1, a2, recopilar(papel)[0].año COMO año, contar(*) COMO colaboraciones DONDE año >= 2006
FUSIONAR (a1)-
[coautor:CO_AUTHOR_LATE {año: año}]->(a2)
SET coautor.colaboraciones = colaboraciones;
```

Antes de crear nuestros conjuntos de entrenamiento y prueba, verifiquemos cuántos pares de nodos tienen enlaces entre sí. La siguiente consulta encontrará el número de pares CO_AUTHOR_EARLY :

```
COINCIDIR ()-[:CO_AUTOR_ANTICIPADO]->()
DEVOLVER count(*) COMO count
```

Al ejecutar esa consulta se devolverá el resultado que se muestra aquí:

contar

81096

Y esta consulta encontrará el número de pares CO_AUTHOR_LATE :

```
COINCIDIR ()-[:CO_AUTHOR_TARDE]->()
DEVOLVER count(*) COMO count
```

Al ejecutar esa consulta se devuelve este resultado:

contar

74128

Ahora estamos listos para construir nuestros conjuntos de datos de entrenamiento y prueba.

Equilibrio y división de datos. Los

pares de nodos con relaciones CO_AUTHOR_EARLY y CO_AUTHOR_LATE entre sí actuarán como ejemplos positivos, pero también necesitaremos crear algunos ejemplos negativos. La mayoría de las redes del mundo real son dispersas, con concentraciones de relaciones, y este grafo no es diferente. El número de ejemplos donde dos nodos no tienen una relación es mucho mayor que el número de los que sí la tienen.

Si consultamos nuestros datos de CO_AUTHOR_EARLY, encontraremos 45.018 autores con ese tipo de relación, pero solo 81.096 relaciones entre ellos. Puede que esto no parezca desequilibrado, pero lo es: el número máximo potencial de relaciones que nuestro grafo podría tener es $(45.018 * 45.017) / 2 = 1.013.287.653$, lo que significa que hay muchos ejemplos negativos (sin vínculos). Si usáramos todos los ejemplos negativos para entrenar nuestro modelo, tendríamos un grave problema de desequilibrio de clases. Un modelo podría alcanzar una precisión extremadamente alta al predecir que no hay relación entre ningún par de nodos.

En su artículo "[Nuevas perspectivas y métodos en la predicción de enlaces](#)", R. Lichtenwalter, J. Lussier y N. Chawla describen varios métodos para abordar este desafío. Uno de estos enfoques consiste en construir ejemplos negativos mediante la búsqueda de nodos dentro de nuestra vecindad a los que no estemos conectados actualmente.

Construiremos nuestros ejemplos negativos buscando pares de nodos con una distancia de entre dos y tres saltos, excluyendo aquellos que ya tienen una relación. Luego, reduciremos la resolución de esos pares de nodos para obtener el mismo número de ejemplos positivos y negativos.



Tenemos 314.248 pares de nodos que no tienen relación entre sí a una distancia de dos saltos. Si aumentamos la distancia a tres saltos, tenemos 967.677 pares de nodos.

La siguiente función se utilizará para reducir la resolución de los ejemplos negativos:

```
def down_sample(df):
    copia = df.copia()
    cero = Contador(copia.etiqueta.valores)[0] un
    = Contador(copia.etiqueta.valores)[1]
    n = cero - un
    copia = copia.drop(copia[copia.label == 0].sample(n=n, random_state=1).index) return
    copia.sample(frac=1)
```

Esta función calcula la diferencia entre el número de ejemplos positivos y negativos, y luego muestrea los ejemplos negativos para que haya números iguales.

Luego podemos ejecutar el siguiente código para construir un conjunto de entrenamiento con ejemplos positivos y negativos equilibrados:

```

entrenar_enlaces_existentes = graph.run("""
COINCIDIR (autor: Autor)-[:CO_AUTOR_TEMPRANO]->(otro: Autor)
DEVUELVE id(autor) COMO nodo1, id(otro) COMO nodo2, 1 COMO etiqueta
""").to_data_frame()

enlaces_perdidos_del_tren = graph.run("""
PARTIDO (autor: Autor)
DONDE (autor)-[:CO_AUTOR_TEMPRANO]-()
COINCIDIR (autor)-[:CO_AUTOR_EARLY*2..3]-()
DONDE no((autor)-[:CO_AUTOR_TEMPRANO]-())
DEVUELVE id(autor) COMO nodo1, id(otro) COMO nodo2, 0 COMO etiqueta
""").to_data_frame()

entrenar_enlaces_faltantes = entrenar_enlaces_faltantes.eliminar_duplicados()
entrenamiento_df = entrenar_enlaces_faltantes.agregar(entrenar_enlaces_existentes, ignorar_indice=Verdadero)
entrenamiento_df['etiqueta'] = entrenamiento_df['etiqueta'].astype('categoría')
entrenamiento_df = down_sample(entrenamiento_df)
entrenamiento_datos = spark.createDataFrame(entrenamiento_df)

```

Hemos convertido la columna de etiqueta en una categoría, donde 1 indica que existe un vínculo entre un par de nodos y 0 indica que no lo hay. Podemos consultar los datos de nuestro DataFrame ejecutando el siguiente código:

```
datos_de_entrenamiento.mostrar(n=5)
```

etiqueta	nodo1	nodo2
----------	-------	-------

	10019	28091	1
	10170	51476	1
	10259	17140	0
	10259	26047	1
	10293	71349	1

Los resultados nos muestran una lista de pares de nodos y si tienen una relación de coautor; por ejemplo, los nodos 10019 y 28091 tienen una etiqueta 1, lo que indica una colaboración.

Ahora ejecutemos el siguiente código para verificar el resumen de contenidos de los datos. Marco:

```
datos_de_entrenamiento.groupby("etiqueta").count().show()
```

Aquí está el resultado:

recuento de etiquetas	
-----------------------	--

0	81096
1	81096

Hemos creado nuestro conjunto de entrenamiento con el mismo número de muestras positivas y negativas. Ahora debemos hacer lo mismo con el conjunto de prueba. El siguiente código generará un conjunto de prueba con ejemplos positivos y negativos equilibrados:

```
prueba_enlaces_existentes = graph.run("""
COINCIDIR (autor: Autor)-[:CO_AUTHOR_LATE]->(otro: Autor)
DEVUELVE id(autor) COMO nodo1, id(otro) COMO nodo2, 1 COMO etiqueta
""").to_data_frame()

prueba_enlaces_faltantes = graph.run("""
PARTIDO (autor: Autor)
DONDE (autor)-[:CO_AUTHOR_LATE]-()
COINCIDIR (autor)-[:CO_AUTHOR*2..3]-()
DONDE no((autor)-[:CO_AUTHOR]-())
DEVUELVE id(autor) COMO nodo1, id(otro) COMO nodo2, 0 COMO etiqueta
""").to_data_frame()

prueba_enlaces_faltantes = prueba_enlaces_faltantes.eliminar_duplicados()
prueba_df = prueba_enlaces_faltantes.agregar(prueba_enlaces_existentes, ignorar_indice=Verdadero)
prueba_df['etiqueta'] = prueba_df['etiqueta'].astype('categoria')
prueba_df = muestra_abajo(prueba_df)
datos_de_prueba = spark.createDataFrame(prueba_df)
```

Podemos ejecutar el siguiente código para comprobar el contenido del DataFrame:

```
test_data.groupby("etiqueta").count().show()
```

Lo que da el siguiente resultado:

recuento de etiquetas	
0	74128
1	74128

Ahora que tenemos conjuntos de datos de entrenamiento y prueba equilibrados, veamos nuestros métodos para predecir vínculos.

Cómo predecimos los eslabones perdidos.

Debemos partir de algunas suposiciones básicas sobre qué elementos de nuestros datos podrían predecir si dos autores se convertirán en coautores posteriormente. Nuestra hipótesis variaría según el dominio y el problema, pero en este caso, creemos que las características más predictivas estarán relacionadas con las comunidades. Comenzaremos suponiendo que los siguientes elementos aumentan la probabilidad de que los autores se conviertan en coautores:

- Más coautores en común
- Posibles relaciones triádicas entre autores • Autores con más relaciones

• Autores en la misma comunidad • Autores

en la misma comunidad más unida

Construiremos características de grafos basándonos en nuestras suposiciones y las usaremos para entrenar un clasificador binario. La clasificación binaria es un tipo de aprendizaje automático (ML) cuya función es predecir a cuál de dos grupos predefinidos pertenece un elemento, basándose en una regla. Utilizamos el clasificador para predecir si un par de autores tendrá un vínculo o no, basándonos en una regla de clasificación. En nuestros ejemplos, un valor de 1 significa que existe un vínculo (coautoría) y un valor de 0 significa que no existe.

Implementaremos nuestro clasificador binario como un bosque aleatorio en Spark. Un bosque aleatorio es un método de aprendizaje conjunto para clasificación, regresión y otras tareas, como se ilustra en la [Figura 8-7](#).

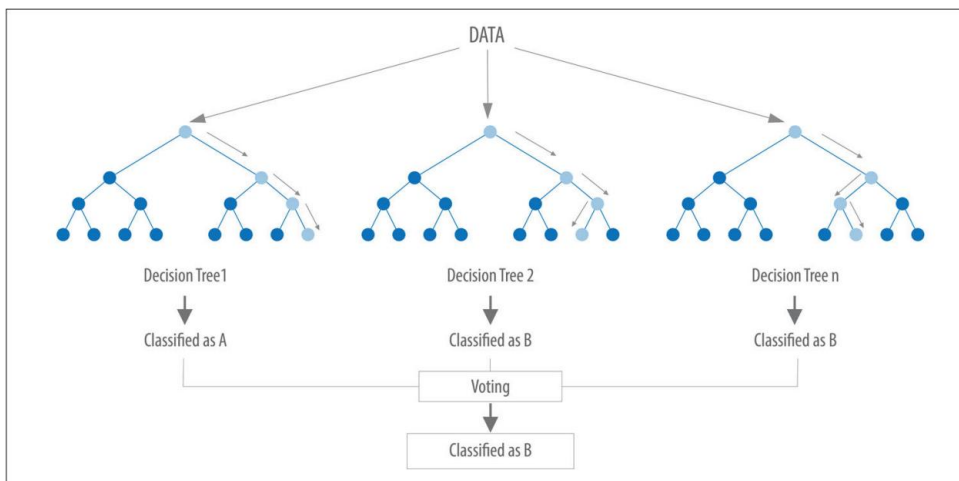


Figura 8-7. Un bosque aleatorio crea una colección de árboles de decisión y luego agrega los resultados para un voto mayoritario (para la clasificación) o un valor promedio (para la regresión).

Nuestro clasificador de bosque aleatorio tomará los resultados de los múltiples árboles de decisión que entrenamos y luego usará la votación para predecir una clasificación (en nuestro ejemplo, si existe un vínculo (coautoría) o no).

Ahora vamos a crear nuestro flujo de trabajo.

Creación de una canalización de aprendizaje automático.

Crearemos nuestra canalización de aprendizaje automático basándonos en un clasificador de bosque aleatorio en Spark. Este método es muy adecuado, ya que nuestro conjunto de datos estará compuesto por una combinación de características fuertes y débiles. Si bien las características débiles a veces serán útiles, el método de bosque aleatorio garantizará que no creemos un modelo que solo se ajuste a nuestros datos de entrenamiento.

Para crear nuestra canalización de aprendizaje automático, pasaremos una lista de características como la variable "campos". Estas son las características que utilizará nuestro clasificador. El clasificador espera recibir estas características como una sola columna llamada "características", por lo que utilizamos VectorAssembler para transformar los datos al formato requerido.

El siguiente código crea una canalización de aprendizaje automático y configura nuestros parámetros usando MLlib:

```
def create_pipeline(campos):
    ensamblador = VectorAssembler(inputCols=campos, outputCol="características")
    rf = RandomForestClassifier(labelCol="etiqueta", featuresCol="características",
                               numTrees=30, maxDepth=10)
    return Pipeline(etapas=[ensamblador, rf])
```

El RandomForestClassifier utiliza estos parámetros:

etiquetaCol

El nombre del campo que contiene la variable que queremos predecir; es decir, si un par de nodos tienen un vínculo

característicasCol

El nombre del campo que contiene las variables que se utilizarán para predecir si un par de nodos tiene un vínculo

numTrees

El número de árboles de decisión que forman el bosque aleatorio

profundidad máxima

La profundidad máxima de los árboles de decisión

Elegimos el número de árboles de decisión y su profundidad con base en la experimentación. Podemos considerar los hiperparámetros como la configuración de un algoritmo que puede ajustarse para optimizar el rendimiento. Los mejores hiperparámetros suelen ser difíciles de determinar con antelación, y ajustar un modelo suele requerir ensayo y error.

Hemos cubierto los conceptos básicos y configurado nuestra canalización, así que profundicemos en la creación de nuestro modelo y la evaluación de su rendimiento.

Predicción de enlaces: características básicas de los gráficos

Comenzaremos creando un modelo simple que intenta predecir si dos autores tendrán una colaboración futura en función de las características extraídas de los autores comunes, el apego preferencial y la unión total de los vecinos:

Autores comunes

Calcula el número de triángulos potenciales entre dos autores. Esto refleja la idea de que dos autores con coautores en común podrían presentarse y colaborar en el futuro.

El apego preferencial

genera una puntuación para cada par de autores multiplicando el número de coautores que cada uno tiene. Se intuye que es más probable que los autores colaboren con alguien que ya es coautor de muchos artículos.

Unión total de vecinos

Encuentra el número total de coautores que tiene cada autor, menos los duplicados.

En Neo4j, podemos calcular estos valores mediante consultas Cypher. La siguiente función calculará estas medidas para el conjunto de entrenamiento:

```
def aplicar_características_de_entrenamiento_gráfico(datos):
    consulta =
        DESBLOQUEAR $pares COMO par
        COINCIDIR (p1) DONDE id(p1) = par.nodo1 COINCIDIR (p2)
        DONDE id(p2) = par.nodo2 DEVOLVER par.nodo1 COMO
        nodo1, par.nodo2 COMO nodo2, tamaño([(p1)-
            [:CO_AUTHOR_EARLY]-(a)-
            [:CO_AUTHOR_EARLY]-(p2) | a]) COMO commonAuthors,
            tamaño((p1)-[:CO_AUTHOR_EARLY]-(a)) * tamaño((p2)-[:CO_AUTHOR_EARLY]-
            (a)) COMO prefAttachment, tamaño(apoc.coll.toSet( [(p1)-[:CO_AUTHOR_EARLY]-
            (a) | id(a)] +
            [(p2)-[:CO_AUTHOR_EARLY]-(a) | id(a)]
            )) AS totalNeighbors

    pares = [{"nodo1": fila["nodo1"], "nodo2": fila["nodo2"]} para la fila en
        datos.collect()] características =
        spark.createDataFrame(graph.run(query, {"pairs":
            pairs}).to_data_frame()) devuelve
        datos.join(características, ["nodo1", "nodo2"])
```

Y la siguiente función los calculará para el conjunto de prueba:

```
def aplicar_grafía_prueba_características(datos):
    consulta =
        DESBLOQUEAR $pares COMO par
        COINCIDIR (p1) DONDE id(p1) = par.nodo1 COINCIDIR (p2)
        DONDE id(p2) = par.nodo2 DEVOLVER par.nodo1 COMO
        nodo1, par.nodo2 COMO nodo2, tamaño([(p1)-
            [:CO_AUTHOR]-(a)-[:CO_AUTHOR]-
            (p2) | a]) COMO commonAuthors, tamaño((p1)-[:CO_AUTHOR]-(a)) * tamaño((p2)-[:CO_AUTHOR]-(a))
            AS prefAdjunto,
            tamaño(apoc.coll.toSet(
            [(p1)-[:CO_AUTHOR]-(a) | id(a)] + [(p2)-[:CO_AUTHOR]-(a) | id(a)]
            )) AS totalNeighbors

    pares = [{"nodo1": fila["nodo1"], "nodo2": fila["nodo2"]}
        para la fila en data.collect()]
    características = spark.createDataFrame(graph.run(consulta,
```

```

{"pares": pares}).to_data_frame()
devolver datos.join(características, ["nodo1", "nodo2"])

```

Ambas funciones toman un DataFrame que contiene pares de nodos en las columnas nodo1 y nodo2. Luego, construimos una matriz de mapas que contiene estos pares y calculamos las medidas de cada par de nodos.



La cláusula UNWIND es particularmente útil en este capítulo para tomar una gran colección de pares de nodos y devolver todas sus características en una consulta.

Podemos aplicar estas funciones en Spark a nuestros DataFrames de entrenamiento y prueba con el siguiente código:

```

datos_de_entrenamiento = aplicar_características_de_entrenamiento_gráfico(datos_de_entrenamiento)
datos_de_prueba = aplicar_características_de_prueba_gráfico(datos_de_prueba)

```

Exploremos los datos de nuestro conjunto de entrenamiento. El siguiente código trazará un histograma de la frecuencia de commonAuthors:

```

plt.style.use('fivethirtyeight') fig, axs =
plt.subplots(1, 2, figsize=(18, 7), sharey=True) charts = [(1, "han
colaborado"), (0, "no han colaborado")]

para índice, gráfico in enumerate(charts):
    etiqueta, título = gráfico
    filtrado = datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"] ==
etiqueta) autores_comunes = filtrado.toPandas()["Autores_comunes"]
    histograma = autores_comunes.value_counts().sort_index()
    histograma /= float(histograma.sum())
    histograma.plot(kind="bar", x='Autores_comunes', color="azul_oscuro",
ax=axs[índice], título=f'Autores que {título} (etiqueta={etiqueta})')
    axs[índice].xaxis.set_label_text(" Autores comunes")

plt.tight_layout()
plt.show()

```

Podemos ver el gráfico generado en la [Figura 8-8](#).

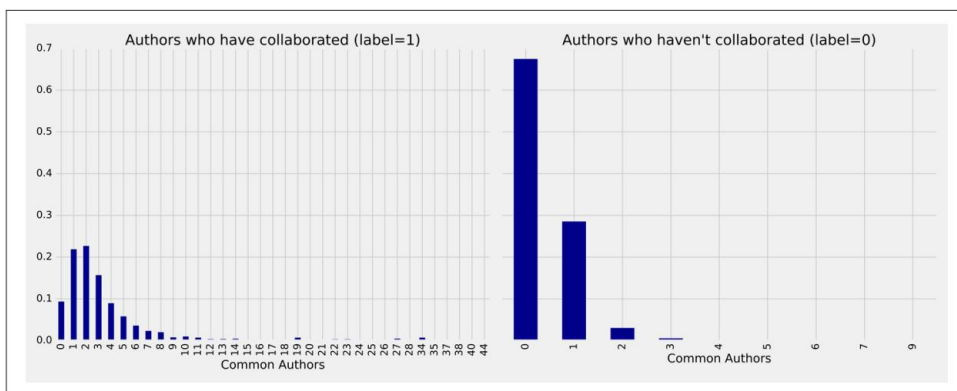


Figura 8-8. Frecuencia de autores comunes

A la izquierda, vemos la frecuencia de autores comunes cuando los autores han colaborado, y a la derecha, la frecuencia de autores comunes cuando no lo han hecho. Para quienes no han colaborado (lado derecho), el número máximo de autores comunes es 9, pero el 95 % de los valores son 1 o 0. No es sorprendente que, de las personas que no han colaborado en un artículo, la mayoría tampoco tenga muchos otros coautores en común. Para quienes han colaborado (lado izquierdo), el 70 % tiene menos de cinco coautores en común, con un pico entre uno y dos coautores adicionales.

Ahora queremos entrenar un modelo para predecir enlaces faltantes. La siguiente función lo hace:

```
def train_model(campos, datos_de_entrenamiento):
    pipeline = create_pipeline(campos) model =
    pipeline.fit(datos_de_entrenamiento) return model
```

Comenzaremos creando un modelo básico que solo use commonAuthors. Podemos crearlo ejecutando este código:

```
modelo_básico = modelo_de_entrenamiento(["Autores_comunes"], datos_de_entrenamiento)
```

Con nuestro modelo entrenado, veamos su rendimiento con datos ficticios. El siguiente código evalúa el código con diferentes valores para commonAuthors:

```
eval_df = spark.createDataFrame(
    [(0.), (1.), (2.), (10.), (100.)], ["autorescomunes"])

(modelo_básico.transform(eval_df)
.select("commonAuthors", "probabilidad", "predicción") .show(truncate=False))
```

Ejecutar ese código dará el siguiente resultado:

	probabilidad de autores comunes	predicción
0	[0,7540494940434322,0,24595050595656787]	0,0
1	[0,7540494940434322,0,24595050595656787]	0,0
2	[0.0536835525078107,0.9463164474921892]	1.0
10	[0.0536835525078107,0.9463164474921892]	1.0

Si tenemos un valor de commonAuthors menor que 2, hay un 75 % de probabilidad de que no haya una relación entre los autores, por lo que nuestro modelo predice 0. Si tenemos un valor de commonAuthors de 2 o más, hay un 94 % de probabilidad de que haya una relación entre los autores, por lo que nuestro modelo predice 1.

Ahora, evaluemos nuestro modelo con respecto al conjunto de prueba. Si bien existen varias maneras de evaluar el rendimiento de un modelo, la mayoría se derivan de unas pocas métricas predictivas de referencia, como se describe en la **Tabla 8-1**:

Tabla 8-1. Métricas predictivas

Medida	Fórmula	Descripción
Exactitud	$\frac{\text{Verdaderos positivos} + \text{Verdaderos negativos}}{\text{Predicciones totales}}$	La fracción de predicciones correctas de nuestro modelo, o el número total de predicciones correctas dividido entre el número total de predicciones. Tenga en cuenta que la precisión por sí sola puede ser engañosa, especialmente cuando nuestros datos no están balanceados. Por ejemplo, si tenemos un conjunto de datos con 95 gatos y 5 perros, y nuestro modelo predice que cada imagen es un gato, tendremos una precisión del 95 % a pesar de no identificar correctamente a ninguno de los perros.
Precisión	$\frac{\text{Verdaderos Positivos}}{\text{Verdaderos positivos} + \text{Falso positivo}}$	La proporción de identificaciones positivas correctas. Una precisión baja indica más falsos positivos. Un modelo sin falsos positivos tiene una precisión de 1.0.
Recordatorio (tasa de positivos verdaderos)	$\frac{\text{Verdaderos Positivos}}{\text{Verdaderos positivos} + \text{falsos negativos}}$	La proporción de positivos reales que se identifican correctamente. Un nivel de recuperación bajo indica más falsos negativos. Un modelo que no produce falsos negativos tiene un nivel de recuperación de 1.0.
Tasa de falsos positivos	$\frac{\text{Falsos positivos}}{\text{Falsos positivos} + \text{Verdaderos negativos}}$	La proporción de positivos incorrectos identificados. Una puntuación alta indica más falsos positivos.
Característica de funcionamiento del receptor Curva (ROC)	Gráfico XY	La curva ROC es un gráfico de la tasa de recuperación (tasa de verdaderos positivos) frente a la tasa de falsos positivos en diferentes umbrales de clasificación. El área bajo la curva (AUC) mide el área bidimensional bajo la curva ROC desde el eje XY (0,0) hasta (1,1).

Utilizaremos exactitud, precisión, recuperación y curvas ROC para evaluar nuestros modelos. La exactitud es una medida aproximada, por lo que nos centraremos en aumentar nuestras medidas generales de precisión y recuperación. Utilizaremos las curvas ROC para comparar cómo las características individuales modifican las tasas de predicción.



Dependiendo de nuestros objetivos podremos querer favorecer diferentes medidas. Por ejemplo, podríamos querer eliminar todos los falsos negativos de los indicadores de enfermedad, pero no querríamos forzar las predicciones de todos los factores a un resultado positivo. Podríamos establecer múltiples umbrales para diferentes modelos que sometán algunos resultados a una inspección secundaria sobre la probabilidad de resultados falsos.

La reducción de los umbrales de clasificación produce más resultados positivos en general, lo que aumenta tanto los falsos positivos como los verdaderos positivos.

Utilicemos la siguiente función para calcular estas medidas predictivas:

```
def evaluar_modelo(modelo, datos_de_prueba):
    # Ejecutar el modelo contra el conjunto de pruebas
    predictions = model.transform(test_data)

    # Calcular recuentos de verdaderos positivos, falsos positivos y falsos negativos tp
    = predicciones[(predictions.label == 1) &
                    (predicciones.prediccion == 1)].count()
    fp = predicciones[(predicciones.etiqueta == 0) &
                      (predicciones.prediccion == 1)].count()
    fn = predicciones[(predicciones.etiqueta == 1) &
                      (predicciones.prediccion == 0)].count()

    # Calcular manualmente la recuperación y la precisión
    recuperación = float(tp) / (tp + fn)
    precisión = float(tp) / (tp + fp)

    # Calcular la precisión utilizando el evaluador de clasificación binaria de Spark MLLib accuracy
    = BinaryClassificationEvaluator().evaluate(predictions)

    # Calcular la tasa de falsos positivos y la tasa de verdaderos positivos usando las funciones de sklearn
    labels = [row["label"] for row in predictions.select("label").collect()] preds = [row["probability"][1] for
    row in predictions.select
        ("probabilidad").collect()] fpr, tpr,
    umbral = roc_curve(etiquetas, preds) roc_auc = auc(fpr, tpr)

    devolver { "fpr": fpr, "tpr": tpr, "roc_auc": roc_auc, "precisión": precisión,
              "recordar": recordar, "precisión": precisión }
```

Luego escribiremos una función para mostrar los resultados en un formato más fácil de consumir:

```
def display_results(resultados):
    resultados = {k: v para k, v en resultados.items() si k no está en ["fpr", "tpr",
        "roc_auc"]} devolver
    pd.DataFrame({"Medida": lista(resultados.keys()),
                  "Puntuación": lista(resultados.valores())})
```

Podemos llamar a la función con este código y mostrar los resultados:

```
resultados_básicos = evaluar_modelo(modelo_básico, datos_de_prueba)
mostrar_resultados(resultados_básicos)
```

Las medidas predictivas para el modelo de autores comunes son:

medir la puntuación

precisión 0,864457

recordar 0,753278

precisión 0,968670

Este no es un mal comienzo, dado que predecimos la colaboración futura basándonos únicamente en el número de autores comunes en nuestros pares de autores. Sin embargo, obtenemos una visión más amplia si consideramos estas medidas en contexto. Por ejemplo, este modelo tiene una precisión de 0,968670, lo que significa que es muy bueno para predecir la existencia de vínculos. Sin embargo, nuestra recuperación es de 0,753278, lo que significa que no es bueno para predecir cuándo no existen vínculos.

También podemos trazar la curva ROC (correlación de verdaderos positivos y falsos positivos) utilizando las siguientes funciones:

```
def create_roc_plot():
    plt.style.use('classic') fig =
    plt.figure(figsize=(13, 8)) plt.xlim([0, 1])
    plt.ylim([0, 1])
    plt.ylabel('Tasa de
verdaderos positivos') plt.xlabel('Tasa
de falsos positivos') plt.rc('ejes',
prop_cycle=(cycler('color',
                    ['r', 'g', 'b', 'c', 'm', 'y', 'k']))) plt.plot([0, 1], [0, 1],
linestyle='--', label='Puntuación aleatoria (AUC = 0,50)') return plt, fig
```

```
def add_curve(plt, título, fpr, tpr, roc): plt.plot(fpr,
tpr, etiqueta=f'{título} (AUC = {roc:0.2})')
```

Lo llamamos así:

```
plt, fig = crear_gráfico_roc()

add_curve(plt, "Autores comunes",
resultados_básicos["fpr"], resultados_básicos["tpr"], resultados_básicos["roc_auc"])

plt.legend(loc='abajo a la derecha')
plt.show()
```

Podemos ver la curva ROC de nuestro modelo básico en la [Figura 8-9](#).

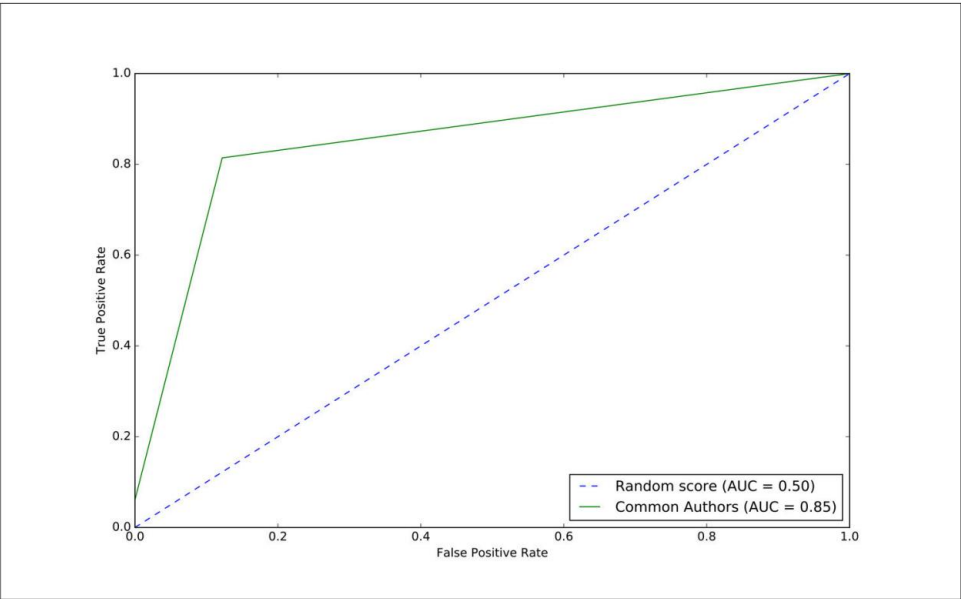


Figura 8-9. Curva ROC del modelo básico

El modelo de autores comunes nos da una puntuación de área bajo la curva (AUC) de 0,86. Aunque esto nos da una medida predictiva general, necesitamos el gráfico (u otro medidas) para evaluar si esto se ajusta a nuestro objetivo. En la Figura 8-9 vemos que a medida que obtenemos cerca de un 80% de tasa de verdaderos positivos (recuerde) nuestra tasa de falsos positivos alcanza aproximadamente el 20%. Esto podría ser problemático en escenarios como la detección de fraudes, donde se producen falsos positivos. caro de perseguir.

Ahora usemos las otras funciones gráficas para ver si podemos mejorar nuestras predicciones. Antes de entrenar nuestro modelo, veamos cómo se distribuyen los datos. Podemos ejecutar lo siguiente: El siguiente código muestra estadísticas descriptivas para cada una de nuestras características gráficas:

```
(datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"]==1)
    .describir()
    .select("resumen", "autorescomunes", "adjuntopref", "vecinostotales")
    .espectáculo())

(datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"]==0)
    .describir()
    .select("resumen", "autorescomunes", "adjuntopref", "vecinostotales")
    .espectáculo())
```

Podemos ver los resultados de ejecutar esos fragmentos de código en las siguientes tablas:

	Resumen de autores comunes	PrefAttachment	Vecinos totales
contar	81096	81096	81096
significar	3.5959233501035808	69.93537289138798	10.082408503502021

Resumen de autores comunes		PrefAttachment	Vecinos totales
desviación estándar	4.715942231635516	171.47092255919472	8.44109970920685
mín.	0	1	2
máximo	44	3150	90

Resumen de autores comunes		PrefAttachment	Vecinos totales
contar	81096	81096	81096
significar	0,37666469369635985	48,18137762651672	12,97586810693499
desviación estándar	0,6194576095461857	94,92635344980489	10,082991078685803
mín.	0	1	1
máximo	9	1849	89

Características con mayores diferencias entre enlaces (coautoría) y sin enlace (sin coautoría)

La titulación) debería ser más predictiva porque la brecha es mayor. El valor promedio

para prefAttachment es mayor para los autores que han colaborado en comparación con los que no lo han hecho.

No lo han hecho. Esa diferencia es aún más sustancial para los autores comunes. Observamos que

No hay mucha diferencia en los valores de totalNeighbors, lo que probablemente significa

Esta característica no será muy predictiva. También es interesante la gran desviación estándar como

así como los valores mínimos y máximos para la fijación preferencial. Esto es lo que

Podríamos esperar que las redes de mundo pequeño con concentradores (superconexiones)

tores).

Ahora vamos a entrenar un nuevo modelo, agregando apego preferencial y unión total de vecinos.

bors, ejecutando el siguiente código:

```
campos = ["autorescomunes", "adjuntopref", "vecinostotales"]
modelo_grafia = modelo_entrenamiento(campos, datos_entrenamiento)
```

Y ahora evaluemos el modelo y mostremos los resultados:

```
resultados_grafia = evaluar_modelo(modelo_grafia, datos_de_prueba)
mostrar_resultados(resultados_gráficos)
```

Las medidas predictivas para el modelo gráfico son:

medir la puntuación	
precisión	0,978351
recordar	0,924226
precisión	0,943795

Nuestra precisión y recuperación han aumentado sustancialmente, pero la precisión ha disminuido un poco.

y seguimos clasificando erróneamente alrededor del 8% de los enlaces. Grafiquemos la curva ROC y

Compare nuestros modelos básicos y gráficos ejecutando el siguiente código:

```
plt, fig = crear_gráfico_roc()

add_curve(plt, "Autores comunes",
           resultados_básicos["fpr"], resultados_básicos["tpr"],
           resultados_básicos["roc_auc"])

add_curve(plt, "Grafía",
           resultados_gráficos["fpr"], resultados_gráficos["tpr"],
           resultados_gráficos["roc_auc"])

plt.legend(loc='abajo a la derecha')
plt.show()
```

Podemos ver el resultado en la Figura 8-10.

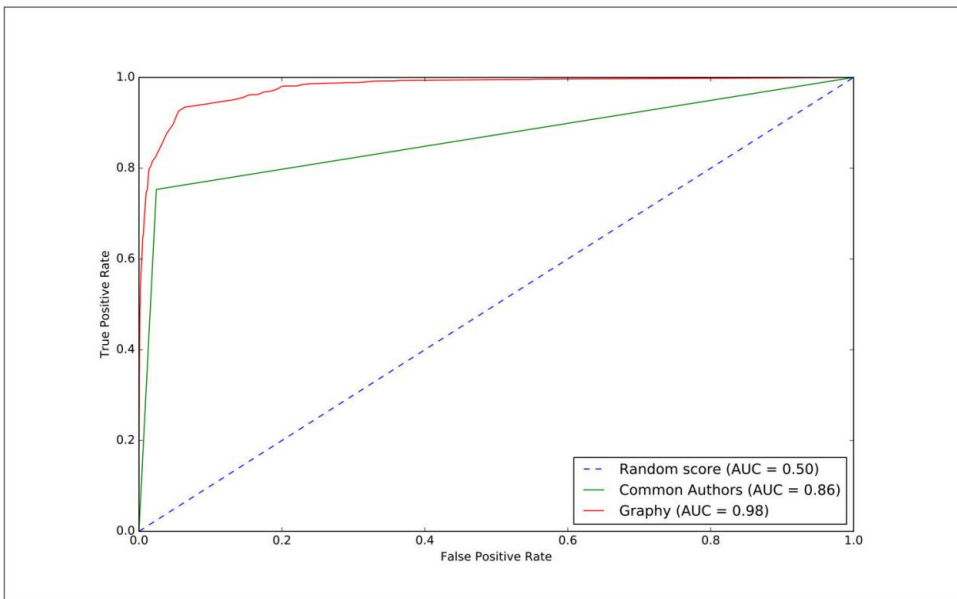


Figura 8-10. Curva ROC del modelo gráfico.

En general, parece que vamos en la dirección correcta y es útil visualizar las comparaciones para tener una idea de cómo los diferentes modelos impactan en nuestros resultados.

Ahora que tenemos más de una característica, queremos evaluar cuáles marcan la diferencia.

Utilizaremos la importancia de la característica para clasificar el impacto de las diferentes características en la predicción de nuestro modelo. Esto nos permite evaluar la influencia de diferentes algoritmos y estadísticas en los resultados.



Para calcular la importancia de las características, el algoritmo de bosque aleatorio de Spark promedia la reducción de impurezas en todos los árboles del bosque. La impureza es la frecuencia con la que las etiquetas asignadas aleatoriamente son incorrectas.

Las clasificaciones de características se comparan con el grupo de características que estamos evaluando, siempre normalizadas a 1. Si clasificamos una característica, su importancia es 1,0 ya que tiene el 100 % de la influencia en el modelo.

La siguiente función crea un gráfico que muestra las características más influyentes:

```
def plot_feature_importance(campos, importancias_de_las_características):
    df = pd.DataFrame({"Característica": campos, "Importancia": importancia_de_las_características})
    df = df.sort_values("Importancia", ascendente=Falso) ax =
    df.plot(kind='bar', x='Característica', y='Importancia', leyenda=Ninguna)
    ax.xaxis.set_label_text("")
    plt.tight_layout()
    plt.show()
```

Y lo llamamos así:

```
rf_model = graphy_model.stages[-1]
plot_feature_importance(campos, rf_model.featureImportances)
```

Los resultados de ejecutar esa función se pueden ver en [la Figura 8-11](#).

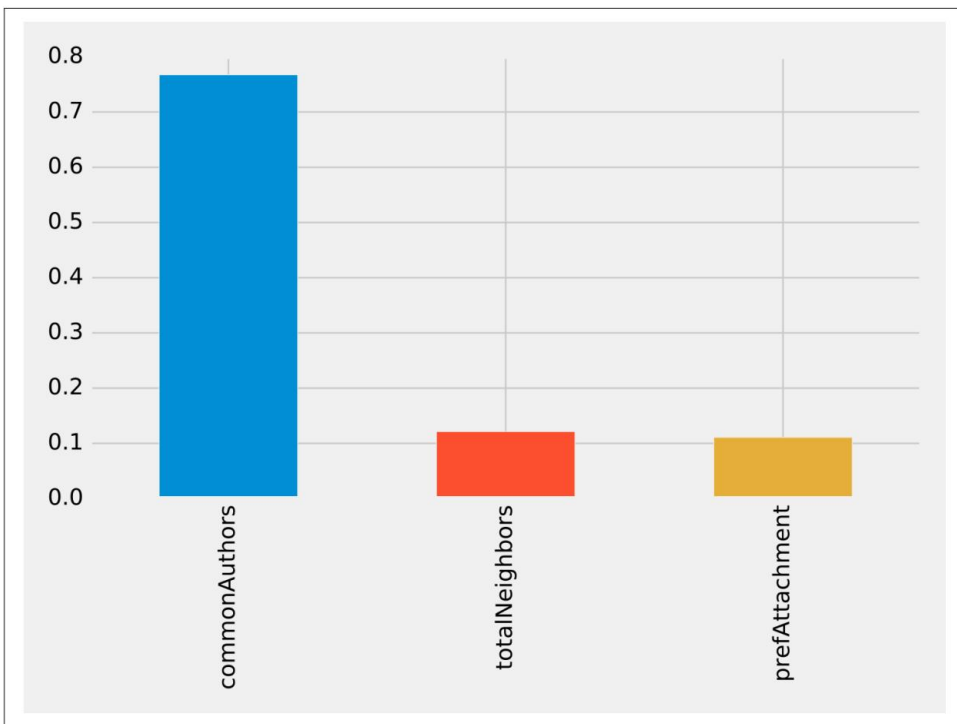


Figura 8-11. Importancia de las características: modelo gráfico

De las tres características que hemos utilizado hasta ahora, commonAuthors es la más importante por un amplio margen.

Para comprender cómo se crean nuestros modelos predictivos, podemos visualizar uno de los árboles de decisión de nuestro bosque aleatorio utilizando la biblioteca `spark-tree-plotting`. El siguiente código genera un archivo `GraphViz`:

```
de spark_tree_plotting importar exportar_graphviz

cadena_de_puntos = export_graphviz(rf_model.trees[0],
    NombresDeCaracteristicas=Campos, NombresDeCategorias=[], NombresDeClase=["Verdadero", "Falso"],
    Relleno=Verdadero, EsquinasRedondeadas=Verdadero, HojasRedondeadas=Verdadero)

con open("/tmp/rf.dot", "w") como archivo:
    file.write(dot_string)
```

Luego podemos generar una representación visual de ese archivo ejecutando el siguiente comando desde la terminal:

```
punto -Tpdf /tmp/rf.punto -o /tmp/rf.pdf
```

La salida de ese comando se puede ver en la [Figura 8-12](#).

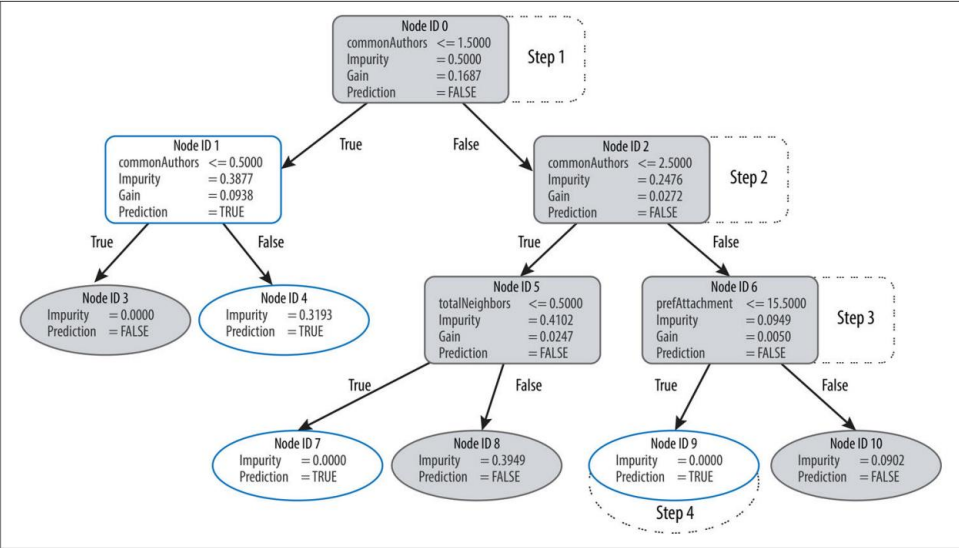


Figura 8-12. Visualización de un árbol de decisiones

Imaginemos que estamos usando este árbol de decisión para predecir si un par de nodos con las siguientes características están vinculados:

Preferencias de autores comunes Adjuntos totales Vecinos		
10	12	5

Nuestro bosque aleatorio recorre varios pasos para crear una predicción:

1. Empezamos desde el nodo 0, donde tenemos más de 1,5 commonAuthors, por lo que seguimos La rama falsa baja hasta el nodo 2.
2. Tenemos más de 2,5 commonAuthors aquí, por lo que seguimos la rama False para nodo 6.
3. Tenemos una puntuación inferior a 15,5 para prefAttachment, lo que nos lleva al nodo 9.
4. El nodo 9 es un nodo hoja en este árbol de decisión, lo que significa que no tenemos que verificar más condiciones: el valor de Predicción (es decir, Verdadero) en este nodo es la predicción del árbol de decisión.
5. Finalmente, el bosque aleatorio evalúa el elemento que se predice frente a una colección de estos árboles de decisión y hace su predicción basándose en el más popular.

Ahora veamos cómo agregar más funciones gráficas.

Predicción de enlaces: Triángulos y el coeficiente de agrupamiento. Las soluciones de recomendación suelen basar las predicciones en algún tipo de métrica triangular, así que veamos si nos ayudan con nuestro ejemplo. Podemos calcular el número de triángulos de los que forma parte un nodo y su coeficiente de agrupamiento ejecutando lo siguiente. consulta:

```
LLAMAR algo.triangleCount('Autor', 'CO_AUTHOR_EARLY', { escritura: verdadero,
propiedad de escritura: 'trianglesTrain', propiedad de clusteringCoefficient:
'coefficientTrain'});
```

```
LLAMAR algo.triangleCount('Autor', 'CO_AUTOR', { escribir:true,
writeProperty:'trianglesTest', clusteringCoefficientProperty: 'coefficientTest'});
```

La siguiente función agregará estas características a nuestros DataFrames:

```
def aplicar_características_triángulos(datos, propiedad_triángulos, propiedad_coeficiente):
    consulta =
        DESBLOQUEAR $pares COMO par
        COINCIDIR (p1) DONDE id(p1) = par.nodo1 COINCIDIR (p2)
        DONDE id(p2) = par.nodo2 DEVOLVER par.nodo1 COMO
        nodo1, par.nodo2 COMO nodo2,
            apoc.coll.min([p1[$triángulosProp],
            p2[$triángulosProp]])
            AS minTriángulos,
            apoc.coll.max([p1[$triángulosProp], p2[$triángulosProp]])
            AS maxTriángulos,
            apoc.coll.min([p1[$coeficientePropiedad], p2[$coeficientePropiedad]])
            Coeficiente mínimo AS,
            apoc.coll.max([p1[$coeficientePropiedad], p2[$coeficientePropiedad]])
            Coeficiente máximo AS
    """"

    params =
        { "pares": [{"nodo1": fila["nodo1"], "nodo2": fila["nodo2"]}
                    para la fila en data.collect()],
          "triángulosProp": triángulos_prop,
          "coeficienteProp": coeficiente_prop

    }
    características = spark.createDataFrame(graph.run(consulta, parámetros).to_data_frame()) return
    datos.join(características, ["nodo1", "nodo2"])
```



Observe que hemos usado prefijos mínimos y máximos para nuestros algoritmos de conteo de triángulos y coeficientes de agrupamiento. Necesitamos una manera de evitar que nuestro modelo aprenda según el orden en que se pasan los autores por pares desde nuestro grafo no dirigido. Para ello, hemos dividido estas características por autores con conteos mínimos y máximos.

Podemos aplicar esta función a nuestros DataFrames de entrenamiento y prueba con el siguiente código:

```
datos_de_entrenamiento = aplicar_caracteristicas_de_triángulos(datos_de_entrenamiento,
                                                                "triángulosTren", "coeficienteTren")
datos_de_prueba = aplicar_caracteristicas_de_triángulos(datos_de_prueba,
                                                         "Pruebadetriángulos", "Pruebadecoficiente")
```

Y ejecute este código para mostrar estadísticas descriptivas para cada una de las características de nuestro triángulo:

```
(datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"]==1)
 .describir()
 .select("resumen", "minTriángulos", "maxTriángulos",
         "coeficiente mínimo", "coeficiente máximo")
 .espectáculo())

(datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"]==0)
 .describir()
 .select("resumen", "minTriángulos", "maxTriángulos", "minCoficiente",
         "coeficiente máximo")
 .espectáculo())
```

Podemos ver los resultados de ejecutar esos fragmentos de código en las siguientes tablas.

Resumen minTriángulos		MaxTriángulos	Coeficiente mínimo	coeficiente máximo
contar	81096	81096	81096	81096
significar	19.478260333431983	27.73590559337082	0.5703773654487051	0.8453786164620439
desviación estándar	65.7615282768483	74.01896188921927	0.3614610553659958	0.2939681857356519
mín.	0	0	0.0	0.0
máximo	622	785	1.0	1.0

Resumen minTriángulos		MaxTriángulos	Coeficiente mínimo	coeficiente máximo
contar	81096	81096	81096	81096
significar	5.754661142349808	35.651980368945445	0.49048921333297446	0.860283935358397
desviación estándar	20.639236521699	85.82843448272624	0.3684138346533951	0.2578219623967906
mín.	0	0	0.0	0.0
máximo	617	785	1.0	1.0

Observe en esta comparación que no hay una diferencia tan grande entre el coautor-datos de envío y no coautoría. Esto podría significar que estas características no son tan predictivas.

Podemos entrenar otro modelo ejecutando el siguiente código:

```
campos = ["autorescomunes", "adjuntopref", "vecinostotales",
          "minTriángulos", "maxTriángulos", "minCoficiente", "maxCoficiente"]
modelo_de_triángulo = modelo_de_entrenamiento(campos, datos_de_entrenamiento)
```

Y ahora evaluemos el modelo y mostremos los resultados:

```
resultados_triángulo = evaluar_modelo(modelo_triángulo, datos_prueba)
mostrar_resultados(triángulo_resultados)
```


Las medidas predictivas para el modelo de triángulos se muestran en esta tabla:

medir la puntuación	
precisión	0,992924
recordar	0.965384
precisión	0,958582

Nuestras medidas predictivas han mejorado notablemente al añadir cada nueva característica al modelo anterior. Agreguemos nuestro modelo de triángulos a nuestro gráfico de curva ROC con el siguiente código:

```
plt, fig = crear_gráfico_roc()

add_curve(plt, "Autores comunes",
           resultados_básicos["fpr"], resultados_básicos["tpr"], resultados_básicos["roc_auc"])

add_curve(plt, "Grafía",
           resultados_gráficos["fpr"], resultados_gráficos["tpr"],
           resultados_gráficos["roc_auc"])

add_curve(plt, "Triángulos",
           resultados_triángulo["fpr"], resultados_triángulo["tpr"],
           resultados_triángulo["roc_auc"])

plt.legend(loc='abajo a la derecha')
plt.show()
```

Podemos ver el resultado en la [Figura 8-13](#).

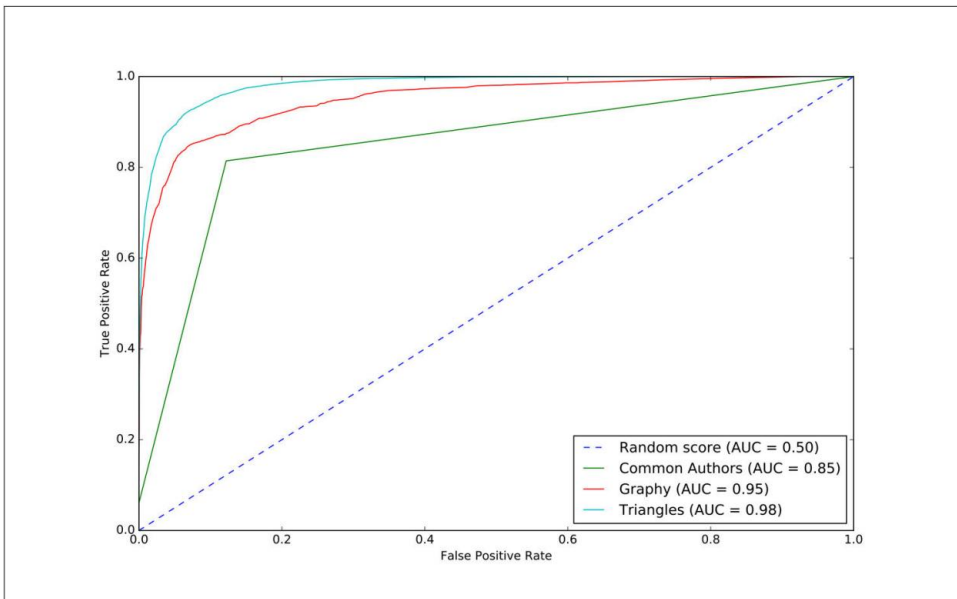


Figura 8-13. Curva ROC para el modelo de triángulos

En general, nuestros modelos han mejorado, y nos encontramos en torno al 90% en medidas predictivas. Aquí es cuando las cosas suelen complicarse, porque se logran los avances más fáciles, pero aún hay margen de mejora. Veamos cómo han cambiado las características importantes:

```
rf_model = triangle_model.stages[-1]
plot_feature_importance(campos, rf_model.featureImportances)
```

Los resultados de ejecutar esa función se pueden ver en [la Figura 8-14](#).

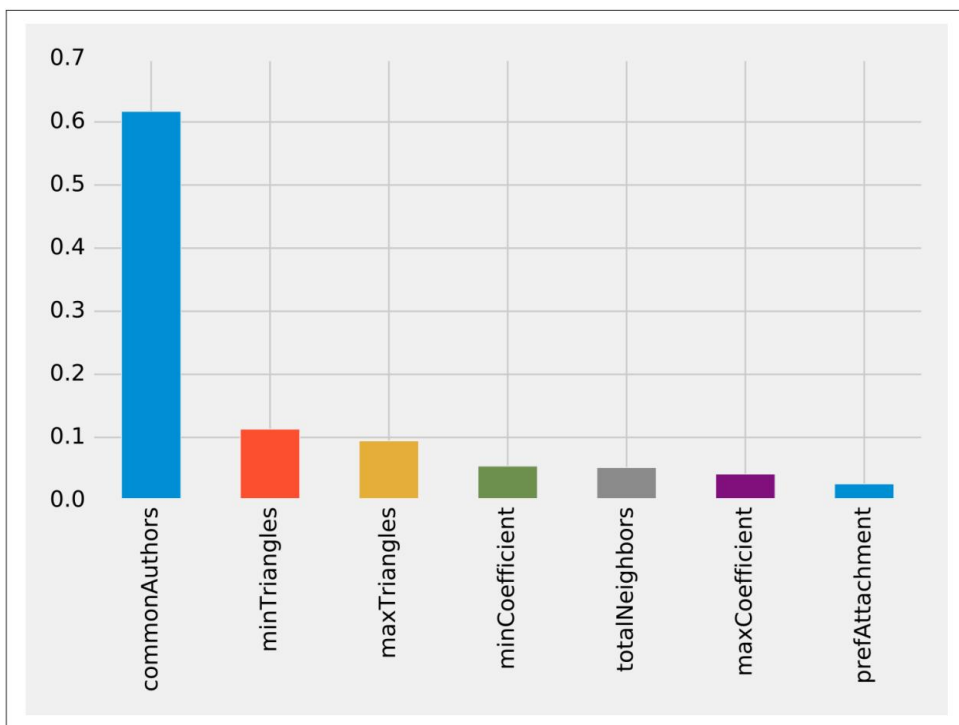


Figura 8-14. Importancia de las características: modelo de triángulos

La función de autores comunes sigue teniendo el mayor impacto en nuestro modelo. Quizás debamos analizar nuevas áreas y ver qué sucede al añadir información de la comunidad.

Predicción de vínculos: Detección de comunidades. Planteamos la

hipótesis de que los nodos que pertenecen a la misma comunidad tienen mayor probabilidad de tener un vínculo entre sí, si aún no lo tienen. Además, creemos que cuanto más unida sea una comunidad, mayor será la probabilidad de que se formen vínculos.

Primero, calcularemos comunidades más detalladas utilizando el algoritmo de Propagación de Etiquetas de Neo4j. Para ello, ejecutamos la siguiente consulta, que almacenará la comunidad en la propiedad "partitionTrain" para el conjunto de entrenamiento y "partitionTest" para el conjunto de prueba:

```
LLAMAR algo.labelPropagation("Autor", "CO_AUTOR_TEMPRANO",
  "AMBOS", {partitionProperty: "partitionTrain"});
```

```
LLAMA algo.labelPropagation("Autor", "CO_AUTOR", "AMBOS",
  {propiedadDePartición: "pruebaDePartición"});
```

También calcularemos grupos más detallados mediante el algoritmo de Louvain. Este algoritmo devuelve clústeres intermedios, y almacenaremos el más pequeño de estos clústeres en la propiedad `louvainTrain` para el conjunto de entrenamiento y `louvainTest` para el conjunto de prueba.

```
LLAMA algo.louvain.stream("Autor", "CO_AUTOR_TEMPRANO",
                          {includeIntermediateCommunities:true})
YIELD nodeld, comunidad, comunidades WITH
algo.getNodeById(nodeld) AS node, comunidades[0] AS lowestCommunity SET node.louvainTrain
= lowestCommunity;

LLAMA algo.louvain.stream("Autor", "CO_AUTOR",
                          {includeIntermediateCommunities:true})
YIELD nodeld, comunidad, comunidades WITH
algo.getNodeById(nodeld) AS node, comunidades[0] AS lowestCommunity SET node.louvainTest
= lowestCommunity;
```

Ahora crearemos la siguiente función para devolver los valores de estos algoritmos:

```
def apply_community_features(datos, propiedad_de_partición, propiedad_de_louvain):
    consulta =
        DESBLOQUEAR $pares COMO par
        COINCIDIR (p1) DONDE id(p1) = par.nodo1 COINCIDIR (p2)
        DONDE id(p2) = par.nodo2 DEVOLVER par.nodo1 COMO
        nodo1, par.nodo2 COMO nodo2, CASO
            CUANDO p1[$particiónProp] =
                p2[$particiónProp] ENTONCES 1 SI NO 0 TERMINA COMO samePartition, CASO CUANDO
                    p1[$louvainProp] = p2[$louvainProp] ENTONCES 1
                SI NO 0 TERMINA COMO sameLouvain
    """

    params =
        { "pares": [{"nodo1": fila["nodo1"], "nodo2": fila["nodo2"]} para
                    fila en data.collect()],
          "particiónProp": partición_prop,
          "louvainProp": louvain_prop

        }
    características = spark.createDataFrame(graph.run(consulta, parámetros).to_data_frame()) return
    datos.join(características, ["nodo1", "nodo2"])
```

Podemos aplicar esta función a nuestros DataFrames de entrenamiento y prueba en Spark con el siguiente código:

```
datos_de_entrenamiento =
    aplicar_funciones_comunitarias(datos_de_entrenamiento,
    "partitionTrain", "louvainTrain")
datos_de_prueba = aplicar_funciones_comunitarias(datos_de_prueba, "partitionTest", "louvainTest")
```

Y podemos ejecutar este código para ver si los pares de nodos pertenecen a la misma partición:

```
plt.style.use('fivethirtyeight') fig, axs =
plt.subplots(1, 2, figsize=(18, 7), sharey=True) charts = [(1, "han colaborado"),
(0, "no han colaborado")]

para índice, gráfico en enumerate(gráficos):
    etiqueta, título = gráfico
```

```

filtrado = datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"] ==
etiqueta) valores = (filtrado.withColumn('samePartition',
F.when(F.col("mismaPartición") == 0, "Falso") .de lo

```

```

contrario("Verdadero")) .groupby("mismaPartición") .agg(F.count("etiqueta").alias("contar")) .select("mismaPartición", "contar")
valores.set_index("misma partición", drop=True, inplace=True)
valores.plot(kind="bar", ax=axes[index], legend=None, title=f"Autores
que {title} (label={label})")
axes[index].xaxis.set_label_text("Misma partición")

```

```

plt.tight_layout()
plt.show()

```

Vemos los resultados de ejecutar ese código en la **Figura 8-15**.

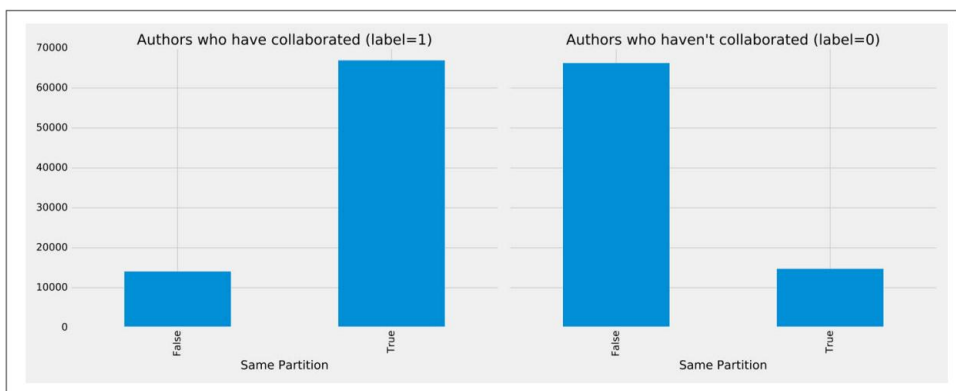


Figura 8-15. Particiones iguales

Parece que esta función podría ser bastante predictiva: es mucho más probable que los autores que han colaborado estén en la misma partición que quienes no lo han hecho. Podemos hacer lo mismo para los clústeres de Lovaina ejecutando el siguiente código:

```

plt.style.use('fivethirtyeight') fig, axes =
plt.subplots(1, 2, figsize=(18, 7), sharey=True) charts = [(1, "han
colaborado"), (0, "no han colaborado")]

para índice, gráfico en enumerate(charts):
etiqueta, título = gráfico
filtrado = datos_de_entrenamiento.filter(datos_de_entrenamiento["etiqueta"] ==
etiqueta) valores = (filtrado.withColumn('sameLovain',
F.when(F.col("sameLovain") == 0,
"False") .otherwise("True")))

.groupby("sameLovain") .agg(F.count("label").alias("count")) .select("sameLovain", "count") .toPandas())

```

```

values.set_index("sameLouvain", drop=True, inplace=True)
values.plot(kind="bar", ax=axes[index], legend=None,
            title=f"Autores que {title} (label={label})")
axes[index].axis.set_label_text("Same Louvain")

plt.tight_layout()
plt.show()

```

Podemos ver los resultados de ejecutar ese código en la [Figura 8-16](#).

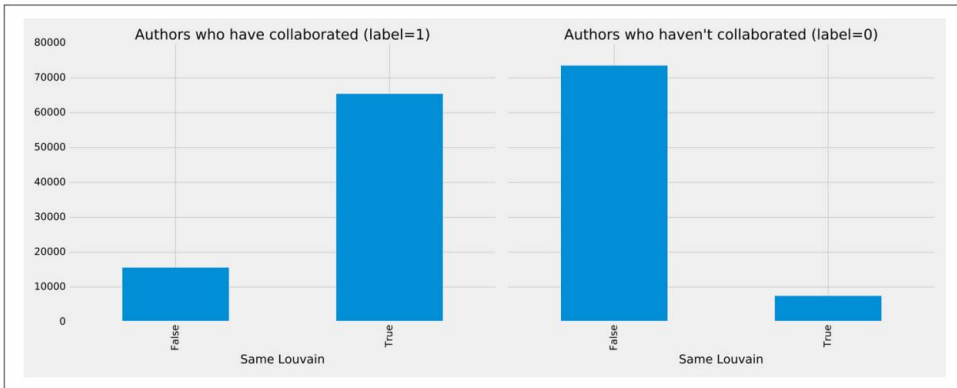


Figura 8-16. Mismos cúmulos de Lovaina

Parece que esta característica también podría ser bastante predictiva: es probable que los autores que han colaborado estén en el mismo grupo, y es muy poco probable que los que no lo han hecho estén en el mismo grupo.

Podemos entrenar otro modelo ejecutando el siguiente código:

```

campos = ["autorescomunes", "adjuntopref", "vecinostotales",
          "minTriángulos", "maxTriángulos", "minCoeficiente", "maxCoeficiente",
          "mismaPartición", "mismaLouvain"]
community_model = train_model(campos, datos_de_entrenamiento)

```

Y ahora evaluemos el modelo y mostremos los resultados:

```

resultados_de_la_comunidad = evaluar_modelo(modelo_de_la_comunidad, datos_de_prueba)
mostrar_resultados(resultados_de_la_comunidad)

```

Las medidas predictivas para el modelo comunitario son:

medir la puntuación

precisión 0,995771

recordar 0.957088

precisión 0,978674

Algunas de nuestras medidas han mejorado, así que a modo de comparación, grafiquemos la curva ROC para todos nuestros modelos ejecutando el siguiente código:

```
plt, fig = crear_gráfico_roc()

add_curve(plt, "Autores comunes",
           resultados_básicos["fpr"], resultados_básicos["tpr"], resultados_básicos["roc_auc"])

add_curve(plt, "Grafía",
           resultados_gráficos["fpr"], resultados_gráficos["tpr"],
           resultados_gráficos["roc_auc"])

add_curve(plt, "Triángulos",
           resultados_triángulo["fpr"], resultados_triángulo["tpr"],
           resultados_triángulo["roc_auc"])

add_curve(plt, "Comunidad",
           resultados_comunidad["fpr"], resultados_comunidad["tpr"],
           resultados_comunidad["roc_auc"])

plt.legend(loc='abajo a la derecha')
plt.show()
```

Podemos ver el resultado en la Figura 8-17.

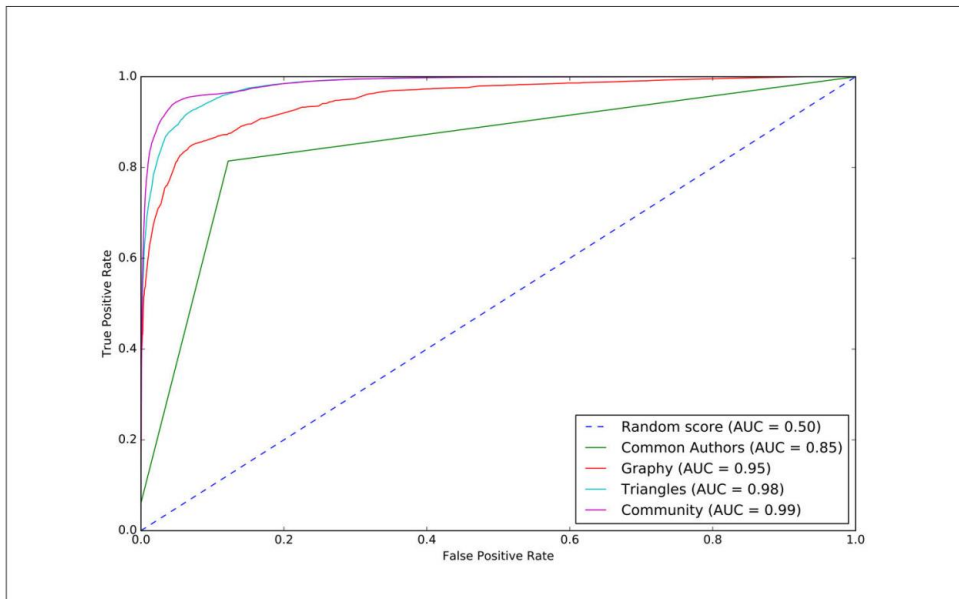


Figura 8-17. Curva ROC del modelo comunitario.

Podemos ver mejoras con la incorporación del modelo comunitario, así que veamos cuáles son las características más importantes:

```
rf_model = community_model.stages[-1]
plot_feature_importance(campos, rf_model.featureImportances)
```

Los resultados de ejecutar esa función se pueden ver en [la Figura 8-18](#).

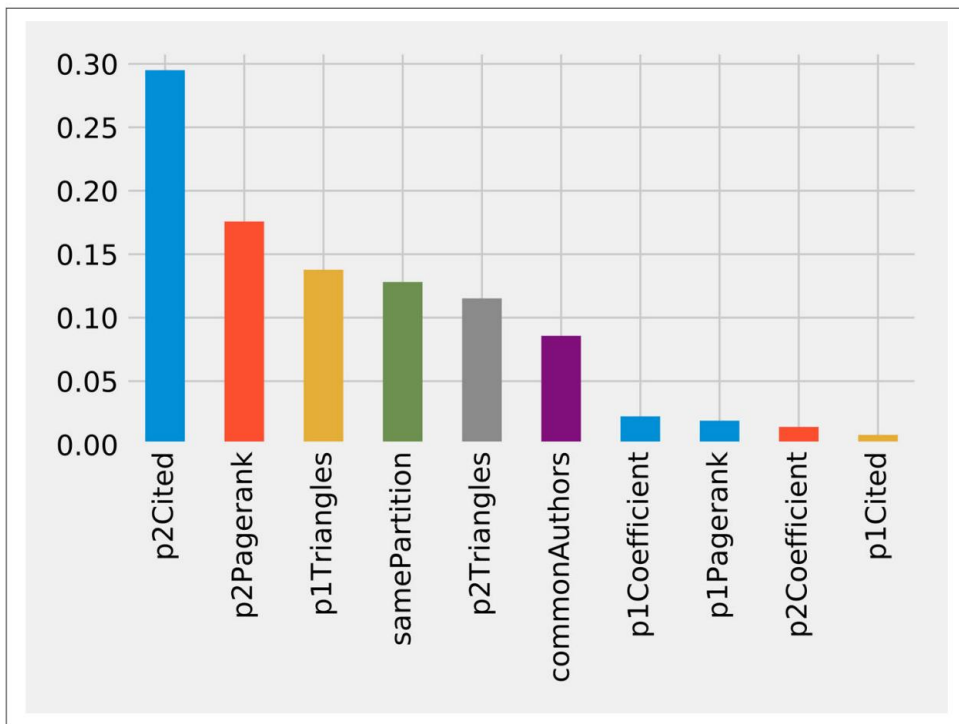


Figura 8-18. Importancia de las características: modelo de comunidad

Si bien el modelo de autores comunes es, en general, muy importante, conviene evitar un elemento excesivamente dominante que pueda sesgar las predicciones basadas en nuevos datos. Los algoritmos de detección de comunidades tuvieron una gran influencia en nuestro último modelo, con todas las características incluidas, y esto contribuye a perfeccionar nuestro enfoque predictivo.

En nuestros ejemplos, hemos visto que las características simples basadas en grafos son un buen punto de partida. A medida que añadimos más características basadas en grafos y algoritmos de grafos, seguimos mejorando nuestras medidas predictivas. Ahora contamos con un modelo sólido y equilibrado para predecir vínculos de coautoría.

El uso de grafos para la extracción de características conectadas puede mejorar significativamente nuestras predicciones. Las características y algoritmos ideales de los grafos varían según los atributos de los datos, incluyendo el dominio de la red y la forma del grafo. Sugerimos considerar primero los elementos predictivos de los datos y probar hipótesis con diferentes tipos de características conectadas antes de realizar ajustes.

Ejercicios de lectura

Hay varias áreas de investigación y maneras de construir otros modelos. Aquí hay algunas ideas para explorar más a fondo:

¿Qué tan predictivo es nuestro modelo con los datos de conferencias que no incluimos? ¿Qué sucede al eliminar algunas características al probar nuevos datos? ¿Dividir los años de forma diferente para el entrenamiento y las pruebas afecta nuestra predicción?

- Este conjunto de datos también tiene citas entre artículos; ¿podemos usar esos datos para generar predicciones? ¿Diferentes características o predecir citas futuras?

Resumen

En este capítulo, analizamos el uso de características y algoritmos de grafos para mejorar el aprendizaje automático. Abordamos algunos conceptos preliminares y, a continuación, presentamos un ejemplo detallado de la integración de Neo4j y Apache Spark para la predicción de enlaces. Ilustramos cómo evaluar modelos de clasificadores de bosque aleatorio e incorporar diversos tipos de características conectadas para mejorar nuestros resultados.

Resumiendo

En este libro, cubrimos conceptos de gráficos, así como plataformas de procesamiento y análisis. Luego, repasamos numerosos ejemplos prácticos sobre cómo usar algoritmos de grafos en Apache Spark y Neo4j. Para finalizar, analizamos cómo los grafos mejoran el aprendizaje automático.

Los algoritmos gráficos son la base del análisis de sistemas del mundo real: desde la prevención del fraude y la optimización del enrutamiento de llamadas hasta la predicción de la propagación de la gripe. Esperamos que se una a nosotros y desarrolle sus propias soluciones únicas que aprovechen los datos altamente conectados de la actualidad.

Información adicional y recursos

En esta sección, abordamos brevemente información adicional que puede ser útil para algunos lectores. Analizaremos otros tipos de algoritmos, otra forma de importar datos a Neo4j y otra biblioteca de procedimientos. También encontrará recursos para encontrar conjuntos de datos, asistencia con la plataforma y capacitación.

Otros algoritmos

Se pueden utilizar muchos algoritmos con datos de grafos. En este libro, nos hemos centrado en los más representativos de los algoritmos de grafos clásicos y los más útiles para los desarrolladores de aplicaciones. Algunos algoritmos, como la coloración y las heurísticas, se han omitido porque son de mayor interés en casos académicos o porque se pueden derivar fácilmente.

Otros algoritmos, como la detección de comunidades basada en bordes, son interesantes, pero aún no se han implementado en Neo4j ni en Apache Spark. Prevemos que la lista de algoritmos gráficos utilizados en ambas plataformas aumentará a medida que se incremente el uso del análisis de gráficos.

También existen categorías de algoritmos que se utilizan con grafos, pero que no son estrictamente de naturaleza gráfica. Por ejemplo, en [el capítulo 8](#) analizamos algunos algoritmos utilizados en el contexto del aprendizaje automático. Otra área importante son los algoritmos de similitud, que suelen aplicarse a recomendaciones y predicción de enlaces. Los algoritmos de similitud determinan qué nodos se parecen más entre sí mediante diversos métodos para comparar elementos como los atributos de los nodos.

Importación masiva de datos de Neo4j y Yelp

La importación de datos a Neo4j con el lenguaje de consulta Cypher utiliza un enfoque transaccional. [La Figura A-1](#) ilustra una descripción general de este proceso.

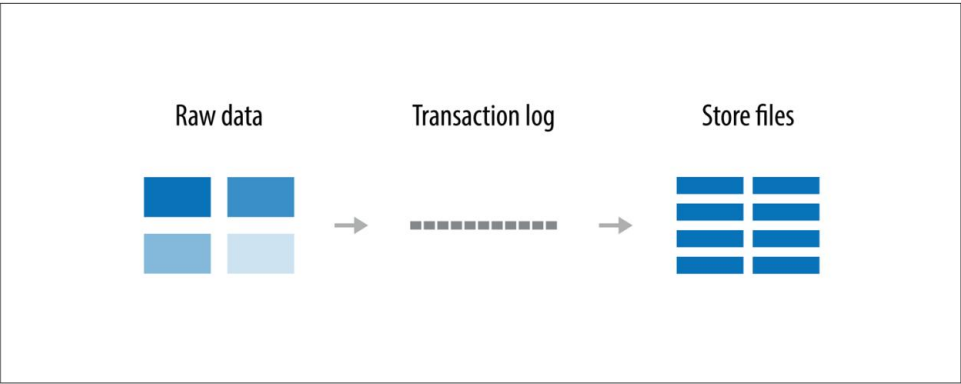


Figura A-1. Importación basada en cifrado

Si bien este método funciona bien para la carga incremental de datos o la carga masiva de hasta 10 millones de registros, la herramienta de importación de Neo4j es una mejor opción para importar conjuntos de datos masivos iniciales. Esta herramienta crea los archivos de almacenamiento directamente, omitiendo el registro de transacciones, como se muestra en la [Figura A-2](#).

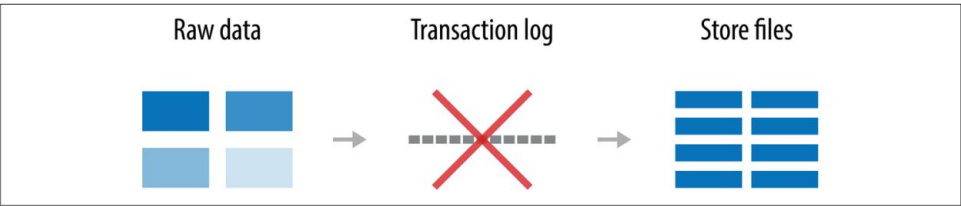


Figura A-2. Uso de la herramienta de importación de Neo4j

La herramienta de importación de Neo4j procesa archivos CSV y espera que estos tengan encabezados específicos. La [Figura A-3](#) muestra un ejemplo de archivos CSV que la herramienta puede procesar.

Nodes

id:ID(User)	name
1234	Bob
1235	Alice
1236	Erika

id:ID(Review)	text	stars
678	Awesome	3
679	Mediocre	2
680	Really bad	1

Relationships

:START_ID(User)	:END_ID(Review)
1234	678
1235	679
1236	680

Figura A-3. Formato de los archivos CSV que procesa Neo4j Import

El tamaño del conjunto de datos de Yelp implica que la herramienta de importación de Neo4j es la mejor opción para importar los datos a Neo4j. Los datos están en formato JSON, por lo que primero debemos convertirlos al formato que la herramienta de importación de Neo4j espera. La Figura A-4 muestra un ejemplo del JSON que necesitamos transformar.

```
{
  "business_id": "UtmSQZThPQlt3SmkAG0jg",
  "name": "Maggie & Stella's Gifts",
  "neighborhood": "Oakland",
  "address": "209 Oakland Ave",
  "city": "Pittsburgh",
  "state": "PA",
  "postal_code": "15213",
  "latitude": 40.4414214,
  "longitude": -79.9564571,
  "stars": 3.5,
  "review_count": 3,
  "is_open": 1,
  "attributes": {
    "BikeParking": "True",
    "BusinessAcceptsCreditCards": "True",
    "BusinessParking": { "garage": False, "street": False, "validated":
False, "lot": False, "valet": False },
    "RestaurantsPriceRange2": "2"
  },
  "categories": "Flowers & Gifts, Gift Shops, Shopping",
  "hours": {
    "Monday": "9:0-18:0",
    "Tuesday": "9:0-18:0",
    "Wednesday": "9:0-18:0",
    "Thursday": "9:0-18:0",
    "Friday": "9:0-17:0",
    "Saturday": "10:0-17:0"
  }
}
```



Python script

Figura A-4. Transformación de JSON a CSV

Usando Python, podemos crear un script sencillo para convertir los datos a un archivo CSV. Una vez transformados los datos a ese formato, podemos importarlos a Neo4j. Encontrará instrucciones detalladas sobre cómo hacerlo en el [repositorio de recursos del libro](#).

APOC y otras herramientas Neo4j

[Awesome Procedures on Cypher \(APOC\)](#) es una biblioteca que contiene más de 450 procedimientos y funciones para facilitar tareas comunes como la integración, limpieza y conversión de datos, así como funciones de ayuda generales. APOC es la biblioteca estándar de Neo4j.

Neo4j también tiene otras herramientas que pueden usarse junto con su biblioteca de algoritmos gráficos, como una aplicación de "juego" de algoritmos para exploración sin código.

Estos se pueden encontrar en su [sitio para desarrolladores para algoritmos gráficos](#).

Encontrar conjuntos de datos

Encontrar un conjunto de datos gráficos que se ajuste a los objetivos o hipótesis de prueba puede ser un desafío. Además de revisar artículos de investigación, considere explorar índices para conjuntos de datos de red:

El [Proyecto de Análisis de Redes de Stanford \(SNAP\)](#) incluye varios conjuntos de datos, junto con artículos relacionados y guías de uso. El [Índice de Redes](#)

[Complejas de Colorado \(ICON\)](#) es un índice de búsqueda de conjuntos de datos de redes de calidad para investigación de diversos dominios de la ciencia de redes. [La Colección de Redes de Koblenz \(KONECT\)](#) incluye grandes conjuntos de datos de redes de diversos tipos para la investigación en ciencia de redes.

La mayoría de los conjuntos de datos requerirán algún ajuste para transformarlos en algo más útil. estera.

Asistencia con las plataformas Apache Spark y Neo4j

Existen numerosos recursos en línea para las plataformas Apache Spark y Neo4j. Si tiene alguna pregunta específica, le animamos a contactar con sus respectivas comunidades:

- Para preguntas generales sobre Spark, suscríbase a users@spark.apache.org en [la página de la Comunidad Spark](#).

- Para preguntas sobre GraphFrames, utilice el [rastreador de problemas de](#)

- [GitHub](#). • Para todas las preguntas sobre Neo4j (incluidas las relacionadas con algoritmos gráficos), visite [Neo4j Comunidad en línea](#).

Capacitación

Existen numerosos recursos excelentes para iniciarse en el análisis de grafos. Una búsqueda de cursos o libros sobre algoritmos de grafos, ciencia de redes y análisis de redes revelará numerosas opciones. Algunos ejemplos excelentes de aprendizaje en línea incluyen:

- [Curso de Análisis de redes sociales aplicadas en Python de Coursera](#) •
- [Serie de YouTube sobre análisis de redes sociales de Leonid Zhukov](#)
- [El curso de Análisis de redes de Stanford](#) incluye videoconferencias, listas de lectura y otros recursos
- [Complexity Explorer](#) ofrece cursos en línea sobre ciencia de la complejidad

Índice

A

A* algoritmo

Shortest Path, 56-58

con Neo4j, 57

densidad real, 24

gráficos acíclicos, 18

gráficos cíclicos vs., 22-23

árboles y, 23

addedMessages, 54, 67, 86 análisis

de datos de vuelos

de aerolíneas con Apache Spark, 166-181

retrasos desde ORD, 170-172

análisis exploratorio, 168

retrasos relacionados con la niebla desde

SFO, 172-174 aeropuertos interconectados por

aerolínea, 174-181 aeropuertos

populares,

168-170 Alexa, 185 procesamiento

centrado en algoritmos, 30 algoritmo All Pairs Shortest Path (APSP), 46, 60-65

secuencia de operaciones, 60-62

cuándo usar, 62

con Apache Spark, 62 con

Neo4j, 63-65

Amazon, 185

análisis antilavado de dinero (ALD), xiv

Apache Spark

aproximadamente, 32-34

Algoritmo de ruta más corta de todos los pares con, 62

analizando datos de vuelos de aerolíneas con, 166-181

(ver también datos de vuelos de aerolíneas)

Búsqueda en amplitud con, 46

Cercanía Centralidad con, 86-88

Componentes conectados con, 125-126

Centralidad de grado con, 83

importar datos de gráficos sociales a, 80

importar datos de gráficos de dependencia de software

a, 114 importar conjunto de datos de

transporte a, 43 instalar, 34

Propagación de etiquetas con,

130 recursos en línea, 228

PageRank con, 103-105

PageRank personalizado con, 107

Algoritmo de ruta más corta (ponderado), 54-56

Algoritmo de ruta más corta de fuente única con, 67-69

Proyecto Spark Graph, 33

Componentes fuertemente conectados con, 120

Triángulo Contar con, 117

cuándo usarlo, 31

Centralidad de intermediación aproximada, 160

inteligencia artificial (IA), 183 grado

promedio, 81 camino

más corto promedio, 27

Biblioteca de procedimientos asombrosos en

Cypher (APOC), 35, 148, 192, 228

B

Número de tocino, 50

Barabási, Albert-László, 11

Algoritmo de centralidad de intermediación, 78, 92-99

puentes y puntos de control, 93

cálculo, 93

cuándo utilizar, 94

con Neo4j, 95-97 con

el conjunto de datos Yelp, 159-162

clasificación binaria, 200

grafos bipartitos, 18, 25
Boruvka, Otakar, 70
Búsqueda en amplitud (BFS), 45-47
puentes, 93
Problema de los puentes de Königsberg, 2, 49
importación de datos en masa, herramienta de importación Neo4j para, 225-227
Paralelo sincrónico masivo (BSP), 30

Investigación sobre el
cáncer, xiv Algoritmos de centralidad, 27 ,
77-108 Centralidad de intermediación,
92-99 Centralidad de cercanía,
84-92 Centralidad de grado,
81-84
descripción general, 78 Brandes aproximado
aleatorio, 98 datos de grafos
sociales para, 79-81 Aeropuerto Internacional O'Hare de
Chicago (ORD), datos en retrasos
de, 170-172 redes de citas,
124 Clauset, A.,
5 camarilla,
23 Algoritmo de centralidad de cercanía, 78, 84-92
Variación de centralidad armónica, 91
Variación de Wasserman-Faust, 89-91
cuándo usar, 85
con Apache Spark, 86-88 con
Neo4j, 88
Algoritmo de coeficiente de agrupamiento, 109
(véase también Recuento y agrupamiento de triángulos)
Algoritmos de coeficientes)
clústeres, definidos, 19
Índice de redes complejas de Colorado (ICON),
228
algoritmos de detección de comunidades, 27, 109-143
Componentes conectados, 124-127 para
predicción de enlaces, 218-223
Algoritmo de propagación de etiquetas, 127-133
Modularidad de Lovaina, 133-143 datos
del gráfico de dependencia del software para,
112-114
Componentes fuertemente conectados, 119-124
Recuento de triángulos y coeficiente de agrupamiento
114-119
comunidades de validación, 143
grafo completo, 23
componentes, definidos, 19

Configuración que supera a una única
Hilo (COSTO), 29
Algoritmo de componentes conectados, 109, 124-127
cuándo
usar, 124 con Apache
Spark, 125-126 con Neo4j, 126
gráficos conectados,
19 contexto, xiv, 183 costos,
50, 57 ciclos, 22
gráficos cíclicos,
18

Cifra, 33, 166, 225
Cifrado para Apache Spark (CAPS), 33

D

D'Orazio, Francesco, 10
amortiguación/factor de amortiguación, 102, 104
DataFrame, 32
conjuntos de datos, fuentes
para, 228 deduplicación, 124
Algoritmo de centralidad de grado, 78, 81-84
alcance de un nodo, 81
cuándo utilizar, 82
con datos del aeropuerto,
168-170 con Apache Spark, 83
distribución de grados, 81
grado de un nodo, 81
algoritmo de pasos delta, 69 gráficos
densos, 18, 23 densidad
de relaciones, 111
Búsqueda en profundidad (DFS), 48-49, 120
diámetro de un gráfico, 27
Dijkstra, Edsger, 50
Algoritmo de Dijkstra (véase algoritmo del camino más
corto)
grafo acíclico dirigido (DAG), 22 grafos
dirigidos, 18, 21 relaciones
direccionales, xiii grafos
desconectados, 19 distancia
(término), 50

E Eguíluz, Víctor M., 4
diagrama entidad-relación (ERD), xiii Erdős,
Paul, 51 Euler,
Leonhard, 2 Camino
euleriano, 49

F

Facebook, 184

Faust, Katherine, 89

extracción de características,

186 importancia de las

características, 210 selección

de características, 186

vectores

de características, 186 características conectadas

extracción/selección de características, 185-189

características del algoritmo gráfico, 188

grafía, 187

Fischer, Michael J., 124

Fleurquin, Pablo, 4 red

alimentaria, 5

Freeman, Linton C., 81, 93

G

Galler, Bernard A., 124 Girvan–

Newman (GN), 143 coeficiente de agrupamiento global,

116 patrones globales, 6 Google PageRank,

99 Pregel, 30 Grandjean,

Martin, 4

algoritmos de

grafos (en

general) acerca de, 3-5

centralidad, 27 detección de comunidad,

27 definido, 3

(ver también

algoritmos específicos) importancia

de, 8-12 en

la práctica, 145-181 búsqueda de rutas,

27 tipos de, 27 análisis de

grafos acerca de, 3-5

definido, 3 casos de

uso, 12 motores

de cómputo de

grafos, 31

bases de

datos de grafos,

31 incrustación de grafos, definida,

186 grafo global, 6, 186

grafo local, 6 , 186 plataformas de grafos

Apache Spark, 32-34

Neo4j, 34-37

consideraciones de

plataforma, 29

plataformas representativas, 31-37

selección de una plataforma, 31

procesamiento de gráficos, 6-8, 30

Algoritmos de búsqueda de gráficos, 39-49

definidos, 39

Algoritmos de búsqueda de gráficos, 40

Búsqueda en amplitud, 45-47

Búsqueda en profundidad, 48-49

transporte de datos de gráficos para, 41-44

teoría de grafos, 15-28

acerca de, 15-28

orígenes de, 2

terminología, 15 tipos

y estructuras, 16 algoritmos de

recorrido de grafos

Búsqueda en amplitud, 45-47

Búsqueda en profundidad, 48-49

procesamiento centrado en grafos, 30

GraphFrames, 32, 103, 114 gráficos

(en general) acerca de,

2 acíclico

vs. cíclico, 18, 22-23 bipartito, 18,

25 atributos comunes,

18 conectado vs. desconectado,

19 sabores de, 18-25 k-partito, 18, 25

monopartito, 18, 24

disperso vs. denso,

23 no dirigido vs. dirigido,

18, 21 no ponderado vs.

ponderado, 18, 19 conjuntos de datos de

grafías, 228 características de grafías, 187

H

Camino hamiltoniano, 49

Algoritmo de cercanía de centralidad armónica, 91

Hart, Peter, 57 hop

(term), 21, 50

procesamiento transaccional y analítico híbrido

(HTAP), 7

1 impureza, 211

enlaces

internos, 21

influencia, 99 islas, 19

K

grafos k-partitos, 18, 25

Colección de la Red de Coblenza (KONECT), 228

Problema de los puentes de Königsberg, 2, 49

Yo

Algoritmo de propagación de etiquetas (LPA), 109, 127-133

 método pull, 128

 método push, 127

 etiquetas de semilla,

 129 aprendizaje semisupervisado, 129

 cuándo usar, 129 con

 Apache Spark, 130 con Neo4j,

 131-133 con el conjunto

 de datos Yelp, 163-165 etiqueta,

definida, 15 modelo

de gráfico de propiedad etiquetado, 15

punto de referencia Lancichinetti–Fortunato–Radicchi (LFR), 143 puntos

de referencia, 62

Latora, V., 91

nodos hoja, 22

Lee, CY, 45

predicción de enlace, 190-223

 equilibrar/dividir datos para entrenamiento/prueba, 197-199

Características básicas de gráficos para,

201-213 gráfico de coautoría,

193 detección de comunidad, 218-223

creación de datos de entrenamiento y prueba equilibrados.

 conjuntos, 194-199

creación de una canalización de aprendizaje automático, 200

definido, 189 importación de datos a

Neo4j, 192 predicción de enlaces

faltantes, 199 herramientas y datos, 190-192

Triángulos y coeficiente de agrupamiento,

214-218

descubrimiento basado en la literatura (LBD), xiv

coeficiente de agrupamiento local, 115, 118

Algoritmo de modularidad de Louvain, 109, 133-143 para

 predicción de enlaces, 219-221

 agrupamiento basado en la calidad mediante

modularidad, 134-137

 cuándo utilizar, 137 con Neo4j, 138-143

Extracción/selección de

 características conectadas con aprendizaje automático (ML),

 185-189

incrustaciones de grafos, 186

grafos, contexto y precisión, 184 importancia

del contexto en, 183 predicción de

enlaces, 183 Marchiori,

M., 91 campañas de

marketing, xiv matplotlib, 148

densidad máxima,

24 algoritmo de árbol de

expansión mínimo, 40, 70-73 cuándo usar, 71 con Neo4j, 72

modularidad, 134

(véase también

algoritmo de

 modularidad de Louvain) cálculo, 134-135 agrupación

 basada en la calidad y,

 134-137 lavado de dinero, xiv grafos monopartitos,

18, 24 Moore, C., 5 Moore,

Edward F., 45 multigrafo, 17

N

pesos negativos, 51

Neo4j

 Algoritmo A* con 57

 Algoritmo de ruta más corta para todos los pares

 con, 63-65

Analizando datos de Yelp con, 145-166 (ver

 también el conjunto de datos de Yelp)

Centralidad de intermediación con, 95-97

Cercanía Centralidad con, 88

Componentes conectados con, 126

importación de conjuntos de datos de Citation Network en,

192

importar datos de gráficos sociales a, 81

importar datos de gráficos de dependencia de software a,

114 importar

conjunto de datos de transporte a, 44

Propagación de etiquetas con, 131-133

coeficiente de agrupamiento local con, 118

Modularidad de Lovaina con, 138-143

Algoritmo de árbol de expansión mínimo con 72 recursos

en línea, 228

PageRank con, 105-107

Algoritmo de paseo aleatorio con 74

Brandes aproximado aleatorio con 98

Algoritmo de ruta más corta (sin ponderar), 51-53

Algoritmo de ruta más corta (ponderado), 53

Algoritmo de ruta más corta de fuente única con, 69

Componentes fuertemente conectados con, 122-124

Triángulos con, 117 cuándo usarlos, 32

Algoritmo de los k caminos más cortos de Yen, 58

Biblioteca de algoritmos Neo4j

Camino más corto (sin ponderar), 51-53

Ruta más corta (ponderada), 53

Escritorio Neo4j, 36

Plataforma de gráficos Neo4j, 34-37

Herramienta de importación de Neo4j, 225-227

Ciencia de redes, 4 redes

gráfico como representación de, 3 tipos y estructuras, 16 Newman, MEJ, 5 Nilsson, Nils, 57

procesamiento centrado en nodos, 30 nodos Centralidad y, 77 definido, 2

O

procesamiento analítico en línea (OLAP), 7 aprendizaje en línea, 229 procesamiento de transacciones en línea (OLTP), 7 enlaces salientes, 21

PAG

Página, Larry, 99 años

PageRank, 78, 99-108 e influencia, 99

implementación de convergencia, 105 fórmula para, 100 iteración/ navegadores aleatorios/sumideros de rango, 102

Variante de PageRank personalizada, 107 cuándo usarla, 103 con Apache Spark, 103-105 con un número fijo de iteraciones, 104 con Neo4j, 105-107 con el conjunto de datos de Yelp, 154-157 biblioteca de pandas, 148

Distribución de Pareto, 11

camino, definido, 16

Algoritmos de búsqueda de rutas, 27, 39-44

Todos los pares camino más corto, 60-65

Algoritmo de árbol de expansión mínimo, 70-73

Algoritmo de paseo aleatorio, 73-75

Camino más corto, 49-60

Ruta más corta de una sola fuente, datos de gráficos de transporte 65-70 para, 41-44

gráficos ponderados y, 20

Pearson, Karl, 73

PageRank personalizado (PPR), 103, 107 nodos pivotales, 93 ley de potencia, 11 apego preferencial, 9

Pregel, 30

Algoritmo de Prim, 70

motores de recomendación de productos, 185

propiedades, definidas, 16

pseudografos, 17

pyspark REPL, 34

Agrupación basada en la calidad Q , 134-137

R

Raghavan, Usha Nandini, 127 Ramasco, José J., 4 bosque aleatorio, 200, 213 red aleatoria, 18 algoritmo Random Walk, 40, 73-75 cuándo usar, 74 con Neo4j, 74 algoritmo de centralidad de Brandes aleatorio aproximado (RA-Brandes), 98 sumidero de rango, 102

Raphael, Bertram, 57 alcance de un nodo, 81

Reif, Jennifer, 37

tipo de relación, 16

procesamiento centrado en la relación, 30

relaciones (término), 1, 2

S

Aeropuerto Internacional de San Francisco (SFO), datos sobre retrasos relacionados con la niebla, 172-174 red sin escala, 18 ley de escala (ley de potencia), 11 motores de búsqueda, xiv

etiquetas de semillas,

129 aprendizaje semisupervizado, 129

Problema de los siete puentes de Königsberg, 2, 49

Algoritmo de la ruta más corta, 40, 49-60

Algoritmo A*, 56-58 cuándo
usarlo, 50 con Apache

Spark (ponderado), 54-56 con Neo4j (no ponderado),
51-53 con Neo4j (ponderado), 53

Variación de los k caminos más cortos de Yen,

58 gráficos simples, 16

Algoritmo de ruta más corta de una sola fuente (SSSP), 40,
65-70 con

Apache Spark, 67-69 con Neo4j, 69

red de mundo

pequeño, 18 datos de gráficos

sociales para algoritmos

de centralidad, 79-81 importar a Apache

Spark, 80 importar a Neo4j, 81 análisis de

redes sociales, 116 datos de

gráficos de dependencia de software,

112-114 importar a Apache Spark, 114 importar a Neo4j,

114 árboles de expansión, 23

Proyecto Spark Graph, 33

gráficos dispersos, 18, 23

Proyecto de análisis de redes de Stanford (SNAP), 228 gráficos
estrictos, 16

Strogatz, Steven, 9

Algoritmo de componentes fuertemente conectados (SCC), 109,

119-124 cuándo utilizarlo,

120 con datos del

aeropuerto, 175-180 con Apache

Spark, 120 con Neo4j, 122-124

agujero estructural, 119

subgrafo, definido, 16

T

teletransportación, 102

conjuntos de datos de prueba,

194-199 conjuntos de datos de

entrenamiento, 194-199 entrenamiento,

recursos en línea para, 229 relaciones

transitivas, xiii

translytics, 7 conjuntos de datos

de transporte, 41-44 importación a Apache

Spark, 43 importación a Neo4j, 44

Problema del viajante (TSP), 49 procesamiento

centrado en el recorrido, 30 árboles, 23

Trémaux, Charles Pierre, 48

Algoritmos de conteo de triángulos y coeficiente de agrupamiento,
109, 114-119 para predicción

de enlaces (ejemplo de aprendizaje automático), 214-218
coeficiente de

agrupamiento global, 116 coeficiente de

agrupamiento local, 115 coeficiente de

agrupamiento local con Neo4j, 118

Conteo de triángulos con Apache Spark, 117

Triángulos con Neo4j, 117 cuándo

usarlo, 116 aplicación

para planificar viajes, 152-159

Gorjeo

Propagación de etiquetas, 129

PageRank personalizado, 103

Grafos no dirigidos U , 18, 21

Unión Hallazgo, 124

gráficos no ponderados, 18, 19

caminos más cortos no ponderados, 51-53

V

vértices, 2

(ver también nodos)

Yo

Algoritmo de cercanía de Wasserman-Faust, 89-91

Wasserman, Stanley, 89

Componentes débilmente conectados, 124 pesos

(términos), 50 gráficos

ponderados, 18, 19

Rutas más cortas ponderadas

con Apache Spark, 54-56 con

Neo4j, 53

weightProperty, 107

Y

Análisis de

conjuntos de datos de Yelp con Neo4j, 145-166

Promoción cruzada de Bellagio, 159-162

encontrar críticos influyentes de hoteles, 154-159 encontrar

categorías similares, 162-166 modelo gráfico,

147 importar a Neo4j,

147

Herramienta de importación Neo4j para,
225-227 descripción
general, 148-151 red
social, 146 consultoría de negocios de viajes, 157-159

Aplicación de planificación de viajes,
152-159 Algoritmo de los k caminos más cortos de
Yen, 58 Yen, Jin Y., 58

Acerca de los autores

Mark Needham es promotor de grafos e ingeniero de relaciones con desarrolladores en Neo4j. Trabaja para ayudar a los usuarios a integrar los grafos y Neo4j, creando soluciones sofisticadas para problemas complejos de datos. Mark tiene una amplia experiencia en datos de grafos, habiendo colaborado previamente en el desarrollo del sistema de agrupamiento causal de Neo4j. Escribe sobre sus experiencias como grafista en su popular blog <https://markhneedham.com/blog/> y tuitea [@markhneedham](#).

Amy E. Hodler es una apasionada de la ciencia de redes y directora del programa de IA y análisis de grafos en Neo4j. Promueve el uso del análisis de grafos para revelar estructuras dentro de redes del mundo real y predecir el comportamiento dinámico. Amy ayuda a los equipos a aplicar enfoques innovadores para generar nuevas oportunidades en empresas como EDS, Microsoft, Hewlett-Packard (HP), Hitachi IoT y Cray Inc. Amy siente pasión por la ciencia y el arte, y le fascinan los estudios de complejidad y la teoría de grafos. Su cuenta de Twitter es [@amyhodler](#).

Colofón

El animal de la portada de Graph Algorithms es la araña de jardín europea (*Araneus diadematus*), una araña común de Europa y también de América del Norte, donde fue introducida inadvertidamente por los colonos europeos.

La araña de jardín europea mide menos de una pulgada de largo y es de color marrón moteado con marcas pálidas, algunas de las cuales en su espalda están dispuestas de tal manera que parecen formar una pequeña cruz, lo que le da a la araña su nombre común de "araña cruzada". Estas arañas son comunes en toda su área de distribución y se notan con mayor frecuencia a fines del verano, cuando crecen hasta su tamaño máximo y comienzan a tejer sus redes.

Las arañas de jardín europeas tejen una red circular en la que atrapan a sus pequeñas presas. La red suele consumirse y volver a tejerse por la noche para asegurar y mantener su eficacia. Mientras la araña permanece oculta, una de sus patas se apoya en una "línea de señalización" conectada a la red, cuyo movimiento la alerta de la presencia de una presa que forcejea. La araña se mueve rápidamente para morder a su presa y matarla, además de inyectarle enzimas especiales que le permiten consumirla. Cuando sus redes son perturbadas por depredadores o por una perturbación involuntaria, la araña de jardín europea usa sus patas para sacudirla y luego cae al suelo sobre un hilo de seda. Cuando pasa el peligro, la araña usa este hilo para volver a ascender a su red.

Viven un año: tras eclosionar en primavera, maduran durante el verano y se aparean a finales de año. Los machos se acercan a las hembras con cautela, ya que a veces las hembras los matan y los consumen. Tras el apareamiento, la araña hembra teje un denso capullo de seda para sus huevos antes de morir en otoño.

Al ser bastante comunes y adaptarse bien a hábitats alterados por el ser humano, estas arañas han sido ampliamente estudiadas. En 1973, dos arañas de jardín hembra, llamadas Arabella y Anita, participaron en un experimento a bordo del orbitador Skylab de la NASA para evaluar el efecto de la gravedad cero en la construcción de telarañas. Tras un período inicial de adaptación a la ingravidez, Arabella construyó una tela parcial y luego una tela circular completa.

Muchos de los animales que aparecen en las portadas de O'Reilly están en peligro de extinción; todos ellos son importantes para el mundo.

La imagen de portada es una ilustración a color de Karen Montgomery, basada en un grabado en blanco y negro del Meyers Kleines Lexicon. Las fuentes de la portada son Gilroy y Guardian Sans. La fuente del texto es Adobe Minion Pro; la fuente del encabezado es Adobe Myriad Condensed; y la fuente del código es Ubuntu Mono de Dalton Maag.